



SAPIENZA
UNIVERSITÀ DI ROMA

MIP Modelling

Mixed-integer linear models
for Management Engineering

Lecture notes on modelling with integer and binary variables: logic and binary variables, links between variables, big-M, dual bounds of the LP relaxation, constructive heuristics; three families of problems and a chapter of mixed models — assignment and scheduling, location and covering, production and lot sizing, selection and knapsack, partitioning and balancing, logistics — with solved exercises, additional modelling questions and complete Python/Gurobi implementations.

Fabio Furini

sites.google.com/view/fabiofurini

Department of Computer, Control and Management Engineering — Sapienza University of Rome

Online version and downloadable material:

fabiofurini.github.io/mip-modelling/
github.com/fabiofurini/mip-modelling

Academic year 2026–2027

Contents

I	Modelling	5
1	What is a MIP model	6
1.1	Data, variables, objective, constraints	6
1.2	Why integrality matters	9
1.3	The LP relaxation and the sandwich of bounds	10
1.4	Three patterns that keep coming back	12
1.5	What the solver does with the two bounds	12
1.6	What this chapter leaves open	13
2	Logic and binary variables	14
2.1	Propositions and binary variables	14
2.2	Literals, clauses, conjunctive normal form	15
2.3	From CNF to linear constraints	16
2.4	Logical implications	17
2.5	Splitting an implication: when it is allowed	19
2.6	Counting: at most one, at least one, exactly one	19
2.7	Logical operators and set operators	20
2.8	Checking the translation, not trusting it	20
2.9	Five exercises	20
2.10	Logical constraints inside an optimisation model	22
2.11	What is left to the next chapter	24
3	Links between variables	25
3.1	Activation: aggregated and disaggregated form	26
3.2	Fixed cost, capacity and continuous flow	27
3.3	Minimum lot size and semicontinuous variables	28
3.4	Integer counts, multiple capacity, rounding up	29
3.5	The maximum auxiliary variable	30
3.6	Min-max, max-min and the range	30
3.7	The absolute value	31
3.8	Big-M: conditional constraints and disjunctions	32

3.9	Precedences and sequencing	33
3.10	"If and only if"	34
3.11	Counting the different types	35
3.12	Alldiff and binary expansion	36
3.13	Soft constraints, deviations and penalties	37
3.14	Piecewise linear functions	38
3.15	The map of techniques	39
4	Relaxations, duality and bounds	41
4.1	What a relaxation is	41
4.2	The primal/dual conversion table	42
4.3	Weak duality, strong duality	42
4.4	Three recipes for building a dual solution by hand	43
4.5	A minimisation problem, in full	43
4.6	A maximisation problem: the roles swap	46
4.7	Valid inequalities and constraints that preserve optimality	48
4.8	Stronger formulations: what it means, and how it is proved	49
4.9	What the solver says	49
4.10	LP duals are not the marginal prices of the MILP	50
4.11	The bound protocol of the course	51
5	Constructive heuristics	52
5.1	The three bin-packing heuristics	52
5.2	LPT: balancing over identical machines	53
5.3	Constructive covering heuristic	54
5.4	Knapsack constructive heuristic: a primal bound	55
5.5	Lot sizing: least unit cost period covering	56
5.6	Local search, and what it does not give	57
5.7	When the constructive heuristic fails	57
5.8	The overview of the heuristics	57
6	From the model to Python/Gurobi	59
6.1	The four classes of variables	59
6.2	The model, one family per block	59
6.3	Reading the results	60
6.4	Tolerances	61
6.5	The relaxation with <code>relax()</code>	62
6.6	The course protocol, from start to finish	62
6.7	Running the code	63
II	The problems	65
	Fifteen numerical models	66

7	Assignment and scheduling	102
	The family	102
7.1	Minimum-cost assignment with availability	103
7.2	Machines with a fixed usage cost	109
7.3	Job selection with revenues and fixed-cost machines	116
7.4	Parallel jobs: the processing time as a maximum	122
7.5	One machine, job classes with setup	127
7.6	Classes with completion bonus and “if and only if” reduction	133
7.7	Total tardiness on one machine: sequencing with big-M	139
8	Location and coverage	146
	The family	146
8.1	Capacitated facility location	147
8.2	p-median: at most k locations	152
8.3	Signal coverage with interference	158
8.4	Hub location with maximum cost	163
9	Production planning	170
	The family	170
9.1	Lot sizing with a fixed set-up cost	171
9.2	Production and workforce: two equivalent formulations	178
9.3	Vehicles: minimum lot and a bonus for variety	186
10	Mixed models	193
	Why a chapter of mixed models	193
10.1	Prizes obtainable in two ways	194
10.2	Combinatorial auction	199
10.3	Diet with a count of the foods and a minimum lot	204
10.4	Christmas trees and boxes of lights	210
10.5	Shipments in boxes	216
10.6	Summer camps	221
10.7	Antitrust split	227
10.8	Songs on CDs	232
10.9	Books on shelves	237
10.10	Two problems to model	242
III	The course	245
11	Course organization	246
11.1	The three-part path	246
11.2	The format of every exercise (and of the exam)	246
11.3	The additional questions and their solutions	247
11.4	Grading criteria	247

11.5 Typical discussion questions	247
11.6 The most common mistakes	247
11.7 Reproducibility	248
Index	249

Part I

Modelling



What is a MIP model

Script: `python/cap01_models.py`

[Online page](#)

1.1 Data, variables, objective, constraints

A *mathematical programming* model translates a decision into four ingredients, always in the same order:

1. the **data**: the numbers describing the problem, known before deciding (costs, capacities, demands, times...);
2. the **variables**: the quantities the decision maker controls;
3. the **objective**: a function of the variables to minimise or maximise;
4. the **constraints**: the equations and inequalities the variables must satisfy for the solution to be feasible.

An *optimisation* problem asks for the feasible solution (satisfying all constraints) with the best objective value.

The classes of models

- **The acronyms**: upright LP is the *class* of problems, italic LP is *one* problem of that class — the one whose $\tilde{x}(LP)$ and $z(LP)$ are written. The same for ILP, BIP and MILP.
- **The size and the data**: n variables, with $j \in \{1, 2, \dots, n\}$, and m constraints, with $i \in \{1, 2, \dots, m\}$; the costs c_j , the coefficients a_{ij} and the right-hand sides b_i .

What a model can contain

- a **linear objective function** of the variables, to be minimised or maximised;
- **linear constraints** of three types: $\geq$, for a requirement to be met; $\leq$, for a capacity to be respected; equality, for a balance or an exact assignment;
- **variables**, in one or more families, each with its own domain, declared at the end of the model.

None of the three has a fixed shape: how many constraints, of which type and with how many families of variables is decided by the problem.

What tells the four classes apart: the domains

- **LP**: all the variables are continuous, $x_j \in \mathbb{R}$, usually non-negative. In this course a model of LP type is never written from scratch: it arrives as the *relaxation* of a MIP model, when the integrality constraints are dropped.
- **ILP**: all the variables are integer, $x_j \in \mathbb{Z}$.
- **BIP**: all the variables are binary, $x_j \in \{0, 1\}$.
- **MILP**: some variables are integer or binary and others continuous.

That is where the difference in difficulty lives: an LP is solved in polynomial time, an ILP and a MILP in general are not.

A model of ILP type

How many units to buy from two suppliers, x_1 and x_2 , in whole lots.

$$\begin{array}{ll} \min & 4x_1 + 7x_2 \\ \text{subject to} & 2x_1 + 3x_2 \geq 12 \\ & x_1 + x_2 \leq 5 \\ & x_1, x_2 \in \mathbb{Z}_{\geq 0} \end{array}$$

The objective minimises the spending. The first constraint, of direction $\geq$, requires the demand of 12 units to be covered; the second, of direction $\leq$, caps the total lots at five. The last row is the domain: all the variables are integer, and the model is an ILP.

A model of BIP type

Which projects to fund: $y_k = 1$ if project k is funded, 0 otherwise.

$$\begin{array}{ll} \max & 5y_1 + 4y_2 + 6y_3 \\ \text{subject to} & y_1 + y_2 + y_3 \leq 2 \\ & y_1 + y_3 \geq 1 \\ & y_1, y_2, y_3 \in \{0, 1\} \end{array}$$

The objective maximises the value of the projects chosen. The first constraint, of direction $\leq$, says that at most two of them can be funded; the second, of direction $\geq$, that at least one of the first and the third must be. The last row is the domain: all the variables are binary, and the model is a BIP.

A model of MILP type

Two products to make, with x_1 and x_2 the quantities produced, and a plant to switch on or not, with $y = 1$ if it is switched on and $y = 0$ otherwise.

$$\begin{array}{rcll}
 \max & 3x_1 & + & 8x_2 & - & 10y \\
 \text{subject to} & x_1 & + & x_2 & & = & 6 \\
 & & & x_2 & - & 4y & \leq & 0 \\
 & x_1 & & & & \geq & 1 \\
 & x_1, & x_2 & & & \geq & 0 \\
 & & & & y & \in & \{0,1\}
 \end{array}$$

The objective maximises the revenue of the two products minus the cost of the plant. The first constraint, an equality, requires exactly the six units demanded to be produced; the second, of direction $\leq$, says that at most four units of the second product can be made, and only with the plant on, because with $y = 0$ it leaves $x_2 \leq 0$; the third, of direction $\geq$, asks for at least one unit of the first product. The last two rows are the domains: two continuous variables and one binary, and the model is a MILP.

Every datum, every set and every symbol is *defined before it is used*. In every model of these notes:

1. the variables are introduced explicitly before the formulation;
2. the objective and each family of constraints carry an equation number;
3. the domain constraints, which declare the variables, close the model;
4. after the model a list explains objective and constraints family by family.

Notation for optimal values

Optimal values name the model they optimise: $z(MILP)$ is the optimum of a model of MILP type. The relaxations and their duals, introduced later in this chapter, follow the same rule. *Feasible* solutions carry a bar ($\bar{x}$, $\bar{\pi}$), however they were built; *optimal* ones carry a tilde ($\tilde{x}$, $\tilde{\pi}$). The bounds are called *LB* (lower bound, $LB \leq z(MILP)$) and *UB* (upper bound, $z(MILP) \leq UB$), whatever the direction of the objective. When the solution a bound comes from matters, it is written in parentheses: a feasible solution $(\bar{x}_1, \bar{x}_2, \dots, \bar{x}_n)$ of a minimisation model gives

$$UB(\bar{x}) = \sum_{j=1}^n c_j \bar{x}_j,$$

and in a maximisation model the same sum is $LB(\bar{x})$.

This course works almost exclusively with MILPs: binary or integer variables for "yes/no" or counting decisions, continuous variables for quantities that are measured (time, money, flow). Chapter 3 teaches how to link them correctly.

1.2 Why integrality matters

An ILP or a MILP is not solved by "rounding" the solution of its continuous relaxation. A minimal counterexample shows this.

Rounding fails

$$\begin{aligned}
 \max \quad & x_1 + x_2 \\
 \text{subject to} \quad & 2x_1 + 2x_2 \leq 3 \\
 & x_1, x_2 \in \{0, 1\}
 \end{aligned} \tag{1.1}$$

The LP relaxation replaces $x_1, x_2 \in \{0, 1\}$ by $0 \leq x_1, x_2 \leq 1$ and has optimal value $z(LP^+) = 3/2$. Careful: that value is attained by *infinitely many* optimal solutions — every point of the segment $x_1 + x_2 = 3/2$ with $0 \leq x_1, x_2 \leq 1$ — among them $(3/4, 3/4)$, $(1, 1/2)$ and $(1/2, 1)$. Which one the solver returns depends on the algorithm and is not a property of the model: none of them is *the* solution of the relaxation. On our installation Gurobi returns the vertex $(1/2, 1)$.

Rounding gives different outcomes depending on the starting point *and* on the direction. From $(3/4, 3/4)$: upwards it gives $(1, 1)$, which violates the constraint ($2 + 2 = 4 > 3$); downwards it gives $(0, 0)$, feasible but of value 0. From this point *neither* direction finds the optimum. From $(1, 1/2)$: upwards it gives $(1, 1)$ again, infeasible; downwards it gives $(1, 0)$, feasible with value 1, which is exactly the integer optimum $z(MILP) = 1$, attained at $(1, 0)$ and at $(0, 1)$. So there is no "right" direction: on this instance downwards works, and only from one of the two points — and nothing, before solving, told us which of the two the solver would return.

Two distinct lessons. First: rounding can produce *infeasible* points, and when it produces feasible ones there is no guarantee whatsoever on their value. Second: here the difference $3/2 - 1 = 1/2$ is not the fault of rounding — *no* feasible integer point is worth more than 1.

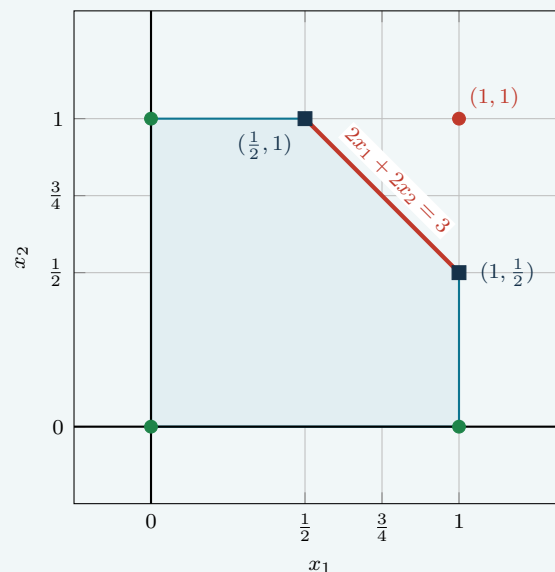


Figure 1.1: Example (1.1). In blue the feasible region of the LP^+ relaxation; in green the three feasible integer points, in red $(1, 1)$, which violates the constraint. The two blue squares are two of the infinitely many optimal solutions of the relaxation, all of value $3/2$.

The lesson: the continuous relaxation gives an indication (a bound, see below), not a solution. MILPs are solved with dedicated algorithms, not with naive heuristics on the continuous problem.

1.3 The LP relaxation and the sandwich of bounds

LP relaxation: two versions not to be confused

The **LP relaxation** of a MILP is obtained by dropping the integrality and binary requirements and keeping everything else. There are two ways of doing this, and the course uses both:

- the **relaxation with the bounds**, $z(LP^+)$: $x \in \{0, 1\}$ becomes $0 \leq x \leq 1$, and $x \in \mathbb{Z}_{\geq 0}$ becomes $x \geq 0$. This is what Gurobi solves with `relax()` and the first relaxation the solver solves;
- the **relaxation without the bounds**, $z(LP)$: $x \in \{0, 1\}$ becomes $x \geq 0$ alone, without carrying over the upper bound. It is weaker than (or equal to) the previous one, but has a more compact dual: this is the one whose dual is written by hand in the exercises.

The relaxation without the bounds is the weaker of the two, that is the bound *furthest* from the integer optimum. The two coincide when the other constraints of the model already imply $x \leq 1$: this happens, for instance, when there is an assignment constraint $\sum_m x_{jm} = 1$. Which side of $z(MILP)$ they sit on is settled by the box below.

The relaxation is a bound: on which side, and why

The relaxation *removes* constraints, so every feasible solution of the MILP stays feasible for the relaxation, while the converse is false: the relaxation optimises over a larger set of solutions. Optimising over a larger set cannot give a worse value, hence:

$$\text{maximisation: } z(LP) \geq z(LP^+) \geq z(MILP);$$

$$\text{minimisation: } z(LP) \leq z(LP^+) \leq z(MILP).$$

In a maximisation the relaxation is an *upper* bound, in a minimisation a *lower* bound. In both cases it is an **optimistic** bound: it promises at least as much as the true optimum, and in general more. It is never a solution, and the value it promises may be unreachable.

The rule is this one, and it holds throughout the course.

- **Primal bounds** come from *feasible integer solutions*: a solution that really exists, built by a heuristic (Chapter 5) or found by the solver. They are the pessimistic side.

- **Dual bounds** come either from the *optimal solution of the LP relaxation* or from any *feasible solution of its dual*. They are the optimistic side.

So the dual of the LP relaxation does *not* give a bound "from the other side": by weak duality, in a minimisation every feasible dual solution is worth at most $z(LP)$, hence at most $z(MILP)$, and it sits on the *same* side as the relaxation — in general weaker. In summary: let $(\bar{x}_1, \bar{x}_2, \dots, \bar{x}_n)$ be a feasible solution of the MILP, of value $\sum_{j=1}^n c_j \bar{x}_j$, and $(\bar{\pi}_1, \bar{\pi}_2, \dots, \bar{\pi}_m)$ a feasible solution of the dual of the relaxation, of value $\sum_{i=1}^m b_i \bar{\pi}_i$. In a **minimisation** problem the first gives $UB(\bar{x})$ and the second $LB(\bar{\pi})$:

$$LB(\bar{\pi}) \leq z(D(LP)) = z(LP) \leq z(LP^+) \leq z(MILP) \leq UB(\bar{x}),$$

in a **maximisation** problem the roles are swapped, the first gives $LB(\bar{x})$ and the second $UB(\bar{\pi})$:

$$LB(\bar{x}) \leq z(MILP) \leq z(LP^+) \leq z(LP) = z(D(LP)) \leq UB(\bar{\pi}).$$

The equality in the middle is strong duality: the dual of the relaxation is worth exactly the relaxation, and the real inequality is the one between the *value of a feasible dual solution* and that common value.

The two bounds meet in the *sandwich* $LB \leq z(MILP) \leq UB$ that runs through the whole course: Chapter 5 builds the pessimistic side, Chapter 4 the optimistic one.

Three "gaps" not to be confused

The word *gap* denotes three different quantities. Let us fix that the denominator is always $|z(MILP)|$ or $|UB|$ as indicated, that the numerator is taken in absolute value and that, when the denominator is zero, the relative gap is not written at all (the absolute difference is reported instead).

1. **Heuristic gap.** If $\bar{x}$ is the solution built by the heuristic, its value $\sum_{j=1}^n c_j \bar{x}_j$ is the bound $UB(\bar{x})$ in a minimisation (and $LB(\bar{x})$ in a maximisation), and the gap is

$$\frac{|UB(\bar{x}) - z(MILP)|}{|z(MILP)|}$$

when the optimum is known. It is the gap reported in the exercise tables, because the course instances are solved to optimality. When the optimum is *not* known it is computed against a dual bound, which takes the place of $z(MILP)$: the number one gets is larger, and it is the only one that can be claimed.

2. **Certified gap** between two known bounds, without knowing $z(MILP)$: $(UB - LB)/|UB|$ for a minimisation with $UB > 0$. It is what one can claim when optimality has not been proved: it guarantees that $z(MILP)$ lies in the interval $[LB, UB]$, not that it is close to either end.
3. **The solver's MIPGap:** the same idea, computed by Gurobi from its own UB (`ObjVal` of the incumbent) and LB (`ObjBound`), with its convention $|UB - LB|/|UB|$ and its tolerances. It is 0 when optimality has been proved.

The formulas in items 2 and 3 require the incumbent to exist and the denominator to be non-zero: they are conventions, not mathematical identities, and must be stated every time a number is reported.

What the two bounds are actually for

In this course's exercises the instances are small on purpose: Gurobi finds $z(MILP)$ in a fraction of a second, and LB and UB serve to *understand* where a bound comes from and to check the reasoning behind the heuristic and the dual. In practice, however, a real MILP can have thousands of variables and the solver may not terminate in useful time: it is stopped, and what is left is the best solution found so far and the best bound proved at the time of the stop. The certified gap between the two is all one can say with certainty about the distance from the optimum, *without* knowing $z(MILP)$. This is why Chapter 5 teaches how to build a feasible solution quickly and Chapter 4 how to build an optimistic bound by hand: they are the tools that remain when the exact optimum is out of reach, and they are also what makes it possible to judge the numbers the solver reports.

1.4 Three patterns that keep coming back

Many MILP constraints with binary selection variables (indexed by a set I) belong to one of three families, depending on the direction:

Name	Constraint	Meaning
set partitioning	$\sum_{i \in I} x_i = 1$	exactly one element of I is chosen
set packing	$\sum_{i \in I} x_i \leq 1$	at most one element of I is chosen
set covering	$\sum_{i \in I} x_i \geq 1$	at least one element of I is chosen

Problem 7.1 (assignment) uses a *set partitioning* for every job; problem 7.3 (selection) a *set packing*; Chapter 2 shows *set covering* as the direct translation of an OR clause. Recognising the pattern immediately tells how the constraint behaves: a *covering* allows several elements to be chosen together, a *packing* does not, a *partitioning* imposes both at once.

1.5 What the solver does with the two bounds

A MILP with a bounded feasible set is solved with dedicated algorithms. How they work — how the solution space is explored, how cuts are separated, what to branch on — is not a topic of this course: solution techniques are the subject of a separate course. Only one thing matters here, and it is worth saying at once: the two bounds this course teaches how to build by hand are exactly the ones the solver works with.

- The **primal bound** is a feasible solution, and it gives a value to beat: any part of the space that cannot do better is discarded without being explored.
- The **dual bound** comes from the relaxation, and says how much one may hope for at most: the closer it is to the integer optimum — that is, the tighter the formulation, see chapter 3 — the less work is left.

The gap between the two is what the solver reports as **MIPGap**, and it is also the only thing one can claim with certainty when the optimum is not reached. That is why chapter 5 teaches how to build a feasible solution quickly and chapter 4 how to build an optimistic bound by hand: writing the model with a tight formulation and being able to produce the two bounds is what, of this course, is of use when the time comes to solve.

1.6 What this chapter leaves open

Question	Where it is answered
How are logical conditions translated into linear constraints?	Chapter 2
How are different families of variables linked to each other?	Chapter 3
How is an optimistic bound built by hand, and how does one prove that a formulation is stronger than another?	Chapter 4
How is a feasible solution built quickly?	Chapter 5
How is all of this written in Python/Gurobi, and how are the results read?	Chapter 6

This table is an orientation map, not a dependency: the chapters are read in the order in which they are written.

Logic and binary variables

Script: `python/cap02_logic.py`, `python/booleane.py`

[Online page](#)

A binary variable is a "yes/no" answer. This chapter shows how to translate logical conditions between those answers into *linear constraints* and — above all — how to *prove* that the translation is exact.

2.1 Propositions and binary variables

One proposition, one variable

A **boolean function** returns one of the two values **TRUE** and **FALSE**; it is represented by a binary variable

$$x \in \{0, 1\}, \quad x = 1 \iff \text{the proposition is TRUE}, \quad x = 0 \iff \text{it is FALSE}.$$

A **boolean expression** is built from binary variables, the three operators **AND** (conjunction, $\wedge$), **OR** (disjunction, $\vee$) and **NOT** (negation, $\neg$), and parentheses.

Two boolean expressions

$$\begin{aligned} &\text{NOT } x_a \text{ OR } ((x_b \text{ OR } x_c) \text{ AND } (x_d \text{ OR } x_e)), \\ &(x_a \text{ OR } x_b) \text{ AND } (x_c \text{ OR } x_d) \text{ AND NOT } x_e. \end{aligned}$$

Satisfiability

The **boolean satisfiability problem** asks whether there is an assignment of logical values to the variables that makes the expression true. If there is, the expression is **satisfiable**; otherwise it is **unsatisfiable**.

Satisfiable and not

- the formula

$$\text{NOT } x_a \text{ OR } ((x_b \text{ OR } x_c) \text{ AND } (x_d \text{ OR } x_e))$$

is satisfied, for instance, by $x_a = 0$ (the others arbitrary).

- the formula

$$(x_a \text{ OR } x_b) \text{ AND } (x_c \text{ OR } x_d) \text{ AND NOT } x_e$$

is satisfied by $x_a = 1, x_c = 1, x_b = x_d = x_e = 0$.

- $(x_a \text{ OR } x_b) \text{ AND } (\text{NOT } x_a \text{ OR } x_b) \text{ AND } (x_a \text{ OR NOT } x_b) \text{ AND } (\text{NOT } x_a \text{ OR NOT } x_b)$ is unsatisfiable: each of the four possible assignments of (x_a, x_b) falsifies one of the four clauses.

2.2 Literals, clauses, conjunctive normal form

The three words

A **literal** is a variable (x_a , positive literal) or its negation ($\text{NOT } x_a$, negative literal). A **clause** is a disjunction (OR) of literals, or a single literal. An expression is in **conjunctive normal form** (CNF) if it is a conjunction (AND) of clauses, or a single clause.

Of the two expressions in the previous example, the second is in CNF and the first is not. Every boolean expression *can* be rewritten in CNF, using the equivalences below.

The equivalences that are needed

For every triple of binary variables $x_a, x_b, x_c \in \{0, 1\}$:

$$x_a \text{ AND } (x_b \text{ OR } x_c) \iff (x_a \text{ AND } x_b) \text{ OR } (x_a \text{ AND } x_c) \quad (2.1)$$

$$x_a \text{ OR } (x_b \text{ AND } x_c) \iff (x_a \text{ OR } x_b) \text{ AND } (x_a \text{ OR } x_c) \quad (2.2)$$

$$\text{NOT } (x_a \text{ OR } x_b) \iff \text{NOT } x_a \text{ AND NOT } x_b \quad (2.3)$$

$$\text{NOT } (x_a \text{ AND } x_b) \iff \text{NOT } x_a \text{ OR NOT } x_b \quad (2.4)$$

$$x_a \text{ OR } (x_a \text{ AND } x_b) \iff x_a \quad (2.5)$$

$$x_a \text{ AND } (x_a \text{ OR } x_b) \iff x_a \quad (2.6)$$

$$\text{NOT } (\text{NOT } x_a) \iff x_a \quad (2.7)$$

Equations (2.1)–(2.2) are the **distributive** properties, (2.3)–(2.4) are **De Morgan's** laws, (2.5)–(2.6) are **absorption**. They hold with more than two terms inside the parentheses and with expressions in place of the variables.

Every equivalence is proved by cases: with two variables there are 4 assignments, with three there are 8, and one checks that the two sides agree on each. The script `cap02_logic.py` runs this check on all seven equivalences.

Collecting: (2.1) read backwards

Read from right to left, the distributive property (2.1) allows two conjunctions sharing a literal to be *collected*:

$$(x_a \text{ AND } x_b) \text{ OR } (x_a \text{ AND } x_c) \iff x_a \text{ AND } (x_b \text{ OR } x_c).$$

It is useful when putting into CNF a disjunction of several conjunctions with common literals: it reduces, step by step, the number of terms to distribute.

Two transformations into CNF

- for the formula

$$\text{NOT } x_a \text{ OR } ((x_b \text{ OR } x_c) \text{ AND } (x_d \text{ OR } x_e))$$

the distributive (2.2) is enough, and it becomes

$$(\text{NOT } x_a \text{ OR } x_b \text{ OR } x_c) \text{ AND } (\text{NOT } x_a \text{ OR } x_d \text{ OR } x_e).$$

- the formula

$$(x_a \text{ AND } x_b) \text{ OR } (x_a \text{ AND } x_c) \text{ OR } (x_b \text{ AND } x_c)$$

says "at least two of a, b, c ". Collect x_a in the first two,

$$[x_a \text{ AND } (x_b \text{ OR } x_c)] \text{ OR } (x_b \text{ AND } x_c),$$

apply (2.2) and obtain the two factors

$$x_a \text{ OR } (x_b \text{ AND } x_c) \quad \text{and} \quad (x_b \text{ OR } x_c) \text{ OR } (x_b \text{ AND } x_c);$$

the second reduces to $(x_b \text{ OR } x_c)$ by absorption (2.5), and the first is distributed once more. Altogether:

$$(x_a \text{ OR } x_b) \text{ AND } (x_a \text{ OR } x_c) \text{ AND } (x_b \text{ OR } x_c).$$

This will be the key to exercise 2.4.

2.3 From CNF to linear constraints

The translation, in three rules

A boolean expression in CNF is equivalent to a system of linear constraints on the same binary variables:

1. every **clause** becomes an inequality constraint ≥ 1 ;
2. every **OR** inside the clause becomes a $+$;
3. every negative literal **NOT** x becomes $1 - x$.

Why it is exact

A clause with positive literals P and negative literals N becomes

$$\sum_{i \in P} x_i + \sum_{i \in N} (1 - x_i) \geq 1 \quad \Longleftrightarrow \quad \sum_{i \in P} x_i - \sum_{i \in N} x_i \geq 1 - |N|.$$

On a binary variable, $x_i = 1$ contributes 1 to the left-hand side if the literal is positive and satisfied, and $1 - x_i = 1$ contributes 1 if the literal is negative and satisfied. The sum counts *how many literals of the clause are true*: asking it to be ≥ 1 is exactly asking the clause to

be true. The conjunction of clauses becomes the system of all the constraints, because they must all hold together.

With two or more negative literals the equivalent form obtained by multiplying by -1 reads better: for instance $1 - x_1 + 1 - x_6 + x_7 \geq 1$ is written $x_1 + x_6 - x_7 \leq 1$. It is the same constraint. What never changes is the number of constraints: *one per clause*, and clauses are counted after simplification (removing the tautologies x OR NOT x and the clauses absorbed by a subclause of theirs).

Three translations

$$(\text{NOT } x_a \text{ OR } x_b \text{ OR } x_c) \text{ AND } (\text{NOT } x_a \text{ OR } x_d \text{ OR } x_e) \iff \begin{cases} -x_a + x_b + x_c \geq 0, \\ -x_a + x_d + x_e \geq 0; \end{cases}$$

$$(x_a \text{ OR } x_b) \text{ AND } (x_c \text{ OR } x_d) \text{ AND NOT } x_e \iff \begin{cases} x_a + x_b \geq 1, \\ x_c + x_d \geq 1, \\ x_e \leq 0; \end{cases}$$

$$(x_a \text{ OR } x_b) \text{ AND } (\text{NOT } x_a \text{ OR } x_b) \text{ AND } (x_a \text{ OR NOT } x_b) \text{ AND } (\text{NOT } x_a \text{ OR NOT } x_b) \\ \iff \begin{cases} x_a + x_b \geq 1, \\ -x_a + x_b \geq 0, \\ x_a - x_b \geq 0, \\ x_a + x_b \leq 1. \end{cases}$$

The last system has no binary solution, just as the expression was unsatisfiable: the first three constraints force $x_a = x_b = 1$, which violates the fourth.

2.4 Logical implications

The implication between two variables

The logical implication $x_a \Rightarrow x_b$ ("if a then b ") is equivalent to the boolean expression

$$\text{NOT } x_a \text{ OR } x_b,$$

already in CNF, and hence to the linear constraint

$$1 - x_a + x_b \geq 1 \iff x_b - x_a \geq 0.$$

The contrapositive is the same thing

By contraposition, $x_a \Rightarrow x_b$ also implies the contrapositive

$$\text{NOT } x_b \Rightarrow \text{NOT } x_a.$$

This is not a second constraint: its boolean expression is

$$\text{NOT}(\text{NOT } x_b) \text{ OR } \text{NOT } x_a,$$

which by double negation (2.7) is the same

$$\text{NOT } x_a \text{ OR } x_b.$$

A single constraint imposes both readings — and when describing a model it is best to write the one that reads better in the application at hand.

$x_b - x_a \geq 0$ says "if a then b ", not " a if and only if b ": it admits $x_a = 0, x_b = 1$. An "if and only if" needs *two* implications, that is $x_a = x_b$, or one implication plus an optimality argument: that is the subject of Section 3.10.

The same recipe works when antecedent and consequent are arbitrary boolean expressions: one writes $\text{NOT}(\text{antecedent}) \text{ OR } (\text{consequent})$, puts it into CNF and translates.

The four basic combinations

Implication	Expression in CNF	Constraints	#
$x_a \text{ AND } x_b \Rightarrow x_c$	$\text{NOT } x_a \text{ OR } \text{NOT } x_b \text{ OR } x_c$	$x_a + x_b - x_c \leq 1$	1
$x_a \text{ OR } x_b \Rightarrow x_c$	$(\text{NOT } x_a \text{ OR } x_c) \text{ AND } (\text{NOT } x_b \text{ OR } x_c)$	$x_c - x_a \geq 0, x_c - x_b \geq 0$	2
$x_a \Rightarrow x_b \text{ AND } x_c$	$(\text{NOT } x_a \text{ OR } x_b) \text{ AND } (\text{NOT } x_a \text{ OR } x_c)$	$x_b - x_a \geq 0, x_c - x_a \geq 0$	2
$x_a \Rightarrow x_b \text{ OR } x_c$	$\text{NOT } x_a \text{ OR } x_b \text{ OR } x_c$	$x_b + x_c - x_a \geq 0$	1

Read the table like this: a *disjunction in the antecedent* and a *conjunction in the consequent* cost two constraints; a conjunction in the antecedent and a disjunction in the consequent cost one. This is why the aggregated constraint $\sum_j x_{jm} \leq k y_m$ and the disaggregated one $x_{jm} \leq y_m$ have different costs in number of rows — and different strengths in the relaxation (Section 3.1).

An antecedent with three literals

$$x_a \text{ AND } \text{NOT } x_b \text{ AND } x_c \Rightarrow \text{NOT } x_d \iff \text{NOT } x_a \text{ OR } x_b \text{ OR } \text{NOT } x_c \text{ OR } \text{NOT } x_d$$

$$\iff x_a - x_b + x_c + x_d \leq 2.$$

2.5 Splitting an implication: when it is allowed

Two valid splits and one that is not

$$(x_a \text{ OR } x_b) \Rightarrow x_c \iff (x_a \Rightarrow x_c) \text{ AND } (x_b \Rightarrow x_c) \quad (2.8)$$

$$x_a \Rightarrow (x_b \text{ AND } x_c) \iff (x_a \Rightarrow x_b) \text{ AND } (x_a \Rightarrow x_c) \quad (2.9)$$

The analogous split with a **conjunctive** antecedent is *not* valid:

$$(x_a \text{ AND } x_b) \Rightarrow x_c \text{ not } \iff (x_a \Rightarrow x_c) \text{ AND } (x_b \Rightarrow x_c).$$

With $x_a = 1$, $x_b = 0$, $x_c = 0$ the implication $(x_a \text{ AND } x_b) \Rightarrow x_c$ is *true* — its antecedent is **FALSE** — but $x_a \Rightarrow x_c$ is *false*. The conjunction of the two split implications is therefore **strictly stronger** than the original: it cuts off solutions the problem allows. Getting this wrong does not produce a "slightly different" model, it produces a model that may exclude the optimum.

The two valid splits are often the shortest route: in the previous table, the rows with two constraints follow directly from (2.8) or (2.9), without going through De Morgan and distributivity.

2.6 Counting: at most one, at least one, exactly one

Cardinality conditions on a set I of binary variables are already linear and need no CNF.

Condition	Constraint	Note
at least one	$\sum_{i \in I} x_i \geq 1$	it is the clause $\bigvee_{i \in I} x_i$: <i>set covering</i>
at most one	$\sum_{i \in I} x_i \leq 1$	<i>set packing</i> ; equivalent to the $\binom{ I }{2}$ clauses $\text{NOT } x_i \text{ OR } \text{NOT } x_j$, but in a single constraint and with a tighter relaxation
exactly one	$\sum_{i \in I} x_i = 1$	<i>set partitioning</i> : the two above together
at least p	$\sum_{i \in I} x_i \geq p$	in CNF it would take $\binom{ I }{p+1}$ clauses
at most p	$\sum_{i \in I} x_i \leq p$	in CNF it would take $\binom{ I }{p+1}$ clauses
none	$\sum_{i \in I} x_i \leq 0$	that is, $x_i = 0$ for every i

One cardinality constraint beats many clauses

"At most one out of three" can be written as three clauses,

$$\text{NOT } x_1 \text{ OR } \text{NOT } x_2, \quad \text{NOT } x_1 \text{ OR } \text{NOT } x_3, \quad \text{NOT } x_2 \text{ OR } \text{NOT } x_3,$$

that is $x_1 + x_2 \leq 1$, $x_1 + x_3 \leq 1$, $x_2 + x_3 \leq 1$; or as the single $x_1 + x_2 + x_3 \leq 1$. The two systems have the *same* binary solutions, but not the same relaxation: $x_1 = x_2 = x_3 = 1/2$ satisfies the three clauses and violates the aggregated constraint. The compact version is shorter *and* stronger: a case with no trade-off. When counting is possible, count; CNF is for conditions that are not cardinalities.

2.7 Logical operators and set operators

Associating with every binary variable a subset of a universe U — $x_a = 1$ means "the chosen element lies in A " — the logical operators become set operators:

Logic	Sets
x_a AND x_b	$A \cap B$ intersection
x_a OR x_b	$A \cup B$ union
NOT x_a	$\overline{A}$ complement

The equivalences of the previous section become the set identities $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$, $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$, $\overline{A \cap B} = \overline{A} \cup \overline{B}$, $\overline{A \cup B} = \overline{A} \cap \overline{B}$, $A \cup (A \cap B) = A$ and $A \cap (A \cup B) = A$. Those who prefer Venn diagrams thus get a second proof of the same rules; those who prefer enumeration have the first. Each checks the other.

2.8 Checking the translation, not trusting it

The operational criterion

A translation into linear constraints is correct when, for *every* binary assignment of the variables involved, the expression is true if and only if all the constraints are satisfied. With few variables this check is an enumeration of 2^n cases, done in an instant: it is the proof, by cases, of the result. The module `python/booleane.py` performs it — and every translation in the following section is checked this way.

The module also does the rest of the chapter's work: it eliminates implications ($A \Rightarrow B$ becomes NOT A OR B), pushes negations onto the literals with De Morgan, distributes OR over AND, discards tautological and absorbed clauses, and finally applies the three translation rules. It is not a SAT solver: it is a line-by-line transcription of this chapter's rules.

2.9 Five exercises

In all five, the projects are numbered and for every project p the binary variable is

$$x_p = \begin{cases} 1 & \text{if project } p \text{ is chosen,} \\ 0 & \text{otherwise.} \end{cases}$$

The solutions — CNF expression, linear constraints, constraint count and the number of assignments on which the equivalence is checked by enumeration — are in the solutions document, reserved for teachers.

Exercise 2.1 — direct implications

Ten projects. Model as linear constraints:

1. if project 2 is chosen then project 3 is chosen;
2. if project 2 is chosen then project 4 is not chosen;
3. if projects 1 and 6 are chosen then project 7 is chosen;

4. if project 1 or project 6 is chosen then project 8 is chosen;
5. if projects 2 and 3 are chosen then project 9 is not chosen;
6. if project 2 or project 3 is chosen then project 10 is not chosen.

Exercise 2.2 — negated antecedents and consequents

Ten projects. Model:

1. if project 3 is not chosen then project 2 is chosen;
2. if project 4 is not chosen then project 2 is not chosen;
3. if project 7 is chosen then projects 1 and 6 are chosen;
4. if project 8 is chosen then project 1 or project 6 is chosen;
5. if project 9 is not chosen then projects 2 and 3 are chosen;
6. if project 10 is not chosen then project 2 or project 3 is chosen.

Exercise 2.3 — compound antecedents and consequents

Eight projects. Model:

1. $x_7 \text{ OR } x_3 \Rightarrow x_1 \text{ AND } x_2$;
2. $x_1 \text{ AND } x_6 \text{ AND } x_7 \Rightarrow x_8$;
3. $x_5 \text{ AND } x_2 \text{ AND NOT } x_4 \Rightarrow \text{NOT } x_3$;
4. $(x_1 \text{ OR } x_4) \text{ AND } x_6 \Rightarrow x_2 \text{ AND } (x_5 \text{ OR } x_7)$;
5. $(x_2 \text{ OR } x_5) \text{ AND NOT } x_8 \Rightarrow x_3 \text{ OR NOT } x_6$;
6. $(x_1 \text{ OR } x_4) \text{ AND } (x_2 \text{ OR } x_5) \text{ AND NOT } x_8 \Rightarrow x_3 \text{ AND } (\text{NOT } x_6 \text{ OR } x_7)$.

Exercise 2.4 — "at least two of"

Nine projects. Model:

1. if project 4 is chosen then at least two of 1, 2, 3;
2. if at least two of 6, 7, 8 then project 5;
3. if project 4 is not chosen then at least two of 1, 2, 3, 9;
4. if project 8 is chosen then (1 and 6) or (1 and 7) or (2 and 6);
5. if at least two of 1, 3, 5 then project 9 is not chosen;
6. if (1 and 2) or (3 and 4) then project 5.

The condition "at least two of 1, 2, 3" is also $x_1 + x_2 + x_3 \geq 2$. As the consequent of the implication governed by x_4 it becomes

$$x_1 + x_2 + x_3 \geq 2x_4 :$$

a single constraint, against the three clauses of item 1. The two systems have the same 16 binary solutions — checked by enumeration — but not the same relaxation, and this time it is the counted form that is **stronger**. Summing the three clauses $x_i + x_j \geq x_4$ gives only $x_1 + x_2 + x_3 \geq \frac{3}{2}x_4$, weaker than $\geq 2x_4$; conversely, with $0 \leq x_i \leq 1$, the counted form implies each clause. On max $x_1 + x_2 + x_3 + 3x_4$ subject to $x_1 + x_2 + x_3 + 2x_4 \leq 3$, where $z(MILP) = 3$, the three clauses give $z(LP^+) = 27/7 \approx 3.86$ and the counted constraint gives $z(LP^+) = 15/4 = 3.75$: closer to the integer optimum.

The moral is not "counting is always better": in the previous section "at most one" counted was stronger than the pairwise clauses, here "at least two" counted is again stronger, but the aggregated activation constraint of Chapter 3 will be *weaker* than the disaggregated one. The moral is that two formulations with the same integer set must be compared on their relaxation, with the criteria of Chapter 4, and not by eye.

Exercise 2.5 — splits

Ten projects. Model:

1. x_1 OR $x_2 \Rightarrow x_3$;
2. $x_4 \Rightarrow x_5$ AND x_6 ;
3. x_1 OR $x_2 \Rightarrow x_3$ AND x_4 ;
4. x_1 AND $x_2 \Rightarrow x_3$;
5. x_5 OR $x_6 \Rightarrow \text{NOT } x_7$;
6. $\text{NOT } x_8$ OR $\text{NOT } x_9 \Rightarrow x_{10}$.

2.10 Logical constraints inside an optimisation model

The constraints of this chapter rarely live alone: they are added to a model that optimises something. With the $n = 10$ projects of exercise 2.9, revenues r_p and costs b_p

project p	1	2	3	4	5	6	7	8	9	10
revenue r_p	9	7	4	8	3	6	2	5	7	6
cost b_p	4	3	2	4	2	3	1	3	4	3

and budget $B = 14$, the model is

Project selection with logical constraints

$$\begin{aligned}
& \max \sum_{p=1}^n r_p x_p \\
& \text{subject to} \quad \sum_{p=1}^n b_p x_p \leq B, \\
& \text{the 8 constraints of exercise 2.9,} \\
& x_p \in \{0, 1\}, \quad \forall p \in \{1, 2, \dots, n\}.
\end{aligned}$$

Description. The objective adds up the revenues of the chosen projects. The first constraint, a single one, is the budget. The eight recalled constraints are the linear translations of the six implications of exercise 2.9: they add no variables, only rows. The domains are the n binary conditions, one per project.

Without the logical constraints the optimum is 30. With them the optimum drops to $z(\text{MILP}) = 28$, obtained by choosing projects 1, 2, 3, 5, 8, of total cost 14: the budget is tight. The relaxation $z(\text{LP}^+)$ is 29. The six implications hold: 2 is chosen and so is 3; 4 is not chosen; 1 and 6 are not both chosen; 1 is chosen and so is 8; 2 and 3 are chosen and 9 is not; 2 is chosen and 10 is not.

How much six implications cut

The $2^{10} = 1024$ assignments of the ten binaries drop to 234 once all six implications of exercise 2.9 are imposed: less than a quarter. Figure 2.1 shows how many survive each implication on its own and how many survive as they accumulate. None of these implications, on its own, cuts more than half the space; together they cut three quarters. This is why a model is written one constraint at a time but judged as a whole.

The model in gurobipy

```

1 from booleans import cnf, vincolo, IMP, AND, OR, NOT, V
2
3 x = {p: V(f"x{p}") for p in range(1, 11)}
4 implications = [IMP(x[2], x[3]), IMP(x[2], NOT(x[4])),
5                 IMP(AND(x[1], x[6]), x[7]), IMP(OR(x[1], x[6]), x[8]),
6                 IMP(AND(x[2], x[3]), NOT(x[9])), IMP(OR(x[2], x[3]),
7                 NOT(x[10]))]
8
9 m = gp.Model("project_selection"); m.Params.OutputFlag = 0
10 xv = m.addVars(range(1, 11), vtype=GRB.BINARY, name="x")
11 m.setObjective(gp.quicksum(r[p] * xv[p] for p in range(1, 11)), GRB.
    MAXIMIZE)
12 m.addConstr(gp.quicksum(b[p] * xv[p] for p in range(1, 11)) <= budget,
    name="budget")
13 for i, formula in enumerate(implications, 1):           # one clause,
    one constraint
    for j, clause in enumerate(cnf(formula), 1):

```

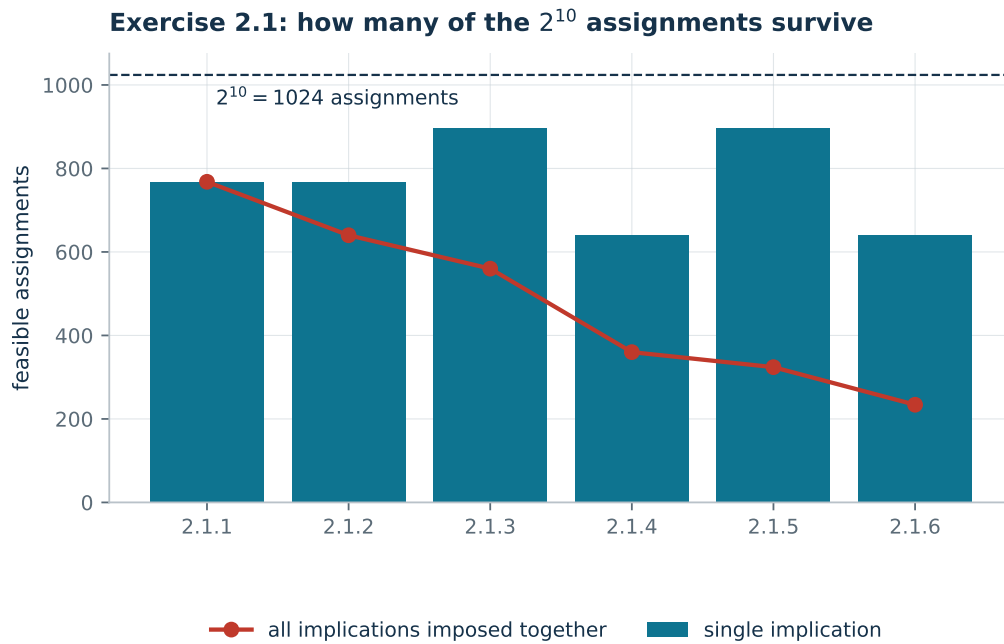


Figure 2.1: Exercise 2.9: assignments of the ten binaries surviving each implication on its own (bars) and all the implications imposed up to that point (line).

```

14     coef, sense, rhs = vincolo(clause)
15     lhs = gp.quicksum(k * xv[int(n[1:])] for n, k in coef.items())
16     m.addConstr(lhs <= rhs if sense == "<=" else lhs >= rhs, name=f
17               "logic{i}_{j}")
17 m.optimize()

```

The two bounds of this course — the primal one and the dual one — are what the solver works with.

2.11 What is left to the next chapter

This chapter deals with links between variables that are *all binary*. When one of the two families is continuous or integer — "if the machine is not on it produces nothing", "this variable equals the maximum of those" — CNF is no longer enough: the coefficients, the big-Ms and the optimality arguments of Chapter 3 are needed.

Links between variables

Script: `python/cap03_links.py`

[Online page](#)

Chapter 2 links variables that are *all binary*. Here we link *different* families: binary with continuous, binary with integer, continuous with each other. There are fourteen techniques, and they are the real content of MIP modelling: almost every model in this course is a combination of them.

How to read each section

Every technique is presented in five steps:

- (a) the **link in words**: what one wants to say;
- (b) the **linear constraints** that say it, with their count;
- (c) the **proof**: in both directions if both are imposed by the constraints; with an exchange argument if one follows from optimality; with a *counterexample* when a converse is false;
- (d) the **strength of the relaxation**, on a minimal instance solved by the script;
- (e) the **gurobipy line** and the pointers to the Part II problems where the technique is seen again.

Step (c) is what tells a correct model from a plausible one. Step (d) is what tells a model that can be solved from one that cannot.

Throughout the chapter we distinguish:

- a property **imposed by the constraints**: it holds for *every* feasible solution, optimal or not. It is proved by looking at the constraints alone;
- a property **of optimality**: it holds only in optimal solutions. It is proved by an *exchange argument*: take a feasible solution that violates it, build a modified solution, check that it stays feasible and compare the values. If the value improves *strictly*, the conclusion is "in every optimum"; if it merely does not worsen, the conclusion is "there exists an optimum", which is weaker.

Writing "in every optimum" when the coefficient is only non-negative is the commonest mistake in this chapter.

3.1 Activation: aggregated and disaggregated form

(a) **The link in words.** A resource (a machine, a site, a plant) must be *activated* before being used. The binaries x_{ij} say "object i uses resource j ", the binaries y_j say "resource j is activated". We want: if anyone uses j , then j is activated.

(b) **The constraints.** Two ways, with n objects and m resources:

$$\text{disaggregated: } x_{ij} \leq y_j, \quad \forall i \in \{1, 2, \dots, n\}, \forall j \in \{1, 2, \dots, m\} \quad (nm \text{ constraints}), \quad (3.1)$$

$$\text{aggregated: } \sum_{i=1}^n x_{ij} \leq n y_j, \quad \forall j \in \{1, 2, \dots, m\} \quad (m \text{ constraints}). \quad (3.2)$$

When the resource has a capacity k_j (at most k_j objects), the aggregated form is written with that coefficient, $\sum_i x_{ij} \leq k_j y_j$, and then it imposes the capacity *as well*: two conditions in one constraint.

(c) **The proof.** On binary points the two forms are **equivalent**.

- *Disaggregated $\Rightarrow$ aggregated*: summing (3.1) over i gives $\sum_i x_{ij} \leq n y_j$.
- *Aggregated $\Rightarrow$ disaggregated, on binary points*: if $y_j = 0$, (3.2) gives $\sum_i x_{ij} \leq 0$ and hence $x_{ij} = 0 \leq y_j$ for every i ; if $y_j = 1$, then $x_{ij} \leq 1 = y_j$ for every i because the x are binary.

The second direction uses the binarity of x and y : *outside* the binary points it does not hold, and that is exactly what makes the two forms different in the relaxation.

The converse is not imposed. "If j is activated then somebody uses it", that is $y_j = 1 \Rightarrow \sum_i x_{ij} \geq 1$, does **not** follow from the constraints: $y_j = 1$ with all $x_{ij} = 0$ is feasible in both forms. It follows from optimality when activating costs something: if the activation cost f_j is *strictly* positive, given an optimal solution with $y_j = 1$ and no $x_{ij} = 1$, setting $y_j = 0$ leaves all constraints satisfied and reduces the cost by $f_j > 0$ — a contradiction. Hence "in every optimum". If instead $f_j = 0$ is allowed, the correct conclusion is the weaker "there exists an optimum with $y_j = 0$ ".

(d) **The strength of the relaxation.** Instance: $n = 3$ customers, $m = 2$ sites, activation costs $f = (8, 6)$, service costs

$$c = \begin{pmatrix} 2 & 5 \\ 4 & 1 \\ 3 & 3 \end{pmatrix}.$$

Formulation	link constraints	$z(LP^+)$
aggregated (3.2)	$m = 2$	$38/3 \approx 12.67$
disaggregated (3.1)	$nm = 6$	15

The integer optimum is $z(MILP) = 15$: the disaggregated form already reaches it in the relaxation, the aggregated one stops at $38/3$. More rows, tighter relaxation: the typical trade-off of this technique. On large instances the choice is not obvious — nm rows can be many — but on hard instances the disaggregated form almost always pays off.

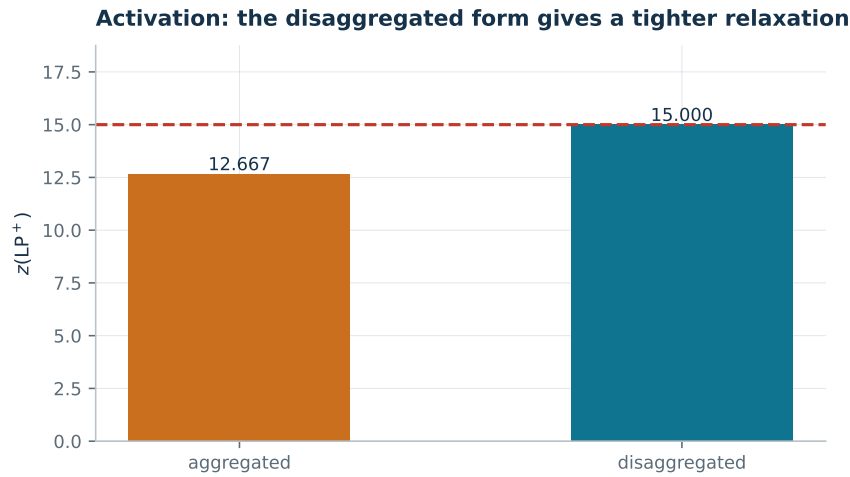


Figure 3.1: The relaxation of the two activation forms on the three-customer, two-site instance. The red line is the integer optimum.

(e) In gurobipy, and where it is seen again.

```

1 m.addConstrs((x[i, j] <= y[j] for i in range(n) for j in range(mm)),
   name="link") # disaggregated
2 m.addConstrs((x.sum("*", j) <= n * y[j] for j in range(mm)), name="link")
   # aggregated

```

Seen again in problems 7.2 (machines with fixed cost), 7.3 (selection), 7.5 (classes with setup) and 8.4 (hubs), where comparing the two forms is a modelling question.

3.2 Fixed cost, capacity and continuous flow

(a) **The link in words.** As above, but the quantity used is *continuous*: $q_j \geq 0$ is how much is produced at plant j , and y_j says whether the plant is open. A closed plant produces nothing; an open one produces at most its capacity c_j .

(b) **The constraints.**

$$q_j \leq c_j y_j, \quad \forall j \in \{1, 2, \dots, m\} \quad (m \text{ constraints}). \quad (3.3)$$

One constraint per plant, with the *right* coefficient: the capacity, not some large number picked at random.

(c) **The proof.** If $y_j = 0$ the constraint gives $q_j \leq 0$ and, together with $q_j \geq 0$, forces $q_j = 0$: the implication "closed $\Rightarrow$ produces nothing" is imposed by the constraint, and with it its contrapositive "produces $\Rightarrow$ open". If $y_j = 1$ the constraint gives $q_j \leq c_j$: the capacity. The direction "open $\Rightarrow$ produces" is not imposed and follows from optimality only if $f_j > 0$, as in the previous section.

(d) **The strength of the relaxation.** Instance: two plants, fixed costs $f = (10, 14)$, unit costs $c = (3, 2)$, capacities $C = (6, 7)$, demand $D = 9$. The optimum is $z(MILP) = 44$ (both open, $q = (2, 7)$).

Coefficient in (3.3)	$z(LP^+)$
the capacity, c_j	$112/3 \approx 37.33$
a big-M = 100 (plus the constraint $q_j \leq c_j$ separately)	$1059/50 = 21.18$

Same integer set, same optimum, relaxations far apart. The rule: **the coefficient of an activation binary is the smallest value the continuous variable can be capped at when the activation equals 1**, and it must be derived from the data.

(e) **In gurobipy, and where it is seen again.**

```
m.addConstrs((q[j] <= c[j] * y[j] for j in range(mm)), name="link")
```

Seen again in problem 8.1 (capacitated location) and throughout Chapter 9.

3.3 Minimum lot size and semicontinuous variables

(a) **The link in words.** "If you produce, produce at least ℓ ": the quantity q_j is either zero or lies between a threshold ℓ and the capacity c_j . It is not an interval: it is the union of a point and an interval. A variable with this domain is called **semicontinuous**.

(b) **The constraints.** With the same binary y_j as in the previous section,

$$\begin{aligned} \ell y_j &\leq q_j, & \forall j \in \{1, 2, \dots, m\} & \quad (m \text{ constraints}), \\ q_j &\leq c_j y_j, & \forall j \in \{1, 2, \dots, m\} & \quad (m \text{ constraints}). \end{aligned} \tag{3.4}$$

(c) **The proof.** Both directions are imposed by the constraints. If $y_j = 0$: $0 \leq q_j \leq 0$, that is $q_j = 0$. If $y_j = 1$: $\ell \leq q_j \leq c_j$. Hence

$$q_j \in \{0\} \cup [\ell, c_j],$$

exactly the domain wanted. One needs $\ell \leq c_j$, otherwise $y_j = 1$ is infeasible and the variable is forced to zero: a data error the solver reports as infeasibility only if y_j is forced to 1 by other constraints.

(d) **The strength of the relaxation.** On the same instance as the previous section with $\ell = 5$: the optimum goes from 44 to $z(MILP) = 49$, with $q = (5, 5)$ instead of $(2, 7)$ — the threshold forces production of 5 at the second plant even though the first is more expensive. But $z(LP^+)$ stays at $112/3$, identical to the case without the threshold.

In the relaxation y_j is free in $[0, 1]$, and the constraint $q_j \geq \ell y_j$ is satisfied by lowering y_j : $y_j \leq q_j/\ell$ suffices. The threshold constraint *never bites* on the continuous problem. All of its effect is discharged onto integrality — which is why models with minimum lot sizes are typically harder than those without, at equal size.

(e) In gurobipy, and where it is seen again.

```
1 m.addConstrs((q[j] >= ell * y[j] for j in range(mm)), name="lot")
2 m.addConstrs((q[j] <= c[j] * y[j] for j in range(mm)), name="capacity")
```

Gurobi also has the type `GRB.SEMICONT`, which declares the domain directly; in this course the formulation is written by hand, because that is what one must be able to prove. Seen again in question 7.2.2 and in exercises 9.1 and 9.3.

3.4 Integer counts, multiple capacity, rounding up

(a) **The link in words.** "How many containers are needed?" The variable is not binary but *integer*: $w \in \mathbb{Z}_{\geq 0}$ counts indivisible objects (boxes, trucks, shifts, workers) and each carries a capacity c .

(b) **The constraints.**

$$\begin{aligned} \sum_i a_i x_i &\leq c w, & (1 \text{ constraint}), \\ w &\in \mathbb{Z}_{\geq 0}. & (1 \text{ integer variable}) \end{aligned} \tag{3.5}$$

(c) **The proof.** Setting $q = \sum_i a_i x_i$, the constraint imposes $w \geq q/c$ and integrality imposes $w \geq \lceil q/c \rceil$. Together with an objective that minimises w (or pays for w), in every optimum w equals exactly that ceiling: if it were larger, lowering it by 1 would stay feasible and reduce the cost. With a zero cost on w the conclusion weakens to "there exists an optimum".

$\lceil t \rceil$ is not a linear function: it cannot be written inside a constraint. The pair "inequality $\leq cw$ + declaration that w is integer" realises it *implicitly*, and that is how it must be read when explaining the model.

(d) **The strength of the relaxation.** 17 unit items, capacity $c = 5$: the relaxation gives $w \geq 17/5 = 3.4$ and the integer optimum is $z(MILP) = 4$. The gap $4 - 17/5 = 3/5$ comes entirely from integrality: no linear cut on the x alone closes it, an inequality using w integer is needed.

(e) In gurobipy, and where it is seen again.

```
1 w = m.addVar(vtype=GRB.INTEGER, name="w")
2 m.addConstr(gp.quicksum(a[i] * x[i] for i in range(n)) <= c * w, name="
   capacity")
```

Seen again in exercises 10.4 (boxes of lights), 10.5 (shipments) and 9.2 (workforce).

3.5 The maximum auxiliary variable

(a) **The link in words.** A variable z must equal the *maximum* of certain quantities: the duration of the longest activity, the highest connection cost, the load of the busiest machine.

(b) **The constraints.** If the quantities are $t_j x_j$ with x_j binary,

$$\begin{aligned} z &\geq t_j x_j, \quad \forall j \in \{1, 2, \dots, n\} && (n \text{ constraints}), \\ z &\geq 0. \end{aligned} \tag{3.6}$$

(c) **The proof, in three steps.**

1. *Imposed by the constraint:* for every j with $x_j = 1$ we get $z \geq t_j$, hence $z \geq \max_{j:x_j=1} t_j$. This holds in every feasible solution.
2. *From optimality:* if z appears in the objective with a strictly positive coefficient in a minimisation (or a strictly negative one in a maximisation) and in no other constraint, then in every optimum z takes the smallest feasible value. Indeed, given an optimal solution with $z > \max_{j:x_j=1} t_j$, lowering z to that maximum leaves all constraints (3.6) satisfied and strictly improves the objective.
3. *Conclusion:* in every optimum, $z = \max_{j:x_j=1} t_j$ exactly (with the convention that the maximum over the empty set is 0, guaranteed by $z \geq 0$).

If z appears in another constraint that wants it *large* — for instance $z \geq$ something else, or a constraint $z \geq \tau_j$ as in question 7.7.2 — the exchange argument no longer works: lowering z may violate that constraint. In that case $z \geq \max$ still holds, but $z = \max$ does not. This is the mistake Part II flags in the makespan variant of problem 7.4.

(d) **The strength of the relaxation.** $\min z$ with $z \geq t_j x_j$, $t = (4, 7, 3)$ and $\sum_j x_j \geq 2$: the integer optimum is $z(MILP) = 4$ (jobs 1 and 3 are chosen, and the maximum is 4). The relaxation is $168/61 \approx 2.75$: the fractional solution $x_j = z/t_j$ spreads the choice over all three jobs and lowers the maximum. The maximum link gives *weak* relaxations: one of the reasons makespan problems are hard.

(e) **In gurobipy, and where it is seen again.**

```
1 z = m.addVar(name="z")
2 m.addConstrs((z >= t[j] * x[j] for j in range(n)), name="maximum")
```

Seen again in problems 7.4, 7.7, 8.4 and 10.9 (books on shelves).

3.6 Min-max, max-min and the range

(a) **The link in words.** Three fairness objectives, often confused: *min-max* (minimise the largest load), *max-min* (maximise the smallest load) and *range* (minimise the gap between the largest and the smallest).

(b) **The constraints.** With ℓ_k the load of resource k ($k \in \{1, 2, \dots, m\}$):

$$\text{min-max: } \min z \quad \text{with } z \geq \ell_k, \forall k \in \{1, 2, \dots, m\} \quad (m \text{ constraints}), \quad (3.7)$$

$$\text{max-min: } \max u \quad \text{with } u \leq \ell_k, \forall k \in \{1, 2, \dots, m\} \quad (m \text{ constraints}), \quad (3.8)$$

$$\text{range: } \min(z - u) \quad \text{with both} \quad (2m \text{ constraints}). \quad (3.9)$$

(c) **The proof.** Each is the maximum (or minimum) auxiliary variable of the previous section, with the exchange argument in the right direction: in a $\min z$ the variable z falls to the maximum of the loads; in a $\max u$ the variable u rises to the minimum of the loads. In the range both pressures are present at once and the two conclusions hold together.

On the five-weight instance $p = (3, 5, 2, 4, 7)$ to be split between two workers (total 21), the three versions choose the *same* split (11, 10) — the best possible, because the total is odd — but their optimal values are 11, 10 and 1. They are three different numbers describing the same solution. Comparing " $z = 11$ " of a min-max with " $z = 1$ " of a range means nothing: before comparing two models one must make sure they optimise the same thing. And the optimal solutions need not coincide either: with more than two resources, min-max and max-min in general choose different splits.

(d) **The strength of the relaxation.** The min-max on that instance gives $z(LP^+) = 21/2 = 10.5$ against $z(MILP) = 11$: the relaxation splits the weights exactly in half, which integrality does not allow.

(e) **In gurobipy, and where it is seen again.**

```
1 z = m.addVar(name="z")
2 m.addConstrs((z >= load[k] for k in range(len(load))), name="max")
3 m.setObjective(z, GRB.MINIMIZE)
```

Seen again in question 7.4.1 (makespan), in exercise 10.7 (antitrust split) and in 10.8 (tracks on CDs).

3.7 The absolute value

(a) **The link in words.** One wants $|u - v|$: a gap, an error, an imbalance. The case in the objective and the case in a constraint behave radically differently.

(b) **The constraints.**

- **In the objective, minimising:** a variable $d \geq 0$ and two constraints,

$$\begin{aligned} d &\geq u - v, & (1 \text{ constraint}), \\ d &\geq v - u, & (1 \text{ constraint}), \end{aligned} \quad (3.10)$$

with d in the objective to be minimised. No binary.

- **As a $\leq$ constraint:** $|u - v| \leq k$ is simply $u - v \leq k$ and $v - u \leq k$ (2 constraints, no binary).

- **As a $\geq$ constraint:** $|u - v| \geq k$ **cannot** be written without binaries. It is the disjunction " $u - v \geq k$ or $v - u \geq k$ ", and needs a binary b and a big-M:

$$\begin{aligned} u - v &\geq k - M(1 - b), & (1 \text{ constraint}), \\ v - u &\geq k - Mb. & (1 \text{ constraint}, 1 \text{ binary}) \end{aligned} \quad (3.11)$$

(c) The proof. In the first case the two constraints (3.10) impose $d \geq |u - v|$ (one of the two right-hand sides is $|u - v|$); the objective, which minimises d and in which d appears nowhere else, drives it to equality by the exchange argument of Section 3.5. In the third case, $b = 1$ switches off the second constraint (provided $M \geq k + \max(v - u)$) and leaves the first, and vice versa: it is a disjunction, not a conjunction, and without the binary both would be imposed — that is, $0 \geq 2k$, infeasible for $k > 0$.

$|u - v| \leq k$ is the intersection of two half-planes: a convex set, written with two linear constraints. $|u - v| \geq k$ is the complement of a strip: it is *not* convex, and no system of linear constraints without integer variables can describe it. The binary is not a trick: it is necessary.

(d) The strength of the relaxation. On the five-weight instance, the problem $\min |\ell_1 - \ell_2|$ has optimum $z(MILP) = 1$ and relaxation $z(LP^+) = 0$: the continuous problem splits 21 into two equal halves and zeroes the gap. The relaxation of an absolute-value objective is typically 0, that is, useless.

(e) In gurobipy, and where it is seen again.

```
1 d = m.addVar(name="d")
2 m.addConstr(d >= u - v, name="abs_plus")
3 m.addConstr(d >= v - u, name="abs_minus")
```

Seen again in exercises 10.8 (CDs) and 10.7 (antitrust), and in Section 3.13 in an equivalent form with two deviations.

3.8 Big-M: conditional constraints and disjunctions

(a) The link in words. "If $y = 1$ then the constraint $a'x \leq b$ holds; if $y = 0$ the constraint is not there." A constraint that switches on and off.

(b) The constraint.

$$a'x \leq b + M(1 - y) \quad (1 \text{ constraint}). \quad (3.12)$$

(c) The proof, and how M is chosen. With $y = 1$ the constraint is $a'x \leq b$: switched on. With $y = 0$ it becomes $a'x \leq b + M$: switched off *provided* M is large enough to cut nothing off, that is

$$M \geq \max\{a'x : x \text{ feasible for the other constraints}\} - b. \quad (3.13)$$

Three things must be told apart:

- a **valid** M : it satisfies (3.13), so the model has the right integer set;

- an **improvable** M : valid but larger than necessary; the integer set is the same, the relaxation is weaker;
- the **smallest proved** M : the smallest value for which validity has been *proved*, typically by computing the maximum in (3.13) from the variable bounds alone. It is not necessarily the absolute minimum, and it is honest to say so.

With $a = (3, 4, 5)$, $b = 6$ and x binary, the maximum of $a'x$ is 12, so $M = 12 - 6 = 6$ is valid. On $\max x_1 + x_2 + x_3 + y$ the optimum is $z(MILP) = 3$ (with $y = 0$ and all the x at 1). With $M = 5$ the constraint with $y = 0$ remains $3x_1 + 4x_2 + 5x_3 \leq 11$, which excludes $x = (1, 1, 1)$: the optimum drops to 2. The model no longer answers the question asked.

(d) **The strength of the relaxation.** On the same instance:

M	$z(LP^+)$	
6 (the smallest proved)	3	coincides with $z(MILP)$
20	$37/10 = 3.7$	
1000	$1997/500 \approx 3.994$	almost the largest possible, 4

The degradation is monotone and fast. The operational rule: **compute M from the data, always, and write it in the text right after the model.**

(e) **In gurobipy, and where it is seen again.**

```

1 M = sum(max(a[j], 0) for j in range(n)) - b          # computed from the
   data, not guessed
2 m.addConstr(gp.quicksum(a[j] * x[j] for j in range(n)) <= b + M * (1 -
   y), name="cond")

```

Seen again in problem 7.7 (total tardiness) and in Section 3.9.

3.9 Precedences and sequencing

(a) **The link in words.** On a machine that runs one job at a time, for every pair of jobs one precedes the other. It is a disjunction per pair, not a precedence fixed by the data.

(b) **The constraints.** With $\kappa_j \geq 0$ the completion time of job j , t_j its duration and $s_{ij} \in \{0, 1\}$ equal to 1 if j precedes i :

$$s_{ij} + s_{ji} = 1, \quad \forall i, j \in \{1, 2, \dots, n\}, i < j \quad \left(\binom{n}{2} \text{ constraints}\right), \quad (3.14)$$

$$\kappa_i \geq \kappa_j + t_i - M(1 - s_{ij}), \quad \forall i, j \in \{1, 2, \dots, n\}, i \neq j \quad (n(n-1) \text{ constraints}), \quad (3.15)$$

$$\kappa_j \geq t_j, \quad \forall j \in \{1, 2, \dots, n\} \quad (n \text{ constraints}). \quad (3.16)$$

(c) **The proof, and the smallest M .** Constraint (3.14) imposes that exactly one of the two orders is chosen. If $s_{ij} = 1$, (3.15) becomes $\kappa_i \geq \kappa_j + t_i$: job i finishes at least t_i after j finishes, so they do not overlap. If $s_{ij} = 0$ it becomes $\kappa_i \geq \kappa_j + t_i - M$, which must always hold. Since $\kappa_i \geq t_i$ and $\kappa_j \leq \sum_h t_h$ in every sensible solution, it suffices to take

$$M = \sum_{h=1}^n t_h,$$

because then $\kappa_j + t_i - M \leq \sum_h t_h + t_i - \sum_h t_h = t_i \leq \kappa_i$. The time horizon $\sum_h t_h$ must be *declared*: without an upper bound on the κ_j , no finite M is valid.

(d) **The strength of the relaxation.** Three jobs of duration (3, 2, 4) on one machine, makespan objective: $z(MILP) = 9 = \sum_h t_h$ (obviously: a single machine), with completion times (3, 5, 9). The relaxation is 4: with $s_{ij} = 1/2$ all the precedence constraints are half switched off and the jobs may overlap. It is the weakest relaxation in the whole chapter, and it explains why big-M sequencing models scale badly.

(e) **In gurobipy, and where it is seen again.**

```

1 M = sum(t)                                     # the horizon, declared
2 for i in range(n):
3     for j in range(i):
4         m.addConstr(s[i, j] + s[j, i] == 1, name=f"disj{i}{j}")
5         m.addConstr(kappa[i] >= kappa[j] + t[i] - M * (1 - s[i, j]),
6                     name=f"prec{i}{j}")
7         m.addConstr(kappa[j] >= kappa[i] + t[j] - M * (1 - s[j, i]),
8                     name=f"prec{j}{i}")

```

Seen again in problem 7.7, where release dates allow M to be reduced.

3.10 "If and only if"

(a) **The link in words.** A binary y must equal 1 *exactly when* a condition holds — for instance "all the jobs of the class are done". One implication is not enough: two are needed.

(b) **The constraints.** With $x_1, \dots, x_p$ binary and the condition "all equal 1":

$$y \leq x_j, \quad \forall j \in \{1, 2, \dots, p\} \quad (p \text{ constraints}), \quad (3.17)$$

$$y \geq \sum_{j=1}^p x_j - (p-1) \quad (1 \text{ constraint}). \quad (3.18)$$

(c) **The proof, in both directions.**

- (3.17) gives $y = 1 \Rightarrow x_j = 1$ for every j ; equivalently, if even one of the x_j is zero then $y = 0$.
- (3.18) gives the converse: if all $x_j = 1$, the right-hand side equals $p - (p-1) = 1$, so $y \geq 1$ and hence $y = 1$. If at least one is zero, the right-hand side is ≤ 0 and the constraint says nothing.

Together, the two impose $y = 1 \iff \sum_j x_j = p$ in *every* feasible solution.

When one direction is enough. If y appears in the objective of a maximisation with a bonus $v > 0$ and in no other constraint, direction (3.18) may be omitted: in every optimum, if all $x_j = 1$ and $y = 0$, raising y to 1 stays feasible for (3.17) and increases the objective by $v > 0$. This is an optimality argument, and it holds only at the optimum.

On the three-job instance, revenues (2, 2, 2), capacity 3:

bonus v	constraints	$z(MILP)$	y, x at the optimum	does y say "class complete"?
9	only (3.17)	15	$y = 1, x = (1, 1, 1)$	yes
9	both	15	$y = 1, x = (1, 1, 1)$	yes
0	only (3.17)	6	$y = 0, x = (1, 1, 1)$	no
0	both	6	$y = 1, x = (1, 1, 1)$	yes

With $v = 0$ the optimal value is the same in the two models, but in the third case y is no longer a faithful indicator. If y only serves to collect a bonus, the missing direction is useless; if y appears in *other* constraints — and this happens as soon as a condition on the number of complete classes is added — it must be written.

(d) The strength of the relaxation. $z(LP^+) = 15 = z(MILP)$ on this instance: with both directions imposed the relaxation is exact. Constraint (3.18) is weak in the relaxation (with $x_j = 1/2$ its right-hand side is negative), but here that does not matter.

(e) In gurobipy, and where it is seen again.

```
1 m.addConstrs((y <= x[j] for j in range(p)), name="iff_up")
2 m.addConstr(y >= gp.quicksum(x[j] for j in range(p)) - (p - 1), name="
   iff_down")
```

Seen again in problem 7.6 (classes with a bonus) and in exercise 9.3 (variety bonus).

3.11 Counting the different types

(a) The link in words. "At least two different types must be produced", "at most three foods in the diet", "no more than four configurations". One counts *how many* elements are active, not how much of them is produced.

(b) The constraints. With $q_j \geq 0$ the quantity and y_j the indicator:

$$q_j \leq c_j y_j, \quad \forall j \in \{1, 2, \dots, n\} \quad (n \text{ constraints}), \quad (3.19)$$

$$q_j \geq \ell y_j, \quad \forall j \in \{1, 2, \dots, n\} \quad (n \text{ constraints}), \quad (3.20)$$

$$\sum_{j=1}^n y_j \geq p \quad (1 \text{ constraint}). \quad (3.21)$$

(c) The proof, and why both thresholds are needed. (3.19) gives " $y_j = 0 \Rightarrow q_j = 0$ "; (3.20) gives the missing direction " $y_j = 1 \Rightarrow q_j \geq \ell$ ". Without (3.20), a solution with $y_j = 1$ and $q_j = 0$ is feasible: the counting constraint (3.21) is satisfied by switching on empty indicators, and the condition "at least p different types" says nothing any more. **Counting types works**

only if every switched-on type actually produces something, and the threshold ℓ is what guarantees it.

(d) **The strength of the relaxation.** Three types, unit revenues $(4, 3, 5)$, resource 12, capacity 10 each, threshold $\ell = 3$, at least two types: $z(MILP) = 57$, with $q = (3, 0, 9)$. The relaxation is also 57: here the counting introduces no gap at all.

(e) **In gurobipy, and where it is seen again.**

```
1 m.addConstrs((q[j] <= c[j] * y[j] for j in range(n)), name="activate")
2 m.addConstrs((q[j] >= ell * y[j] for j in range(n)), name="lot")
3 m.addConstr(y.sum() >= p, name="at_least_p_types")
```

Seen again in exercises 9.3 (vehicles), 10.3 (diet) and 10.4 (trees).

3.12 Alldiff and binary expansion

(a) **The link in words.** Two different but related needs: giving n objects n *all distinct* values (*alldiff*), and representing a bounded integer variable with binary variables (*binary expansion*).

(b) **The constraints.** *Alldiff*, with $p_{iv} = 1$ if object i gets value v :

$$\sum_{v=1}^n p_{iv} = 1, \quad \forall i \in \{1, 2, \dots, n\} \quad (n \text{ constraints}), \quad (3.22)$$

$$\sum_{i=1}^n p_{iv} = 1, \quad \forall v \in \{1, 2, \dots, n\} \quad (n \text{ constraints}). \quad (3.23)$$

Binary expansion of $v \in \{0, 1, \dots, 2^p - 1\}$:

$$v = \sum_{k=0}^{p-1} 2^k b_k, \quad (1 \text{ constraint}), \quad (3.24)$$

$$b_k \in \{0, 1\}, \quad \forall k \in \{0, 1, \dots, p-1\} \quad (p \text{ binaries}).$$

(c) **The proof.** *Alldiff* is a double *set partitioning*: the first group gives "every object one value", the second "every value to one object". Together they impose a bijection, that is, all distinct values. Binary expansion is the base-2 representation, unique for every integer in that range: the correspondence between v and $(b_0, \dots, b_{p-1})$ is one-to-one.

Alldiff has an exact relaxation, expansion does not help

The matrix of the double partitioning is the one of the assignment problem: it is totally unimodular, so every vertex of the relaxation is integer and $z(LP^+) = z(MILP)$. On the 3×3 instance of the script both are 7: integrality is free. Binary expansion, by contrast, adds no strength: $\sum_k 2^k b_k$ with $b_k \in [0, 1]$ covers all of $[0, 2^p - 1]$ continuously, exactly like $v \geq 0, v \leq 2^p - 1$. It serves to *reformulate*, not to tighten — for instance when another part of the model needs binary indicators rather than an integer variable.

(d) **The example.** $v \in \{0, \dots, 7\}$ with $v \geq 5$, $\min v$: the model gives $v = 5 = 1 + 4$, that is $(b_0, b_1, b_2) = (1, 0, 1)$.

(e) In gurobipy, and where it is seen again.

```
1 m.addConstrs((p.sum(i, "*") == 1 for i in range(n)), name="one_value")
2 m.addConstrs((p.sum("v", v) == 1 for v in range(n)), name="alldiff")
3 m.addConstr(v == gp.quicksum(2 ** k * b[k] for k in range(p)), name="
    expansion")
```

Alldiff is seen again in the numerical model of the queens (EX 9) and in the music-school timetable (EX 15).

3.13 Soft constraints, deviations and penalties

(a) **The link in words.** A constraint one would *prefer* to satisfy, but which may be violated at a price: the demand of a period, a timetable preference, a service target. Turning a hard constraint into a soft one is often the difference between an infeasible model and a useful one.

(b) **The constraint.** In place of $a'x = \beta$ one writes

$$\begin{aligned} a'x + s^- - s^+ &= \beta, & (1 \text{ constraint}), \\ s^- &\geq 0, & (1 \text{ continuous variable}), \\ s^+ &\geq 0, & (1 \text{ continuous variable}), \end{aligned} \tag{3.25}$$

and adds $\pi^- s^- + \pi^+ s^+$ to the objective, with penalties $\pi^-, \pi^+ > 0$. The variable s^- measures how far *below* the target one is, s^+ how far above.

(c) **The proof.** The constraint alone is always satisfiable: any $a'x$ can be compensated with one of the two deviations. The interesting property is that **in every optimum at least one of the two deviations is zero**: if $s^- = \sigma > 0$ and $s^+ = \tau > 0$, subtracting $\min(\sigma, \tau)$ from both leaves (3.25) satisfied (the difference $s^- - s^+$ does not change) and reduces the objective by $(\pi^- + \pi^+) \min(\sigma, \tau) > 0$. So s^- and s^+ really are the positive and negative parts of the deviation, and $s^- + s^+ = |a'x - \beta|$ in every optimum: the same thing as Section 3.7, written with an equality instead of two inequalities.

If $\pi^- = 0$, the exchange argument is no longer strict and an optimal solution may have both deviations positive: the reading " s^- is the shortfall" is no longer guaranteed. The same rule as everywhere in this chapter: the strength of the conclusion depends on the sign of the coefficient in the objective.

(d) **The example.** Demand 6 in each of three periods, total availability $15 < 18$, penalties $\pi^+ = 3$ and $\pi^- = 2$. The hard model would be infeasible; the soft one gives $z(MILP) = 6$, with $q = (3, 6, 6)$ and a shortfall of 3 concentrated in the first period. Nothing in the data says it should be concentrated: any split of the total shortfall 3 has the same cost, and the solver returns one of them.

(e) In gurobipy, and where it is seen again.

```
1 sm = m.addVars(T, name="s_minus"); sp = m.addVars(T, name="s_plus")
```

```

2 m.addConstrs((q[t] + sm[t] - sp[t] == demand[t] for t in range(T)),
   name="target")
3 m.setObjective(cost + gp.quicksum(pen_down * sm[t] + pen_up * sp[t] for
   t in range(T)),
4               GRB.MINIMIZE)

```

Seen again in the music-school timetable (EX 15) and in exercise 9.1.

3.14 Piecewise linear functions

(a) **The link in words.** The cost is not proportional to the quantity: it changes in brackets — discounts above a threshold, surcharges beyond a capacity, banded tariffs. The function $g(q)$ is continuous and piecewise linear, with breakpoints $q_0 < q_1 < \dots < q_K$ and values $g_0, \dots, g_K$.

(b) **The constraints.** With $\lambda_k \geq 0$ the weights of the convex combination and $w_t \in \{0, 1\}$ the indicator of piece t (between q_{t-1} and q_t):

$$\sum_{k=0}^K \lambda_k = 1, \quad (1 \text{ constraint}), \quad (3.26)$$

$$q = \sum_{k=0}^K q_k \lambda_k, \quad (1 \text{ constraint}), \quad (3.27)$$

$$g(q) = \sum_{k=0}^K g_k \lambda_k, \quad (1 \text{ constraint}), \quad (3.28)$$

$$\sum_{t=1}^K w_t = 1, \quad (1 \text{ constraint}), \quad (3.29)$$

$$\lambda_k \leq \sum_{t: k \in \{t-1, t\}} w_t, \quad \forall k \in \{0, 1, \dots, K\} \quad (K + 1 \text{ constraints}). \quad (3.30)$$

(c) **The proof.** Constraint (3.30) is **adjacency**: λ_k can be positive only if the chosen piece has q_k as an endpoint. With (3.29) the piece is unique, so at most two consecutive λ are positive, and then $(q, g(q))$ is a point of the segment between (q_{t-1}, g_{t-1}) and (q_t, g_t) : that is, a point of *the graph*. Without adjacency one could mix non-consecutive nodes, and (q, g) would land in the **lower convex envelope** of the graph — below the function.

When g is convex and one *minimises*, the lower convex envelope coincides with the function and the optimal λ turn out adjacent automatically: the model without binaries is already correct. When g is not convex — the case of quantity discounts, where marginal cost *falls* — the difference shows immediately. With breakpoints $(0, 4, 10, 16)$, values $(0, 12, 30, 36)$ and demand $q \geq 13$:

Formulation	optimal value
without adjacency (free convex combination)	$117/4 = 29.25$
with adjacency (3.30)	33

The first mixes nodes 0 and 3 (the only nonzero λ) and returns a cost the function *attains*

nowhere: it is the value of the envelope, not of the graph. The second mixes nodes 2 and 3, adjacent, and gives the exact value $g(13) = 30 + 6 \cdot \frac{3}{6} = 33$.

(d) The strength of the relaxation. Two things must not be confused: adjacency changes the *integer set* (the two models have different optima, 29.25 and 33), but not the strength of the relaxation — with w_t fractional, constraint (3.30) does not bite and the two models share the same $z(LP^+) = 117/4$.

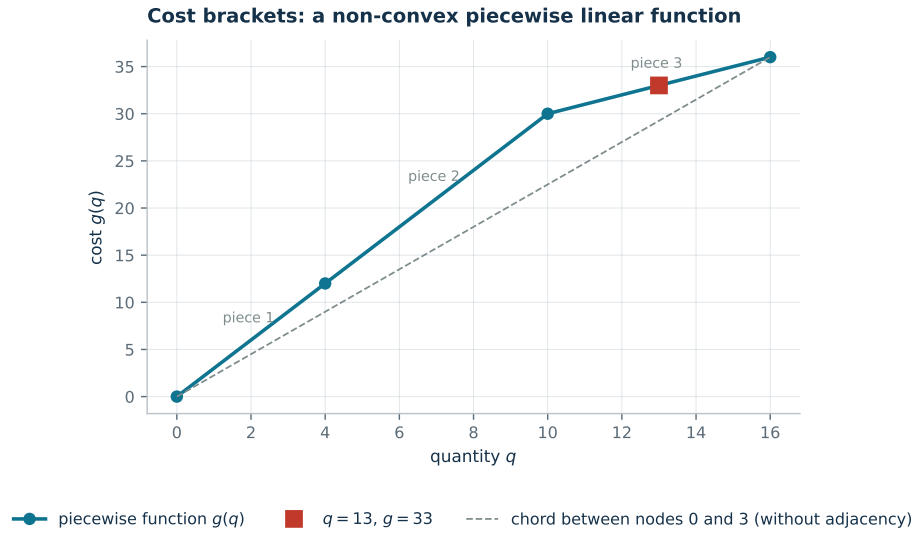


Figure 3.2: The piecewise function with a quantity discount. The dashed chord between the extreme nodes is what the formulation without adjacency can reach: it lies *below* the graph.

(e) In gurobipy, and where it is seen again.

```

1 m.addConstr(lam.sum() == 1, name="convex")
2 m.addConstr(q == gp.quicksum(nodes[k] * lam[k] for k in range(K + 1)),
   name="abscissa")
3 m.addConstr(w.sum() == 1, name="one_piece")
4 for k in range(K + 1):
   on the pieces touching k
   # adjacency: lambda_k only
5     m.addConstr(lam[k] <= gp.quicksum(w[t] for t in (k - 1, k) if 0 <=
       t < K),
6         name=f"adjacency{k}")

```

Gurobi also offers `addGenConstrPWL` and `SOS2` types, which do the same job internally; here the manual formulation remains the main material, because it is the one whose correctness must be provable. Seen again in exercise 10.1 (prizes with two modes).

3.15 The map of techniques

The following table is meant for consultation: read from the first column ("what I want to say") to the second ("how it is written"), while the last two recall what must be declared and where the technique is seen at work.

Technique	Formulation	To be declared	Seen again in
3.1 activation	$x_{ij} \leq y_j$ (disagg.) or $\sum_i x_{ij} \leq k_j y_j$ (agg.)	which form, and why; sign of f_j for the optimality direction	7.2, 7.3, 7.5, 8.4
3.2 fixed cost	$q_j \leq c_j y_j$	that c_j is the capacity, not a big-M	8.1, ch. 9
3.3 minimum lot	$\ell y_j \leq q_j \leq c_j y_j$	that $\ell \leq c_j$; that the relaxation is unaffected	7.2.2, 9.1, 9.3
3.4 integer count	$\sum_i a_i x_i \leq Kw$, w integer	that integrality realises the ceiling	9.2, 10.4, 10.5
3.5 maximum auxiliary	$z \geq t_j x_j$, $z \geq 0$	that z appears nowhere else; its sign in the objective	7.4, 7.7, 8.4, 10.9
3.6 min-max / max-min	$T \geq L_k$ and $\min T$; $U \leq L_k$ and $\max U$	which of the three objectives, and that they are not comparable	7.4.1, 10.7, 10.8
3.7 absolute value	$d \geq \pm(u - v)$ in the objective; disjunction if $\geq k$	whether it is objective or constraint, and in which direction	10.7, 10.8
3.8 big-M	$a'x \leq b + M(1 - y)$	the value of M computed from the data	7.7, 3.9
3.9 precedences	$s_{ij} + s_{ji} = 1$, $\kappa_i \geq \kappa_j + t_i - M(1 - s_{ij})$	the horizon and $M = \sum_h t_h$	7.7
3.10 if and only if	$y \leq x_j$ and $y \geq \sum_j x_j - (p - 1)$	whether the second direction is needed or follows from optimality	7.6, 9.3
3.11 counting types	$\ell y_j \leq q_j \leq c_j y_j$, $\sum_j y_j \geq p$	that without the threshold ℓ the count is empty	9.3, 10.3, 10.4
3.12 alldiff / expansion	double partitioning; $v = \sum_k 2^k b_k$	that alldiff has an exact relaxation	EX 9, EX 15
3.13 soft constraints	$a'x + s^- - s^+ = \beta$ with penalties	the two signs of the penalties	9.1, EX 15
3.14 piecewise function	convex combination + adjacency	whether g is convex; otherwise adjacency is mandatory	10.1

The three questions to ask about a new link

1. **Which direction is imposed by the constraints and which is not?** Answered by looking at the constraints alone, one case per value of the binary.
2. **Does the missing direction follow from optimality?** Answered by the exchange argument, and the answer depends on the *sign* of the coefficient in the objective: strictly nonzero gives "in every optimum", zero gives at most "there exists an optimum".
3. **How strong is the relaxation?** Answered by comparing $z(LP^+)$ with $z(MILP)$ on a small instance and — when there are two formulations — by comparing them with each other. The two formulations must first be proved equivalent on integer points: without that step one is not comparing two models of the same problem.

Relaxations, duality and bounds

Script: `python/cap04_bounds.py`

[Online page](#)

This chapter teaches how to produce, *by hand*, a number that certainly lies on one side of the integer optimum. It serves three purposes: understanding how good a model is, understanding how good a heuristic is, and reading with judgement the numbers a solver reports when it has not finished.

4.1 What a relaxation is

Operational definition

A **relaxation** of a minimisation problem with feasible set X is a problem with the same objective and a larger feasible set, $X \subseteq \hat{X}$: the only thing relaxed are the constraints. Then

$$\min_{x \in \hat{X}} \sum_{j=1}^n c_j x_j \leq \min_{x \in X} \sum_{j=1}^n c_j x_j,$$

because the minimum over a larger set cannot be higher. In a maximisation the inequality is reversed.

The three relaxations used in this course, in increasing order of strength:

Name	What is dropped	Note
$z(LP)$, relaxation without the bounds	$x \in \{0, 1\}$ becomes $x \geq 0$	this is the one whose dual is written by hand: it has fewer constraints and so a dual with fewer variables
$z(LP^+)$, with the bounds	$x \in \{0, 1\}$ becomes $0 \leq x \leq 1$	this is Gurobi's <code>relax()</code> , the first relaxation the solver solves
$z(LP^{++})$, with the bounds and the valid inequalities	as above, plus valid inequalities	Section 4.7 explains which ones

In a minimisation, $z(LP) \leq z(LP^+) \leq z(LP^{++}) \leq z(MILP)$.

If the model contains an assignment constraint $\sum_m x_{jm} = 1$ with $x \geq 0$, then $x_{jm} \leq 1$ is already implied and the two relaxations are *equal*. In the bound table of Chapter 7 this happens in problems 1, 4, 6 and 7. When the two do differ, the difference must be reported: it is information about the model.

4.2 The primal/dual conversion table

From a minimisation primal to its dual

Let the primal be a minimisation, with m constraints indexed by i and n variables indexed by j .

In the primal (min)	In the dual (max)
constraint i of type $\geq$	variable $\pi_i \geq 0$
constraint i of type $\leq$	variable $\pi_i \leq 0$
equality constraint i	free variable π_i
variable $x_j \geq 0$	constraint j of type $\leq c_j$
free variable x_j	equality constraint $j, = c_j$

The dual objective is $\max \sum_i b_i \pi_i$ and dual constraint j is $\sum_i a_{ij} \pi_i \leq c_j$ with the direction given by the table. If the primal is a **maximisation**, every direction in the table is reversed and the dual is a minimisation.

How to read the dual

Dual constraint j says: "the value I attach to the resources consumed by activity j cannot exceed its cost". With this reading, every recipe for building a feasible dual solution has an economic meaning, and that is what makes it memorable: Section 4.4 gives three.

4.3 Weak duality, strong duality

The two theorems, and what they are for here

Let $(\bar{x}_1, \dots, \bar{x}_n)$ be a feasible solution of a minimisation model of LP type and $(\bar{\pi}_1, \dots, \bar{\pi}_m)$ a feasible solution of its dual.

- **Weak duality:** $\sum_{i=1}^m b_i \bar{\pi}_i \leq \sum_{j=1}^n c_j \bar{x}_j$. *Always*, with no assumptions. This is the one we need: it gives a lower bound from *any* feasible dual solution, even one built by hand and far from optimal.
- **Strong duality:** if the LP has a finite optimum, then $z(D(LP)) = z(LP)$. It serves as a *check*: the optimum of the dual written by hand must coincide with $z(LP)$; if it does not, the dual is written wrong. The course scripts verify it with an `assert` in `due_rilassamenti`.

And then the step to the integer problem, which is trivial but must be said: since every

feasible solution of the MILP is feasible for the relaxation too,

$$\sum_{i=1}^m b_i \bar{\pi}_i \leq z(LP) \leq z(MILP).$$

A dual bound of the LP is a bound for the MILP too, on the same side.

The dual one writes is that of the *relaxation*, not of the MILP. A MILP has no linear dual, and strong duality between a MILP and any linear program does not hold in general: the jump $z(MILP) - z(LP)$ is precisely what is missing. When this course says "the dual", it always means the dual of the stated relaxation.

4.4 Three recipes for building a dual solution by hand

The recipes

1. **Zero out and saturate.** Set all dual variables to zero except one family, and push those to the largest value (in a maximisation dual) compatible with the dual constraints. Example: in problem 7.1 one sets $\bar{\pi} = 0$ and obtains $\bar{\mu}_j = \min_m c_{jm}$, that is "every job costs at least its cheapest option".
2. **Constructive heuristic on the constraints.** Scan the primal constraints one at a time, raise the corresponding dual variable until the first dual constraint that opposes it becomes tight, and update the residuals. This is the recipe used below for the covering problem.
3. **The best ratio.** With a single capacity constraint in a maximisation, the dual variable $\bar{v} = \max_j p_j/w_j$ is feasible and gives the bound $b\bar{v}$: "all the capacity used at the best efficiency".

Whichever recipe is used, the constructed solution must be *checked* feasible for the dual — that is the only thing that makes the bound valid — and its value compared with $z(LP)$ to see how much was left on the table.

4.5 A minimisation problem, in full

Minimum-cost zone covering

A town has $n \in \mathbb{Z}_{\geq 1}$ districts; each has an emergency team, and activating the team of district $j \in \{1, 2, \dots, n\}$ costs $c_j \in \mathbb{Q}_{>0}$ thousand euros. There are $m \in \mathbb{Z}_{\geq 1}$ sensitive zones: zone $i \in \{1, 2, \dots, m\}$ is covered if at least one of the teams in the set S_i of the districts bordering it is active. All zones must be covered at minimum cost.

Data: $n = 4$ districts of cost $c = (4, 3, 5, 3)$, and $m = 6$ zones, which correspond to the six pairs of districts, in the order $\{1, 2\}$, $\{2, 3\}$, $\{1, 3\}$, $\{1, 4\}$, $\{2, 4\}$, $\{3, 4\}$. With $x_j = 1$ if team j is active (n binary variables) and S_i the set of teams covering zone i :

Minimum-cost covering

$$\min \sum_{j=1}^n c_j x_j \quad (4.1a)$$

$$\text{subject to } \sum_{j \in S_i} x_j \geq 1, \quad \forall i \in \{1, 2, \dots, m\}, \quad (4.1b)$$

$$x_j \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, n\}. \quad (4.1c)$$

It is a *set covering*: m covering constraints, one per zone.

The dual of the relaxation without the bounds. Relaxing (4.1c) to $x_j \geq 0$ and attaching to each constraint (4.1b) a dual variable $\pi_i \geq 0$ ($\geq$ direction in a minimisation, by the conversion table):

Dual of the LP relaxation

$$\max \sum_{i=1}^m \pi_i \quad (4.2a)$$

$$\text{subject to } \sum_{i: j \in S_i} \pi_i \leq c_j, \quad \forall j \in \{1, 2, \dots, n\}, \quad (4.2b)$$

$$\pi_i \geq 0, \quad \forall i \in \{1, 2, \dots, m\}. \quad (4.2c)$$

Description. The variable π_i is the price attached to zone i . The objective (4.2a) adds up the prices of the m zones. Constraints (4.2b) are the columns of the x_j , one per team: the zones team j covers, priced at those values, cannot be worth more than what it costs to activate it, namely c_j . This is the economic reading of the previous section: no team may turn out to be a bargain.

For the instance, every team covers three zones and the four dual constraints are

$$\pi_1 + \pi_3 + \pi_4 \leq 4, \quad \pi_1 + \pi_2 + \pi_5 \leq 3, \quad \pi_2 + \pi_3 + \pi_6 \leq 5, \quad \pi_4 + \pi_5 + \pi_6 \leq 3.$$

Covering on the instance — primal

$$\min 4x_1 + 3x_2 + 5x_3 + 3x_4$$

$$x_1 + x_2 \geq 1$$

$$x_2 + x_3 \geq 1$$

$$x_1 + x_3 \geq 1$$

$$x_1 + x_4 \geq 1$$

$$x_2 + x_4 \geq 1$$

$$x_3 + x_4 \geq 1$$

$$x_1, x_2, x_3, x_4 \in \{0, 1\}$$

Covering on the instance — dual of the LP relaxation

$$\begin{aligned}
\max \quad & \pi_1 + \pi_2 + \pi_3 + \pi_4 + \pi_5 + \pi_6 \\
& \pi_1 + \pi_3 + \pi_4 \leq 4 \\
& \pi_1 + \pi_2 + \pi_5 \leq 3 \\
& \pi_2 + \pi_3 + \pi_6 \leq 5 \\
& \pi_4 + \pi_5 + \pi_6 \leq 3 \\
& \pi_1, \pi_2, \pi_3, \pi_4, \pi_5, \pi_6 \geq 0
\end{aligned}$$

A dual solution by hand (recipe 2). Scan the zones in order; for zone i raise π_i by the most the residuals of the dual constraints of the teams in S_i allow, then update the residuals.

- **Zone 1** ($S_1 = \{1, 2\}$): residuals $(4, 3, 5, 3)$, the smallest among teams 1 and 2 is 3. Set $\bar{\pi}_1 = 3$; residuals $(1, 0, 5, 3)$.
- **Zone 2** ($\{2, 3\}$): the residual of team 2 is 0, so $\bar{\pi}_2 = 0$.
- **Zone 3** ($\{1, 3\}$): the smallest of 1 and 5 is 1. Set $\bar{\pi}_3 = 1$; residuals $(0, 0, 4, 3)$.
- **Zones 4 and 5** ($\{1, 4\}$ and $\{2, 4\}$): teams 1 and 2 have zero residual, so $\bar{\pi}_4 = \bar{\pi}_5 = 0$.
- **Zone 6** ($\{3, 4\}$): the smallest of 4 and 3 is 3. Set $\bar{\pi}_6 = 3$; residuals $(0, 0, 1, 0)$.

All dual constraints hold by construction, and the value is

$$LB = \bar{\pi}_1 + \dots + \bar{\pi}_6 = 3 + 0 + 1 + 0 + 0 + 3 = 7.$$

A primal upper bound. The covering constructive heuristic picks, at each step, the team with the best ratio between cost and zones still uncovered: it takes teams 1, 2 and 4, of cost $4 + 3 + 3 = 10$. The solution is feasible and *integer*, so $UB = 10$.

The sandwich, and what the solver says. $z(LP) = z(LP^+) = 15/2$ (the two relaxations coincide, and the optimum of the dual confirms it: strong duality) and $z(MILP) = 10$, attained with teams 1, 2, 4 — the same solution as the heuristic, which here is optimal.

UB (constructive heuristic)	LB (dual by hand)	$z(LP)$	$z(MILP)$	heuristic gap
10	7	15/2	10	0.0%

Read it this way: the hand-built bound is 7, the relaxation 7.5, the optimum 10. The certified gap between our two hand-built bounds is $(10 - 7)/10 = 30\%$: without solving the MILP we would know only that the optimum lies between 7 and 10. The heuristic was already optimal, but the bounds cannot tell us that.

4.6 A maximisation problem: the roles swap

Knapsack

There are $n \in \mathbb{Z}_{\geq 1}$ items: item $j \in \{1, 2, \dots, n\}$ has value $p_j \in \mathbb{Q}_{>0}$ and weight $w_j \in \mathbb{Q}_{>0}$, and the knapsack has capacity $b \in \mathbb{Q}_{>0}$. Choose the subset of maximum value that fits.

The data of the instance are $n = 4$ items of value $p = (10, 7, 6, 4)$ and weight $w = (5, 4, 3, 3)$, and a knapsack of capacity $b = 9$.

The following n binary variables are introduced:

$$x_j = \begin{cases} 1 & \text{if item } j \text{ is put in the knapsack,} \\ 0 & \text{otherwise,} \end{cases} \quad \forall j \in \{1, 2, \dots, n\}.$$

Binary knapsack

$$\max \sum_{j=1}^n p_j x_j \quad (4.3a)$$

$$\text{subject to } \sum_{j=1}^n w_j x_j \leq b, \quad (4.3b)$$

$$x_j \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, n\}. \quad (4.3c)$$

Description. The objective (4.3a) adds up the value of the chosen items. Constraint (4.3b) is the capacity of the knapsack: there is one of them, and it says that the total weight of the chosen items does not exceed b . The domains (4.3c) are the four binary conditions, one per item: an item is either taken whole or not taken.

The dual of the relaxation without the bounds. The domain (4.3c) is relaxed to $x_j \geq 0$. The primal is a maximisation with a single constraint of type $\leq$: the conversion table gives a single dual variable $v \geq 0$, attached to the capacity, and one dual constraint for each of the n primal variables.

Dual of the LP relaxation of the knapsack model

$$\min \quad b v \quad (4.4a)$$

$$\text{subject to } w_j v \geq p_j, \quad \forall j \in \{1, 2, \dots, n\}, \quad (4.4b)$$

$$v \geq 0. \quad (4.4c)$$

Description. The variable v is the price of one unit of capacity. The objective (4.4a) is the value attached to the whole available capacity. The n constraints (4.4b), one per item, say that the capacity consumed by item j , priced at v , is worth no less than the profit that item brings.

Written out on the data of the instance, the primal and the dual are as follows.

Knapsack on the instance — primal

$$\begin{aligned}
& \max 10x_1 + 7x_2 + 6x_3 + 4x_4 \\
& 5x_1 + 4x_2 + 3x_3 + 3x_4 \leq 9 \\
& x_1, \quad x_2, \quad x_3, \quad x_4 \in \{0, 1\}
\end{aligned}$$

Knapsack on the instance — dual of the LP relaxation

$$\begin{aligned}
& \min 9v \\
& 5v \geq 10 \\
& 4v \geq 7 \\
& 3v \geq 6 \\
& 3v \geq 4 \\
& v \geq 0
\end{aligned}$$

Each dual constraint has a single term, because each item consumes a single resource: the column of variable x_j in the primal has a single row.

The two bounds.

- **Heuristic** (constructive heuristic by ratio p_j/w_j): the ratios are 2, 7/4, 2, 4/3; item 1 is taken ($w = 5$) and then item 3 ($w = 3$), for a weight of $8 \leq 9$; item 2 no longer fits. Value $10 + 6 = 16$. In a **maximisation** the heuristic gives a **lower** bound: $LB = 16 \leq z(MILP)$.
- **Dual by hand** (recipe 3): $\bar{v} = \max_j p_j/w_j = 2$ satisfies $w_j \bar{v} \geq p_j$ for every j , so it is feasible, and its value is $b\bar{v} = 9 \cdot 2 = 18$. In a **maximisation** the dual gives an **upper** bound: $z(MILP) \leq UB = 18$.

The sandwich. $z(MILP) = 17$ (items 1 and 2, weight exactly 9), $z(LP) = 18$ and $z(LP^+) = 71/4 = 17.75$:

$$16 \leq z(MILP) = 17 \leq z(LP^+) = \frac{71}{4} \leq z(LP) = 18 = UB.$$

LB (constructive heuristic)	$z(MILP)$	$z(LP^+)$	$z(LP)$	UB (dual by hand)	heuristic gap
16	17	71/4	18	18	5.9%

Here the dual by hand is *optimal* for the relaxation without the bounds ($UB = z(LP) = 18$), and the relaxation with the bounds is strictly better ($71/4 < 18$): the constraint $x_j \leq 1$ bites, because without it the LP takes $9/5$ units of item 1.

The sandwich, written once and for all

$$\text{minimisation: } LB(\bar{\pi}) \leq z(D(LP)) = z(LP) \leq z(LP^+) \leq z(MILP) \leq UB(\bar{x})$$

$$\text{maximisation: } LB(\bar{x}) \leq z(MILP) \leq z(LP^+) \leq z(LP) = z(D(LP)) \leq UB(\bar{\pi})$$

where $(\bar{\pi}_1, \bar{\pi}_2, \dots, \bar{\pi}_m)$ is a feasible dual solution of the relaxation, of value $\sum_{i=1}^m b_i \bar{\pi}_i$, and $(\bar{x}_1, \bar{x}_2, \dots, \bar{x}_n)$ a feasible solution of the MILP, of value $\sum_{j=1}^n c_j \bar{x}_j$. The chains read as

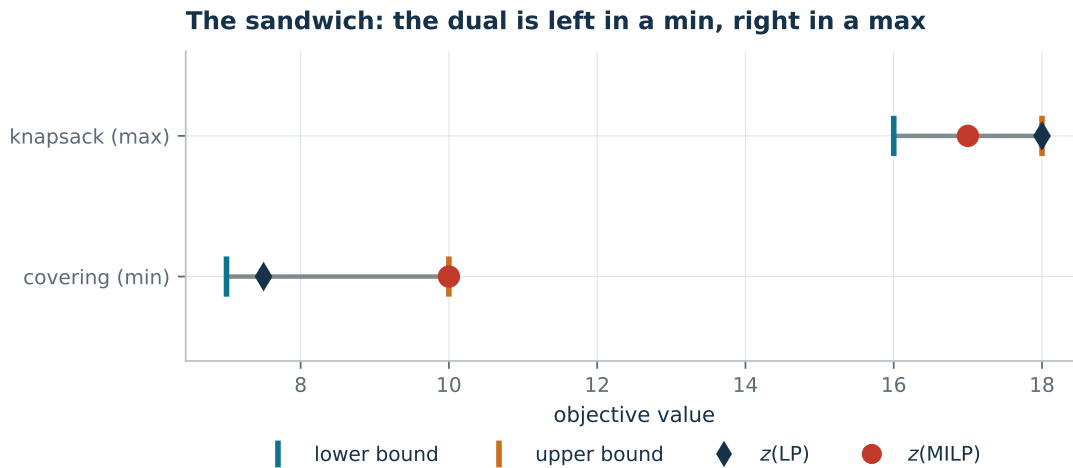


Figure 4.1: The two examples. In a minimisation the dual bound lies to the left of the optimum and the heuristic to the right; in a maximisation the roles swap.

follows: the *relaxation side* is optimistic and holds all the dual bounds; the *heuristic side* is pessimistic and holds all the feasible solutions. The name (*LB* or *UB*) depends on the direction of the objective, the role does not.

4.7 Valid inequalities and constraints that preserve optimality

Two different notions

- A **valid inequality** for a MILP is satisfied by *all* feasible integer solutions. Adding it does not change $z(MILP)$; it may reduce $z(LP^+)$, and then it is called a **cut**.
- A **constraint that preserves optimality** cuts off some feasible solutions, but not all the optimal ones: after adding it an optimum still exists and $z(MILP)$ does not change. It is not a valid inequality, and it must be declared as such. Example: $z_j \leq M_j y_j$ in problem 8.4.

The difference matters: the first can always be added; the second only after proving that at least one optimum survives, and never if one then wants to enumerate *all* the optimal solutions.

The cover cut for the knapsack

A set S of items is a **cover** if $\sum_{j \in S} w_j > b$: they do not all fit together. Then

$$\sum_{j \in S} x_j \leq |S| - 1$$

is a valid inequality. The cover is **minimal** if no proper subset of it is already a cover; the cuts of minimal covers are the strongest.

On the knapsack instance ($w = (5, 4, 3, 3)$, $b = 9$) the minimal covers are the four triples

$\{1, 2, 3\}$, $\{1, 2, 4\}$, $\{1, 3, 4\}$, $\{2, 3, 4\}$, of weight 12, 12, 11, 10: all > 9 , and no pair is. The four cuts are $\sum_{j \in S} x_j \leq 2$.

The optimal solution of the relaxation $z(LP^+) = 71/4$ is $\tilde{x} = (1, 1/4, 1, 0)$. On it:

Cover S	$\sum_{j \in S} \tilde{x}_j$	$ S - 1$	
$\{1, 2, 3\}$	9/4	2	violated: the cut is needed
$\{1, 2, 4\}$	5/4	2	satisfied
$\{1, 3, 4\}$	2	2	satisfied (with equality)
$\{2, 3, 4\}$	5/4	2	satisfied

Adding all four cuts, $z(LP^+)$ drops from $71/4 = 17.75$ to $69/4 = 17.25$, and $z(MILP)$ stays 17: the relaxation has moved closer to the integer optimum without changing it. This is the mechanism by which a modern solver closes gaps.

4.8 Stronger formulations: what it means, and how it is proved

The criterion

Two formulations A and B of the same problem are compared in two steps.

1. **Same integer set:** $X_A \cap \mathbb{Z}^n = X_B \cap \mathbb{Z}^n$. Without this step nothing is being compared. It is proved in both directions, using binarity or integrality where needed.
2. **Relaxation:** B is *stronger* than A if $X_B \subseteq X_A$ as polyhedra, and then $z_B(LP^+)$ is at least as good as $z_A(LP^+)$. If the inclusion is strict *and* one exhibits a point of $X_A \setminus X_B$ that is optimal for the relaxation of A , the improvement is proved on that instance.

The reference case is the activation of Section 3.1: $x_{ij} \leq y_j$ implies $\sum_i x_{ij} \leq ny_j$, so the disaggregated form is stronger; on three customers and two sites the relaxation goes from $38/3$ to 15, which is already the integer optimum.

A stronger formulation has fewer nodes but more rows, and every node costs more. The balance depends on the instance. What can be *proved* is the strength of the relaxation; speed is *measured*.

4.9 What the solver says

The four attributes to read, and in which order

1. **Status:** 2 is OPTIMAL, 3 INFEASIBLE, 9 TIME_LIMIT. To be read *before* anything else.
2. **SolCount:** how many integer solutions were found. If it is 0, **ObjVal** does not exist and reading it is an error.
3. **ObjVal** and **ObjBound:** the best solution found and the best bound proved. In a minimisation, $\text{ObjBound} \leq z(MILP) \leq \text{ObjVal}$.

4. **MIPGap** and **NodeCount**: the certified gap between the two and how many nodes were explored.

On the covering instance, the relaxation of the model *as we wrote it* is $15/2$ and the integer optimum is 10. Yet Gurobi reports **ObjBound** = 10 and **NodeCount** = 0: it closed the gap at the root, with presolve, its own cuts and heuristics, without ever branching. Switching those off (**Presolve** = **Cuts** = **Heuristics** = 0) the same model gives the same optimum 10 but with 5 nodes. Two consequences:

- "how hard a model is" is not a property of the model alone: it also depends on what the solver brings;
- the relaxation we speak of in the hand-built bounds is that of the model as written, and it is obtained with **relax()**: reading **ObjBound** at the end of the solve gives the optimum, not the relaxation.

4.10 LP duals are not the marginal prices of the MILP

The claim, and the counterexample

In an LP, the dual $\tilde{\pi}_i$ of a resource constraint is its *marginal price*: increasing b_i by a small amount changes the optimum by $\tilde{\pi}_i$ per unit (as long as the basis does not change). **For a MILP this is false**, and not by approximation.

On the knapsack, as the capacity b varies:

b	$z(MILP)$	$z(LP^+)$	LP dual	true change in $z(MILP)$
8	16	16	2	—
9	17	$71/4$	$7/4$	+1
10	17	$39/2$	$7/4$	0
11	20	$85/4$	$7/4$	+3
12	23	23	$7/4$	+3

The LP dual is the ratio p_j/w_j of the "critical" item: it is 2 when the capacity runs out on item 1 and $7/4$ when there is room left for item 2. The true change of the integer optimum comes in jumps: from $b = 9$ to $b = 10$ it does not change *at all* (no subset exploits the extra unit), while the dual promises $7/4$; from 10 to 11 it jumps by 3.

Of the LP dual, the only use this course makes of it remains true: it is a **bound**. $\tilde{\pi}_i$ says how much the resource is worth *in the relaxation*, and the sum $b'\tilde{u}$ is a valid bound for the MILP. As managerial advice ("is it worth buying one more unit?") it must instead be checked by solving the MILP again: the difference $z(MILP)(b_i + 1) - z(MILP)(b_i)$ is the only correct answer, and there is no closed formula for it.

4.11 The bound protocol of the course

What every Part II exercise produces

1. a **feasible and integer** solution from a constructive heuristic, checked on constraints, bounds *and* integrality (the integrality check is what makes the value a bound of the MILP and not of the relaxation alone);
2. the **dual of the relaxation without the bounds**, written in general form and for the instance, with the directions taken from the conversion table;
3. a **feasible dual solution built by hand**, with the recipe declared, checked feasible and evaluated;
4. the two **relaxations** $z(LP)$ and $z(LP^+)$ from the solver, with a note on whether they coincide and why;
5. the **optimum** $z(MILP)$ and the table $UB \cdot LB \cdot z(LP) \cdot z(LP^+) \cdot z(MILP) \cdot \text{gap}$, with the gap defined as in Chapter 1;
6. the **additional considerations**: valid inequalities with their count, constraints that preserve optimality declared as such, comparison of the relaxations of alternative formulations.

Every number in the table exists in a CSV produced by the problem's script, and an **assert** in `check_numbers.py` compares it with the value quoted in the text.

Constructive heuristics

Script: `python/cap05_heuristics.py`, `python/euristiche.py`

[Online page](#)

A constructive heuristic builds *one* solution, quickly, adding one element at a time and never backtracking. It proves nothing about the quality of that solution, and it is not even guaranteed to reach a feasible one: it can get stuck part-way, with an element that fits nowhere — precisely because it never backtracks. When it does end with a feasible solution, that solution is the other half of the sandwich of Chapter 4: the pessimistic side, the one guaranteed by a solution that really exists; when it fails, there is no primal bound.

What a heuristic must produce in this course

1. a readable **pseudocode**, with the scanning order, the choice criterion, the tie-breaking rule and the failure case stated explicitly;
2. the corresponding **Python function**, line by line (in `python/euristiche.py`);
3. the **trace** of the execution on an instance, which is the text appearing in the notes;
4. the **feasibility check** of the solution produced: constraints, bounds *and* integrality;
5. the resulting **bound**, with the right name: *UB* in a minimisation, *LB* in a maximisation.

Point 4 is not a formality. A solution satisfying the linear constraints but with a fractional component is feasible for the *relaxation*, not for the MILP, and its value is not a primal bound. The function `ammissibile` of `python/mip.py` checks integrality too, precisely for this reason.

A heuristic produces a feasible solution, and its value is attainable. In a **minimisation** that value is an *upper* bound: $z(MILP) \leq UB$. In a **maximisation** it is a *lower* bound: $LB \leq z(MILP)$. Calling *UB* the result of a constructive heuristic on a maximisation is the commonest sign error in the course.

5.1 The three bin-packing heuristics

These are the three of family 7: jobs to be assigned to machines with limited availability.

Next-fit, first-fit, best-fit

Input: n, k , durations $t[1..n][1..k]$, availability $a[1..k]$, criterion γ

Output: a feasible solution x or "no solution found"

```

1 Function Build( $n, k, t, a, \gamma$ )
2    $x[j][m] \leftarrow 0$  for every  $j, m$ ;    $ra[m] \leftarrow a[m]$  for every  $m$ 
3   for  $j := 1$  to  $n$  do
4     // next-fit: the current machine only, then the next one
5     // first-fit: the first  $m$  with  $t[j][m] \leq ra[m]$ 
6     // best-fit: among the feasible  $m$ , the one with smallest
7      $\gamma(j, m, ra)$ 
8     choose  $m^*$  by the rule
9     if no  $m$  is feasible then return "no solution found"
10     $x[j][m^*] \leftarrow 1$ ;    $ra[m^*] \leftarrow ra[m^*] - t[j][m^*]$ 
11  end
12  return  $x$ 

```

All three scan the jobs *in the given order*: changing the order changes the result, and this must be said when a value is reported. Ties in the criterion γ are broken on the smallest index, so the run is reproducible.

On the instance of problem 7.1 ($n = k = 3$, durations and costs of Section 7.1, availability $a = (5, 6, 7)$, a *minimisation*):

Heuristic	UB	$z(MILP)$	gap
next-fit	14	11	27.3%
first-fit	14	11	27.3%
best-fit on cost	11	11	0.0%

The best-fit on cost finds the optimum; but no bound certifies it — that takes the solver, or a dual bound reaching 11, and in problem 7.1 the hand-built dual stops at 10.

5.2 LPT: balancing over identical machines

The idea

To minimise the *makespan* — the load of the busiest machine — over k identical machines: sort the jobs by **decreasing** duration and send each to the machine with the smallest current load. The decreasing order is essential: leaving the long jobs for last makes them impossible to place.

LPT — longest processing time

Input: n, k , durations $t[1..n]$ **Output:** an assignment x and its makespan

```

1 Function  $LPT(n, k, t)$ 
2    $L[m] \leftarrow 0$  for every  $m$                                 // current loads
3   for  $j$  in order of decreasing  $t[j]$  do
4      $m^* \leftarrow \arg \min_m L[m]$                             // ties: the smallest index
5      $x[j][m^*] \leftarrow 1$ ;  $L[m^*] \leftarrow L[m^*] + t[j]$ 
6   end
7   return  $x, \max_m L[m]$ 

```

Seven jobs on three machines

$t = (5, 5, 4, 4, 3, 3, 3)$, $k = 3$, total 27. Trace:

- **Steps 1–3.** The jobs 5, 5, 4 go to the three empty machines: $L = (5, 5, 4)$.
- **Step 4.** Job 4: the smallest load is machine 3, which goes to 8. $L = (5, 5, 8)$.
- **Steps 5–6.** The two jobs of length 3 go to machines 1 and 2: $L = (8, 8, 8)$.
- **Step 7.** The last job of length 3 finds all loads equal to 8; by the tie rule it goes to machine 1, which reaches 11.

LPT makespan: $UB = 11$, with loads $(11, 8, 8)$.

The elementary bound. The makespan is at least $\max(\max_j t_j, \sum_j t_j/k) = \max(5, 27/3) = 9$: no machine can do less than the longest job, and at least one must carry the average. The optimum is exactly $z(MILP) = 9$ — attained with $\{5, 4\}$, $\{5, 4\}$, $\{3, 3, 3\}$ — and LPT is off by 22.2%.

Two free bounds, to be compared

$\max_j t_j$ and $\sum_j t_j/k$ are two lower bounds computable without solving anything, and the better of the two is often already close to the optimum. An "obvious" bound that nobody writes down is a wasted bound: the dual of Chapter 4 is for when the obvious ones are not enough, not instead of them.

5.3 Constructive covering heuristic

Constructive covering heuristic

Input: costs $c[1..n]$, sets $S[1..m]$ (who covers zone i)

Output: a set of elements covering all zones

```

1 Function CoveringConstructive heuristic( $c, S$ )
2    $uncovered \leftarrow \{1, 2, \dots, m\}; \quad y[j] \leftarrow 0$  for every  $j$ 
3   while  $uncovered \neq \emptyset$  do
4     for every  $j$  not yet chosen:  $new(j) \leftarrow |\{i \in uncovered : j \in S_i\}|$ 
5     if  $new(j) = 0$  for every  $j$  then return "no solution found"
6      $j^* \leftarrow \arg \min_{j: new(j) > 0} c[j]/new(j)$ 
7      $y[j^*] \leftarrow 1; \quad uncovered \leftarrow uncovered \setminus \{i : j^* \in S_i\}$ 
8   end
9   return  $y$ 

```

The criterion is the **cost per newly covered zone**, not the absolute cost: picking the cheapest element regardless of how much it covers leads to far worse solutions.

The four teams of Chapter 4

Costs $c = (4, 3, 5, 3)$, six zones (the six pairs of districts).

- **Step 1.** All six zones uncovered; ratios $4/3, 3/3 = 1, 5/3, 3/3 = 1$; a tie between elements 2 and 4, element 2 is chosen and covers zones 1, 2 and 5.
- **Step 2.** Zones 3, 4, 6 remain; ratios $4/2 = 2, 5/2, 3/2$; element 4 is chosen and covers zones 4 and 6.
- **Step 3.** Zone 3 remains; ratios $4/1$ and $5/1$; element 1 is chosen.

Solution $\{1, 2, 4\}$, of cost $UB = 10$, which here is the optimum: $z(MILP) = 10$. The dual bound built in Chapter 4 is 7: the certified gap is 30%, even though the heuristic was already optimal.

5.4 Knapsack constructive heuristic: a primal bound

Constructive heuristic by value/weight ratio

Input: values $p[1..n]$, weights $w[1..n]$, capacity C

Output: a feasible subset

```

1 Function KnapsackConstructive heuristic( $p, w, C$ )
2    $residual \leftarrow C; \quad y[j] \leftarrow 0$  for every  $j$ 
3   for  $j$  in order of decreasing  $p[j]/w[j]$  do
4     if  $w[j] \leq residual$  then  $y[j] \leftarrow 1; \quad residual \leftarrow residual - w[j]$ 
5   end
6   return  $y$ 

```

On the instance $p = (10, 7, 6, 4)$, $w = (5, 4, 3, 3)$, $C = 9$: the ratios are 2, $7/4$, 2, $4/3$; item 1

and then item 3 are taken (weight 8), items 2 and 4 no longer fit. Value 16. Since the problem is a *maximisation*,

$$LB = 16 \leq z(MILP) = 17,$$

with a gap of 5.9%: the optimum takes items 1 and 2, filling the knapsack exactly. The ratio constructive heuristic goes wrong precisely because item 3, better by ratio, leaves an unusable residual.

5.5 Lot sizing: least unit cost period covering

Least unit cost

Input: demands $d[1..T]$, setup cost f , unit holding cost h
Output: the periods in which production is launched, and the quantities

```

1 Function LeastUnitCost( $d, f, h$ )
2    $t \leftarrow 1$ 
3   while  $t \leq T$  do
4     skip the periods with  $d[t] = 0$ 
5     for  $k := 1$  to  $T - t + 1$  do
6        $q_k \leftarrow \sum_{s=t}^{t+k-1} d[s]; \quad c_k \leftarrow (f + h \sum_{s=t}^{t+k-1} (s - t) d[s]) / q_k$ 
7     end
8      $k^* \leftarrow \arg \min_k c_k$  // the lowest average cost per unit
9     produce  $Q_{k^*}$  in period  $t$ ;  $t \leftarrow t + k^*$ 
10  end
```

Wagner–Whitin is an **exact** dynamic-programming algorithm for the *uncapacitated* lot-sizing model: it solves that model to optimality in polynomial time. The procedure above is a heuristic, and its value is only a bound. Calling it "the Wagner–Whitin constructive heuristic" confuses two different things.

Five periods

Demands $d = (20, 10, 30, 40, 10)$, setup $f = 50$, holding $h = 1$.

- **Step 1.** From period 1 it pays to cover 2 periods (quantity 30, unit cost 2).
- **Step 2.** From period 3 it pays to cover 2 periods (quantity 70, unit cost ≈ 1.286).
- **Step 3.** From period 5 only that period remains (quantity 10, unit cost 5).

Cost of the heuristic solution: $UB = 200$. The optimum of the corresponding MILP is $z(MILP) = 170$, a gap of 17.6%: it is also the value Wagner–Whitin would give, being exact on this model.

5.6 Local search, and what it does not give

One step beyond construction

A **local search** starts from a feasible solution and tries elementary moves (move a job, swap two items), accepting those that improve. It stops at a **local optimum**: a solution that no move of the chosen family improves.

On the LPT solution of the example ($L = (11, 8, 8)$, makespan 11), the move "move one job to another machine" improves nothing: moving one of the two jobs of length 3 off machine 1 brings its load to 8 but raises the receiving machine to 11. The local search stops at 11, while the optimum is 9: to get there a *swap* between two machines is needed, not a simple move.

Local search returns a feasible solution, hence a bound on the pessimistic side, and nothing else. It proves nothing about the optimum, produces no bound on the optimistic side, and the fact that it stopped does not mean it has arrived. The only way to know how good it is, is to compare it with a bound from the other side.

5.7 When the constructive heuristic fails

Three jobs of duration (3, 3, 2) on two machines with availability (5, 3). Next-fit: job 1 goes to machine 1 (residual 2); job 2 does not fit and moves to machine 2 (residual 0); job 3 does not fit and there are no more machines: **failure**. But the problem is feasible: jobs 2 and 3 fit together on machine 1 ($3 + 2 = 5$) and job 1 on machine 2 ($3 \leq 3$).

A constructive heuristic is *myopic*: it decides one thing at a time and never backtracks. Its failure is information about the heuristic, not about the problem. To prove that a model is infeasible one needs the solver (**Status** = **INFEASIBLE**) or a proof, not a constructive heuristic that went wrong.

5.8 The overview of the heuristics

Heuristic	Direction	value	$z(MILP)$	gap
next-fit / first-fit (assignment)	min (UB)	14	11	27.3%
best-fit on cost (assignment)	min (UB)	11	11	0.0%
LPT (makespan)	min (UB)	11	9	22.2%
covering constructive heuristic	min (UB)	10	10	0.0%
ratio constructive heuristic (knapsack)	max (LB)	16	17	5.9%
least unit cost (lot sizing)	min (UB)	200	170	17.6%

What this table teaches

Two heuristics find the optimum and four do not, and *before* solving the MILP there is no way of knowing which. A 0% gap and a 27% gap are told apart only *afterwards*. This is exactly why the course always asks for two bounds: a heuristic on its own says how much

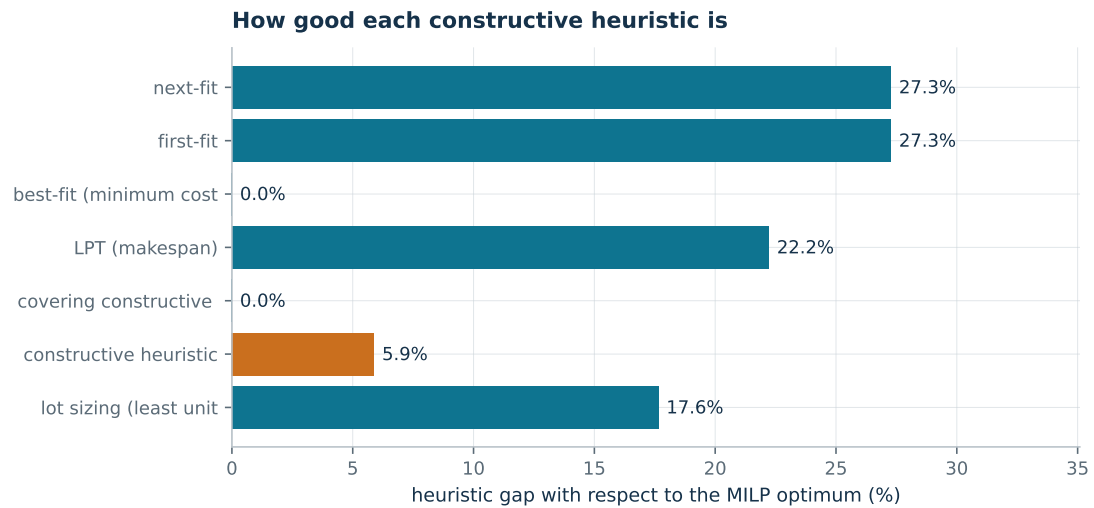


Figure 5.1: The gap of each heuristic with respect to the MILP optimum, on the minimal instances of this chapter. Teal for minimisation problems, orange for maximisation.

a solution one can actually implement costs, not how much is being lost.

From the model to Python/Gurobi

Script: `python/cap06_gurobi.py`

[Online page](#)

The course uses **one solver only**, Gurobi from Python. This chapter shows how a model is written — one family of constraints per block, with the names of the mathematical model — and above all how the results are *read*, including the case where the solver has not finished.

6.1 The four classes of variables

What	vtype	Domain	Typical use
yes/no decision	GRB.BINARY	$\{0,1\}$	selection, activation, assignment
count	GRB.INTEGER	$\mathbb{Z}$ between <code>lb</code> and <code>ub</code>	boxes, shifts, workers
measurable quantity	(default)	$[lb, ub] \subseteq \mathbb{R}$	time, money, flow
free variable	(default) with <code>lb=-GRB.INFINITY</code>	$\mathbb{R}$	duals of equalities

GRB.BINARY already implies `lb = 0` and `ub = 1`: there is no need to repeat them. A continuous variable has `lb = 0` by default: a variable that must be allowed to go negative — typically the dual of an equality, or a signed deviation — must be declared with `lb=-GRB.INFINITY`, otherwise the model is silently wrong.

6.2 The model, one family per block

```

1 def model(t, c, a):
2     """Problem 7.1: one addConstrs per family, with the label as its
      name."""
3     m = gp.Model("assignment"); m.Params.OutputFlag = 0
4     x = m.addVars(n, k, vtype=GRB.BINARY, name="x")           # data ->
      variables
5     m.setObjective(gp.quicksum(c[j][h] * x[j, h] for j in range(n)
6                             for h in range(k)), GRB.MINIMIZE)
7     m.addConstrs((x.sum(j, "*") == 1 for j in range(n)), name="assign")

```

```

8     m.addConstrs((gp.quicksum(t[j][h] * x[j, h] for j in range(n)) <= a
9         [h]
10         for h in range(k)), name="availability")
    return m, x

```

The three writing rules of the course

1. **One addConstrs per family**, in the order of the mathematical model, and the **name** is the label of the family ("assign", "availability"). The generated names are assign[0], assign[1], ...: they reappear in the duals and in the solver messages.
2. **Variables are declared all at once**, with addVars and the indices of the model. `x.sum(j, "*")` is the sum over all the indices of the second dimension: it is the writing of the model's *dummy index*.
3. **Data come in as arguments**, not as globals: the same function serves the base instance and all the variants of the modelling questions.

The model as written has 9 variables, 6 constraints and 18 nonzero coefficients. To check that it is what was written on paper, `m.write("model.lp")` produces the readable LP format:

```

Minimize
  5 x[0,0] + 10 x[0,1] + 2 x[0,2] + 5 x[1,0] + 4 x[1,1] + 6 x[1,2]
  + 5 x[2,0] + 4 x[2,1] + 6 x[2,2]
Subject To
  assign[0]: x[0,0] + x[0,1] + x[0,2] = 1
  ...

```

It is the quickest way to spot a wrong coefficient: the instance table in the notes and this output must agree line by line.

6.3 Reading the results

The reading order, which never changes

1. **Status**. Always first.
2. **SolCount**: if it is 0, *no* solution exists and **ObjVal** cannot be read.
3. **ObjVal** and **ObjBound**: the best solution found and the best bound proved.
4. **MIPGap**, **NodeCount**, **Runtime**: how much remains uncertain, and how much it cost.

Status	value	SolCount	What can be said
OPTIMAL	2	≥ 1	$z(MILP) = \text{ObjVal}$, proved
INFEASIBLE	3	0	the model has no feasible solution
UNBOUNDED	5	0	a constraint, or a bound on a variable, is missing
TIME_LIMIT	9	0	nothing: neither a solution nor, in general, a useful bound
TIME_LIMIT	9	≥ 1	$\text{ObjBound} \leq z(MILP) \leq \text{ObjVal}$
SOLUTION_LIMIT	10	≥ 1	as above

The four cases on the instance of problem 7.1

- **Normal solve.** Status = 2, SolCount = 2, ObjVal = ObjBound = 11, MIPGap = 0, NodeCount = 0: Gurobi closed everything at the root.
- **Infeasible.** With availability $a = (1, 1, 1)$ no assignment is possible: Status = 3, SolCount = 0.
- **Stopped immediately.** With TimeLimit = 0: Status = 9, SolCount = 0 and ObjBound = $-\infty$. There is nothing to report: neither a solution nor a bound.
- **Stopped at the first solution.** With SolutionLimit = 1: Status = 10, SolCount = 1, ObjVal = 12, ObjBound = 10, MIPGap = 0.1667. This is the case where an *interval* is reported: the optimum lies between 10 and 12, and the 16.7% measures its width. Saying "the optimum is 12" would be false.

One does not write "the optimum is ObjVal" if Status is not OPTIMAL. One does not write a gap if SolCount is 0. One does not compare the Runtime of two models solved with different settings. And one does not read ObjBound at the end of the solve thinking it is the root relaxation: for that, `relax()` is needed, as in Section 4.9.

6.4 Tolerances

Parameter	Default	Meaning
IntFeasTol	10^{-5}	how far an integer variable may be from an integer
FeasibilityTol	10^{-6}	violation allowed on a linear constraint
OptimalityTol	10^{-6}	tolerance on the reduced costs
MIPGap	10^{-4}	relative gap below which the solver stops

A binary may come back as 0.9999999997. In the text one writes 1: values are rounded *when they are reported*, and comparisons always use a tolerance — in this course 10^{-6} , the constant TOL of `python/mip.py`. Writing `if x.X == 1` is a mistake; one writes `if x.X > 0.5`.

The default MIPGap = 10^{-4} also means the solver may stop *before* the exact optimum while

declaring OPTIMAL: on instances with large values it is worth lowering it.

6.5 The relaxation with relax()

```

1 m.update()                # relax() copies the model: pending changes must
    be applied first
2 r = m.relax()             # binaries become 0 <= x <= 1, integers x >= lb
3 r.Params.OutputFlag = 0
4 r.optimize()
5 zlp = r.ObjVal
6 duals = {c.ConstrName: c.Pi for c in r.getConstrs()}

```

On the instance of problem 7.1, $z(LP^+) = z(LP) = 53/5$ — the two relaxations coincide, because the assignment constraints already imply $x_{jm} \leq 1$ — and the nonzero duals are

$$\tilde{\mu} = (2, 4.8, 5), \quad \tilde{\pi}_2 = -0.2,$$

that is, the three dual variables of the `assign` constraints and the one of `availability[1]`: machine 2 is the only tight resource.

The relaxation without the bounds, when it is needed

The function `rilassamento(m, rafforzato=False)` of `python/mip.py` also drops the upper bound of the binaries: it is the relaxation whose dual the notes write by hand. Both versions are reported, with a note on whether they coincide and why.

6.6 The course protocol, from start to finish

The sequence of every Part II exercise

data → model → heuristic and check → LP and dual
→ MIP → table → figures and notebook

```

1 m, x = model(t, c, a)                # (1) data,
    (2) model
2
3 e = best_fit(t, a, lambda j, h, ra: c[j][h], "cost")    # (3)
    heuristic
4 ub = sum(c[j][h] for (j, h) in e.x)
5 assert ammissibile(m, {f"x[{j},{h}]" : 1 for (j, h) in e.x})    #
    constraints, bounds AND integrality
6
7 d = dual(t, c, a)                    # (4) the dual
    written by hand
8 by_hand = {f"mu[{j}]" : min(c[j]) for j in range(n)}
9 lb, viol = valuta(d, by_hand); assert viol <= 1e-9
10 zlp, zlp_str, pi = due_rilassamenti(m, d)                # checks
    strong duality
11

```

```

12 z = risolti(m) # (5) the MIP
13 row = registra_bound("7.1 assignment", ub, lb, zlp, zlp_str, z) # (6)
    the table
14 salva_dati(pd.DataFrame([row]), "fam07_1_bound") # -> data
    /fam07_1_bound.csv

```

On the instance of problem 7.1 the protocol produces

$$LB = 10, \quad z(LP) = z(LP^+) = \frac{53}{5}, \quad z(MILP) = 11, \quad UB = 11,$$

and the row ends up in `data/sched1_bound.csv`. That is where the notes, the website page and `check_numbers.py` read it from: **one single place where the number exists**.

The four numbers every Part II exercise produces

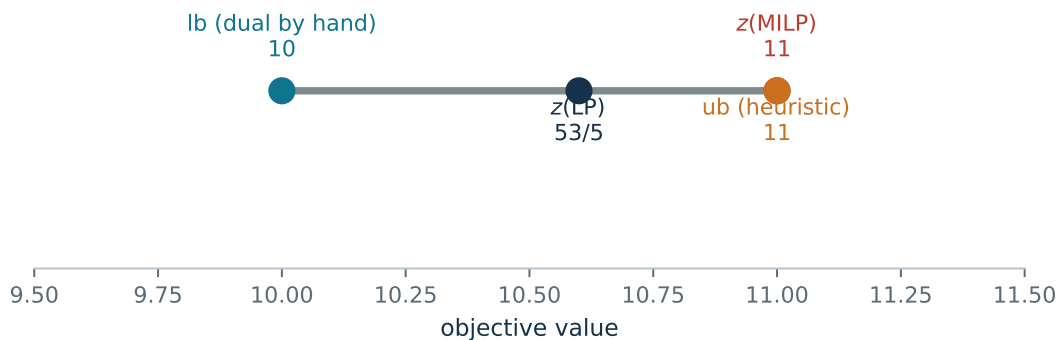


Figure 6.1: The four numbers every Part II exercise produces, on the instance of problem 7.1.

6.7 Running the code

Locally

From the `python/` folder:

```

python3 -m venv .venv && source .venv/bin/activate
pip install gurobipy pandas matplotlib
python3 fam07_1_assignment.py # one chapter
python3 run_all.py           # all of them, plus the notebooks
python3 check_numbers.py     # the asserts on every quoted number

```

On Colab

Every chapter has a notebook that opens from the badge on the website page. The first cell installs `gurobipy` and downloads the three shared modules (`stile.py`, `mip.py`, `euristiche.py`) if they are not already there: nothing needs to be installed on the computer.

The *size-limited* licence of the pip package allows models of up to **2000 variables and 2000 constraints**. The instances of this course are tiny — the largest model of Part II has a few dozen variables — and all fit with an enormous margin. For larger instances the free academic licence is activated from portal.gurobi.com. If a model exceeds the limit, Gurobi reports it with an explicit error at `optimize()`: it does not silently produce a wrong result.

Part II

The problems

Fifteen numerical models

This chapter collects the fifteen numerical models of the course, from NUM 1 to NUM 15. They are the easier examples: explicit data, few variables, one technique at a time. They come before the three families of problems for exactly that reason — one reads them to get the measure of things, and then tackles the general problems of chapters 7–10.

The format is reduced but always keeps the same five pieces:

- the statement, with the data of the instance;
- the **symbolic model**, with its variables and its constraints;
- the **model of the instance**, primal and dual, in the violet box;
- a feasible solution built by hand, which gives the primal bound;
- a dual solution built by hand, which gives the dual bound, and the comparison with the solver optimum.

Every model has its own script and its own notebook, one per model, listed at the beginning of each section.

EX 1 — The eight-seat van

Script: `python/ex01_van.py`

[Notebook in Colab](#)

EX 1 — which groups of tourists to accept

A tour operator rents an eight-seat van to guide tourists around a scenic location. On a given day four groups show up, of 2, 3, 4 and 5 people. Since the van cannot take all of them at once, the operator asks each group (whose members do not want to be separated) to make an offer: the four groups offer 30, 50, 80 and 70 euros. For operational reasons at most two groups can be accepted; moreover, if group 2 is accepted then group 4 must be accepted as well. The goal is to maximise the total earnings.

The model.

With $x_j = 1$ if group j is accepted:

EX 1 on the instance — primal

$$\begin{aligned}
& \max 30x_1 + 50x_2 + 80x_3 + 70x_4 \\
& \text{subject to } 2x_1 + 3x_2 + 4x_3 + 5x_4 \leq 8 \\
& \quad x_1 + x_2 + x_3 + x_4 \leq 2 \\
& \quad \quad x_2 - x_4 \leq 0 \\
& \quad x_1, x_2, x_3, x_4 \in \{0, 1\}
\end{aligned}$$

Description. Three techniques of chapter 3 in four lines: the capacity (section 3.1), the count (3.4) and the logical precedence (3.9). The third constraint reads $x_2 \leq x_4$: if $x_2 = 1$ then $x_4 = 1$. The other direction is false and must not hold: group 4 can be accepted without group 2.

Heuristic: the primal bound.

Constructive heuristic on the decreasing offer, accepting a group only if all the constraints stay satisfied (the implication included).

- Group 3 (offer 80): accepted, 4 seats, 1 group.
- Group 4 (offer 70): rejected, 9 seats would be needed out of 8.
- Group 2 (offer 50): rejected, it would force accepting group 4.
- Group 1 (offer 30): accepted, 6 seats, 2 groups.

$$z(MILP) \geq LB = 110.$$

Dual: the dual bound.

With $\alpha \geq 0$ on the seats, $\beta \geq 0$ on the number of groups and $\gamma \geq 0$ on the implication, the dual is

$$\min 8\alpha + 2\beta \quad \text{s.t.} \quad a_j \alpha + \beta (+\gamma \text{ if } j = 2, -\gamma \text{ if } j = 4) \geq p_j.$$

The recipe: $\beta = \gamma = 0$ and only the seat is priced, at the price no group can beat,

$$\bar{\alpha} = \max_j \frac{p_j}{a_j} = \max\left(\frac{30}{2}, \frac{50}{3}, \frac{80}{4}, \frac{70}{5}\right) = 20, \quad UB = 8 \cdot 20 = 160.$$

EX 1 on the instance — dual of the LP relaxation

$$\begin{aligned}
& \min 8\alpha + 2\beta \\
& \text{subject to } 2\alpha + \beta \geq 30 \\
& \quad 3\alpha + \beta + \gamma \geq 50 \\
& \quad 4\alpha + \beta \geq 80 \\
& \quad 5\alpha + \beta - \gamma \geq 70 \\
& \quad \alpha \geq 0 \\
& \quad \beta \geq 0 \\
& \quad \gamma \geq 0
\end{aligned}$$

Results.

	LB	UB	$z(LP)$	$z(LP^+)$	$z(MILP)$
EX 1 van (max)	110	160	160	140	120

The optimum accepts groups 2 and 4, filling exactly the eight seats, for earnings of 120. The hand-built dual bound is *optimal* for the relaxation without the bounds (160); the strengthened one, which adds $x_j \leq 1$, drops to 140 and gets closer to the optimum. Removing the implication the optimum would rise to 130 (groups 2 and 3): the implication costs 10 euros. Removing the limit of two groups changes nothing, because it is the seat capacity that bites.

EX 2 — Bus lines

Script: `python/ex02_buslines.py`[Notebook in Colab](#)

EX 2 — assignment with a capacity in number of lines

A city council is evaluating the offers of three bus companies for four bus lines. Each company has stated the cost at which it would operate each line:

Company	Line 1	Line 2	Line 3	Line 4
1	10	4	9	7
2	1	2	3	10
3	8	9	10	1

Every line must be assigned to exactly one company and every company may operate at most two lines. The total cost is to be minimised.

The model.

With $x_{ij} = 1$ if company i operates line j ($3 \cdot 4 = 12$ binary variables):

EX 2

$$\begin{aligned}
 \min \quad & \sum_{i=1}^n \sum_{j=1}^m c_{ij} x_{ij} \\
 \text{subject to} \quad & \sum_{i=1}^n x_{ij} = 1, & \forall j \in \{1, 2, \dots, m\}, \\
 & \sum_{j=1}^m x_{ij} \leq p, & \forall i \in \{1, 2, \dots, n\}, \\
 & x_{ij} \in \{0, 1\}, & \forall i \in \{1, 2, \dots, n\}, \forall j \in \{1, 2, \dots, m\}.
 \end{aligned}$$

Description. With $n = 3$ companies, $m = 4$ lines and $p = 2$: four *set partitioning* constraints (one per line) and three capacity constraints (one per company). It is problem 7.1 with the capacity counted in *number* of jobs instead of time.

Heuristic.

Constructive heuristic on the lines, in order: every line goes to the cheapest company among those not yet full.

- **Line 1:** costs 10, 1, 8; the cheapest is company 2.
- **Line 2:** costs 4, 2, 9; company 2 again, which is now full.
- **Line 3:** companies 1 and 3 remain, costs 9 and 10: company 1.
- **Line 4:** companies 1 and 3 remain, costs 7 and 1: company 3.

Value $1 + 2 + 9 + 1 = 13$, so $UB = 13$.

Dual and lower bound.

Relaxing $x_{ij} \in \{0, 1\}$ to $x_{ij} \geq 0$, with α_j free for every line constraint and $\pi_i \leq 0$ for every capacity constraint:

Dual of the LP relaxation of EX 2

$$\begin{aligned}
 \max \quad & \sum_{j=1}^m \alpha_j + p \sum_{i=1}^n \pi_i \\
 \text{subject to} \quad & \alpha_j + \pi_i \leq c_{ij}, & \forall i \in \{1, 2, \dots, n\}, \forall j \in \{1, 2, \dots, m\}, \\
 & \alpha_j \geq 0, & \forall j \in \{1, 2, \dots, m\}, \\
 & \pi_i \leq 0, & \forall i \in \{1, 2, \dots, n\}.
 \end{aligned}$$

Description. The variable α_j is the value of line j , which has to be assigned anyway, and π_i the (non-positive) price of one of the p lines company i may run. The objective prices the lines and the available slots. The dual constraints are the columns of the x_{ij} : giving line j to company i satisfies its assignment constraint and takes one slot, and the balance cannot exceed the cost c_{ij} of that pairing.

With the "zero out and saturate" recipe, $\bar{\pi} = 0$ and $\bar{\alpha}_j = \min_i c_{ij} = (1, 2, 3, 1)$, feasible by construction: $LB = 7$. Meaning: "every line costs at least the lowest offer it received".

EX 2 on the instance — dual of the LP relaxation

$$\begin{array}{llll}
\max & \alpha_1 + \alpha_2 + \alpha_3 + \alpha_4 + 2\beta_1 + 2\beta_2 + 2\beta_3 \\
\text{subject to} & \alpha_1 & +\beta_1 & \leq 10 \\
& \alpha_2 & +\beta_1 & \leq 4 \\
& \alpha_3 & +\beta_1 & \leq 9 \\
& \alpha_4 & +\beta_1 & \leq 7 \\
& \alpha_1 & & +\beta_2 \leq 1 \\
& \alpha_2 & & +\beta_2 \leq 2 \\
& \alpha_3 & & +\beta_2 \leq 3 \\
& \alpha_4 & & +\beta_2 \leq 10 \\
& \alpha_1 & & +\beta_3 \leq 8 \\
& \alpha_2 & & +\beta_3 \leq 9 \\
& \alpha_3 & & +\beta_3 \leq 10 \\
& \alpha_4 & & +\beta_3 \leq 1 \\
& \alpha_1, \alpha_2, \alpha_3, \alpha_4 & & \geq 0 \\
& \beta_1, \beta_2, \beta_3 & & \geq 0
\end{array}$$

The bounds.

The optimum is $z(MILP) = 9$ (lines 1 and 3 to company 2, line 2 to company 1, line 4 to company 3) and both relaxations coincide with it.

UB (constructive heuristic)	LB (dual by hand)	$z(LP)$	$z(MILP)$	heuristic gap
13	7	9	9	44.4%

Worth noting: $z(LP) = z(MILP) = 9$, yet the hand-built bound stops at 7 and the heuristic is off by 44%. The relaxation is exact, the dual recipe is not: they are two different things.

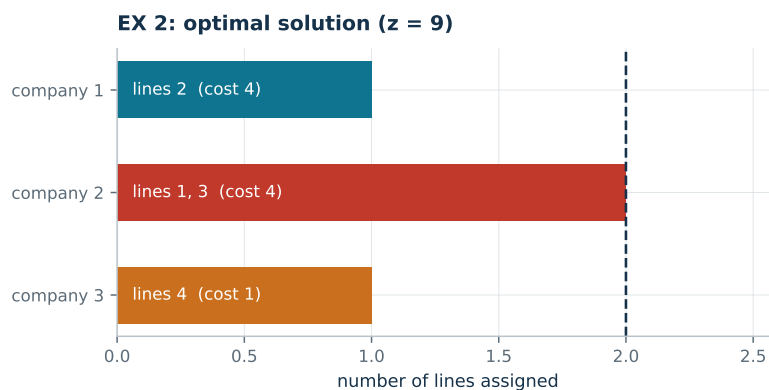


Figure 6.2: EX 2: the optimal solution. No company exceeds two lines.

EX 3 — Relay

Script: `python/ex03_relay.py`

[Notebook in Colab](#)

EX 3 — assignment with more resources than tasks

A coach must form a relay team by choosing three swimmers out of four. The race has three 50-metre legs: backstroke, breaststroke and butterfly. The recorded times (seconds) are:

Stroke	Carlo	Paolo	Fabio	Marco
Backstroke	37	32	33	37
Breaststroke	43	33	42	34
Butterfly	33	28	38	30

The total relay time is to be minimised.

The model.

With $x_{sn} = 1$ if stroke s is swum by n : $\sum_n x_{sn} = 1$ for every stroke (3 constraints), $\sum_s x_{sn} \leq 1$ for every swimmer (4 constraints), objective $\min \sum_{s,n} t_{sn} x_{sn}$. One swimmer stays out: it is an assignment with more resources than tasks.

EX 3 on the instance — primal

$$\begin{aligned}
 & \min 37x_{11} + 32x_{12} + 33x_{13} + 37x_{14} + 43x_{21} + 33x_{22} + 42x_{23} + 34x_{24} + 33x_{31} + 28x_{32} + 38x_{33} + 30x_{34} \\
 \text{subject to } & x_{11} + x_{12} + x_{13} + x_{14} = 1 \\
 & x_{21} + x_{22} + x_{23} + x_{24} = 1 \\
 & x_{31} + x_{32} + x_{33} + x_{34} = 1 \\
 & x_{11} + x_{21} + x_{31} \leq 1 \\
 & x_{12} + x_{22} + x_{32} \leq 1 \\
 & x_{13} + x_{23} + x_{33} \leq 1 \\
 & x_{14} + x_{24} + x_{34} \leq 1 \\
 & x_{11}, x_{12}, x_{13}, x_{14}, x_{21}, x_{22}, x_{23}, x_{24}, x_{31}, x_{32}, x_{33}, x_{34} \in \{0, 1\}
 \end{aligned}$$

Heuristic.

Constructive heuristic on the strokes in the given order: backstroke to Paolo (32), breaststroke to Marco (34, the fastest of the three left), butterfly to Carlo (33). Total $UB = 99$ seconds.

Dual and lower bound.

The same structure as EX 2 with $p = 1$: $\bar{\pi} = 0$ and $\bar{\alpha}_s = \min_n t_{sn} = (32, 33, 28)$, so $LB = 93$. Meaning: "every stroke costs at least the time of its best specialist". The bound is weak exactly when two strokes share the same best specialist — here Paolo is the fastest in both backstroke and butterfly, and indeed the bound is not attainable.

EX 3 on the instance — dual of the LP relaxation

$$\begin{aligned}
& \max \alpha_1 + \alpha_2 + \alpha_3 + \beta_1 + \beta_2 + \beta_3 + \beta_4 \\
& \text{subject to } \alpha_1 + \beta_1 \leq 37 \\
& \quad \alpha_1 + \beta_2 \leq 32 \\
& \quad \alpha_1 + \beta_3 \leq 33 \\
& \quad \alpha_1 + \beta_4 \leq 37 \\
& \quad \alpha_2 + \beta_1 \leq 43 \\
& \quad \alpha_2 + \beta_2 \leq 33 \\
& \quad \alpha_2 + \beta_3 \leq 42 \\
& \quad \alpha_2 + \beta_4 \leq 34 \\
& \quad \alpha_3 + \beta_1 \leq 33 \\
& \quad \alpha_3 + \beta_2 \leq 28 \\
& \quad \alpha_3 + \beta_3 \leq 38 \\
& \quad \alpha_3 + \beta_4 \leq 30 \\
& \alpha_1, \alpha_2, \alpha_3 \geq 0 \\
& \beta_1, \beta_2, \beta_3, \beta_4 \geq 0
\end{aligned}$$

The bounds.

The optimum is $z(MILP) = 95$ seconds: backstroke to Fabio (33), breaststroke to Marco (34), butterfly to Paolo (28); Carlo stays out.

UB (constructive heuristic)	LB (dual by hand)	$z(LP)$	$z(MILP)$	heuristic gap
99	93	95	95	4.2%

Why the relaxation is exact here

The matrix of the model is that of the assignment problem: totally unimodular. All the vertices of the relaxation are integer, so $z(LP) = z(MILP)$ and integrality costs nothing — the same phenomenon as alldiff in Section 3.12. This stops being true as soon as a capacity constraint with coefficients other than 0 and 1 is added.

EX 4 — Shoes: production, inventory and hirings

Script: `python/ex04_shoes.py`

[Notebook in Colab](#)

EX 4 — production and workforce over three months

Over the next three months a shoe factory must meet monthly demands of 3000, 5000 and 7000 pairs of shoes. At the start of the first month it has 500 pairs in stock and employs 100 workers. Every worker is paid 1500 euros a month and works 160 hours; producing one pair requires 4 labour hours and 15 euros of raw materials. At the start of every month workers may be hired, at a cost of 100 euros each for training. Storing one unsold pair at the end of the month costs 3 euros. The factory wants the production and workforce plan of minimum total cost.

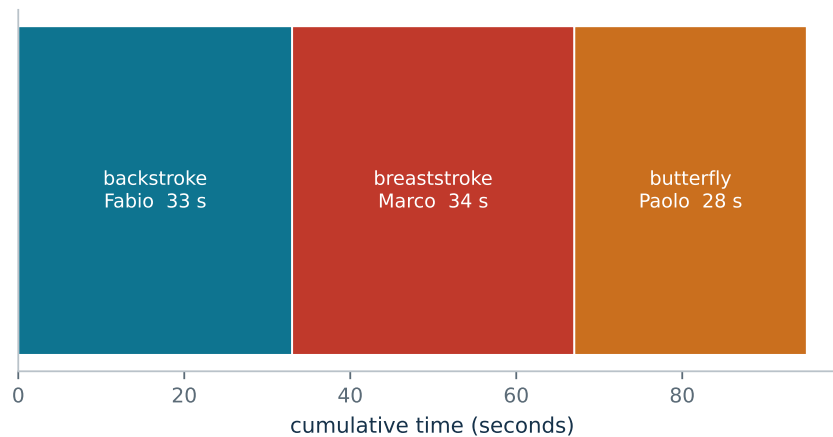
EX 3: the optimal relay (z = 95 s); Carlo stays out

Figure 6.3: EX 3: the optimal relay, leg by leg.

The model.

It is model 9.2A with one more datum, the initial inventory. With x_t pairs produced, s_t inventory at the end of the month, y_t workers on duty and z_t workers hired:

EX 4 on the instance

$$\begin{aligned}
 &\min 15x_1 + 15x_2 + 15x_3 + 3s_1 + 3s_2 + 1500y_1 + 1500y_2 + 1500y_3 + 100z_1 + 100z_2 + 100z_3 \\
 \text{subject to } &x_1 - s_1 = 2500 \\
 &x_2 + s_1 - s_2 = 5000 \\
 &x_3 + s_2 = 7000 \\
 &-4x_1 + 160y_1 \geq 0 \\
 &-4x_2 + 160y_2 \geq 0 \\
 &-4x_3 + 160y_3 \geq 0 \\
 &y_1 - z_1 = 100 \\
 &-y_1 + y_2 - z_2 = 0 \\
 &-y_2 + y_3 - z_3 = 0 \\
 &x_1, x_2, x_3 \geq 0 \\
 &s_1, s_2 \geq 0 \\
 &y_1, y_2, y_3 \in \mathbb{Z}_{\geq 0} \\
 &z_1, z_2, z_3 \in \mathbb{Z}_{\geq 0}
 \end{aligned}$$

Description. The demand of the first month is $3000 - 500 = 2500$ net pairs; 14 500 pairs must be produced in total. Here the workforce appears in two families of variables at once, y_t and z_t , tied by the staff balance: it is the mixed formulation one obtains by keeping both readings of problem 9.2.

Heuristic: the primal bound.

"Just in time" production: every month the net demand is produced, with no inventory, hiring as many workers as needed. Month 1: 10 000 hours, 63 workers, the 100 already on duty are

enough. Month 2: 20 000 hours, 125 workers, 25 are hired. Month 3: 28 000 hours, 175 workers, another 50 are hired. The cost is

$$z(MILP) \leq UB = 825\,000.$$

Dual: the dual bound.

With α_t free on the demand balances, $\beta_t \geq 0$ on the hours and γ_t free on the workforce balances, the recipe is: $\gamma = 0$, and one hour of work is worth what it really costs,

$$\bar{\beta}_t = \frac{w}{r} = \frac{1500}{160} = \frac{75}{8},$$

so that one pair is worth at most

$$\bar{\alpha}_t = 15 + 4 \cdot \frac{75}{8} = \frac{105}{2} = 52.5.$$

One checks that all the dual columns are satisfied, and

$$LB = \frac{105}{2} \cdot 14\,500 = 761\,250.$$

Dual of EX 4 on the instance

$$\begin{array}{llllll}
 \max & 2500\alpha_1 + 5000\alpha_2 + 7000\alpha_3 & & & +100\gamma_1 & \\
 \text{subject to} & \alpha_1 & & -4\beta_1 & & \leq 15 \\
 & & \alpha_2 & & -4\beta_2 & \leq 15 \\
 & & & \alpha_3 & & -4\beta_3 \leq 15 \\
 & -\alpha_1 & +\alpha_2 & & & \leq 3 \\
 & & -\alpha_2 & +\alpha_3 & & \leq 3 \\
 & & & 160\beta_1 & +\gamma_1 - \gamma_2 & \leq 1500 \\
 & & & & 160\beta_2 & +\gamma_2 - \gamma_3 \leq 1500 \\
 & & & & & 160\beta_3 & +\gamma_3 \leq 1500 \\
 & & & & & & -\gamma_1 \leq 100 \\
 & & & & & & -\gamma_2 \leq 100 \\
 & & & & & & -\gamma_3 \leq 100 \\
 & \alpha_1, & \alpha_2, & \alpha_3 & & & \geq 0 \\
 & & & & \beta_1, & \beta_2, & \beta_3 & \geq 0 \\
 & & & & & & & \gamma_1, \gamma_2, \gamma_3 \geq 0
 \end{array}$$

Results.

	UB	LB	$z(LP)$	$z(LP^+)$	$z(MILP)$
EX 4 shoes	825 000	761 250	773 500	773 500	774 180

The optimal plan produces 3980, 4000 and 6520 pairs with 100, 100 and 163 workers: 1480 pairs are kept in stock at the end of the first month and 480 at the end of the second, and 63 workers are hired once only, in the third month. Producing early costs 3 euros a pair and saves wages: raising the storage cost to 20 euros the inventories drop to 1000 and 0 (optimum 807 500), at 60 euros they disappear altogether and one falls back on the heuristic (825 000).

EX 5 — Vehicles with a minimum quantity

Script: `python/ex05_vehicles.py`

[Notebook in Colab](#)

EX 5 — five types of vehicle

A company is considering the production of five vehicle types: compact car, midsize car, large car, midsize minivan and large minivan. It has 1000 tons of steel and 2000 labour hours available. The table reports, for every type, the steel per unit, the hours per unit, the unit profit and the minimum quantity to be produced if the type is selected:

Type	steel	hours	profit	minimum
compact car	2	30	200	10
midsize car	3	25	250	10
large car	5	40	300	10
midsize minivan	6	45	550	5
large minivan	8	55	700	5

The company wants the plan of maximum profit.

The model.

It is model 9.3 without the bonus: the same semicontinuous variables, with $M_j = \min_i \lfloor b_i / a_{ij} \rfloor$. On the instance $M = (66, 80, 50, 44, 36)$, and in every case the limiting resource is the labour time.

EX 5 on the instance — primal

$$\begin{array}{llllll}
 \max & 200x_1 + 250x_2 + 300x_3 + 550x_4 + 700x_5 \\
 \text{subject to} & 2x_1 + 3x_2 + 5x_3 + 6x_4 + 8x_5 & & & & \leq 1000 \\
 & 30x_1 + 25x_2 + 40x_3 + 45x_4 + 55x_5 & & & & \leq 2000 \\
 & x_1 & & & -10y_1 & \geq 0 \\
 & & x_2 & & -10y_2 & \geq 0 \\
 & & & x_3 & & -10y_3 \geq 0 \\
 & & & & x_4 & -5y_4 \geq 0 \\
 & & & & & x_5 - 5y_5 \geq 0 \\
 & x_1 & & & -66y_1 & \leq 0 \\
 & & x_2 & & -80y_2 & \leq 0 \\
 & & & x_3 & & -50y_3 \leq 0 \\
 & & & & x_4 & -44y_4 \leq 0 \\
 & & & & & x_5 - 36y_5 \leq 0 \\
 & x_1, & x_2, & x_3, & x_4, & x_5 & \in \mathbb{Z}_{\geq 0} \\
 & & & & y_1, & y_2, & y_3, & y_4, & y_5 & \in \{0, 1\}
 \end{array}$$

Heuristic: the primal bound.

Constructive heuristic on the profit per labour hour: 20/3, 10, 15/2, 110/9 and 140/11 for the five types. One starts from the large minivan (140/11), produces 36 of them (the maximum

allowed by the hours), and is left with 712 tons of steel but only 20 hours: no other type reaches its own minimum quantity and the constructive heuristic stops.

$$z(MILP) \geq LB = 25\,200.$$

Dual: the dual bound.

With $\ell_j = \beta_j = 0$ (the minimum lot is not priced) and a single resource priced at the highest unit price among the types:

$$\begin{aligned}\bar{\pi}_{\text{steel}} &= \max_j \frac{\bar{p}_j}{a_{1j}} = 100 \Rightarrow 1000 \cdot 100 = 100\,000, \\ \bar{\pi}_{\text{hours}} &= \max_j \frac{\bar{p}_j}{a_{2j}} = \frac{140}{11} \Rightarrow 2000 \cdot \frac{140}{11} = \frac{280\,000}{11} \approx 25\,454.5.\end{aligned}$$

The better bound comes from the hours:

$$z(MILP) \leq UB = \frac{280\,000}{11}.$$

EX 5 on the instance — dual

min1000π ₁ +2000π ₂												
subject to	2π ₁	+30π ₂	+λ ₁				+μ ₁				≥ 200	
	3π ₁	+25π ₂	+λ ₂				+μ ₂				≥ 250	
	5π ₁	+40π ₂	+λ ₃				+μ ₃				≥ 300	
	6π ₁	+45π ₂	+λ ₄				+μ ₄				≥ 550	
	8π ₁	+55π ₂	+λ ₅				+μ ₅				≥ 700	
			-10λ ₁				-66μ ₁				≥ 0	
			-10λ ₂				-80μ ₂				≥ 0	
			-10λ ₃				-50μ ₃				≥ 0	
			-5λ ₄				-44μ ₄				≥ 0	
			-5λ ₅				-36μ ₅				≥ 0	
	π ₁ ,	π ₂									≥ 0	
			λ ₁ ,	λ ₂ ,	λ ₃ ,	λ ₄ ,	λ ₅					≥ 0
								μ ₁ ,	μ ₂ ,	μ ₃ ,	μ ₄ ,	μ ₅ ≥ 0

Results.

	LB	UB	$z(LP)$	$z(LP^+)$	$z(MILP)$
EX 5 vehicles (max)	25 200	280000/11	280000/11	229000/9	25 250

The optimum produces 9 midsize minivans and 29 large minivans, using all 2000 hours and 286 tons of steel out of 1000. The dual bound is *optimal* for the relaxation without the bounds, and the heuristic is off by a mere 0.2%. Without minimum quantities the optimum would be 25 300: the minimum lot costs 50 euros of profit. Doubling the minimum quantities one drops to 25 200, and doubling the labour hours one reaches 50 750 with the hours still saturated.

EX 6 — Hub-and-spoke

Script: `python/ex06_hub.py`

[Notebook in Colab](#)

EX 6 — the minimum number of hubs

An airline organises its network with a *hub-and-spoke* system: a hub serves directly every city within 1000 miles. The company operates on eight cities; for each of them, the cities within 1000 miles are:

City	Cities within 1000 miles
Atlanta	Atlanta, Chicago, Houston, New York
Chicago	Atlanta, Chicago, New York
Denver	Denver, Los Angeles
Houston	Atlanta, Houston
Los Angeles	Denver, Los Angeles, San Francisco
New York	Atlanta, Chicago, New York
San Francisco	Los Angeles, San Francisco, Seattle
Seattle	San Francisco, Seattle

The minimum number of hubs is wanted, such that every city is within 1000 miles of at least one hub.

The model.

The data are $n = 8$ cities and $m = 8$ candidate hubs. With $y_j = 1$ if city j becomes a hub (m binaries) and S_i the set of cities covering city i :

EX 6

$$\begin{aligned}
 & \min \sum_{j=1}^m y_j \\
 & \text{subject to} \quad \sum_{j \in S_i} y_j \geq 1, & \forall i \in \{1, 2, \dots, n\}, \\
 & y_j \in \{0, 1\}, & \forall j \in \{1, 2, \dots, m\}.
 \end{aligned}$$

Description. A pure *set covering*: eight covering constraints, all costs equal to 1. Note that "within 1000 miles" is a symmetric relation, so $j \in S_i \iff i \in S_j$: the matrix of the model is symmetric.

EX 6 on the instance — primal

$$\begin{aligned}
& \min y_1 + y_2 + y_3 + y_4 + y_5 + y_6 + y_7 + y_8 \\
& \text{subject to } y_1 + y_2 + y_4 + y_6 \geq 1 \\
& \quad y_1 + y_2 + y_6 \geq 1 \\
& \quad y_3 + y_5 \geq 1 \\
& \quad y_1 + y_4 \geq 1 \\
& \quad y_3 + y_5 + y_7 \geq 1 \\
& \quad y_1 + y_2 + y_6 \geq 1 \\
& \quad y_5 + y_7 + y_8 \geq 1 \\
& \quad y_7 + y_8 \geq 1 \\
& \quad y_1, y_2, y_3, y_4, y_5, y_6, y_7, y_8 \in \{0, 1\}
\end{aligned}$$

Heuristic.

Constructive covering heuristic (Chapter 5), with the criterion "cost per newly covered city".

- **Step 1.** Ratios 1/4 for Atlanta, 1/3 for Chicago, New York and San Francisco, 1/2 for the others: **Atlanta** is chosen, covering Atlanta, Chicago, Houston and New York.
- **Step 2.** Denver, Los Angeles, San Francisco and Seattle remain; ratios 1/2 for Denver, 1/3 for Los Angeles and San Francisco, 1/2 for Seattle: **Los Angeles** is chosen, covering Denver, Los Angeles and San Francisco.
- **Step 3.** Seattle remains; ratio 1/1 for San Francisco and for Seattle: **San Francisco** is chosen.

Three hubs, so $UB = 3$.

Dual and lower bound.

Relaxing to $y_j \geq 0$, with $\pi_i \geq 0$ for every covering constraint ($\geq$ direction in a minimisation):

Dual of the LP relaxation of EX 6

$$\begin{aligned}
& \max \sum_{i=1}^n \pi_i \\
& \text{subject to } \sum_{i: j \in S_i} \pi_i \leq 1, & \forall j \in \{1, 2, \dots, m\}, \\
& \quad \pi_i \geq 0, & \forall i \in \{1, 2, \dots, n\}.
\end{aligned}$$

Description. The variable π_i is the price attached to city i . The objective adds up the prices of the eight cities. The constraints are the columns of the x_j , one per hub: the cities hub j covers, priced at those values, cannot be worth more than what it costs to open it, namely 1.

Dual constructive heuristic on the cities, in order: π_i is raised until the first opposing dual constraint becomes tight, and the residuals are updated.

- **Atlanta** is covered by four hubs, all with residual 1: $\bar{\pi}_1 = 1$, and the residuals of Atlanta, Chicago, Houston and New York drop to zero.
- **Chicago, Houston** and **New York** have all their hubs at residual zero: $\bar{\pi} = 0$.
- **Denver** is covered by Denver and Los Angeles, both with residual 1: $\bar{\pi}_3 = 1$, and their residuals drop to zero.
- **Los Angeles** and **San Francisco** have one hub at residual zero: $\bar{\pi} = 0$.
- **Seattle** is covered by San Francisco and Seattle, both with residual 1: $\bar{\pi}_8 = 1$.

Value $LB = 1 + 1 + 1 = 3$.

What this bound proves, in words

The three cities that received $\pi_i = 1$ — Atlanta, Denver, Seattle — are pairwise "far apart": no city can serve as a hub for two of them. Hence at least three hubs are needed. The dual has formalised an argument one could tell without writing it: that is the point of the constructive heuristic recipe.

EX 6 on the instance — dual of the LP relaxation

$$\begin{aligned}
 & \max \quad u_1 + u_2 + u_3 + u_4 + u_5 + u_6 + u_7 + u_8 \\
 & \text{subject to} \quad u_1 + u_2 \quad \quad + u_4 \quad \quad + u_6 \quad \leq 1 \\
 & \quad \quad \quad u_1 + u_2 \quad \quad \quad + u_6 \quad \leq 1 \\
 & \quad \quad \quad \quad u_3 \quad \quad + u_5 \quad \leq 1 \\
 & \quad \quad \quad u_1 \quad \quad + u_4 \quad \leq 1 \\
 & \quad \quad \quad \quad u_3 \quad + u_5 \quad + u_7 \leq 1 \\
 & \quad \quad \quad u_1 + u_2 \quad \quad \quad + u_6 \quad \leq 1 \\
 & \quad \quad \quad \quad \quad \quad u_5 \quad + u_7 + u_8 \leq 1 \\
 & \quad \quad \quad \quad \quad \quad \quad u_7 + u_8 \leq 1 \\
 & \quad \quad \quad u_1, u_2, u_3, u_4, u_5, u_6, u_7, u_8 \geq 0
 \end{aligned}$$

The bounds.

$z(MILP) = 3$, with hubs in Atlanta, Los Angeles and San Francisco — the same solution as the constructive heuristic.

UB (constructive heuristic)	LB (dual by hand)	$z(LP)$	$z(MILP)$	heuristic gap
3	3	3	3	0.0%

The two hand-built bounds coincide: optimality is proved without the solver.

EX 7 — Custom aircraft with a fixed set-up cost

Script: `python/ex07_aircraft.py`

[Notebook in Colab](#)

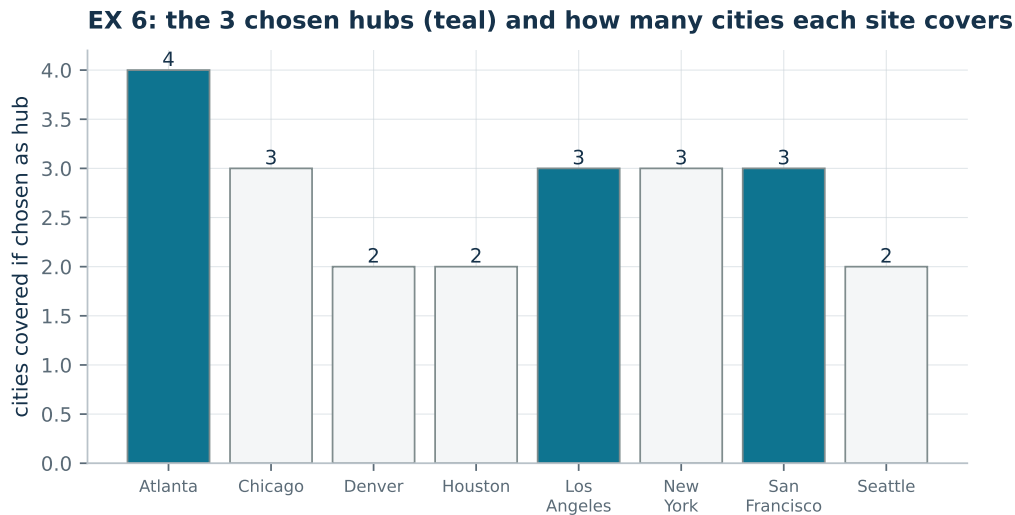


Figure 6.4: EX 6: for every city, how many it would cover as a hub; in teal the three chosen ones.

EX 7 — which orders to accept

A company builds custom-designed aircraft; before producing for a client a fixed set-up cost has to be paid. Three clients have ordered, respectively, 3, 2 and 5 aircraft. The production capacity is 1000 hours. The company decides which orders to accept and how many units to produce for each (possibly fewer than requested). Set-up costs, unit profits and hours per aircraft:

	client 1	client 2	client 3
set-up (millions)	3	2	0
unit profit (millions)	2	3	1
hours per aircraft	200	400	200

The goal is to maximise the total profit net of the set-up costs.

The model.

The data are $n = 3$ clients, with unit profit p_j , setup cost f_j , hours per aircraft h_j and maximum order M_j , plus the production capacity $H = 1000$ hours. With x_j aircraft produced for client j (integer) and y_j the indicator of an accepted order:

EX 7

$$\begin{aligned}
& \max \sum_{j=1}^n (p_j x_j - f_j y_j) \\
& \text{subject to } \sum_{j=1}^n h_j x_j \leq H, \\
& \quad x_j - M_j y_j \leq 0, \quad \forall j \in \{1, 2, \dots, n\}, \\
& \quad x_j \in \mathbb{Z}_{\geq 0}, \quad \forall j \in \{1, 2, \dots, n\}, \\
& \quad y_j \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, n\}.
\end{aligned}$$

EX 7 on the instance — primal

$$\begin{aligned}
& \max \quad 2x_1 + 3x_2 + x_3 - 3y_1 - 2y_2 \\
& \text{subject to } 200x_1 + 400x_2 + 200x_3 \leq 1000 \\
& \quad x_1 - 3y_1 \leq 0 \\
& \quad x_2 - 2y_2 \leq 0 \\
& \quad x_3 - 5y_3 \leq 0 \\
& \quad x_1, x_2, x_3 \in \mathbb{Z}_{\geq 0} \\
& \quad y_1, y_2, y_3 \in \{0, 1\}
\end{aligned}$$

Description. A single constraint per client does two things at once: it caps the quantity at the number of aircraft ordered and it charges the set-up. Client 3 has a zero set-up, so y_3 does not appear in the objective and equals 1 in every optimal solution with $x_3 > 0$.

Heuristic: the primal bound.

Constructive heuristic on the net profit per hour, computed on the whole order: $3/600$, $4/800$ and $5/1000$, that is $1/200$ for all three. With equal ratios one scans in the natural order: client 1 is accepted with 3 aircraft (600 hours, net profit 3), then client 2 with a single aircraft (400 hours, net profit $3 - 2 = 1$), and the hours run out.

$$z(MILP) \geq LB = 4.$$

Dual: the dual bound.

With $\alpha \geq 0$ on the capacity and $\beta_j \geq 0$ on the links:

$$\min 1000 \alpha \quad \text{s.t.} \quad h_j \alpha + \beta_j \geq p_j, \quad M_j \beta_j \leq f_j.$$

The recipe: $\bar{\beta}_j = f_j/M_j$, that is the set-up spread over the aircraft ordered (1, 1 and 0), whence $\alpha \geq (p_j - \bar{\beta}_j)/h_j$ for every j , that is $\alpha \geq 1/200$ in all three cases. With $\bar{\alpha} = 1/200$,

$$z(MILP) \leq UB = 1000 \cdot \frac{1}{200} = 5.$$

Results.

	LB	UB	$z(LP)$	$z(LP^+)$	$z(MILP)$
EX 7 aircraft (max)	4	5	5	5	5

The optimum is 5 and is attained by *three* different plans: five aircraft for client 3; two for client 2 and one for the third; three for client 1 and two for the third. All three use exactly the 1000 hours. The dual bound is exact, and in this case the certificate settles the problem on its own: there is nothing left for the solver to do. Without set-up costs the optimum would rise to 9; with 1400 hours available, to 7.

EX 8 — Seminars

Script: `python/ex08_seminars.py`[Notebook in Colab](#)

EX 8 — selection with exact cardinality and non-adjacency

A student must attend *exactly two* sessions of a summer school. There are three seminars, each offered in four time slots (9–10, 10–11, 11–12, 12–13); every seminar is attended at most once and two sessions cannot occupy the same slot. The preference score of each session is:

	9–10	10–11	11–12	12–13
Seminar 1	8	6	5	3
Seminar 2	7	9	4	6
Seminar 3	5	7	8	9

The student never wants *two consecutive hours* of class. The total preference is to be maximised.

The model.

The data are $n = 3$ seminars, $m = 4$ slots, $q = 2$ sessions to attend and the score p_{sk} . With $x_{sk} = 1$ if seminar s is attended in slot k ($nm = 12$ binaries):

EX 8

$$\begin{aligned}
& \max \sum_{s=1}^n \sum_{k=1}^m p_{sk} x_{sk} \\
& \text{subject to} \quad \sum_{s=1}^n x_{sk} \leq 1, & \forall k \in \{1, 2, \dots, m\}, \\
& \quad \sum_{k=1}^m x_{sk} \leq 1, & \forall s \in \{1, 2, \dots, n\}, \\
& \quad \sum_{s=1}^n \sum_{k=1}^m x_{sk} = q, \\
& \quad \sum_{s=1}^n (x_{sk} + x_{s,k+1}) \leq 1, & \forall k \in \{1, 2, \dots, m-1\}, \\
& \quad x_{sk} \in \{0, 1\}, & \forall s \in \{1, 2, \dots, n\}, \forall k \in \{1, 2, \dots, m\}.
\end{aligned}$$

EX 8 on the instance — primal

$$\begin{aligned}
& \max 8x_{11} + 6x_{12} + 5x_{13} + 3x_{14} + 7x_{21} + 9x_{22} + 4x_{23} + 6x_{24} + 5x_{31} + 7x_{32} + 8x_{33} + 9x_{34} \\
& \text{subject to} \quad x_{11} + x_{21} + x_{31} \leq 1 \\
& \quad x_{12} + x_{22} + x_{32} \leq 1 \\
& \quad x_{13} + x_{23} + x_{33} \leq 1 \\
& \quad x_{14} + x_{24} + x_{34} \leq 1 \\
& \quad x_{11} + x_{12} + x_{13} + x_{14} \leq 1 \\
& \quad x_{21} + x_{22} + x_{23} + x_{24} \leq 1 \\
& \quad x_{31} + x_{32} + x_{33} + x_{34} \leq 1 \\
& \quad x_{11} + x_{12} + x_{13} + x_{14} + x_{21} + x_{22} + x_{23} + x_{24} + x_{31} + x_{32} + x_{33} + x_{34} = 2 \\
& \quad x_{11} + x_{12} + x_{21} + x_{22} + x_{31} + x_{32} \leq 1 \\
& \quad x_{12} + x_{13} + x_{22} + x_{23} + x_{32} + x_{33} \leq 1 \\
& \quad x_{13} + x_{14} + x_{23} + x_{24} + x_{33} + x_{34} \leq 1 \\
& \quad x_{11}, x_{12}, x_{13}, x_{14}, x_{21}, x_{22}, x_{23}, x_{24}, x_{31}, x_{32}, x_{33}, x_{34} \in \{0, 1\}
\end{aligned}$$

Description. with $q = 2$: four slot constraints, three seminar constraints, one *exact* cardinality constraint and three non-adjacency constraints between consecutive slots.

Heuristic.

Constructive heuristic: take the feasible session with the highest score, then the best compatible one. Step 1: seminar 3 in slot 4 (score 9). Step 2: among the sessions that do not use seminar 3, do not occupy slot 4 and are not adjacent to it, the best is seminar 2 in slot 2 (score 9). In a **maximisation** the value is a *lower* bound: $LB = 18$.

Dual and upper bound.

With $\alpha_k \geq 0$ for the slots, $\beta_s \geq 0$ for the seminars, γ free for the exact cardinality and $\delta_k \geq 0$ for non-adjacency:

Dual of the LP relaxation of EX 8

$$\begin{aligned}
 \min \quad & \sum_{k=1}^m \alpha_k + \sum_{s=1}^n \beta_s + q\gamma + \sum_{k=1}^{m-1} \delta_k \\
 \text{subject to} \quad & \alpha_k + \beta_s + \gamma + \sum_{k': k \in \{k', k'+1\}} \delta_{k'} \geq p_{sk}, \quad \forall s \in \{1, 2, \dots, n\}, \forall k \in \{1, 2, \dots, m\}, \\
 & \alpha_k \geq 0, \quad \forall k \in \{1, 2, \dots, m\}, \\
 & \beta_s \geq 0, \quad \forall s \in \{1, 2, \dots, n\}, \\
 & \gamma \geq 0, \\
 & \delta_k \geq 0, \quad \forall k \in \{1, 2, \dots, m-1\}.
 \end{aligned}$$

Description. The variable α_k is the price of time slot k , β_s that of seminar s , γ the price of the cardinality constraint and δ_k that of the non-adjacency constraint between slots k and $k+1$. The objective adds up the right-hand sides of those constraints at those prices. The dual constraints are the columns of the x_{sk} : attending seminar s in slot k takes the slot, takes the seminar, consumes one unit of cardinality and touches the non-adjacency constraints containing k ; the total must cover the preference p_{sk} .

The shortest recipe: everything at zero except $\bar{\gamma} = \max_{s,k} p_{sk} = 9$. Every dual constraint becomes $9 \geq p_{sk}$, true by construction, and the value is $q\bar{\gamma} = 2 \cdot 9 = 18$. It reads: "two sessions are attended, and none is worth more than the best one".

Dual recipe	UB
everything at zero except $\gamma = \max_{s,k} p_{sk}$	18
$\gamma = 0, \alpha_k = \max_s p_{sk}$ ("every slot is worth at most its best")	34

The first is optimal for the relaxation; the second, apparently more natural, is almost twice as large. The recipe matters.

The bounds.

The optimum is $z(MILP) = 18$: seminar 2 in slot 2 and seminar 3 in slot 4 — with a break in between, as required.

LB (constructive heuristic)	$z(MILP)$	$z(LP)$	UB (dual by hand)	heuristic gap
18	18	18	18	0.0%

Here the two hand-built bounds *close* the problem: the heuristic is worth 18, the dual 18, so $z(MILP) = 18$ is proved without the solver.

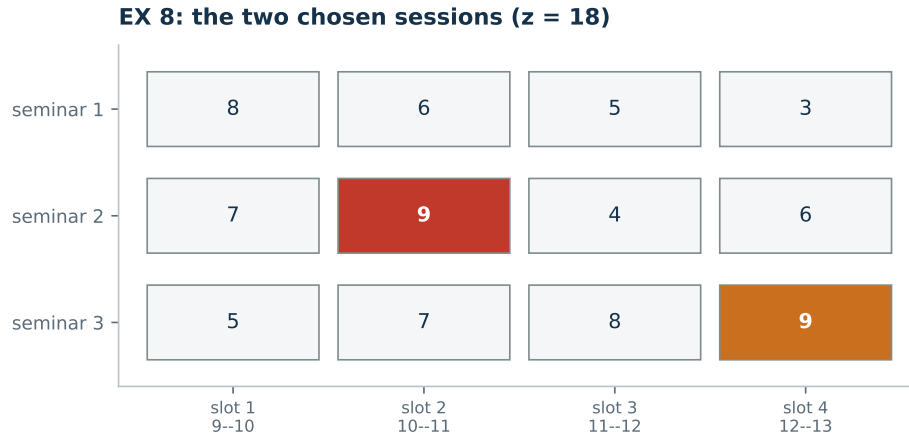


Figure 6.5: EX 8: the grid of preferences; in teal the two chosen sessions.

EX 9 — The eight queens

Script: `python/ex09_queens.py`

[Notebook in Colab](#)

EX 9 — the largest number of queens

On an 8×8 chessboard one wants to place the largest number of queens so that no queen can capture another in a single move. In chess a queen moves any number of squares along the same row, the same column or either diagonal.

The model.

The board is $n \times n$, with $n = 8$. With $x_{ij} = 1$ if there is a queen on square (i, j) ($n^2 = 64$ binaries):

EX 9

$$\begin{aligned}
 & \max \sum_{i=1}^n \sum_{j=1}^n x_{ij} \\
 & \text{subject to} \quad \sum_{j=1}^n x_{ij} \leq 1, & \forall i \in \{1, 2, \dots, n\}, \\
 & \quad \sum_{i=1}^n x_{ij} \leq 1, & \forall j \in \{1, 2, \dots, n\}, \\
 & \quad \sum_{i-j=k} x_{ij} \leq 1, & \forall k \in \{-(n-1), -(n-2), \dots, n-1\}, \\
 & \quad \sum_{i+j=k} x_{ij} \leq 1, & \forall k \in \{2, 3, \dots, 2n\}, \\
 & \quad x_{ij} \in \{0, 1\}, & \forall i \in \{1, 2, \dots, n\}, \forall j \in \{1, 2, \dots, n\}.
 \end{aligned}$$

Description. A set packing on four families of lines: 8 rows, 8 columns and 15+15 diagonals, 46 constraints in total. The indexing trick is all here: the "descending" diagonals are the sets with $i - j$ constant, the "ascending" ones the sets with $i + j$ constant.

Heuristic: the primal bound.

Constructive heuristic row by row: in row i one picks the first free column that is not attacked by the queens already placed; if there is none, the row stays empty. It never backtracks. On the instance the constructive heuristic places queens on $(1, 1)$, $(2, 3)$, $(3, 5)$, $(4, 2)$, $(5, 4)$ and then gets stuck: rows 6, 7 and 8 have no free squares left.

$$z(MILP) \geq LB = 5.$$

Dual: the dual bound.

The dual is a set covering:

$$\begin{aligned} \min \quad & \sum_{i=1}^8 \alpha_i + \sum_{j=1}^8 \beta_j + \sum_{k=-7}^7 \gamma_k + \sum_{k=2}^{16} \delta_k \\ \text{s.t.} \quad & \alpha_i + \beta_j + \gamma_{i-j} + \delta_{i+j} \geq 1, \quad \forall i \in \{1, 2, \dots, 8\}, \forall j \in \{1, 2, \dots, 8\}. \end{aligned}$$

For example one can use a single line: $\bar{\alpha}_i = 1$ for every row, everything else zero. Every square (i, j) has $\alpha_i = 1 \geq 1$, so the solution is feasible and is worth 8. It is the dual translation of the sentence "at most one queen sits in each row".

$$z(MILP) \leq UB = 8.$$

Results.

	LB	UB	$z(LP)$	$z(LP^+)$	$z(MILP)$
EX 9 queens (max)	5	8	8	8	8

The dual bound is attained: the solution found is optimal, and one knows it without trusting the solver. The constructive heuristic, on the other hand, stops at 5: it is the heuristic, not the bound, that leaves the 37.5% gap.

Two variants.

On $n \times n$ boards with $n = 4, 5, 6$ the optimum is always n , and the certificate $\bar{\alpha}_i = 1$ proves it. For $n = 2$ and $n = 3$, instead, the optimum is 1 and 2: the bound n stays valid but is not attainable, and it is the first case in which this dual recipe does not settle the problem. Removing the diagonal constraints one gets *rooks* instead of queens: the model becomes an assignment, the matrix is totally unimodular and the LP relaxation already gives an integer value.

EX 10 — Tools of a CNC machine

Script: `python/ex10_tools.py`

[Notebook in Colab](#)

EX 10 — selection with a tool set

A CNC machine can perform five operations; each one requires a subset of six tools and, if performed, yields a profit:

Operation	Required tools	Profit (€)
1	1, 3, 4, 5	2000
2	1, 2, 6	1500
3	1, 2, 3, 5	1800
4	2, 4, 5	1700
5	5, 6	800

The magazine of the machine holds at most $k = 4$ tools at a time. The configuration of maximum profit is wanted.

The model.

The data are $n = 5$ operations of profit p_i , $m = 6$ tools and a magazine holding $k = 4$ tools; T_i is the set of tools required by operation i . With $x_i = 1$ if operation i is performed (n binaries) and $y_j = 1$ if tool j is loaded (m binaries):

EX 10

$$\begin{aligned}
 & \max \sum_{i=1}^n p_i x_i \\
 & \text{subject to} \quad \sum_{j=1}^m y_j \leq k, \\
 & \quad x_i - y_j \leq 0, \quad \forall i \in \{1, 2, \dots, n\}, \forall j \in T_i, \\
 & \quad x_i \in \{0, 1\}, \quad \forall i \in \{1, 2, \dots, n\}, \\
 & \quad y_j \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, m\}.
 \end{aligned}$$

EX 10 on the instance — primal

$$\begin{array}{ll}
\max & 2000x_1 + 1500x_2 + 1800x_3 + 1700x_4 + 800x_5 \\
\text{subject to} & y_1 + y_2 + y_3 + y_4 + y_5 + y_6 \leq 4 \\
& x_1 - y_1 \leq 0 \\
& x_1 - y_3 \leq 0 \\
& x_1 - y_4 \leq 0 \\
& x_1 - y_5 \leq 0 \\
& x_2 - y_1 \leq 0 \\
& x_2 - y_2 \leq 0 \\
& x_2 - y_6 \leq 0 \\
& x_3 - y_1 \leq 0 \\
& x_3 - y_2 \leq 0 \\
& x_3 - y_3 \leq 0 \\
& x_3 - y_5 \leq 0 \\
& x_4 - y_2 \leq 0 \\
& x_4 - y_4 \leq 0 \\
& x_4 - y_5 \leq 0 \\
& x_5 - y_5 \leq 0 \\
& x_5 - y_6 \leq 0 \\
& x_1, x_2, x_3, x_4, x_5 \in \{0, 1\} \\
& y_1, y_2, y_3, y_4, y_5, y_6 \in \{0, 1\}
\end{array}$$

Description. One magazine constraint and $\sum_i |T_i| = 4 + 3 + 4 + 3 + 2 = 16$ link constraints. It is the *disaggregated* activation of Section 3.1 read the other way round: here the operation is the object and the tools are the resources, and the operation needs *all* of its tools — a conjunction in the consequent, which by the table of Chapter 2 costs one constraint per tool.

Heuristic.

Constructive heuristic by decreasing profit: the operations are scanned from the most profitable and the tool set of each is loaded, if the magazine allows it.

- **Operation 1** (2000, tools 1, 3, 4, 5): 4 new tools are needed and the magazine reaches exactly 4: it runs.
- **Operations 3, 4, 2, 5**: each would need at least one more tool, and the magazine is full: they are discarded.

In a **maximisation** the value is a *lower* bound: $LB = 2000$.

Dual and upper bound.

Relaxing to $x_i, y_j \geq 0$, with $\alpha \geq 0$ for the magazine constraint and $\beta_{ij} \geq 0$ for each link constraint:

Dual of the LP relaxation of EX 10

$$\begin{aligned}
& \min \quad k \alpha \\
& \text{subject to} \quad \sum_{j \in T_i} \beta_{ij} \geq p_i, & \forall i \in \{1, 2, \dots, n\}, \\
& \quad \sum_{i: j \in T_i} \beta_{ij} \leq \alpha, & \forall j \in \{1, 2, \dots, m\}, \\
& \quad \alpha \geq 0, \\
& \quad \beta_{ij} \geq 0, & \forall i \in \{1, 2, \dots, n\}, \forall j \in T_i.
\end{aligned}$$

Description. It reads like this: β_{ij} is the share of the profit of operation i attributed to tool j ; the first group asks the shares to cover the whole profit, the second that no tool receives more than α ; the objective pays α for each of the k slots in the magazine.

The recipe. The profit of each operation is spread in equal parts over the tools it needs, $\bar{\beta}_{ij} = p_i/|T_i|$: the first group is satisfied with equality by construction. Then $\bar{\alpha} = \max_j \sum_{i: j \in T_i} \bar{\beta}_{ij}$, the load of the most demanded tool.

$\bar{\beta}_{ij} = p_i/ T_i $	load of the tool					
	1	2	3	4	5	6
op. 1: $2000/4 = 500$	500		500	500	500	
op. 2: $1500/3 = 500$	500	500				500
op. 3: $1800/4 = 450$	450	450	450		450	
op. 4: $1700/3 = 1700/3$		1700/3		1700/3	1700/3	
op. 5: $800/2 = 400$					400	400
total	1450	4550/3	950	3200/3	5750/3	900

The critical tool is number 5, required by four operations out of five: hence $\bar{\alpha} = 5750/3$ and

$$UB = k \bar{\alpha} = 4 \cdot \frac{5750}{3} = \frac{23000}{3} \approx 7666.7.$$

A choice that comes naturally is $\bar{\alpha} = 2000$ with all $\bar{\beta}_{ij} = 900$: the constraints on the operations are satisfied ($2 \cdot 900 = 1800 \geq 800$ is the tightest case) and the value would be $4 \cdot 2000 = 8000$. **But that solution is not feasible.** The second group of constraints asks $\sum_{i: j \in T_i} \beta_{ij} \leq \alpha$, and tool 5 serves *four* operations: it would receive $4 \cdot 900 = 3600 > 2000$. The violation is 1600. The number 8000 is not a bound: it does not come from any feasible dual solution.

This is why the course protocol does not ask to *propose* a dual solution but to *check* it: the script verifies every constraint, and an **assert** fails if the violation exceeds the tolerance. With the correct recipe the bound is $23000/3 \approx 7666.7$, that is, *better* than the one hoped for by getting it wrong.

The bounds.

$z(MILP) = 2500$: tools 2, 4, 5, 6 are loaded and operations 4 (1700) and 5 (800) are performed. No four-tool configuration yields more: with $\{1, 3, 4, 5\}$ only operation 1 runs (2000), with $\{1, 2, 3, 5\}$ only operation 3 (1800).

LB (constructive heuristic)	$z(MILP)$	$z(LP)$	$z(LP^+)$	UB (dual by hand)	heuristic gap
2000	2500	5200	5200	23000/3	20.0%

Why the relaxation is so weak here

$z(LP) = 5200$ against $z(MILP) = 2500$: more than double. In the continuous problem one can load "a bit" of every tool — for instance $y_j = 2/3$ for six tools, which respects $\sum_j y_j \leq 4$ — and run a fraction of *every* operation. The integrality of the magazine is everything: this is the case where hand-built bounds, however well constructed, are not enough and the work has to be done by the solver. One way to tighten is to add the valid inequalities $\sum_{j \in T_i} y_j \geq |T_i|x_i$ (the aggregated form of the links), or to observe that any operation with $|T_i| > k$ can be fixed to zero.

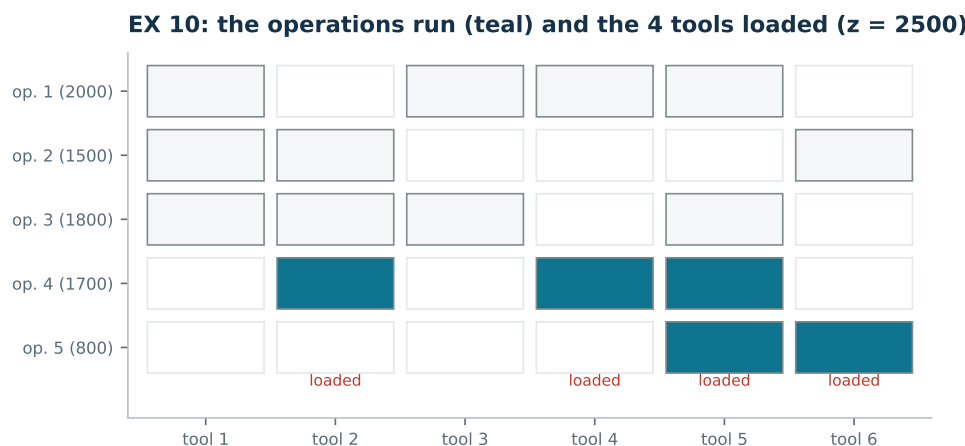


Figure 6.6: EX 10: the operation-tool matrix. In teal the operations performed; the annotated columns are the four loaded tools.

EX 11 — Balancing between two workers

Script: `python/ex11_balancing.py`

[Notebook in Colab](#)

EX 11 — workloads as balanced as possible

Four indivisible jobs of duration 2, 3, 6 and 7 must be assigned to two workers so that their workloads are as balanced as possible.

"Balanced workloads" is not an objective: it is a word. The two linear translations are those of Section 3.6: minimise the *maximum* of the loads, or minimise their *difference*.

Since the total $q = 18$ is constant, the identity

$$\max(w_1, w_2) = \frac{q}{2} + \frac{|w_1 - w_2|}{2}$$

holds, so the two models have the *same* optimal solutions — but different optimal values: 9 for the first, 0 for the second. Reporting "9" as the "difference between the loads" is a mistake, and so is reporting "0" as the "maximum load".

The model (min-max).

The data are $n = 4$ jobs of duration d_j and the total load $q = \sum_{j=1}^n d_j = 18$. With $x_j = 1$ if job j goes to worker 1 and $z \geq 0$ the maximum load:

EX 11, min-max version

$$\begin{aligned} & \min \quad z \\ & \text{subject to} \quad \sum_{j=1}^n d_j x_j - z \leq 0, \\ & \quad \quad - \sum_{j=1}^n d_j x_j - z \leq -q, \quad (\text{load of worker 2, equal to } q - \sum_j d_j x_j) \\ & \quad \quad x_j \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, n\}, \\ & \quad \quad z \geq 0. \end{aligned}$$

EX 11 on the instance — primal

$$\begin{aligned} & \min \quad z \\ & \text{subject to} \quad 2x_1 + 3x_2 + 6x_3 + 7x_4 - z \leq 0 \\ & \quad \quad -2x_1 - 3x_2 - 6x_3 - 7x_4 - z \leq -18 \\ & \quad \quad x_1, \quad x_2, \quad x_3, \quad x_4 \in \{0, 1\} \\ & \quad \quad z \geq 0 \end{aligned}$$

Description. The objective is the load of the busier worker. The first constraint says that z is no smaller than the load of worker 1; the second that it is no smaller than that of worker 2, which equals $q - \sum_j d_j x_j$. Together they impose $z \geq \max(w_1, w_2)$, and the minimisation turns the inequality into an equality on the busier of the two.

Heuristic.

LPT (Chapter 5): the jobs in decreasing order of duration, each to the least loaded worker. Job 4 (7) to worker 1; job 3 (6) to worker 2; job 2 (3) to worker 2, which reaches 9; job 1 (2) to worker 1, which reaches 9. Loads (9, 9), so $UB = 9$.

Dual and lower bound.

Relaxing to $x_j \geq 0$ and using the conversion table — a *minimisation* primal with $\leq$ constraints, hence dual variables $\pi_1, \pi_2 \leq 0$:

Dual of the LP relaxation of EX 11

$$\begin{aligned}
 & \max \quad -q \pi_2 \\
 & \text{subject to} \quad d_j (\pi_1 - \pi_2) \leq 0, & \forall j \in \{1, 2, \dots, n\}, \\
 & \quad \quad \quad -\pi_1 - \pi_2 \leq 1, & \text{(from the column of } z\text{)} \\
 & \quad \quad \quad \pi_1, \pi_2 \leq 0.
 \end{aligned}$$

Description. The variables π_1 and π_2 are the (non-positive) prices of the two min-max constraints. The objective prices the total load q at π_2 . The first group of constraints are the columns of the x_j : moving job j from one worker to the other shifts both loads by d_j , and the balance cannot be positive because the x_j do not appear in the objective. The last constraint is the column of z , which appears in both constraints with coefficient -1 and in the primal objective with cost 1.

Since $d_j > 0$, the first constraints say $\pi_1 \leq \pi_2$. Setting $\pi_1 = \pi_2 = t$, the constraint from z gives $-2t \leq 1$, that is $t \geq -1/2$; the objective $-qt$ is largest at $t = -1/2$ and equals $LB = q/2 = 9$. Meaning: "the two loads add up to q , so the larger one is at least $q/2$ ".

The same thing with non-negative dual variables

Many treatments prefer dual variables ≥ 0 . It suffices to set $\alpha = -\pi_1$ and $\beta = -\pi_2$: the dual becomes $\max q\beta$ with $\alpha \leq \beta$ and $\alpha + \beta \leq 1$, and $\alpha = \beta = 1/2$ gives 9 again. It is the *same* solution written with the other sign; what one cannot do is declare variables ≥ 0 and then use one of them negative.

The bounds.

$z(MILP) = 9$ with jobs $\{1, 4\}$ to the first worker and $\{2, 3\}$ to the second: loads $(9, 9)$, perfectly balanced.

UB (LPT)	LB (dual by hand)	$z(LP)$	$z(MILP)$	heuristic gap
9	9	9	9	0.0%

All the numbers coincide: here the two hand-built bounds meet, and optimality is proved without solving the MILP. This happens because q is even and the jobs can be split exactly in half; with q odd the bound would become $\lceil q/2 \rceil$ and one more argument would be needed.

EX 12 — Shoes with a minimum production threshold

Script: `python/ex12_shoes_threshold.py`

[Notebook in Colab](#)

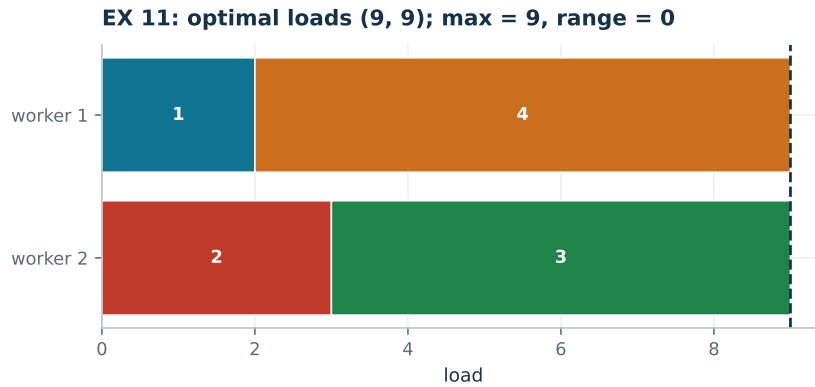


Figure 6.7: EX 11: the optimal loads. The dashed line is $q/2$, the dual bound.

EX 12 — three types of shoe with a threshold

A shoe company produces hiking boots, loafers and walking shoes. The table reports the resources needed for one pair, the minimum quantity to be produced if the type is produced, and the selling price:

Type	leather (g)	hours	nails	minimum	price
hiking boots	850	3	20	100	150
loafers	600	2	15	200	120
walking shoes	700	2.5	20	150	130

There are 120 000 grams of leather, 7000 machine hours and 40 000 nails available. The goal is to maximise the revenue.

The model.

It is again model 9.3 without the bonus, with three resources. The big-Ms read off the data are $M = (141, 200, 171)$, all limited by the leather. For the loafers threshold and maximum coincide (200): the variable x_2 is in fact a binary multiplied by 200.

Heuristic: the primal bound.

Constructive heuristic on the price per gram of leather: $3/17$ for the boots, $1/5$ for the loafers, $13/70$ for the walking shoes. One starts from the loafers, produces 200 of them and the leather runs out exactly. No other type reaches its own threshold.

$$z(MILP) \geq LB = 24\,000.$$

Dual: the dual bound.

With $\ell = \beta = 0$ and a single resource priced:

$$\text{leather: } \frac{1}{5} \Rightarrow 24\,000, \quad \text{hours: } 60 \Rightarrow 420\,000, \quad \text{nails: } 8 \Rightarrow 320\,000.$$

The better bound is the one from the leather:

$$z(MILP) \leq UB = 24\,000.$$

Results.

	LB	UB	$z(LP)$	$z(LP^+)$	$z(MILP)$
EX 12 shoes with threshold (max)	24 000	24 000	24 000	24 000	24 000

All the numbers coincide: the sandwich closes immediately. The reason is that a single resource is tight, the leather, and the type that gets the most out of it per gram uses it up on its own, staying within its own threshold and its own maximum. It is a didactically useful case precisely because it is easy: it shows that when the dual bound matches a feasible solution, the problem is over.

Two variations change the picture. Doubling the thresholds, the one for the loafers becomes 400 pairs while the leather allows at most 200: no type reaches its own threshold any more and production stops altogether, $z(MILP) = 0$. Raising the leather to 200 000 grams, 333 loafers are produced for 39 960 euros, and the big-Ms must be recomputed: they become (235, 333, 285). Big-M values are not constants of the model, but functions of the data of the instance.

EX 13 — Mutual funds bought in lotsScript: `python/ex13_funds.py`[Notebook in Colab](#)**EX 13 — two mutual funds**

After a lottery win, an investor must decide how to allocate 100 million lire. A friend suggests two types of mutual fund. Shares can only be bought in indivisible lots: one lot of the first type costs 12 million, one of the second 20 million. The estimated annual return is $1/6$ of the capital invested for the first type and 15% for the second. Moreover the number of lots of the second type cannot exceed 50% of the total number of lots bought. The goal is to maximise the expected annual return.

The model.

The return of one lot is $12 \cdot 1/6 = 2$ million for the first fund and $20 \cdot 0.15 = 3$ for the second. The share constraint $x_2 \leq \frac{1}{2}(x_1 + x_2)$, multiplied by 2 and moved to the left, becomes $-x_1 + x_2 \leq 0$. With x_j the number of lots of fund j :

EX 13 — the model written out in full

$$\begin{aligned}
 &\max \quad 2x_1 + 3x_2 \\
 &\text{subject to} \quad 12x_1 + 20x_2 \leq 100 \\
 &\quad \quad \quad -x_1 + x_2 \leq 0 \\
 &\quad \quad \quad x_1, \quad x_2 \in \mathbb{Z}_{\geq 0}
 \end{aligned}$$

Description. An integer, non-binary knapsack: the variables count lots, not choices.

Heuristic: the primal bound.

Constructive heuristic on the return per million invested: $2/12 = 1/6$ for the first fund and $3/20$ for the second. One starts from the first, buys 8 lots (96 million) and is left with 4 million,

not enough for another lot of either type.

$$z(MILP) \geq LB = 16.$$

Dual: the dual bound.

With $\alpha \geq 0$ on the budget and $\beta \geq 0$ on the share:

$$\min 100\alpha \quad \text{s.t.} \quad 12\alpha - \beta \geq 2, \quad 20\alpha + \beta \geq 3.$$

The original draft of this exercise proposed $\bar{\alpha} = 5/32$, $\bar{\beta} = 1/8$. Let us check the first constraint:

$$12 \cdot \frac{5}{32} - \frac{1}{8} = \frac{60}{32} - \frac{4}{32} = \frac{56}{32} = \frac{7}{4} < 2.$$

It is violated by $1/4$: that pair is *not* a feasible dual solution, and the value $100 \cdot 5/32$ is not a bound. A dual value may be read as a bound only after checking *all* the constraints, one by one.

The correct recipe is $\bar{\beta} = 0$ and

$$\bar{\alpha} = \max_j \frac{p_j}{c_j} = \max\left(\frac{2}{12}, \frac{3}{20}\right) = \frac{1}{6},$$

that is, a million is worth what it returns in the better fund. Both constraints become $c_j \bar{\alpha} \geq p_j$ and are satisfied, and

$$z(MILP) \leq UB = 100 \cdot \frac{1}{6} = \frac{50}{3} \approx 16.67.$$

Results.

	LB	UB	$z(LP)$	$z(LP^+)$	$z(MILP)$
EX 13 funds (max)	16	50/3	50/3	50/3	16

The heuristic finds the optimum, 8 lots of the first fund and none of the second, for a return of 16 million. The difference $50/3 - 16 = 2/3$ is the price of integrality: the relaxation would buy $25/3$ lots of the first fund, which cannot be bought in pieces.

On the data of the instance the share constraint does not bite, because the first fund returns more per million invested and at the optimum the second one is not bought at all. The share becomes active as soon as the second fund returns 20%, that is 4 million a lot: then without the share the optimum would be 20 (five lots of the second fund) and with the share it drops to 18 (three and three). It is the only way to see what a constraint really does: change the data until it bites.

EX 14 — Emergency department shifts

Script: `python/ex14_shifts.py`

[Notebook in Colab](#)

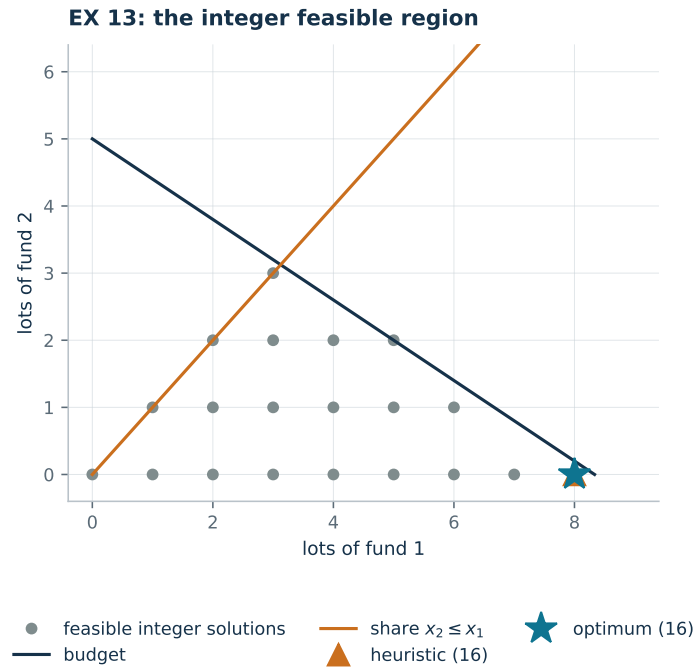


Figure 6.8: EX 13: the integer feasible region, with the budget line, the share line, the heuristic solution and the optimum (coinciding).

EX 14 — weekly shifts

In an emergency department the number of full-time staff members required is given for every day of the week:

Day	Mon	Tue	Wed	Thu	Fri	Sat	Sun
staff	10	8	12	9	9	7	8

One full-time member can be replaced by two part-time members. The shift pattern is fixed: four consecutive full-time days, then one part-time day, then two rest days. The weekly cost of a worker depends on the days worked: 100 euros for every weekday (Monday to Friday), 110 for Saturday and 130 for Sunday; these costs are halved on part-time days. The goal is the weekly plan of minimum cost.

The model.

There are $n = 7$ shift patterns, one per starting day, and $m = 7$ days to cover, with requirement b_i . With x_j the number of workers following pattern j (integer variables), c_j the weekly cost of pattern j and a_{ij} the share of day i covered by pattern j (1 on the four full days, 1/2 on the part-time day, 0 on the two rest days):

EX 14

$$\begin{aligned}
& \min \sum_{j=1}^n c_j x_j \\
& \text{subject to} \quad \sum_{j=1}^n a_{ij} x_j \geq b_i, & \forall i \in \{1, 2, \dots, m\}, \\
& & x_j \in \mathbb{Z}_{\geq 0}, & \forall j \in \{1, 2, \dots, n\}.
\end{aligned}$$

EX 14 on the instance — primal

$$\begin{aligned}
& \min 450x_1 + 455x_2 + 475x_3 + 490x_4 + 490x_5 + 490x_6 + 480x_7 \\
& \text{subject to} \quad x_1 + \frac{1}{2}x_4 + x_5 + x_6 + x_7 \geq 10 \\
& \quad x_1 + x_2 + \frac{1}{2}x_5 + x_6 + x_7 \geq 8 \\
& \quad x_1 + x_2 + x_3 + \frac{1}{2}x_6 + x_7 \geq 12 \\
& \quad x_1 + x_2 + x_3 + x_4 + \frac{1}{2}x_7 \geq 9 \\
& \quad \frac{1}{2}x_1 + x_2 + x_3 + x_4 + x_5 \geq 9 \\
& \quad \quad \frac{1}{2}x_2 + x_3 + x_4 + x_5 + x_6 \geq 7 \\
& \quad \quad \quad \frac{1}{2}x_3 + x_4 + x_5 + x_6 + x_7 \geq 8 \\
& \quad x_1, x_2, x_3, x_4, x_5, x_6, x_7 \in \mathbb{Z}_{\geq 0}
\end{aligned}$$

Description. It is a *set covering* with coefficients 1 and 1/2 and integer, non-binary variables. The costs are computed from the data:

Pattern	full days	part-time	cost
1 (starts Mon)	Mon, Tue, Wed, Thu	Fri	450
2 (starts Tue)	Tue, Wed, Thu, Fri	Sat	455
3 (starts Wed)	Wed, Thu, Fri, Sat	Sun	475
4 (starts Thu)	Thu, Fri, Sat, Sun	Mon	490
5 (starts Fri)	Fri, Sat, Sun, Mon	Tue	490
6 (starts Sat)	Sat, Sun, Mon, Tue	Wed	490
7 (starts Sun)	Sun, Mon, Tue, Wed	Thu	480

For instance pattern 3 costs $100 + 100 + 100 + 110$ for the four full days (Wed, Thu, Fri, Sat) plus $130/2 = 65$ for the part-time Sunday, that is 475 in total.

Heuristic: the primal bound.

Constructive covering heuristic: while some demand is uncovered one adds a copy of the pattern with the lowest ratio

$$\frac{\text{cost}}{\text{demand actually covered}}.$$

The ratio is recomputed at every step, because the residual demand changes. On the instance the constructive heuristic adds 18 shifts: eight of pattern 1 (at 100 euros per unit of demand covered), then four of pattern 3, two of pattern 5, one of pattern 4 and three of pattern 7, with steadily worsening ratios, up to 480 for the last unit.

$$z(\text{MILP}) \leq UB = 8410.$$

Dual: the dual bound.

The dual is

$$\max \sum_{i=1}^7 b_i \pi_i \quad \text{s.t.} \quad \sum_{i=1}^7 a_{ij} \pi_i \leq c_j, \quad \forall j \in \{1, 2, \dots, 7\}, \quad \pi \geq 0.$$

For example one can use the *best ratio* one: put the same price t on all the days. Every pattern covers $4 + 1/2 = 9/2$ person-days, so the dual constraint becomes $\frac{9}{2} t \leq c_j$ for every pattern, and the largest feasible value is

$$t = \min_j \frac{c_j}{9/2} = \min \left(100, \frac{910}{9}, \frac{950}{9}, \frac{980}{9}, \frac{980}{9}, \frac{980}{9}, \frac{320}{3} \right) = 100,$$

attained by pattern 1. The total demand is $10 + 8 + 12 + 9 + 9 + 7 + 8 = 63$, so

$$z(MILP) \geq LB = 100 \cdot 63 = 6300.$$

Results.

	UB	LB	$z(LP)$	$z(LP^+)$	$z(MILP)$
EX 14 shifts	8 410	6 300	115970/17	115970/17	7 060

The optimal plan hires 5 workers on pattern 1, 4 on pattern 3, 1 on pattern 4, 2 on pattern 5 and 3 on pattern 7, at a cost of 7060. The coverage is saturated on Wednesday, Saturday and Sunday, and in surplus on the other days. The heuristic gap is 19.1%; the difference $z(MILP) - z(LP) = 7060 - 115970/17 \approx 238$ is the price of integrality, that is of the fact that people are hired one at a time.

Is the part-time day worth it?

Let us compare the contract of the statement with an alternative one: four full days and *three* rest days, with no part-time. The minimum price per day covered is the same in both cases, 100 euros. And yet the optimum of the second contract is 6710, against the 7060 of the first: the part-time day is worth less. The reason is that the half day falls on a fixed day and often ends up where the coverage is already there; the flexibility lost costs more than the extra coverage is worth. It is an example in which the conclusion "more coverage at the same unit price is always better" is false, and only the model shows it.

EX 15 — The music school timetable

Script: `python/ex15_timetable.py`

[Notebook in Colab](#)

EX 15 — weekly timetable

A music school needs to build its weekly timetable. There are four afternoons (Monday, Tuesday, Wednesday and Thursday), each of three hours. 6 hours of guitar, 3 of violin, 2 of piano and 1 of harp must be placed. Every day must include lessons of at least two different instruments. Piano and harp lessons cannot be in consecutive hours of the same day, because of a conflict between the two teachers. Every teacher has preferences to be avoided, if possible: the guitar teacher prefers not to teach at hour 1 on Monday and Tuesday, the

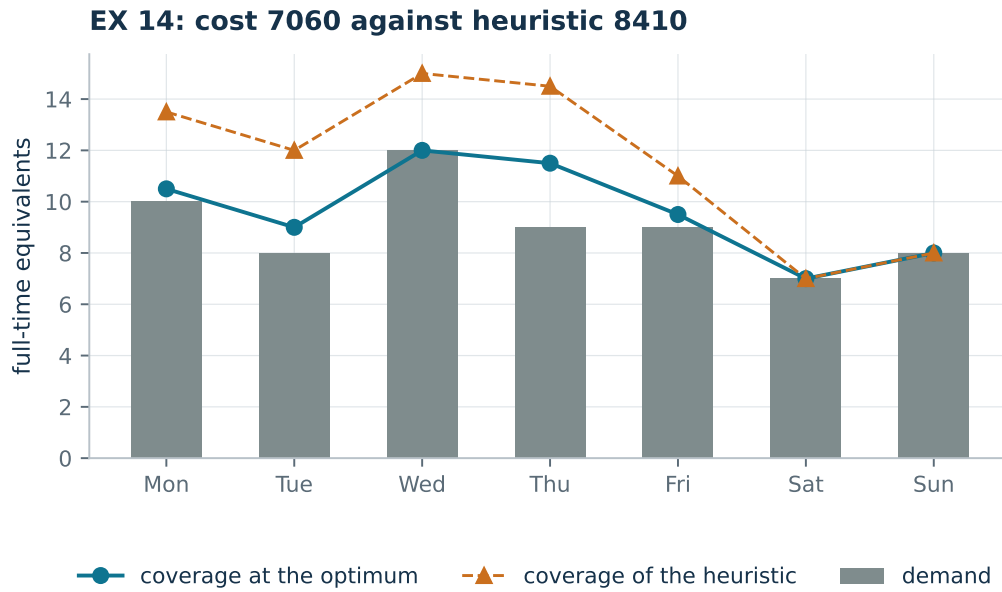


Figure 6.9: EX 14: daily demand and coverage, in the heuristic solution and in the optimal one.

violin teacher at hour 2 on Wednesday and Thursday, the piano teacher at hour 3 on any day, the harp teacher on Tuesday. The goal is to minimise the number of hours that violate a preference.

The model.

With D the days, T the hours, I the instruments, $x_{dti} = 1$ if at hour t of day d instrument i is taught, and $y_{di} = 1$ if instrument i appears on day d :

EX 15

$$\begin{aligned}
& \min \sum_{d \in D} \sum_{t \in T} \sum_{i \in I} c_{dti} x_{dti} \\
& \text{subject to} \quad \sum_{d \in D} \sum_{t \in T} x_{dti} = h_i, & \forall i \in I, \\
& \quad \sum_{i \in I} x_{dti} \leq 1, & \forall d \in D, \forall t \in T, \\
& \quad \sum_{i \in I} y_{di} \geq 2, & \forall d \in D, \\
& \quad x_{dti} - y_{di} \leq 0, & \forall d \in D, \forall t \in T, \forall i \in I, \\
& \quad y_{di} - \sum_{t \in T} x_{dti} \leq 0, & \forall d \in D, \forall i \in I, \\
& \quad x_{dt3} + x_{d,t+1,4} \leq 1, & \forall d \in D, \forall t \in \{1, 2\}, \\
& \quad x_{dt4} + x_{d,t+1,3} \leq 1, & \forall d \in D, \forall t \in \{1, 2\}, \\
& \quad x_{dti} \in \{0, 1\}, & \forall d \in D, \forall t \in T, \forall i \in I, \\
& \quad y_{di} \in \{0, 1\}, & \forall d \in D, \forall i \in I.
\end{aligned}$$

Description. The hours to place are $6 + 3 + 2 + 1 = 12$ and the slots are $4 \cdot 3 = 12$: the two figures coincide, so every slot holds exactly one lesson.

The original draft of this exercise contained only the link $x_{dti} \leq y_{di}$, and not its converse. With that direction alone the constraint $\sum_i y_{di} \geq 2$ is *always* satisfied: it is enough to set $y_{di} = 1$ without teaching. Solving the model written that way, the solver returns a timetable in which Thursday has a single instrument — three hours of guitar — and yet all the constraints hold.

The link $x_{dti} \leq y_{di}$ says "if there is a lesson then the indicator is on", not the converse. One also needs

$$y_{di} \leq \sum_{t \in T} x_{dti},$$

that is technique 3.10 (if and only if). It is the same mistake, and the same correction, as in problem 10.3: when a constraint *counts* indicators, the link must hold in both directions.

A feasible solution.

With the corrected model one builds by hand a timetable that violates no preference:

	hour 1	hour 2	hour 3
Monday	piano	guitar	guitar
Tuesday	violin	guitar	guitar
Wednesday	piano	guitar	guitar
Thursday	violin	harp	violin

One checks: 6 hours of guitar, 3 of violin, 2 of piano, 1 of harp; at least two instruments a day; piano and harp never on the same day, hence never consecutive; no preference violated.

$$z(MILP) \leq UB = 0.$$

The lower bound.

All the costs c_{dti} are 0 or 1, so the objective is a sum of non-negative terms and $z(MILP) \geq 0$ with no dual needed. The hand-built timetable is worth 0: it is optimal.

$$LB = z(LP) = z(LP^+) = z(MILP) = UB = 0.$$

The model therefore serves to *prove* that a conflict-free timetable exists, not to find a less bad one.

What happens if the preferences get tighter.

Two variants make the problem non-trivial.

- If the guitar teacher prefers not to teach at hour 1 nor at hour 2 of any day, only 4 slots stay penalty-free for the guitar (the hour 3 of the four days), but the hours to place are 6: at least $6 - 4 = 2$ lessons will violate the preference. It is a lower bound read off the data alone, and the solver confirms $z(MILP) = 2$: the count is exact.
- If at least *three* different instruments a day are required, the guitar can take at most one slot a day, that is 4 in total, while the guitar hours are 6. The model is infeasible, and one proves it by counting: Gurobi returns the status **INFEASIBLE**, as expected.

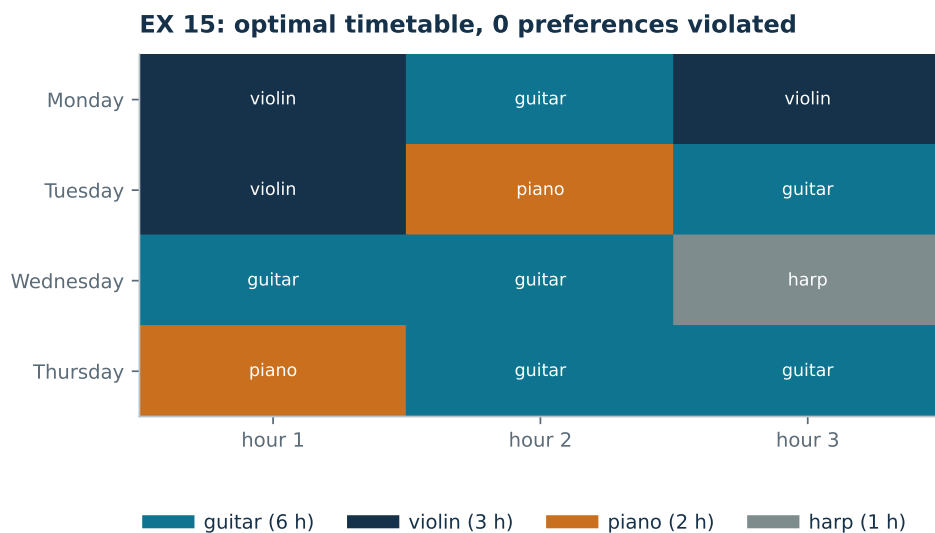


Figure 6.10: EX 15: the optimal timetable found by the solver, one colour per instrument.

Assignment and scheduling

Class: BIP / MILP

Script: one per problem, `python/fam07_1_assignment.py` ...

[Online page](#)

The family

Seven problems with the same skeleton: some *jobs* must be assigned to some *machines* with limited availability. What changes from problem to problem is what is paid and what is decided: the cost of the assignment (problem 1), the fixed cost of every machine switched on (2), the revenue of the jobs one chooses to execute (3), the processing time when the jobs on the same machine run in parallel (4), the setup of a class of jobs (5), a bonus collected *if and only if* a class is complete (6), the tardiness with respect to the due dates when the jobs follow one another on a single machine (7).

The links between variables of Chapter 3 revisited here:

- **activation** (problems 2, 3, 5): the binary variable “machine used” or “class activated” that drives the assignments, with the aggregated constraint $\sum_j t_{jm} x_{jm} \leq a_m y_m$ or the disaggregated one $x_j \leq y_c$;
- **maximum variable** (problem 4): the processing time $y_m \geq t_{jm} x_{jm}$ for every job, which at the optimum equals exactly the maximum;
- **if and only if** (problem 6): one direction is imposed by the constraint, the other by the objective;
- **big-M and disjunctions** (problem 7): “either j before i or i before j ”, with M equal to the sum of the times.

For each problem: model, proof of the links, instance, constructive heuristic (a bound from the side of feasible solutions), dual of the LP relaxation with a hand-built dual solution (a bound from the other side), solver solution, additional considerations and two modelling questions with solutions. All the numbers come from the scripts `fam07_1_assignment.py` ... `fam07_7_tardiness.py` in the `python/` folder, one per problem; each with its own notebook.

Notation of the family

Symbol	Type	Meaning
n	$\in \mathbb{Z}_{\geq 1}$	number of jobs, indexed by $j \in \{1, 2, \dots, n\}$
k	$\in \mathbb{Z}_{\geq 1}$	number of machines, indexed by $m \in \{1, 2, \dots, k\}$
t_{jm}	$\in \mathbb{Q}_{>0}$	processing time (minutes) of job j on machine m ; t_j if it does not depend on the machine
c_{jm}	$\in \mathbb{Q}_{>0}$	cost (euros) of executing job j on machine m
c_m	$\in \mathbb{Q}_{>0}$	fixed cost (euros) if machine m is used
a_m	$\in \mathbb{Q}_{>0}$	availability (minutes) of machine m ; a if there is a single machine
p_m	$\in \mathbb{Z}_{\geq 1}$	maximum number of jobs machine m can execute
r_j	$\in \mathbb{Q}_{>0}$	revenue (euros) if job j is executed
d_j	$\in \mathbb{Q}_{>0}$	due date (minutes) of job j
q	$\in \mathbb{Z}_{\geq 2}$	number of job classes, indexed by $c \in \{1, 2, \dots, q\}$
$\mathcal{J}_c$	$\subseteq \{1, 2, \dots, n\}$	jobs of class c ; the classes partition the jobs
f_c, s_c	$\in \mathbb{Q}_{\geq 0}$	setup cost (euros) and setup time (minutes) of class c
v_c	$\in \mathbb{Q}_{>0}$	bonus (euros) if all the jobs of class c are executed
u	$\in \mathbb{Q}_{>0}$	reduction (minutes) of the availability if jobs of at least two classes are executed

7.1 Minimum-cost assignment with availability

Script: `python/fam07_1_assignment.py`

[Notebook in Colab](#)

Problem 7.1 — minimum-cost assignment

A company has to process $n \in \mathbb{Z}_{\geq 1}$ jobs on $k \in \mathbb{Z}_{\geq 1}$ machines. For each job $j \in \{1, 2, \dots, n\}$ and each machine $m \in \{1, 2, \dots, k\}$, the value $t_{jm} \in \mathbb{Q}_{>0}$ is the processing time in minutes and the value $c_{jm} \in \mathbb{Q}_{>0}$ is the cost in euros of processing job j on machine m . For each machine $m \in \{1, 2, \dots, k\}$, the value $a_m \in \mathbb{Q}_{>0}$ is the available processing time in minutes. Each machine handles one job at a time (no parallel processing). The company seeks a minimum-cost assignment of all jobs to the machines.

The problem in words. We decide on which machine each job is processed. The objective: minimum total cost. The constraints: every job is processed by exactly one machine; the total time of the jobs assigned to a machine does not exceed its availability. It is the *generalised assignment* problem: an assignment with a knapsack constraint per machine.

Data (input of the model)

Symbol	Type	Meaning
n	$\in \mathbb{Z}_{\geq 1}$	number of jobs, $j \in \{1, 2, \dots, n\}$
k	$\in \mathbb{Z}_{\geq 1}$	number of machines, $m \in \{1, 2, \dots, k\}$
t_{jm}	$\in \mathbb{Q}_{>0}$	processing time of job j on machine m
c_{jm}	$\in \mathbb{Q}_{>0}$	cost of processing job j on machine m
a_m	$\in \mathbb{Q}_{>0}$	availability of machine m

Decision variables

We introduce the following $n k$ binary variables:

$$x_{jm} = \begin{cases} 1 & \text{if job } j \text{ is processed by machine } m, \\ 0 & \text{otherwise,} \end{cases}$$

$$\forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}.$$

Using these variables, a BIP model for the problem reads as follows:

Model 7.1 — minimum-cost assignment

$$\min \sum_{j=1}^n \sum_{m=1}^k c_{jm} x_{jm} \quad (7.1a)$$

$$\text{subject to } \sum_{m=1}^k x_{jm} = 1, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.1b)$$

$$\sum_{j=1}^n t_{jm} x_{jm} \leq a_m, \quad \forall m \in \{1, 2, \dots, k\}, \quad (7.1c)$$

$$x_{jm} \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}. \quad (7.1d)$$

Description of the objective function and constraints

- the linear objective function (7.1a) minimises the total processing cost, the sum of the costs of the chosen assignments;
- the linear constraints (7.1b) ensure that each job is assigned to exactly one machine, so that all jobs are processed (n linear constraints);
- the linear constraints (7.1c) guarantee that the total processing time of the jobs assigned to each machine does not exceed its availability (k linear constraints);
- the constraints (7.1d) define the variables of the model.

The model has a single family of variables: there are no links to prove. The constraints (7.1c) are of the “capacity/resource” kind: two quantities of the same nature (minutes required and minutes available) compared, no logical implication to write. The two bounds built below — the primal one, from the heuristic, and the dual one, from the relaxation — are what the solver works with.

The model in gurobipy

Every family of constraints is one `addConstrs` named after its label; the matrix t_{jm} is a list of lists, a_m a list.

```
1 m = gp.Model("assignment"); m.Params.OutputFlag = 0
2 x = m.addVars(n, k, vtype=GRB.BINARY, name="x")
3 m.setObjective(gp.quicksum(c[j][mm] * x[j, mm] for j in range(n)
4                      for mm in range(k)), GRB.MINIMIZE)
```

```

5 m.addConstrs((x.sum(j, "*") == 1 for j in range(n)), name="assign")
6 m.addConstrs((gp.quicksum(t[j][mm] * x[j, mm] for j in range(n)) <= a[
    mm]
7                 for mm in range(k)), name="availability")
8 m.optimize()

```

The instance

$n = 3$ jobs, $k = 3$ machines:

t_{jm}	$m = 1$	$m = 2$	$m = 3$	c_{jm}	$m = 1$	$m = 2$	$m = 3$
$j = 1$	2	1	3	$j = 1$	5	10	2
$j = 2$	3	4	2	$j = 2$	5	4	6
$j = 3$	4	5	3	$j = 3$	5	4	6
					$m = 1$	$m = 2$	$m = 3$
				a_m	5	6	7

Model (7.1) for the instance:

The model of the instance

$$\begin{aligned}
 & \min 5x_{11} + 10x_{12} + 2x_{13} + 5x_{21} + 4x_{22} + 6x_{23} + 5x_{31} + 4x_{32} + 6x_{33} \\
 & \text{subject to } x_{11} + x_{12} + x_{13} = 1 \\
 & \quad \quad \quad x_{21} + x_{22} + x_{23} = 1 \\
 & \quad \quad \quad x_{31} + x_{32} + x_{33} = 1 \\
 & \quad \quad 2x_{11} + 3x_{21} + 4x_{31} \leq 5 \\
 & \quad \quad \quad x_{12} + 4x_{22} + 5x_{32} \leq 6 \\
 & \quad \quad \quad 3x_{13} + 2x_{23} + 3x_{33} \leq 7 \\
 & \quad x_{11}, x_{12}, x_{13}, x_{21}, x_{22}, x_{23}, x_{31}, x_{32}, x_{33} \in \{0, 1\}
 \end{aligned}$$

Constructive heuristic: the primal bound

Three heuristics inspired by bin packing, all linear in the number of job-machine pairs. *Next-fit*: one machine is loaded at a time and the next one is opened when a job no longer fits. *First-fit*: each job goes on the first machine with enough residual availability. *Best-fit*: among the machines with enough availability, the one with minimum cost is chosen. We report the best-fit, the most natural here since the objective is a cost.

It is a constructive heuristic of the family seen in chapter 5: a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

Run on the instance (output of the script):

- **Step 1.** Job 1: residual availabilities $ra = (5, 6, 7)$; every machine is enough ($2 \leq 5$, $1 \leq 6$, $3 \leq 7$); costs 5, 10, 2: the minimum is machine 3, hence $x[1][3] = 1$ and $ra[3] = 7 - 3 = 4$.
- **Step 2.** Job 2: $ra = (5, 6, 4)$; every machine is enough; costs 5, 4, 6: the minimum is machine 2, hence $x[2][2] = 1$ and $ra[2] = 6 - 4 = 2$.

- **Step 3.** Job 3: $ra = (5, 2, 4)$; machine 2 is not enough ($5 > 2$); among the others, costs 5 and 6: the minimum is machine 1, hence $x[3][1] = 1$ and $ra[1] = 5 - 4 = 1$.

The feasible solution found is

$$\bar{x}_{13} = 1, \quad \bar{x}_{22} = 1, \quad \bar{x}_{31} = 1, \quad \text{all the other } \bar{x}_{jm} = 0,$$

of value $2 + 4 + 5 = 11$. Hence $UB = 11$ is an upper bound on the optimum:

$$z(MILP) \leq 11.$$

Next-fit and first-fit both find the assignment $x_{11} = x_{21} = x_{32} = 1$, of value 14: a worse bound.

LP relaxation and dual: the dual bound

The LP relaxation of model (7.1) replaces constraints (7.1d) with

$$x_{jm} \geq 0, \quad \forall j \in \{1, 2, \dots, n\}, \quad \forall m \in \{1, 2, \dots, k\}. \quad (7.2)$$

(The constraint $x_{jm} \leq 1$ is implied by constraints (7.1b): adding it would change nothing.) We associate a free dual variable μ_j with each constraint (7.1b) (equality) and a non-positive dual variable π_m with each constraint (7.1c) ($\leq$ in a minimisation problem). The dual of the LP relaxation is:

Dual of the LP relaxation of model 7.1

$$\max \sum_{j=1}^n \mu_j + \sum_{m=1}^k a_m \pi_m \quad (7.3a)$$

$$\text{subject to } \mu_j + t_{jm} \pi_m \leq c_{jm}, \quad \forall j \in \{1, 2, \dots, n\}, \quad \forall m \in \{1, 2, \dots, k\}, \quad (7.3b)$$

$$\mu_j \geq 0, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.3c)$$

$$\pi_m \leq 0, \quad \forall m \in \{1, 2, \dots, k\}. \quad (7.3d)$$

Description. The variable μ_j is the value attached to job j , which has to be assigned anyway, and π_m the (non-positive) price of one unit of capacity of machine m . The objective (7.3a) prices the jobs and the available capacity. Constraints (7.3b) are the columns of the x_{jm} : putting job j on machine m satisfies its assignment constraint, and so is worth μ_j , and consumes t_{jm} of capacity at price π_m ; the balance cannot exceed the cost c_{jm} of that pairing. The μ_j are free because the assignment constraints are equalities, the π_m non-positive because the capacity ones are of type $\leq$ in a minimisation.

For the instance:

The model of the instance

$$\begin{aligned}
& \max \mu_1 + \mu_2 + \mu_3 + 5\pi_1 + 6\pi_2 + 7\pi_3 \\
& \text{subject to } \mu_1 + 2\pi_1 \leq 5 \\
& \quad \mu_1 + \pi_2 \leq 10 \\
& \quad \mu_1 + 3\pi_3 \leq 2 \\
& \quad \mu_2 + 3\pi_1 \leq 5 \\
& \quad \mu_2 + 4\pi_2 \leq 4 \\
& \quad \mu_2 + 2\pi_3 \leq 6 \\
& \quad \mu_3 + 4\pi_1 \leq 5 \\
& \quad \mu_3 + 5\pi_2 \leq 4 \\
& \quad \mu_3 + 3\pi_3 \leq 6 \\
& \mu_1, \mu_2, \mu_3 \geq 0 \\
& \pi_1, \pi_2, \pi_3 \geq 0
\end{aligned}$$

A hand-built dual solution. With $\bar{\pi}_m = 0$ for every machine, constraints (7.3b) become $\mu_j \leq c_{jm}$ for every m : the largest feasible value is

$$\bar{\mu}_1 = \min\{5, 10, 2\} = 2, \quad \bar{\mu}_2 = \min\{5, 4, 6\} = 4, \quad \bar{\mu}_3 = \min\{5, 4, 6\} = 4,$$

with value $2 + 4 + 4 + 0 = 10$. By weak duality every feasible dual solution is worth at most the LP relaxation, which in turn is worth at most the MILP:

$$10 \leq z(LP) \leq z(MILP) \leq 11.$$

The recipe has a meaning: “every job costs at least its minimum cost” is a lower bound anyone would write down; the dual formalises it and says how to improve it, with $\pi_m < 0$ where the availability is tight.

What the solver says. The optimum of the LP relaxation is $z(LP) = 53/5 = 10.6$ (equal to the optimum of the dual: strong duality), with duals $\tilde{\mu} = (2, 4.8, 5)$ and $\tilde{\pi} = (0, -0.2, 0)$: machine 2 is the tight resource. The MILP has optimum $z(MILP) = 11$ with $\tilde{x}_{13} = \tilde{x}_{22} = \tilde{x}_{31} = 1$: the best-fit heuristic had found the optimal solution, but only the solver certifies it, because the dual bound stopped at 10 (and the relaxation at 10.6: no integer between 10.6 and 11, so in fact $\lceil 53/5 \rceil = 11$ closes the gap — an argument that works because the costs are integer).

UB (best-fit)	LB (hand dual)	$z(LP)$	$z(MILP)$	heuristic gap
11	10	53/5	11	0.0%

Additional considerations

- For every job j and machine m , $x_{jm} \in \{0, 1\}$ implies $x_{jm} \leq 1$: the $n \times k$ inequalities $x_{jm} \leq 1$ are valid for model (7.1), but they are implied by constraints (7.1b) and do not strengthen the relaxation (indeed $z(LP) = z(LP^+)$).
- If a job j does not fit on a machine m ($t_{jm} > a_m$), the variable x_{jm} can be fixed to zero before solving: constraint (7.1c) would force it anyway, but the model is smaller and the relaxation does not get worse.

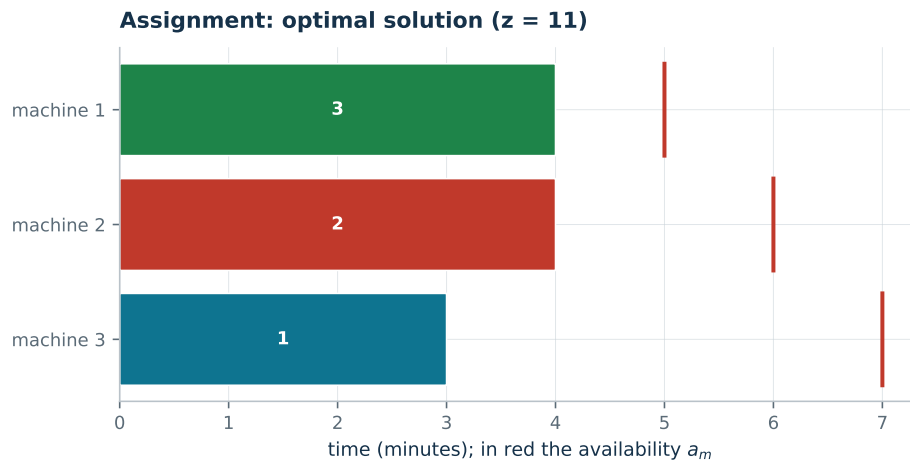


Figure 7.1: The optimal solution of the instance: the jobs assigned to each machine and, in red, the availability a_m .

Additional modelling questions

Modelling question 7.1 — jobs 1 and 3 on the same machine

Jobs 1 and 3 use the same tool and must be processed by the same machine. How does the model change? What is the new optimum for the instance?

Modelling question 7.2 — fixed cost per used machine

Every machine that processes at least one job costs an extra $g_m = 3$ euros to switch on. Model the fixed cost and find the new optimum. Which link of Chapter 3 comes into play?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
UB	11	heuristic solution (UB)
LB	10	certified bound (LB)
$z(MILP)$	11	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 0.0\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 9.1\%.$$

The sandwich on variant 1a

The constraint "jobs 1 and 3 on the same machine" adds one free variable σ_m per machine to the dual, and with it the two jobs can be priced *together*: the constraints of their columns give

$\mu_1 + \mu_3 \leq \min_m (c_{1m} + c_{3m})$, which is more than pricing them separately. The heuristic tries the pair on every machine that can hold it and places job 2 last; the two bounds meet and the problem closes without the solver.

	value	what it is
UB	12	heuristic solution
LB	12	dual certificate built by hand
$z(LP)$	12	relaxation without the bounds
$z(LP^+)$	12	relaxation with the bounds
$z(MILP)$	12	optimum of the MILP

7.2 Machines with a fixed usage cost

Script: `python/fam07_2_fixedcost.py`

[Notebook in Colab](#)

Problem 7.2 — machines with fixed cost

A company needs to process $n \in \mathbb{Z}_{\geq 1}$ jobs and has $k \in \mathbb{Z}_{\geq 1}$ machines. For each job $j \in \{1, 2, \dots, n\}$ and each machine $m \in \{1, 2, \dots, k\}$, the value $t_{jm} \in \mathbb{Q}_{>0}$ is the processing time in minutes of job j on machine m . For each machine $m \in \{1, 2, \dots, k\}$, the value $a_m \in \mathbb{Q}_{>0}$ is the availability in minutes and the value $c_m \in \mathbb{Q}_{>0}$ is the cost in euros if the machine is used. Each machine handles one job at a time. The company wants to assign all jobs to the machines minimising the cost of the machines used.

The problem in words. We decide which machines to switch on and on which machine each job is processed. *The objective:* total cost of the machines switched on. *The constraints:* every job on exactly one machine; a machine switched off processes no job; a machine switched on does not exceed its availability. Compared with problem 7.1 the cost is no longer on the assignments but on the machines: *activation* variables are needed.

Data (input of the model)

Symbol	Type	Meaning
n	$\in \mathbb{Z}_{\geq 1}$	number of jobs, $j \in \{1, 2, \dots, n\}$
k	$\in \mathbb{Z}_{\geq 1}$	number of machines, $m \in \{1, 2, \dots, k\}$
t_{jm}	$\in \mathbb{Q}_{>0}$	processing time of job j on machine m
a_m	$\in \mathbb{Q}_{>0}$	availability of machine m
c_m	$\in \mathbb{Q}_{>0}$	fixed cost if machine m is used

Decision variables

We introduce the following $n k + k$ binary variables:

$$\begin{cases} x_{jm} = 1 \text{ if job } j \text{ is processed by machine } m, 0 \text{ otherwise,} \\ y_m = 1 \text{ if machine } m \text{ is used, } 0 \text{ otherwise,} \end{cases}$$

$$\forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}.$$

Using these variables, a BIP model for the problem reads as follows:

Model 7.2 — machines with fixed cost

$$\min \sum_{m=1}^k c_m y_m \quad (7.4a)$$

$$\text{subject to } \sum_{m=1}^k x_{jm} = 1, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.4b)$$

$$-\sum_{j=1}^n t_{jm} x_{jm} + a_m y_m \geq 0, \quad \forall m \in \{1, 2, \dots, k\}, \quad (7.4c)$$

$$x_{jm} \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}, \quad (7.4d)$$

$$y_m \in \{0, 1\}, \quad \forall m \in \{1, 2, \dots, k\}. \quad (7.4e)$$

Description of the objective function and constraints

- the linear objective function (7.4a) minimises the total cost of the machines used;
- the linear constraints (7.4b) ensure that each job is assigned to exactly one machine (n linear constraints);
- the linear constraints (7.4c) link the assignment variables with the usage variables and impose the capacity restrictions. In particular, for each machine, they ensure that if at least one job is assigned to it then the machine is used, that no job is assigned to an unused machine and that, if the machine is used, the total processing time of the assigned jobs does not exceed its availability (k linear constraints);
- the constraints (7.4d) and (7.4e) define the variables of the model.

Link between the variables

- For each machine m , if the total processing time of the jobs assigned to the machine is positive, then the machine must be used. This logical relation can be written as the implication

$$\sum_{j=1}^n t_{jm} x_{jm} > 0 \implies y_m = 1.$$

From the contrapositive we have

$$y_m = 0 \implies \sum_{j=1}^n t_{jm} x_{jm} = 0 :$$

if machine m is not used, the total processing time of the jobs assigned to it must be zero — that is, since $t_{jm} > 0$ for every j , no job can be assigned to it.

The implication is imposed directly by constraints (7.4c) of model (7.4). Indeed, for each machine m , constraint (7.4c) gives $\sum_{j=1}^n t_{jm} x_{jm} \leq a_m y_m$; if $\sum_{j=1}^n t_{jm} x_{jm} > 0$, then $a_m y_m \geq \sum_{j=1}^n t_{jm} x_{jm} > 0$, hence $y_m > 0$ and, y_m being binary, $y_m = 1$. Conversely, if $y_m = 0$, the constraint gives $\sum_{j=1}^n t_{jm} x_{jm} \leq a_m \cdot 0 = 0$ and, since $t_{jm} > 0$ and $x_{jm} \geq 0$ for every j , this forces $x_{jm} = 0$ for every j , that is $\sum_{j=1}^n t_{jm} x_{jm} = 0$.

- Conversely, for each machine m , if the total processing time of the jobs assigned to the machine is zero, then the machine is not used:

$$\sum_{j=1}^n t_{jm} x_{jm} = 0 \implies y_m = 0,$$

and by contraposition

$$y_m = 1 \implies \sum_{j=1}^n t_{jm} x_{jm} > 0 :$$

if the machine is used, at least one job is assigned to it.

This implication is *not* imposed by the constraints of model (7.4) (the solution $y_m = 1$ with no jobs is feasible), but it follows from the objective function in every optimal solution. Indeed, since $c_m > 0$, if $x_{jm} = 0$ for every j and $y_m = 1$, setting $y_m = 0$ keeps all constraints satisfied (constraint (7.4c) becomes $0 \geq 0$) and decreases the objective by the strictly positive amount c_m : the starting solution was not optimal. Therefore, in every optimal solution, a machine with no jobs is not used.

The two bounds built below — the primal one, from the heuristic, and the dual one, from the relaxation — are what the solver works with.

The model in gurobipy

```

1 m = gp.Model("fixed_cost"); m.Params.OutputFlag = 0
2 x = m.addVars(n, k, vtype=GRB.BINARY, name="x")
3 y = m.addVars(k, vtype=GRB.BINARY, name="y")
4 m.setObjective(gp.quicksum(c[mm] * y[mm] for mm in range(k)),
5                 GRB.MINIMIZE)
6 m.addConstrs((x.sum(j, "*") == 1 for j in range(n)), name="assign")
7 m.addConstrs((-gp.quicksum(t[j][mm] * x[j, mm] for j in range(n))
8               + a[mm] * y[mm] >= 0 for mm in range(k)),
9               name="link")
10 m.optimize()
```

The instance

$n = 3$ jobs, $k = 3$ machines:

t_{jm}	$m = 1$	$m = 2$	$m = 3$
$j = 1$	6	5	3
$j = 2$	5	10	2
$j = 3$	20	13	10

	$m = 1$	$m = 2$	$m = 3$
c_m	8	7	5
a_m	25	20	12

Model (7.4) for the instance:

Model 7.2 on the instance

$$\begin{array}{llllll}
\min & & & & & 8y_1 + 7y_2 + 5y_3 \\
\text{subject to} & x_{11} + x_{12} + x_{13} & & & & = 1 \\
& & x_{21} + x_{22} + x_{23} & & & = 1 \\
& & & x_{31} + x_{32} + x_{33} & & = 1 \\
& -6x_{11} & -5x_{21} & -20x_{31} & +25y_1 & \geq 0 \\
& & -5x_{12} & -10x_{22} & -13x_{32} & +20y_2 \geq 0 \\
& & & -3x_{13} & -2x_{23} & -10x_{33} +12y_3 \geq 0 \\
& x_{11}, & x_{12}, & x_{13}, & x_{21}, & x_{22}, & x_{23}, & x_{31}, & x_{32}, & x_{33} & \in \{0, 1\} \\
& & & & & & & & & & y_1, & y_2, & y_3 \in \{0, 1\}
\end{array}$$

Constructive heuristic: the primal bound

The same three families as in problem 7.1, with the best-fit criterion becoming the *minimum time* (there are no assignment costs: it pays to consume little availability). The best-fit finds $x_{13} = x_{23} = x_{32} = 1$:

- **Step 1.** Job 1: $ra = (25, 20, 12)$; times 6, 5, 3, all feasible; minimum on machine 3: $x[1][3] = 1$, $ra[3] = 12 - 3 = 9$.
- **Step 2.** Job 2: $ra = (25, 20, 9)$; times 5, 10, 2; minimum on machine 3: $x[2][3] = 1$, $ra[3] = 9 - 2 = 7$.
- **Step 3.** Job 3: $ra = (25, 20, 7)$; machine 3 is not enough ($10 > 7$); between 20 and 13 the minimum is machine 2: $x[3][2] = 1$, $ra[2] = 20 - 13 = 7$.

The machines used are 2 and 3: $\bar{y} = (0, 1, 1)$, value $7 + 5 = 12$, hence $z(MILP) \leq 12$. Next-fit and first-fit use machines 1 and 2 (value 15); the “opened machines first” variants also give 15. The minimum-time best-fit is the only one of the five to find 12.

It is a constructive heuristic of the family seen in chapter 5: a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

LP relaxation and dual: the dual bound

The LP relaxation replaces (7.4d) and (7.4e) with $x_{jm} \geq 0$ and $y_m \geq 0$. With a free dual variable μ_j for each constraint (7.4b) and a non-negative π_m for each constraint (7.4c) ($\geq$ in a minimisation), the dual is:

Dual of the LP relaxation of model 7.2

$$\max \sum_{j=1}^n \mu_j \quad (7.5a)$$

$$\text{subject to } \mu_j - t_{jm} \pi_m \leq 0, \quad \forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}, \quad (7.5b)$$

$$a_m \pi_m \leq c_m, \quad \forall m \in \{1, 2, \dots, k\}, \quad (7.5c)$$

$$\mu_j \geq 0, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.5d)$$

$$\pi_m \geq 0, \quad \forall m \in \{1, 2, \dots, k\}. \quad (7.5e)$$

Description. The variable μ_j is the value of job j and π_m the price of one unit of capacity of machine m . The objective (7.5a) prices the jobs only: capacity by itself is not paid for. Constraints (7.5b) are the columns of the x_{jm} : assigning job j to machine m collects μ_j and consumes t_{jm} of capacity at price π_m , and the balance cannot be positive because the x_{jm} do not appear in the primal objective. Constraints (7.5c) are the columns of the y_m : all the capacity a_m that switching on machine m makes available, priced at π_m , cannot be worth more than the fixed cost c_m .

For the instance:

The model of the instance

$$\begin{array}{llll}
 \max & \mu_1 + \mu_2 + \mu_3 \\
 \text{subject to} & \mu_1 & -6\pi_1 & \leq 0 \\
 & \mu_1 & -5\pi_2 & \leq 0 \\
 & \mu_1 & -3\pi_3 & \leq 0 \\
 & \mu_2 & -5\pi_1 & \leq 0 \\
 & \mu_2 & -10\pi_2 & \leq 0 \\
 & \mu_2 & -2\pi_3 & \leq 0 \\
 & \mu_3 - 20\pi_1 & & \leq 0 \\
 & \mu_3 & -13\pi_2 & \leq 0 \\
 & \mu_3 & -10\pi_3 & \leq 0 \\
 & & 25\pi_1 & \leq 8 \\
 & & 20\pi_2 & \leq 7 \\
 & & 12\pi_3 & \leq 5 \\
 & \mu_1, \mu_2, \mu_3 & & \geq 0 \\
 & & \pi_1, \pi_2, \pi_3 & \geq 0
 \end{array}$$

A hand-built dual solution. Constraints (7.5c) say how large each π_m can be at most: take the maximum, $\bar{\pi}_m = c_m/a_m$, that is

$$\bar{\pi}_1 = \frac{8}{25}, \quad \bar{\pi}_2 = \frac{7}{20}, \quad \bar{\pi}_3 = \frac{5}{12}$$

(the cost per minute of each machine). Constraints (7.5b) then give $\mu_j \leq t_{jm}\bar{\pi}_m$ for every m , and the largest feasible value is

$$\begin{aligned}
 \bar{\mu}_1 &= \min\left\{6 \cdot \frac{8}{25}, 5 \cdot \frac{7}{20}, 3 \cdot \frac{5}{12}\right\} = \min\left\{\frac{48}{25}, \frac{7}{4}, \frac{5}{4}\right\} = \frac{5}{4}, \\
 \bar{\mu}_2 &= \min\left\{5 \cdot \frac{8}{25}, 10 \cdot \frac{7}{20}, 2 \cdot \frac{5}{12}\right\} = \min\left\{\frac{8}{5}, \frac{7}{2}, \frac{5}{6}\right\} = \frac{5}{6}, \\
 \bar{\mu}_3 &= \min\left\{20 \cdot \frac{8}{25}, 13 \cdot \frac{7}{20}, 10 \cdot \frac{5}{12}\right\} = \min\left\{\frac{32}{5}, \frac{91}{20}, \frac{25}{6}\right\} = \frac{25}{6}.
 \end{aligned}$$

Value: $\frac{5}{4} + \frac{5}{6} + \frac{25}{6} = \frac{25}{4}$. Hence

$$\frac{25}{4} \leq z(MILP) \leq 12.$$

The reading: every job “pays” at least the cost per minute of the machine that is cheapest for it, times its time. It is a weak bound: the fixed cost of a machine is paid in full as soon as it is used, but the relaxation spreads it over the minutes.

What the solver says. The optimum of the relaxation without the bounds is exactly $z(LP) = 25/4$: the hand-built solution is optimal for the dual. The relaxation strengthened

with $y_m \leq 1$ and $x_{jm} \leq 1$ is worth $z(LP^+) = 1273/200 = 6.365$; adding the disaggregated constraints $x_{jm} \leq y_m$ (valid, see below) it rises to $440/67 = 6.567$. The integer optimum is $z(MILP) = 12$, with machines 2 and 3 on and $\tilde{x}_{12} = \tilde{x}_{23} = \tilde{x}_{33} = 1$: the heuristic had found the optimum, and the gap between 6.25 and 12 measures how far the relaxation is from the integer in this family of models, where the “fractional” fixed costs fool the LP.

LB (best-fit)	UB (hand dual)	$z(LP)$	$z(LP^+)$	$z(MILP)$	heuristic gap
12	25/4	25/4	1273/200	12	0.0%

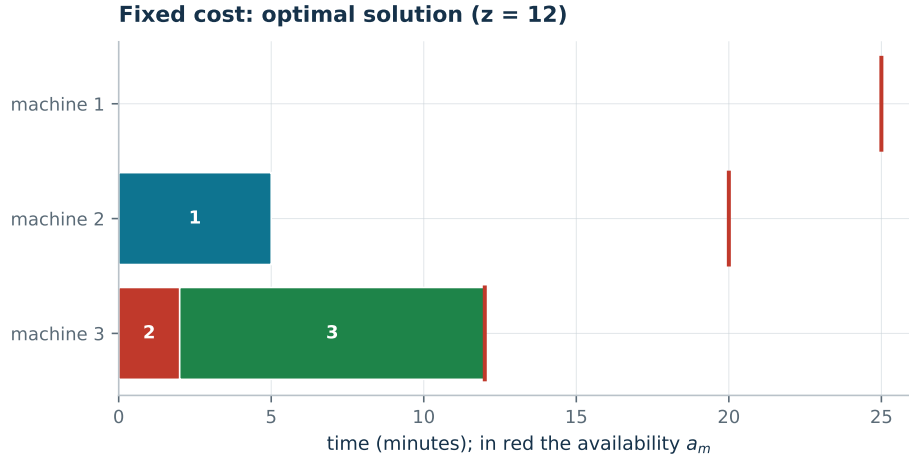


Figure 7.2: The optimal solution of problem 7.2: machines 2 and 3 are switched on.

Additional considerations

- The inequalities $y_m \leq 1$ (k) and $x_{jm} \leq 1$ ($n \cdot k$) are valid; the latter are implied by (7.4b), the former are not: indeed they strengthen the relaxation ($6.25 \rightarrow 6.365$).
- For each machine m , “if at least one job is assigned to m then m is used” is the Boolean implication

$$(x_{1m} \text{ OR } x_{2m} \text{ OR } \dots \text{ OR } x_{nm}) \implies y_m,$$

whose contrapositive is

$$\text{NOT } y_m \implies (\text{NOT } x_{1m} \text{ AND } \dots \text{ AND } \text{NOT } x_{nm}) :$$

if the machine is not used, no job is assigned to it. As a Boolean expression,

$$\text{NOT } (x_{1m} \text{ OR } \dots \text{ OR } x_{nm}) \text{ OR } y_m;$$

by De Morgan and the distributive property it becomes the CNF

$$(\text{NOT } x_{1m} \text{ OR } y_m) \text{ AND } (\text{NOT } x_{2m} \text{ OR } y_m) \text{ AND } \dots \text{ AND } (\text{NOT } x_{nm} \text{ OR } y_m),$$

i.e. the constraints $1 - x_{jm} + y_m \geq 1$:

$$x_{jm} \leq y_m, \quad \forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}$$

($n \cdot k$ valid inequalities, the *disaggregated* form of the link). The model implies them — we proved it above — but the relaxation does not: added, they bring $z(LP^+)$ to $440/67$. It is the concrete example of Section 3.1: same set of integer solutions, tighter relaxation.

- The opposite direction, “if no job is assigned then the machine is not used”, is the clause

$$\text{NOT } y_m \text{ OR } x_{1m} \text{ OR } \dots \text{ OR } x_{nm},$$

i.e.

$$\sum_{j=1}^n x_{jm} \geq y_m, \quad \forall m \in \{1, 2, \dots, k\} :$$

not a valid inequality (the model allows $y_m = 1$ with no jobs) but it can be added without losing the optimum, because $c_m > 0$.

Additional modelling questions

Modelling question 7.3 — minimum usage of a machine switched on

Switching a machine on for a few minutes makes no sense: every machine used must work at least $\ell = 8$ minutes. Model the constraint and find the new optimum.

Modelling question 7.4 — link between two activations

Machine 3 shares the power supply with machine 1: if machine 1 is used, machine 3 must be used too. Write the constraint and discuss what it imposes and what it does not.

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
UB	12	heuristic solution (UB)
LB	$\frac{25}{4}$	certified bound (LB)
$z(MILP)$	12	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 0.0\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 47.9\%.$$

The sandwich on variant 2b

The link "if machine 1 is used then machine 3 is used too" adds a variable $\rho \leq 0$ to the dual, which loosens the column of y_1 and tightens that of y_3 . On the data of the instance it does not pay to move it: machine 3 is the minimum for every job, and lowering its price would lower every μ_j . The certificate stays the one of the base problem — a link between *activations* does not touch the relaxation, which can switch on half a machine. What grows is the integer optimum, and therefore the gap.

	value	what it is
UB	12	heuristic solution
LB	$\frac{25}{4}$	dual certificate built by hand
$z(LP)$	$\frac{25}{4}$	relaxation without the bounds
$z(LP^+)$	$\frac{1273}{200}$	relaxation with the bounds
$z(MILP)$	12	optimum of the MILP

7.3 Job selection with revenues and fixed-cost machines

Script: `python/fam07_3_selection.py`

[Notebook in Colab](#)

Problem 7.3 — job selection

A company can execute $n \in \mathbb{Z}_{\geq 1}$ jobs and has $k \in \mathbb{Z}_{\geq 1}$ machines. For each job $j \in \{1, 2, \dots, n\}$, the value $t_j \in \mathbb{Q}_{>0}$ is the processing time in minutes (the same on every machine) and the value $r_j \in \mathbb{Q}_{>0}$ is the revenue in euros if the job is executed. For each machine $m \in \{1, 2, \dots, k\}$, the value $a_m \in \mathbb{Q}_{>0}$ is the availability in minutes and the value $c_m \in \mathbb{Q}_{>0}$ is the cost in euros if the machine is used. Each machine handles one job at a time. The company wants to choose which jobs to execute, and on which machines, to maximise the profit: revenues of the executed jobs minus costs of the machines used.

The problem in words. We decide which jobs to execute, on which machines, and which machines to switch on. *The objective:* maximum profit. *The constraints:* every job on at most one machine; no job on a machine switched off; availability respected. It is problem 7.2 where the jobs are no longer compulsory and have a revenue: a *maximisation* problem, and the roles of the bounds swap.

Data (input of the model)

Symbol	Type	Meaning
n	$\in \mathbb{Z}_{\geq 1}$	number of jobs, $j \in \{1, 2, \dots, n\}$
k	$\in \mathbb{Z}_{\geq 1}$	number of machines, $m \in \{1, 2, \dots, k\}$
t_j	$\in \mathbb{Q}_{>0}$	processing time of job j
r_j	$\in \mathbb{Q}_{>0}$	revenue if job j is executed
a_m	$\in \mathbb{Q}_{>0}$	availability of machine m
c_m	$\in \mathbb{Q}_{>0}$	fixed cost if machine m is used

Decision variables

We introduce the following $n \cdot k + k$ binary variables:

$$\begin{cases} x_{jm} = 1 \text{ if job } j \text{ is executed by machine } m, 0 \text{ otherwise,} \\ y_m = 1 \text{ if machine } m \text{ is used, 0 otherwise,} \end{cases}$$

$$\forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}.$$

Model 7.3 — job selection

$$\max \sum_{j=1}^n \sum_{m=1}^k r_j x_{jm} - \sum_{m=1}^k c_m y_m \quad (7.6a)$$

$$\text{subject to } \sum_{m=1}^k x_{jm} \leq 1, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.6b)$$

$$\sum_{j=1}^n t_j x_{jm} - a_m y_m \leq 0, \quad \forall m \in \{1, 2, \dots, k\}, \quad (7.6c)$$

$$x_{jm} \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}, \quad (7.6d)$$

$$y_m \in \{0, 1\}, \quad \forall m \in \{1, 2, \dots, k\}. \quad (7.6e)$$

Description of the objective function and constraints

- the linear objective function (7.6a) maximises the total profit, the difference between the revenue of the executed jobs and the cost of the machines used;
- the linear constraints (7.6b) ensure that each job is assigned to at most one machine, so that no job is executed twice (n linear constraints);
- the linear constraints (7.6c) link the assignment variables with the usage variables and impose the capacity restrictions: if at least one job is assigned to a machine then the machine is used, no job is assigned to an unused machine, and the total time of the jobs of a used machine does not exceed its availability (k linear constraints);
- the constraints (7.6d) and (7.6e) define the variables of the model.

Link between the variables

The link is that of problem 7.2, with t_j in place of t_{jm} : we restate the two implications, because the “optimality” direction changes sign.

- The implication

$$\sum_{j=1}^n t_j x_{jm} > 0 \implies y_m = 1,$$

$$\text{by contraposition: } y_m = 0 \implies \sum_{j=1}^n t_j x_{jm} = 0,$$

says that an unused machine has no jobs). Imposed by constraints (7.6c): if $\sum_{j=1}^n t_j x_{jm} > 0$ then $a_m y_m \geq \sum_{j=1}^n t_j x_{jm} > 0$, hence $y_m = 1$; conversely, if $y_m = 0$ then $\sum_{j=1}^n t_j x_{jm} \leq 0$ and, with $t_j > 0$ and $x_{jm} \geq 0$, all the x_{jm} are zero.

- The opposite implication

$$\sum_{j=1}^n t_j x_{jm} = 0 \implies y_m = 0,$$

$$\text{by contraposition: } y_m = 1 \implies \sum_{j=1}^n t_j x_{jm} > 0,$$

says that a used machine has at least one job). Not imposed by the constraints, it follows from the objective in every optimum: since $c_m > 0$, if $y_m = 1$ with no jobs, setting $y_m = 0$ stays feasible and *increases* the profit by c_m — in a maximisation problem the direction of the improvement is reversed, the structure of the argument is not.

The two bounds built below — the primal one, from the heuristic, and the dual one, from the relaxation — are what the solver works with.

The model in gurobipy

```

1 m = gp.Model("selection"); m.Params.OutputFlag = 0
2 x = m.addVars(n, k, vtype=GRB.BINARY, name="x")
3 y = m.addVars(k, vtype=GRB.BINARY, name="y")
4 m.setObjective(gp.quicksum(r[j] * x[j, mm] for j in range(n)
5                          for mm in range(k))
6                - gp.quicksum(c[mm] * y[mm] for mm in range(k)),
7                GRB.MAXIMIZE)
8 m.addConstrs((x.sum(j, "*") <= 1 for j in range(n)), name="at_most_one"
9              )
9 m.addConstrs((gp.quicksum(t[j] * x[j, mm] for j in range(n))
10              - a[mm] * y[mm] <= 0 for mm in range(k)), name="link")
11 m.optimize()
```

The instance

$n = 3$ jobs, $k = 3$ machines:

	$m = 1$	$m = 2$	$m = 3$
a_m	105	110	100
c_m	20	30	15

	$j = 1$	$j = 2$	$j = 3$
t_j	25	40	75
r_j	10	15	30

Model (7.6) for the instance:

Model 7.3 on the instance

$$\begin{array}{llllllllll}
\max & 10x_{11} + 10x_{12} + 10x_{13} + 15x_{21} + 15x_{22} + 15x_{23} + 30x_{31} + 30x_{32} + 30x_{33} & -20y_1 & -30y_2 & -15y_3 & & & & & \\
\text{subject to} & x_{11} & +x_{12} & +x_{13} & & & & & & \leq 1 \\
& & & & x_{21} & +x_{22} & +x_{23} & & & \leq 1 \\
& & & & & & & x_{31} & +x_{32} & +x_{33} \leq 1 \\
& 25x_{11} & & & +40x_{21} & & & +75x_{31} & & -105y_1 \leq 0 \\
& & 25x_{12} & & & +40x_{22} & & +75x_{32} & & -110y_2 \leq 0 \\
& & & 25x_{13} & & & +40x_{23} & & +75x_{33} & -100y_3 \leq 0 \\
& x_{11}, & x_{12}, & x_{13}, & x_{21}, & x_{22}, & x_{23}, & x_{31}, & x_{32}, & x_{33} \in \{0, 1\} \\
& & & & & & & & y_1, & y_2, & y_3 \in \{0, 1\}
\end{array}$$

Constructive heuristic: the primal bound

In a maximisation problem a feasible solution gives a *lower* bound. The three heuristics of problem 7.1 adapt with one more rule: a job that fits on no machine is *skipped* (it is not compulsory). The best-fit chooses the *fullest* machine among those that are enough (minimum residual availability), to concentrate the jobs on few machines:

- **Step 1.** Job 1 ($t_1 = 25$): $ra = (105, 110, 100)$, all enough; the fullest is machine 3: $x[1][3] = 1$, $ra[3] = 75$.
- **Step 2.** Job 2 ($t_2 = 40$): $ra = (105, 110, 75)$; the fullest is still machine 3: $x[2][3] = 1$, $ra[3] = 35$.
- **Step 3.** Job 3 ($t_3 = 75$): $ra = (105, 110, 35)$; machine 3 is not enough; between 1 and 2 the fullest is machine 1: $x[3][1] = 1$, $ra[1] = 30$.

Solution: $\bar{x}_{13} = \bar{x}_{23} = \bar{x}_{31} = 1$, $\bar{y} = (1, 0, 1)$, profit $10 + 15 + 30 - 20 - 15 = 20$, hence $z(MILP) \geq 20$. Next-fit and first-fit fill machine 1 first (jobs 1 and 2) and put job 3 on machine 2: profit $55 - 50 = 5$.

It is a constructive heuristic of the family seen in chapter 5: a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

LP relaxation and dual: the dual bound

Relaxing $x_{jm} \geq 0$ and $y_m \geq 0$, with non-negative dual variables μ_j for constraints (7.6b) and π_m for constraints (7.6c) ($\leq$ in a maximisation), the dual is:

Dual of the LP relaxation of model 7.3

$$\min \sum_{j=1}^n \mu_j \quad (7.7a)$$

$$\text{subject to } \mu_j + t_j \pi_m \geq r_j, \quad \forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}, \quad (7.7b)$$

$$-a_m \pi_m \geq -c_m, \quad \forall m \in \{1, 2, \dots, k\}, \quad (7.7c)$$

$$\mu_j \geq 0, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.7d)$$

$$\pi_m \geq 0, \quad \forall m \in \{1, 2, \dots, k\}. \quad (7.7e)$$

Description. The variable μ_j is the price of the right to run job j and π_m that of one unit of capacity of machine m . The objective (7.7a) adds up the prices of the rights on the jobs. Constraints (7.7b) are the columns of the x_{jm} : running job j on machine m consumes the right, at price μ_j , and t_j of capacity, at price π_m ; together they must cover the revenue r_j that job brings. Constraints (7.7c) are the columns of the y_m : the capacity a_m that switching the machine on makes available, priced at π_m , cannot cost more than the fixed cost c_m paid to switch it on.

For the instance:

The model of the instance

$$\begin{array}{llll}
 \min & \mu_1 + \mu_2 + \mu_3 \\
 \text{subject to} & \mu_1 & +25\pi_1 & \geq 10 \\
 & \mu_1 & +25\pi_2 & \geq 10 \\
 & \mu_1 & +25\pi_3 & \geq 10 \\
 & \mu_2 & +40\pi_1 & \geq 15 \\
 & \mu_2 & +40\pi_2 & \geq 15 \\
 & \mu_2 & +40\pi_3 & \geq 15 \\
 & \mu_3 & +75\pi_1 & \geq 30 \\
 & \mu_3 & +75\pi_2 & \geq 30 \\
 & \mu_3 & +75\pi_3 & \geq 30 \\
 & -105\pi_1 & & \geq -20 \\
 & & -110\pi_2 & \geq -30 \\
 & & & -100\pi_3 \geq -15 \\
 & \mu_1, \mu_2, \mu_3 & & \geq 0 \\
 & & \pi_1, \pi_2, \pi_3 & \geq 0
 \end{array}$$

A hand-built dual solution. Constraints (7.7c) bound each π_m from above: $\bar{\pi}_m = c_m/a_m$, that is $\bar{\pi}_1 = \frac{20}{105} = \frac{4}{21}$, $\bar{\pi}_2 = \frac{30}{110} = \frac{3}{11}$, $\bar{\pi}_3 = \frac{15}{100} = \frac{3}{20}$. Each μ_j must then satisfy $\mu_j \geq r_j - t_j \bar{\pi}_m$ for every m and $\mu_j \geq 0$: the smallest feasible value is the maximum of these,

$$\begin{aligned}
 \bar{\mu}_1 &= \max\left\{10 - 25 \cdot \frac{4}{21}, 10 - 25 \cdot \frac{3}{11}, 10 - 25 \cdot \frac{3}{20}, 0\right\} = \max\left\{\frac{110}{21}, \frac{35}{11}, \frac{25}{4}, 0\right\} = \frac{25}{4}, \\
 \bar{\mu}_2 &= \max\left\{15 - 40 \cdot \frac{4}{21}, 15 - 40 \cdot \frac{3}{11}, 15 - 40 \cdot \frac{3}{20}, 0\right\} = \max\left\{\frac{155}{21}, \frac{45}{11}, 9, 0\right\} = 9, \\
 \bar{\mu}_3 &= \max\left\{30 - 75 \cdot \frac{4}{21}, 30 - 75 \cdot \frac{3}{11}, 30 - 75 \cdot \frac{3}{20}, 0\right\} = \max\left\{\frac{110}{7}, \frac{105}{11}, \frac{75}{4}, 0\right\} = \frac{75}{4}.
 \end{aligned}$$

Value: $\frac{25}{4} + 9 + \frac{75}{4} = 34$. By weak duality

$$20 \leq z(MILP) \leq 34.$$

The reading: μ_j is “how much job j is worth at most” once the spread fixed cost of the cheapest machine per minute (machine 3, at $\frac{3}{20}$ euros per minute) has been paid for every minute.

What the solver says. The optimum of the relaxation without the bounds is $z(LP) = 34$: here too the hand-built solution is optimal for the dual. With the bounds $x_{jm} \leq 1$ and $y_m \leq 1$ the relaxation with the bounds drops to $z(LP^+) = 680/21 = 32.38$. The integer optimum is $z(MILP) = 25$: jobs 1 and 3 on machine 3 ($25 + 75 = 100 \leq 100$, exactly the availability), profit $40 - 15$; job 2 is not worth it, since it would require a second machine ($c_1 = 20 > r_2 = 15$). The heuristic stopped at 20: it had put job 2 on machine 3, leaving out job 3, which then opened machine 1. A 20% gap.

LB (best-fit)	UB (hand dual)	$z(LP)$	$z(LP^+)$	$z(MILP)$	heuristic gap
20	34	34	680/21	25	20.0%

Additional considerations

- $y_m \leq 1$ (k) and $x_{jm} \leq 1$ ($n k$) are valid inequalities; the latter are implied by (7.6b), the former strengthen the relaxation ($34 \rightarrow 32.38$).
- The disaggregated constraints $x_{jm} \leq y_m$ (from

$$(x_{1m} \text{ OR } \dots \text{ OR } x_{nm}) \implies y_m,$$

as in problem 7.2) are valid and implied by the model, but strengthen the relaxation.

- If $r_j < \min_m c_m$ and job j is the only one on a machine, executing it never pays: this is what happens to job 2 in the instance. A dominance argument the solver makes by itself, but which helps to guess the optimum by hand.

Additional modelling questions

Modelling question 7.5 — all jobs compulsory

The company has already signed the contracts: all jobs must be executed. How does the model change and how much does the obligation cost on the instance?

Modelling question 7.6 — a job conditional on another

Job 3 requires a semi-finished product made by job 2: job 3 can be executed only if job 2 is executed too (on any machine). Write the constraint and find the new optimum.

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
LB	20	heuristic solution (LB)
UB	34	certified bound (UB)
$z(MILP)$	25	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|LB - z(MILP)|}{|z(MILP)|} = 20.0\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|LB - UB|}{|z(MILP)|} = 56.0\%.$$

The sandwich on variant 3b

"Job 3 only if job 2 as well" adds a multiplier $\lambda \geq 0$ to the dual, which discounts the columns of job 3 and loads those of job 2. One raises λ as far as job 2 can bear for free: on this instance there is no such margin, λ stays at zero and the certificate coincides with the one of the base problem.

	value	what it is
UB	34	dual certificate built by hand
LB	20	heuristic solution
$z(LP)$	34	relaxation without the bounds
$z(LP^+)$	$\frac{680}{21}$	relaxation with the bounds
$z(MILP)$	20	optimum of the MILP

7.4 Parallel jobs: the processing time as a maximum

Script: `python/fam07_4_parallel.py`

[Notebook in Colab](#)

Problem 7.4 — parallel jobs

A company needs to execute $n \in \mathbb{Z}_{\geq 1}$ jobs using $k \in \mathbb{Z}_{\geq 1}$ machines. For each job $j \in \{1, 2, \dots, n\}$ and each machine $m \in \{1, 2, \dots, k\}$, the value $t_{jm} \in \mathbb{Q}_{>0}$ is the processing time in minutes of job j on machine m . Each machine $m \in \{1, 2, \dots, k\}$ can execute at most $p_m \in \mathbb{Z}_{\geq 1}$ jobs, and executes them in parallel: all the jobs assigned to the same machine start together. The company wants to assign the jobs minimising the sum of the processing times of the machines, where the processing time of a machine is the time of the longest job among those assigned to it.

The problem in words. We decide on which machine each job goes. The objective: the sum of the processing times of the machines, each equal to the *maximum* of the times of the assigned jobs. The constraints: every job on one machine; at most p_m jobs on machine m . The novelty is the objective: a maximum is not linear, but it is linearised with a continuous variable and n " $\geq$ " constraints per machine.

Data (input of the model)

Symbol	Type	Meaning
n	$\in \mathbb{Z}_{\geq 1}$	number of jobs, $j \in \{1, 2, \dots, n\}$
k	$\in \mathbb{Z}_{\geq 1}$	number of machines, $m \in \{1, 2, \dots, k\}$
t_{jm}	$\in \mathbb{Q}_{>0}$	processing time of job j on machine m
p_m	$\in \mathbb{Z}_{\geq 1}$	maximum number of (parallel) jobs on machine m

Decision variables

We introduce the following nk binary variables and k non-negative variables:

$$\begin{cases} x_{jm} = 1 \text{ if job } j \text{ is executed by machine } m, 0 \text{ otherwise,} \\ y_m = \text{processing time of machine } m, \end{cases}$$

$$\forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}.$$

Model 7.4 — parallel jobs

$$\min \sum_{m=1}^k y_m \quad (7.8a)$$

$$\text{subject to } \sum_{m=1}^k x_{jm} = 1, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.8b)$$

$$\sum_{j=1}^n x_{jm} \leq p_m, \quad \forall m \in \{1, 2, \dots, k\}, \quad (7.8c)$$

$$-t_{jm} x_{jm} + y_m \geq 0, \quad \forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}, \quad (7.8d)$$

$$x_{jm} \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}, \quad (7.8e)$$

$$y_m \geq 0, \quad \forall m \in \{1, 2, \dots, k\}. \quad (7.8f)$$

Description of the objective function and constraints

- the linear objective function (7.8a) minimises the sum of the processing times of the machines;
- the linear constraints (7.8b) ensure that each job is assigned to exactly one machine (n linear constraints);
- the linear constraints (7.8c) limit the number of jobs assigned to each machine (k linear constraints);
- the linear constraints (7.8d) link the assignment variables with the processing-time variables. In particular, for each job and each machine, they guarantee that if the job is assigned to the machine then the processing time of the machine is at least the time required by that job on that machine ($n k$ linear constraints);
- the constraints (7.8e) and (7.8f) define the variables of the model.

Link between the variables

It is the pattern of the *variable defined as a maximum* (Section 3.5), in three steps.

- For each machine m and each job j , if the job is assigned to the machine then the processing time of the machine is at least the time of the job:

$$x_{jm} = 1 \implies y_m \geq t_{jm},$$

$$\text{and by contraposition: } y_m < t_{jm} \implies x_{jm} = 0 :$$

if the time of the machine is strictly smaller than t_{jm} , job j cannot be assigned to it. The implication is imposed directly by constraints (7.8d), which give $y_m \geq t_{jm} x_{jm}$: if $x_{jm} = 1$, $y_m \geq t_{jm}$. Conversely, if $y_m < t_{jm}$, then x_{jm} cannot equal 1, since the constraint would force $y_m \geq t_{jm}$; being binary, $x_{jm} = 0$. Since it holds for every j , the family (7.8d) imposes, for every machine, $y_m \geq \max\{t_{jm} x_{jm} : j \in \{1, 2, \dots, n\}\}$.

- Conversely, if no job is assigned to machine m , its processing time is zero:

$$\sum_{j=1}^n x_{jm} = 0 \implies y_m = 0,$$

$$\text{and by contraposition: } y_m > 0 \implies \sum_{j=1}^n x_{jm} > 0.$$

It is not imposed by the constraints (with $x_{jm} = 0$ for every j constraints (7.8d) reduce to $y_m \geq 0$), but it follows from the objective in every optimum: y_m has coefficient $1 > 0$ in the objective; if no job is on the machine and $y_m > 0$, setting $y_m = 0$ keeps the constraints ($0 \geq 0$) and decreases the objective by the strictly positive amount y_m .

- *Synthesis*: in every optimal solution

$$y_m = \max\{t_{jm}x_{jm} : j \in \{1, 2, \dots, n\}\},$$

i.e. y_m is exactly the processing time of the machine. Indeed the first point gives $y_m \geq$ maximum in every feasible solution; if in some solution $y_m > \text{maximum}$, setting y_m equal to the maximum would keep all constraints (7.8d) (by definition of maximum) and decrease the objective by the difference, strictly positive: the solution was not optimal. With no jobs the maximum is 0, and the second point is recovered.

The two bounds built below — the primal one, from the heuristic, and the dual one, from the relaxation — are what the solver works with.

The model in gurobipy

```

1 m = gp.Model("parallel"); m.Params.OutputFlag = 0
2 x = m.addVars(n, k, vtype=GRB.BINARY, name="x")
3 y = m.addVars(k, name="y") #
   continuous, >= 0
4 m.setObjective(y.sum(), GRB.MINIMIZE)
5 m.addConstrs((x.sum(j, "*") == 1 for j in range(n)), name="assign")
6 m.addConstrs((x.sum("*, mm) <= p[mm] for mm in range(k)),
7              name="cardinality")
8 m.addConstrs((-t[j][mm] * x[j, mm] + y[mm] >= 0
9              for j in range(n) for mm in range(k)), name="maximum")
10 m.optimize()
```

The instance

$n = 3$ jobs, $k = 3$ machines, with the times of problem 7.2:

t_{jm}	$m = 1$	$m = 2$	$m = 3$
$j = 1$	6	5	3
$j = 2$	5	10	2
$j = 3$	20	13	10

	$m = 1$	$m = 2$	$m = 3$
p_m	1	2	2

Model (7.8) for the instance has 9 constraints (7.8d), one per pair: $-6x_{11} + y_1 \geq 0$, $-5x_{12} + y_2 \geq 0$, $-3x_{13} + y_3 \geq 0$, $-5x_{21} + y_1 \geq 0$, $-10x_{22} + y_2 \geq 0$, $-2x_{23} + y_3 \geq 0$, $-20x_{31} + y_1 \geq 0$, $-13x_{32} + y_2 \geq 0$, $-10x_{33} + y_3 \geq 0$; the three assignment constraints as in problem 7.2; the cardinalities $x_{11} + x_{21} + x_{31} \leq 1$, $x_{12} + x_{22} + x_{32} \leq 2$, $x_{13} + x_{23} + x_{33} \leq 2$; objective $\min y_1 + y_2 + y_3$.

Constructive heuristic: the primal bound

Next-fit on the cardinalities: machine 1 is filled up to p_1 jobs, then machine 2, and so on; the time of every machine is the maximum of the assigned times.

- **Step 1.** Job 1 on machine 1 (one job $\leq p_1 = 1$): $y[1] = \max(0, 6) = 6$.
- **Step 2.** Machine 1 is full: job 2 on machine 2: $y[2] = \max(0, 10) = 10$.
- **Step 3.** Job 3 on machine 2 (two jobs $\leq p_2 = 2$): $y[2] = \max(10, 13) = 13$.

Solution $\bar{x}_{11} = \bar{x}_{22} = \bar{x}_{32} = 1$, $\bar{y} = (6, 13, 0)$, value 19: $z(MILP) \leq 19$.

It is a constructive heuristic of the family seen in chapter 5: a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

LP relaxation and dual: the dual bound

Relaxing (7.8e) to $x_{jm} \geq 0$, with duals μ_j free (7.8b), $\pi_m \leq 0$ (7.8c, $\leq$ in a minimisation) and $\lambda_{jm} \geq 0$ (7.8d, $\geq$):

Dual of the LP relaxation of model 7.4

$$\max \sum_{j=1}^n \mu_j + \sum_{m=1}^k p_m \pi_m \quad (7.9a)$$

$$\text{subject to } \mu_j + \pi_m - t_{jm} \lambda_{jm} \leq 0, \quad \forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}, \quad (7.9b)$$

$$\sum_{j=1}^n \lambda_{jm} \leq 1, \quad \forall m \in \{1, 2, \dots, k\}, \quad (7.9c)$$

$$\mu_j \geq 0, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.9d)$$

$$\pi_m \leq 0, \quad \forall m \in \{1, 2, \dots, k\}, \quad (7.9e)$$

$$\lambda_{jm} \geq 0, \quad \forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}. \quad (7.9f)$$

Description. The variable μ_j is the value of job j , π_m the (non-positive) price of one of the p_m slots of machine m , and λ_{jm} the price of the link that pushes the maximum y_m above t_{jm} . The objective (7.9a) prices the jobs and the available slots. Constraints (7.9b) are the columns of the x_{jm} : putting job j on machine m satisfies the assignment and takes one slot, but forces the maximum of the machine up to t_{jm} ; the balance cannot be positive, because the x_{jm} do not appear in the objective. Constraints (7.9c) are the columns of the y_m : the maximum of machine m costs 1 in the primal objective, and the prices of the links that push it up cannot be worth more.

Constraint (7.9c) is the reduced cost of y_m : the coefficient 1 in the primal objective limits the sum of the λ_{jm} . **A hand-built dual solution.** Split that unit in equal parts, $\bar{\lambda}_{jm} = 1/n = 1/3$ for every pair, and set $\bar{\pi}_m = 0$; constraints (7.9b) become $\mu_j \leq t_{jm}/3$ for every m :

$$\bar{\mu}_1 = \min\{6, 5, 3\}/3 = 1, \quad \bar{\mu}_2 = \min\{5, 10, 2\}/3 = \frac{2}{3}, \quad \bar{\mu}_3 = \min\{20, 13, 10\}/3 = \frac{10}{3},$$

value $1 + \frac{2}{3} + \frac{10}{3} = 5$. Hence $5 \leq z(MILP) \leq 19$.

What the solver says. $z(LP) = 520/49 = 10.61$ (equal to the optimum of the dual; the relaxation with the bounds coincides, because $x_{jm} \leq 1$ is implied). The integer optimum is $z(MILP) = 15$: job 1 on machine 2, jobs 2 and 3 on machine 3, $\tilde{y} = (0, 5, 10)$. The hand bound (5) is far from the relaxation (10.61): the uniform split $\lambda_{jm} = 1/3$ is the first that comes to mind, not the best — the optimal dual distributes the unit of every machine differently.

LB (next-fit)	UB (hand dual)	$z(LP)$	$z(MILP)$	heuristic gap
19	5	520/49	15	26.7%

Additional considerations

- $x_{jm} \leq 1$ ($n k$ valid inequalities) is implied by (7.8b).
- The direction “no job $\Rightarrow y_m = 0$ ” can be imposed explicitly with an upper bound: with $\bar{t}_m = \max_j t_{jm}$, the constraints

$$\bar{t}_m \sum_{j=1}^n x_{jm} - y_m \geq 0, \quad \forall m \in \{1, 2, \dots, k\}$$

(k constraints) force $y_m = 0$ if the machine is empty and leave y_m room up to $\bar{t}_m$ otherwise. They are *not* valid inequalities (the model allows $y_m > 0$ on an empty machine) but they are *optimality-preserving constraints*: they can be added because the objective minimises $\sum_{m=1}^k y_m$. The distinction between “valid” and “optimality-preserving” is the same as between the two bullets of the link.

Additional modelling questions

Modelling question 7.7 — minimising the time of the slowest machine

The machines all work together and the company wants to finish as early as possible: minimise the *maximum* of the processing times of the machines (the makespan), not the sum. Model and solve.

Modelling question 7.8 — fixed cost if the machine works

Switching a machine on costs $g_m = 4$ euros (once) and one minute of processing costs 1 euro: minimise the total cost. Which link is needed and what is the smallest big-M?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
UB	19	heuristic solution (UB)
LB	5	certified bound (LB)
$z(MILP)$	15	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 26.7\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 93.3\%.$$

The sandwich on variant 4a

Minimising the *maximum* of the times instead of the sum removes the cost from the y_m and adds a family $\nu_m \geq 0$ for the constraints $w \geq y_m$, with the column of w imposing $\sum_m \nu_m \leq 1$. It pays to price a single job, the longest one: the ν maximising $\min_m t_{jm} \nu_m$ makes that product constant, that is ν_m proportional to $1/t_{jm}$, and the bound is the harmonic mean of the times of that job. The recipe turns out to be *optimal* for the relaxation.

	value	what it is
UB	13	heuristic solution
LB	$\frac{260}{59}$	dual certificate built by hand
$z(LP)$	$\frac{260}{59}$	relaxation without the bounds
$z(LP^+)$	$\frac{260}{59}$	relaxation with the bounds
$z(MILP)$	10	optimum of the MILP

7.5 One machine, job classes with setup

Script: `python/fam07_5_classessetup.py`

[Notebook in Colab](#)

Problem 7.5 — classes with setup

A company has $n \in \mathbb{Z}_{\geq 1}$ jobs that can be executed on a machine with an available processing time of $a \in \mathbb{Q}_{>0}$ minutes. For each job $j \in \{1, 2, \dots, n\}$, the value $t_j \in \mathbb{Q}_{>0}$ is the processing time in minutes and the value $r_j \in \mathbb{Q}_{>0}$ is the revenue in euros if the job is executed. The jobs are partitioned into $q \in \mathbb{Z}_{\geq 2}$ classes: for each class $c \in \{1, 2, \dots, q\}$, the set $\mathcal{J}_c \subseteq \{1, 2, \dots, n\}$ contains the jobs of the class. If the machine executes jobs of a class c , a setup cost of $f_c \in \mathbb{Q}_{\geq 0}$ euros is incurred and a setup time of $s_c \in \mathbb{Q}_{\geq 0}$ minutes is needed to prepare the machine. The machine cannot execute jobs in parallel. The company wants to choose the jobs to execute maximising the profit.

The problem in words. We decide which jobs to execute and, consequently, which classes to activate. *The objective:* revenues minus setup costs. *The constraint:* times of the jobs plus setup times of the activated classes within the availability. A knapsack with fixed costs per group: the activation link, this time *disaggregated* from the start.

Data (input of the model)

Symbol	Type	Meaning
n	$\in \mathbb{Z}_{\geq 1}$	number of jobs, $j \in \{1, 2, \dots, n\}$
a	$\in \mathbb{Q}_{>0}$	availability of the machine
t_j, r_j	$\in \mathbb{Q}_{>0}$	time and revenue of job j
q	$\in \mathbb{Z}_{\geq 2}$	number of classes, $c \in \{1, 2, \dots, q\}$
$\mathcal{J}_c$	$\subseteq \{1, 2, \dots, n\}$	jobs of class c (partition)
f_c, s_c	$\in \mathbb{Q}_{\geq 0}$	setup cost and setup time of class c

Decision variables

We introduce the following $n + q$ binary variables:

$$\begin{cases} x_j = 1 \text{ if job } j \text{ is executed, } 0 \text{ otherwise,} & \forall j \in \{1, 2, \dots, n\}, \\ y_c = 1 \text{ if at least one job of class } c \text{ is executed, } 0 \text{ otherwise,} & \forall c \in \{1, 2, \dots, q\}. \end{cases}$$

Model 7.5 — classes with setup

$$\max \sum_{j=1}^n r_j x_j - \sum_{c=1}^q f_c y_c \quad (7.10a)$$

$$\text{subject to } \sum_{j=1}^n t_j x_j + \sum_{c=1}^q s_c y_c \leq a, \quad (7.10b)$$

$$x_j - y_c \leq 0, \quad \forall c \in \{1, 2, \dots, q\}, \forall j \in \mathcal{J}_c, \quad (7.10c)$$

$$x_j \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.10d)$$

$$y_c \in \{0, 1\}, \quad \forall c \in \{1, 2, \dots, q\}. \quad (7.10e)$$

Description of the objective function and constraints

- the linear objective function (7.10a) maximises the profit, the difference between the revenue of the executed jobs and the setup cost of the activated classes;
- the linear constraint (7.10b) ensures that the time of the executed jobs, plus the setup times of the activated classes, does not exceed the availability of the machine (1 linear constraint);
- the linear constraints (7.10c) link the execution variables with the class-activation variables: if a job of a class is executed, the class is activated (n linear constraints, one per job, because the classes partition the jobs);
- the constraints (7.10d) and (7.10e) define the variables of the model.

Link between the variables

Both families are binary: the link constraint is the direct linearisation of a Boolean implication (Section 2.3).

- For each class c , if at least one job of the class is executed, the class is activated:

$$\underbrace{(x_j \text{ OR } x_s \text{ OR } \dots)}_{j,s,\dots \in \mathcal{J}_c} \implies y_c,$$

and by contraposition (De Morgan)

$$\text{NOT } y_c \implies \underbrace{(\text{NOT } x_j \text{ AND } \text{NOT } x_s \text{ AND } \dots)}_{j,s,\dots \in \mathcal{J}_c} :$$

if the class is not activated, none of its jobs is executed.

The implication is equivalent to the Boolean expression

$$\text{NOT}(x_j \text{ OR } x_s \text{ OR } \dots) \text{ OR } y_c;$$

by De Morgan and the distributive property it becomes the CNF

$$(\text{NOT } x_j \text{ OR } y_c) \text{ AND } (\text{NOT } x_s \text{ OR } y_c) \text{ AND } \dots,$$

which translates into the $|\mathcal{J}_c|$ linear constraints $1 - x_j + y_c \geq 1$, i.e. $x_j - y_c \leq 0$ for every $j \in \mathcal{J}_c$: exactly constraints (7.10c). Check in both directions: if $x_j = 1$ for some $j \in \mathcal{J}_c$, the constraint $x_j \leq y_c$ forces $y_c = 1$; if $y_c = 0$, the constraints $x_j \leq y_c$ force $x_j = 0$ for every $j \in \mathcal{J}_c$.

- Conversely, if no job of the class is executed, the class is not activated:

$$\text{NOT}(x_j \text{ OR } x_s \text{ OR } \dots) \implies \text{NOT } y_c,$$

$$\text{by contraposition: } y_c \implies (x_j \text{ OR } x_s \text{ OR } \dots).$$

An activated class has at least one executed job.

It is not imposed by the constraints, but it follows *without loss of optimality* from the objective and the capacity constraint: if $x_j = 0$ for every $j \in \mathcal{J}_c$ and $y_c = 1$, setting $y_c = 0$ stays feasible, frees s_c minutes in constraint (7.10b) and *does not decrease* the objective, because $f_c \geq 0$. Since f_c may be zero, the improvement is not necessarily strict: the correct conclusion is “there always exists an optimum in which classes with no jobs are not activated”, not “in every optimum”. The sign of the data decides the strength of the conclusion.

The two bounds built below — the primal one, from the heuristic, and the dual one, from the relaxation — are what the solver works with.

The model in gurobipy

The classes are lists of indices: $J = [[0, 1], [2, 3], [4, 5, 6]]$.

```
1 m = gp.Model("classes_setup"); m.Params.OutputFlag = 0
2 x = m.addVars(n, vtype=GRB.BINARY, name="x")
3 y = m.addVars(q, vtype=GRB.BINARY, name="y")
4 m.setObjective(gp.quicksum(r[j] * x[j] for j in range(n))
5                 - gp.quicksum(f[c] * y[c] for c in range(q)), GRB.
                   MAXIMIZE)
```

```

6 m.addConstr(gp.quicksum(t[j] * x[j] for j in range(n))
7             + gp.quicksum(s[c] * y[c] for c in range(q)) <= a,
8             name="availability")
9 m.addConstrs((x[j] - y[c] <= 0 for c in range(q) for j in J[c]),
10             name="link")
11 m.optimize()

```

The instance

$n = 7$ jobs, $q = 3$ classes $\mathcal{J}_1 = \{1, 2\}$, $\mathcal{J}_2 = \{3, 4\}$, $\mathcal{J}_3 = \{5, 6, 7\}$, availability $a = 50$:

	$j = 1$	$j = 2$	$j = 3$	$j = 4$	$j = 5$	$j = 6$	$j = 7$
r_j	10	6	8	6	7	9	5
t_j	5	10	8	6	9	5	6

	$c = 1$	$c = 2$	$c = 3$
f_c	10	5	4
s_c	10	12	6

Model (7.10) for the instance:

The model of the instance

$$\begin{aligned}
 & \max 10x_1 + 6x_2 + 8x_3 + 6x_4 + 7x_5 + 9x_6 + 5x_7 - 10y_1 - 5y_2 - 4y_3 \\
 & \text{subject to } 5x_1 + 10x_2 + 8x_3 + 6x_4 + 9x_5 + 5x_6 + 6x_7 + 10y_1 + 12y_2 + 6y_3 \leq 50 \\
 & \quad x_1 \qquad \qquad \qquad -y_1 \qquad \qquad \leq 0 \\
 & \quad \quad x_2 \qquad \qquad \qquad -y_1 \qquad \qquad \leq 0 \\
 & \quad \quad \quad x_3 \qquad \qquad \qquad -y_2 \qquad \leq 0 \\
 & \quad \quad \quad \quad x_4 \qquad \qquad \qquad -y_2 \qquad \leq 0 \\
 & \quad \quad \quad \quad \quad x_5 \qquad \qquad \qquad -y_3 \leq 0 \\
 & \quad \quad \quad \quad \quad \quad x_6 \qquad \qquad \qquad -y_3 \leq 0 \\
 & \quad \quad \quad \quad \quad \quad \quad x_7 \qquad \qquad \qquad -y_3 \leq 0 \\
 & \quad x_1, \quad x_2, \quad x_3, \quad x_4, \quad x_5, \quad x_6, \quad x_7 \qquad \in \{0, 1\} \\
 & \quad \quad \quad \quad \quad \quad y_1, \quad y_2, \quad y_3 \in \{0, 1\}
 \end{aligned}$$

Constructive heuristic: the primal bound

Class by class, job by job: the first job of a class is executed if its time and the setup of the class fit; the following ones if their time fits.

- **Step 1.** Class 1 not active: $s_1 + t_1 = 10 + 5 = 15 \leq 50$; $y[1] = 1$, $x[1] = 1$, $ra = 35$.
- **Step 2.** Class 1 active: $t_2 = 10 \leq 35$; $x[2] = 1$, $ra = 25$.
- **Step 3.** Class 2 not active: $s_2 + t_3 = 12 + 8 = 20 \leq 25$; $y[2] = 1$, $x[3] = 1$, $ra = 5$.
- **Step 4.** Class 2 active: $t_4 = 6 > 5$; job 4 is skipped.
- **Steps 5–7.** Class 3 not active: $s_3 + t_j \in \{15, 11, 12\}$, all > 5 ; jobs 5, 6, 7 are skipped.

Solution $\bar{x} = (1, 1, 1, 0, 0, 0, 0)$, $\bar{y} = (1, 1, 0)$, profit $10 + 6 + 8 - 10 - 5 = 9$: $z(MILP) \geq 9$.

It is a constructive heuristic of the family seen in chapter 5: a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

LP relaxation and dual: the dual bound

Relaxing $x_j \geq 0$ and $y_c \geq 0$, with duals $\pi \geq 0$ for constraint (7.10b) and $\lambda_j \geq 0$ for each constraint (7.10c):

Dual of the LP relaxation of model 7.5

$$\min \quad a\pi \quad (7.11a)$$

$$\text{subject to} \quad t_j\pi + \lambda_j \geq r_j, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.11b)$$

$$s_c\pi - \sum_{j \in \mathcal{J}_c} \lambda_j \geq -f_c, \quad \forall c \in \{1, 2, \dots, q\}, \quad (7.11c)$$

$$\pi \geq 0, \quad (7.11d)$$

$$\lambda_j \geq 0, \quad \forall j \in \{1, 2, \dots, n\}. \quad (7.11e)$$

Description. The variable π is the price of one unit of time and λ_j the price of the link between job j and the setup of its class. The objective (7.11a) is the value of all the available time a . Constraints (7.11b) are the columns of the x_j : the time t_j that job j consumes, priced at π , plus the price of its link, must cover the revenue r_j . Constraints (7.11c) are the columns of the y_c : the setup of class c consumes s_c of time and releases the links of all the jobs in the class; the balance cannot fall below $-f_c$, that is, below the cost of that setup.

For the instance: $\min 50\pi$ subject to $5\pi + \lambda_1 \geq 10$, $10\pi + \lambda_2 \geq 6$, $8\pi + \lambda_3 \geq 8$, $6\pi + \lambda_4 \geq 6$, $9\pi + \lambda_5 \geq 7$, $5\pi + \lambda_6 \geq 9$, $6\pi + \lambda_7 \geq 5$, $10\pi - \lambda_1 - \lambda_2 \geq -10$, $12\pi - \lambda_3 - \lambda_4 \geq -5$, $6\pi - \lambda_5 - \lambda_6 - \lambda_7 \geq -4$, $\pi, \lambda \geq 0$.

A hand-built dual solution. With $\bar{\lambda}_j = 0$ constraints (7.11b) ask $\pi \geq r_j/t_j$ for every job, and constraints (7.11c) are satisfied for every $\pi \geq 0$:

$$\bar{\pi} = \max_j \frac{r_j}{t_j} = \max\left\{\frac{10}{5}, \frac{6}{10}, \frac{8}{8}, \frac{6}{6}, \frac{7}{9}, \frac{9}{5}, \frac{5}{6}\right\} = 2,$$

value $50 \cdot 2 = 100$: $9 \leq z(MILP) \leq 100$. The reading: π is the value of one minute of machine; if every minute were worth as much as the most profitable job per minute, the whole machine would be worth 100 euros. A coarse bound, as “knapsack” bounds often are: it ignores setups and costs.

What the solver says. $z(LP) = 425/13 = 32.7$ (optimum of the dual, reached with $\pi = \frac{17}{26}$ and some $\lambda_j > 0$: the solver does not settle for $\lambda = 0$); with the upper bounds, $z(LP^+) = 329/13 = 25.3$. The integer optimum is $z(MILP) = 21$: classes 2 and 3 activated, jobs 3, 4, 5, 6 executed ($8 + 6 + 9 + 5 + 12 + 6 = 46 \leq 50$), profit $30 - 9$. The heuristic, which opens the classes in the given order and spends 10 + 10 minutes on class 1, stays at 9: a 57% gap, the worst of the family — the scanning order matters, and a constructive heuristic that orders the classes by potential profit would do better.

LB (constructive heuristic)	UB (hand dual)	$z(LP)$	$z(LP^+)$	$z(MILP)$	heuristic gap
9	100	425/13	329/13	21	57.1%

Additional considerations

- $y_c \leq 1$ (q) are valid inequalities that strengthen the relaxation; $x_j \leq 1$ (n) are implied by $y_c \leq 1$ and (7.10c).
- The direction “no job $\Rightarrow$ class not activated” is the clause

$$\text{NOT } y_c \text{ OR } x_j \text{ OR } x_s \text{ OR } \dots,$$

i.e. $\sum_{j \in \mathcal{J}_c} x_j \geq y_c$ (q constraints): not valid, but it can be added without losing the optimum (for $f_c \geq 0$ and $s_c \geq 0$).

- The aggregated form of the link, $\sum_{j \in \mathcal{J}_c} x_j \leq |\mathcal{J}_c| y_c$, has the same set of integer solutions and a weaker relaxation: here the model is born already in the disaggregated form, the right one.

Additional modelling questions

Modelling question 7.9 — a single class

Changing class requires a cleaning of the machine the company wants to avoid altogether: at most one class can be activated. Model and solve.

Modelling question 7.10 — a class subordinate to another

The jobs of class 3 use a tool that is mounted during the setup of class 1: class 3 can be activated only if class 1 is activated too. Write the constraint and find the new optimum.

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
LB	9	heuristic solution (LB)
UB	100	certified bound (UB)
$z(MILP)$	21	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|LB - z(MILP)|}{|z(MILP)|} = 57.1\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|LB - UB|}{|z(MILP)|} = 433.3\%.$$

The sandwich on variant 5a

"At most one class activated" adds a variable $\theta \geq 0$ with right-hand side 1 to the dual: it enters the objective and loosens the columns of the y_c . Once the price of time π is fixed, the λ_j and θ are forced by the dual constraints, so the recipe reduces to searching π among the ratios r_j/t_j and keeping the lowest value. The bound improves a great deal over the recipe of the base problem.

	value	what it is
UB	$\frac{157}{5}$	dual certificate built by hand
LB	17	heuristic solution
$z(LP)$	17	relaxation without the bounds
$z(LP^+)$	17	relaxation with the bounds
$z(MILP)$	17	optimum of the MILP

7.6 Classes with completion bonus and “if and only if” reduction

Script: `python/fam07_6_classesbonus.py`

[Notebook in Colab](#)

Problem 7.6 — completion bonus

A company has $n \in \mathbb{Z}_{\geq 1}$ jobs that can be executed on a machine with an available processing time of $a \in \mathbb{Q}_{>0}$ minutes. For each job $j \in \{1, 2, \dots, n\}$, $t_j \in \mathbb{Q}_{>0}$ is the processing time in minutes and $r_j \in \mathbb{Q}_{>0}$ the revenue in euros if executed. The jobs are partitioned into $q \in \mathbb{Z}_{\geq 2}$ classes $\mathcal{J}_1, \dots, \mathcal{J}_q$. The availability of the machine is reduced by $u \in \mathbb{Q}_{>0}$ minutes *if and only if* the machine executes jobs of at least two different classes. For each class $c \in \{1, 2, \dots, q\}$, an extra revenue of $v_c \in \mathbb{Q}_{>0}$ euros is obtained *if and only if* all the jobs of the class are executed. The machine does not execute jobs in parallel. The company wants to choose the jobs to execute maximising the total revenue.

The problem in words. We decide which jobs to execute. *The objective:* revenues of the jobs plus bonuses of the complete classes. *The constraint:* times of the jobs, plus the reduction u if the classes involved are at least two, within the availability. Two “if and only if”s: one rewards (completion), one penalises (mixing). We shall see that for each it suffices to impose one direction: the objective imposes the other.

Data (input of the model)

Symbol	Type	Meaning
n	$\in \mathbb{Z}_{\geq 1}$	number of jobs, $j \in \{1, 2, \dots, n\}$
a	$\in \mathbb{Q}_{>0}$	availability of the machine
t_j, r_j	$\in \mathbb{Q}_{>0}$	time and revenue of job j
q	$\in \mathbb{Z}_{\geq 2}$	number of classes, $c \in \{1, 2, \dots, q\}$
$\mathcal{J}_c$	$\subseteq \{1, 2, \dots, n\}$	jobs of class c (partition)
v_c	$\in \mathbb{Q}_{>0}$	bonus if all the jobs of class c are executed
u	$\in \mathbb{Q}_{>0}$	reduction of the availability if jobs of at least two classes are executed

Decision variables

We introduce the following $n + q + 1$ binary variables:

$$\begin{cases} x_j = 1 \text{ if job } j \text{ is executed, } 0 \text{ otherwise,} & \forall j \in \{1, 2, \dots, n\}, \\ y_c = 1 \text{ if all the jobs of class } c \text{ are executed, } 0 \text{ otherwise,} & \forall c \in \{1, 2, \dots, q\}, \\ z = 1 \text{ if jobs of at least two different classes are executed, } 0 \text{ otherwise.} \end{cases}$$

Model 7.6 — completion bonus

$$\max \sum_{j=1}^n r_j x_j + \sum_{c=1}^q v_c y_c \quad (7.12a)$$

$$\text{subject to } x_j - y_c \geq 0, \quad \forall c \in \{1, 2, \dots, q\}, \forall j \in \mathcal{J}_c, \quad (7.12b)$$

$$x_j + x_i - z \leq 1, \quad \forall c, g \in \{1, 2, \dots, q\}, c < g, \forall j \in \mathcal{J}_c, \forall i \in \mathcal{J}_g, \quad (7.12c)$$

$$\sum_{j=1}^n t_j x_j + u z \leq a, \quad (7.12d)$$

$$x_j \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.12e)$$

$$y_c \in \{0, 1\}, \quad \forall c \in \{1, 2, \dots, q\}, \quad (7.12f)$$

$$z \in \{0, 1\}. \quad (7.12g)$$

Description of the objective function and constraints

- the linear objective function (7.12a) maximises the total revenue, the sum of the revenues of the executed jobs and of the bonuses of the complete classes;
- the linear constraints (7.12b) link the execution variables with the completion variables: if a class is declared complete, all its jobs are executed (n linear constraints);
- the linear constraints (7.12c) link the execution variables with the mixing variable: if two jobs of different classes are executed, then $z = 1$ ($\sum_{c < g} |\mathcal{J}_c| |\mathcal{J}_g|$ linear constraints, one per pair of jobs of different classes);
- the linear constraint (7.12d) ensures that the time of the executed jobs, plus the reduction u when classes are mixed, does not exceed the availability (1 linear constraint);
- the constraints (7.12e)–(7.12g) define the variables of the model.

Link between the variables

Four implications: two for y_c , two for z . In each pair one is imposed by the constraint, the other by the optimum — and that is how an “if and only if” is modelled (Section 3.10).

- For each class c , if not all the jobs of the class are executed, the class is not complete:

$$\text{NOT}(x_j \text{ AND } x_i \text{ AND } \dots) \implies \text{NOT } y_c,$$

$$\text{by contraposition: } y_c \implies (x_j \text{ AND } x_i \text{ AND } \dots),$$

with $j, i, \dots \in \mathcal{J}_c$. The Boolean expression becomes, by distributivity, the CNF

$$(x_j \text{ AND } x_i \text{ AND } \dots) \text{ OR NOT } y_c \\ \equiv (x_j \text{ OR NOT } y_c) \text{ AND } (x_i \text{ OR NOT } y_c) \text{ AND } \dots,$$

i.e. $x_j + (1 - y_c) \geq 1$: these are constraints (7.12b), $x_j - y_c \geq 0$ for every $j \in \mathcal{J}_c$. Check: if $y_c = 1$, every constraint $x_j \geq y_c$ forces $x_j = 1$; if some $x_j = 0$ with $j \in \mathcal{J}_c$, the constraint $x_j \geq y_c$ forces $y_c = 0$.

- Conversely, if all the jobs of the class are executed, the class is complete:

$$(x_j \text{ AND } x_i \text{ AND } \dots) \implies y_c, \\ \text{contrapositive: NOT } y_c \implies (\text{NOT } x_j \text{ OR NOT } x_i \text{ OR } \dots).$$

Not imposed by the constraints, it follows from the objective in every optimum: since $v_c > 0$, if all the x_j of the class equal 1 and $y_c = 0$, setting $y_c = 1$ stays feasible (constraints (7.12b) ask $x_j \geq 1$, true) and increases the objective by v_c . Hence in every optimum the bonus is collected as soon as the class is complete.

- For each pair of jobs j, i of different classes, if both are executed then $z = 1$:

$$x_j \text{ AND } x_i \implies z, \\ \text{contrapositive: NOT } z \implies \text{NOT } (x_j \text{ AND } x_i).$$

If $z = 0$, two jobs of different classes are not both executed. Boolean expression

$$\text{NOT } (x_j \text{ AND } x_i) \text{ OR } z \text{ and, by De Morgan, NOT } x_j \text{ OR NOT } x_i \text{ OR } z,$$

i.e. $(1 - x_j) + (1 - x_i) + z \geq 1$: these are constraints (7.12c), $x_j + x_i - z \leq 1$. Check: if $x_j = x_i = 1$ the constraint forces $z = 1$; if $z = 0$ it becomes $x_j + x_i \leq 1$.

- Conversely, if the executed jobs all lie in one class, $z = 0$; by contraposition, $z = 1$ implies that for every class c at least one executed job lies outside $\mathcal{J}_c$. Not imposed by the constraints, it follows from the capacity constraint *without loss of optimality*: if the executed jobs lie in a single class and $z = 1$, setting $z = 0$ stays feasible (constraints (7.12c) become $x_j + x_i \leq 1$, true because no mixed pair is executed), frees $u > 0$ minutes in constraint (7.12d) and does not change the objective, where z does not appear. Hence there always exists an optimum with $z = 1$ only if classes are mixed.

The two bounds built below — the primal one, from the heuristic, and the dual one, from the relaxation — are what the solver works with.

The model in gurobipy

```
1 pairs = [(j, i) for c in range(q) for g in range(c + 1, q)
2           for j in J[c] for i in J[g]]
3 m = gp.Model("classes_bonus"); m.Params.OutputFlag = 0
4 x = m.addVars(n, vtype=GRB.BINARY, name="x")
5 y = m.addVars(q, vtype=GRB.BINARY, name="y")
```

```

6 z = m.addVar(vtype=GRB.BINARY, name="z")
7 m.setObjective(gp.quicksum(r[j] * x[j] for j in range(n))
8               + gp.quicksum(v[c] * y[c] for c in range(q)), GRB.
               MAXIMIZE)
9 m.addConstrs((x[j] - y[c] >= 0 for c in range(q) for j in J[c]),
10             name="all")
11 m.addConstrs((x[j] + x[i] - z <= 1 for (j, i) in pairs), name="mixed")
12 m.addConstr(gp.quicksum(t[j] * x[j] for j in range(n)) + u * z <= a,
13             name="availability")
14 m.optimize()

```

The instance

$n = 6$ jobs, $q = 3$ classes $\mathcal{J}_1 = \{1, 2\}$, $\mathcal{J}_2 = \{3, 4\}$, $\mathcal{J}_3 = \{5, 6\}$, $a = 50$, $u = 10$:

	$j = 1$	$j = 2$	$j = 3$	$j = 4$	$j = 5$	$j = 6$		$c = 1$	$c = 2$	$c = 3$
r_j	10	5	20	12	10	22	v_c	5	4	10
t_j	5	15	25	15	10	38				

The model for the instance has objective $\max 10x_1 + 5x_2 + 20x_3 + 12x_4 + 10x_5 + 22x_6 + 5y_1 + 4y_2 + 10y_3$, six constraints (7.12b) ($x_1 \geq y_1$, $x_2 \geq y_1$, $x_3 \geq y_2$, $x_4 \geq y_2$, $x_5 \geq y_3$, $x_6 \geq y_3$), twelve constraints (7.12c) ($x_1 + x_3 - z \leq 1$, $x_1 + x_4 - z \leq 1$, $x_2 + x_3 - z \leq 1$, $x_2 + x_4 - z \leq 1$, $x_1 + x_5 - z \leq 1$, $x_1 + x_6 - z \leq 1$, $x_2 + x_5 - z \leq 1$, $x_2 + x_6 - z \leq 1$, $x_3 + x_5 - z \leq 1$, $x_3 + x_6 - z \leq 1$, $x_4 + x_5 - z \leq 1$, $x_4 + x_6 - z \leq 1$) and the capacity constraint $5x_1 + 15x_2 + 25x_3 + 15x_4 + 10x_5 + 38x_6 + 10z \leq 50$.

Constructive heuristic: the primal bound

Class by class; from the second class on, the first executed job also pays the reduction u ; at the end of a class, if all its jobs are executed, the bonus is collected.

- **Steps 1–2.** Class 1: $t_1 = 5 \leq 50$, $x[1] = 1$, $ra = 45$; $t_2 = 15 \leq 45$, $x[2] = 1$, $ra = 30$. Class complete: $y[1] = 1$.
- **Step 3.** Class 2, reduction not applied yet: $t_3 + u = 35 > 30$, job 3 is skipped.
- **Step 4.** $t_4 + u = 25 \leq 30$: $x[4] = 1$, $z = 1$, $ra = 5$. Class 2 incomplete.
- **Steps 5–6.** Class 3: $t_5 = 10 > 5$ and $t_6 = 38 > 5$, skipped.

Solution $\bar{x} = (1, 1, 0, 1, 0, 0)$, $\bar{y} = (1, 0, 0)$, $\bar{z} = 1$, revenue $10 + 5 + 12 + 5 = 32$: $z(MILP) \geq 32$. It is a constructive heuristic of the family seen in chapter 5: a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

LP relaxation and dual: the dual bound

Relaxing all the binaries to ≥ 0 , with duals $\pi_j \leq 0$ for constraints (7.12b) ($\geq$ in a maximisation), $\lambda_{ji} \geq 0$ for constraints (7.12c) and $\mu \geq 0$ for constraint (7.12d):

Dual of the LP relaxation of model 7.6

$$\min \sum_{c < g} \sum_{j \in \mathcal{J}_c} \sum_{i \in \mathcal{J}_g} \lambda_{ji} + a \mu \quad (7.13a)$$

$$\text{subject to } \pi_j + \sum_{i \notin \mathcal{J}_c} \lambda_{ji} + t_j \mu \geq r_j, \quad \forall c \in \{1, 2, \dots, q\}, \forall j \in \mathcal{J}_c, \quad (7.13b)$$

$$- \sum_{j \in \mathcal{J}_c} \pi_j \geq v_c, \quad \forall c \in \{1, 2, \dots, q\}, \quad (7.13c)$$

$$- \sum_{c < g} \sum_{j \in \mathcal{J}_c} \sum_{i \in \mathcal{J}_g} \lambda_{ji} + u \mu \geq 0, \quad (7.13d)$$

$$\pi_j \leq 0, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.13e)$$

$$\lambda_{ji} \geq 0, \quad \forall c, g \in \{1, 2, \dots, q\}, c < g, \forall j \in \mathcal{J}_c, \forall i \in \mathcal{J}_g, \quad (7.13f)$$

$$\mu \geq 0. \quad (7.13g)$$

Description. The variable π_j is the (non-positive) price of the link "if the class is rewarded, then job j must be done", λ_{ji} the price of the constraint on the mixed pair (j, i) , and μ the price of one unit of time. The objective (7.13a) adds up the prices of all the mixed pairs and the value of the available time a . Constraints (7.13b) are the columns of the x_j : doing job j releases the link with its class, touches all the mixed-pair constraints that contain it and consumes t_j of time; the total must cover the revenue r_j . Constraints (7.13c) are the columns of the y_c : the bonus v_c of class c must be covered by the links of the jobs that make it up. Constraint (7.13d) is the column of z : the variable that flags the presence of two different classes releases all the mixed-pair constraints and consumes u of time.

(In (7.13b) λ_{ji} denotes the variable of the pair $\{j, i\}$, whatever the order of the two indices.)

A hand-built dual solution. Constraints (7.13c) ask the π_j of every class to "cover" the bonus: it suffices to load it on a single job per class, $\bar{\pi}_1 = -5$, $\bar{\pi}_3 = -4$, $\bar{\pi}_5 = -10$, the others 0. With $\bar{\lambda} = 0$ constraint (7.13d) is satisfied for every $\mu \geq 0$ and constraints (7.13b) ask $\mu \geq (r_j - \bar{\pi}_j)/t_j$:

$$\bar{\mu} = \max \left\{ \frac{10+5}{5}, \frac{5}{15}, \frac{20+4}{25}, \frac{12}{15}, \frac{10+10}{10}, \frac{22}{38} \right\} = \max \left\{ 3, \frac{1}{3}, \frac{24}{25}, \frac{4}{5}, 2, \frac{11}{19} \right\} = 3,$$

value $a\bar{\mu} = 150$: $32 \leq z(MILP) \leq 150$. The bound is dominated by job 1, short and with the bonus loaded on it: spreading the bonus over the two jobs of the class ($\pi_1 = \pi_2 = -2.5$) would lower it — a good exercise.

What the solver says. $z(LP) = z(LP^+) = 5280/113 = 46.7$. The integer optimum is $z(MILP) = 42$: class 3 alone, complete, jobs 5 and 6 ($10 + 38 = 48 \leq 50$, $z = 0$), revenue $10 + 22 + 10$. The heuristic, started from class 1, stops at 32: a 24% gap.

LB (constructive heuristic)	UB (hand dual)	$z(LP)$	$z(MILP)$	heuristic gap
32	150	5280/113	42	23.8%

Additional considerations

- The optimality direction for y_c can be imposed explicitly with the clause

$$\text{NOT } x_j \text{ OR NOT } x_i \text{ OR } \dots \text{ OR } y_c,$$

i.e.

$$\sum_{j \in \mathcal{J}_c} x_j - y_c \leq |\mathcal{J}_c| - 1, \quad \forall c \in \{1, 2, \dots, q\}$$

(q optimality-preserving constraints, not valid: the model allows the complete class with $y_c = 0$).

- The optimality direction for z : for every class c , $z \leq \sum_{j \notin \mathcal{J}_c} x_j$ (q constraints), from $z \implies (x_j \text{ OR } x_i \text{ OR } \dots)$ over the jobs outside $\mathcal{J}_c$. These too preserve the optimum without being valid.
- Constraints (7.12c) are as many as the mixed pairs. An aggregated form with class variables, $w_c \geq x_j$ for $j \in \mathcal{J}_c$ and $\sum_{c=1}^q w_c - 1 \leq (q-1)z$, uses q more variables and $n+1$ constraints instead of the pairs: same set of integer solutions, different relaxation. Comparing them is question 7.6.2.

Additional modelling questions

Modelling question 7.11 — at least one job per class

A union agreement requires executing at least one job of every class. Model, say what happens to z and find the new optimum.

Modelling question 7.12 — penalty for a class started and not finished

Starting a class without completing it costs $w = 3$ euros of reorganisation. Model the penalty: which new variable is needed, and in which direction must its link be proved?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
LB	32	heuristic solution (LB)
UB	150	certified bound (UB)
$z(MILP)$	42	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|LB - z(MILP)|}{|z(MILP)|} = 23.8\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|LB - UB|}{|z(MILP)|} = 281.0\%.$$

The sandwich on variant 6a

"At least one job per class" adds a family $\omega_c \leq 0$ to the dual, one per class, which enters the objective: being non-positive, it *lowers* it. Each ω_c is pushed down as far as the columns of that class allow. The heuristic starts from the shortest job of each class — so it is feasible by construction — and then fills the remaining time.

	value	what it is
UB	107	dual certificate built by hand
LB	32	heuristic solution
$z(LP)$	$\frac{309}{7}$	relaxation without the bounds
$z(LP^+)$	$\frac{309}{7}$	relaxation with the bounds
$z(MILP)$	40	optimum of the MILP

7.7 Total tardiness on one machine: sequencing with big-M

Script: `python/fam07_7_tardiness.py`

[Notebook in Colab](#)

Problem 7.7 — total tardiness

A company needs to process $n \in \mathbb{Z}_{\geq 1}$ jobs on a single machine. For each job $j \in \{1, 2, \dots, n\}$, $t_j \in \mathbb{Q}_{>0}$ is the processing time in minutes and $d_j \in \mathbb{Q}_{>0}$ the due date in minutes. The tardiness of job j is $\tau_j = \max\{0, \kappa_j - d_j\}$, where κ_j is the completion time. The machine processes one job at a time. The company wants a sequence of the jobs that minimises the total tardiness.

The problem in words. *We decide* the order of the jobs. *The objective:* the sum of the tardiness values. *The constraints:* for every pair, one of the two precedes the other; whoever comes later finishes at least t minutes after the completion of whoever comes first. The order is a *disjunction* ("either j before i or i before j "): it is linearised with a binary variable and a big-M.

Data (input of the model)

Symbol	Type	Meaning
n	$\in \mathbb{Z}_{\geq 1}$	number of jobs, $j \in \{1, 2, \dots, n\}$
t_j	$\in \mathbb{Q}_{>0}$	processing time of job j
d_j	$\in \mathbb{Q}_{>0}$	due date of job j
M	$\in \mathbb{Q}_{>0}$	big-M constant, $M = \sum_{j=1}^n t_j$

Decision variables

We introduce $n(n-1)$ binary precedence variables and $2n$ non-negative variables:

$$\begin{cases} s_{ji} = 1 \text{ if job } j \text{ precedes job } i, 0 \text{ otherwise,} & \forall j, i \in \{1, 2, \dots, n\}, j \neq i, \\ \kappa_j = \text{completion time of job } j, & \forall j \in \{1, 2, \dots, n\}, \\ \tau_j = \text{tardiness of job } j, & \forall j \in \{1, 2, \dots, n\}. \end{cases}$$

Model 7.7 — total tardiness

$$\min \sum_{j=1}^n \tau_j \quad (7.14a)$$

$$\text{subject to } s_{ji} + s_{ij} = 1, \quad \forall j, i \in \{1, 2, \dots, n\}, j < i, \quad (7.14b)$$

$$-M s_{ji} - \kappa_j + \kappa_i \geq t_i - M, \quad \forall j, i \in \{1, 2, \dots, n\}, j \neq i, \quad (7.14c)$$

$$-\kappa_j + \tau_j \geq -d_j, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.14d)$$

$$\kappa_j \geq t_j, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.14e)$$

$$s_{ji} \in \{0, 1\}, \quad \forall j, i \in \{1, 2, \dots, n\}, j \neq i, \quad (7.14f)$$

$$\kappa_j \geq 0, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.14g)$$

$$\tau_j \geq 0, \quad \forall j \in \{1, 2, \dots, n\}. \quad (7.14h)$$

Description of the objective function and constraints

- the linear objective function (7.14a) minimises the total tardiness;
- the linear constraints (7.14b) link the precedence variables of every pair: either j precedes i or i precedes j ($n(n-1)/2$ linear constraints);
- the linear constraints (7.14c) link the precedence variables with the completion variables through the big-M: if j precedes i , job i cannot be completed before j is completed and i has been processed ($n(n-1)$ linear constraints). A valid value is $M = \sum_{j=1}^n t_j$: there always exists an optimal sequence without idle time, hence no completion needs to exceed the total time;
- the linear constraints (7.14d), together with constraints (7.14h), link the completions with the tardiness values and correctly define the tardiness of every job (n linear constraints);
- the linear constraints (7.14e) impose that every job completes no earlier than its processing time (n linear constraints);
- the constraints (7.14f)–(7.14h) define the variables of the model.

Link between the variables

- For each pair j, i : if j precedes i , the completion of i is at least the completion of j plus the time of i :

$$s_{ji} = 1 \implies \kappa_i \geq \kappa_j + t_i,$$

$$\text{and by contraposition: } \kappa_i < \kappa_j + t_i \implies s_{ji} = 0.$$

It is imposed with the big-M: $\kappa_i \geq \kappa_j + t_i - M(1 - s_{ji})$, i.e. constraints (7.14c). If $s_{ji} = 1$ the constraint becomes $\kappa_i \geq \kappa_j + t_i$; if $s_{ji} = 0$ it becomes $\kappa_i \geq \kappa_j + t_i - M$, always true for the feasible values of κ when M is large enough: with $M = \sum_{j=1}^n t_j$

and completions within M , the right-hand side is $\leq t_i \leq \kappa_i$. This is the pattern of Section 3.8: the big-M “switches off” the constraint.

- For each job j , the tardiness variable satisfies the quantitative bound

$$\tau_j \geq \max\{0, \kappa_j - d_j\},$$

imposed directly by constraints (7.14d) ($\tau_j \geq \kappa_j - d_j$) and (7.14h) ($\tau_j \geq 0$), whatever κ_j and d_j : there is no implication to prove, the link *is* the inequality. There is instead an optimality implication: if the job finishes on time, the tardiness is zero:

$$\kappa_j \leq d_j \implies \tau_j = 0,$$

$$\text{contrapositive: } \tau_j > 0 \implies \kappa_j > d_j.$$

The constraints do not impose it (with $\kappa_j \leq d_j$ they only ask $\tau_j \geq 0$), but it follows from the objective in every optimum: if $\kappa_j \leq d_j$ and $\tau_j > 0$, setting $\tau_j = 0$ stays feasible ($0 \geq \kappa_j - d_j$ because $\kappa_j - d_j \leq 0$) and decreases the objective by $\tau_j > 0$.

- *Synthesis*: in every optimum $\tau_j = \max\{0, \kappa_j - d_j\}$, exactly the tardiness. The lower bound holds in every feasible solution; if τ_j exceeded it, lowering it to the maximum would keep both constraints and decrease the objective by the difference, positive. No additional constraint is needed.

The two bounds built below — the primal one, from the heuristic, and the dual one, from the relaxation — are what the solver works with.

The model in gurobipy

```

1 M = sum(t)
2 m = gp.Model("tardiness"); m.Params.OutputFlag = 0
3 s = m.addVars([(j, i) for j in range(n) for i in range(n) if j != i],
4               vtype=GRB.BINARY, name="s")
5 kappa = m.addVars(n, name="kappa"); tau = m.addVars(n, name="tau")
6 m.setObjective(tau.sum(), GRB.MINIMIZE)
7 m.addConstrs((s[j, i] + s[i, j] == 1 for j in range(n)
8               for i in range(j + 1, n)), name="order")
9 m.addConstrs((-M * s[j, i] - kappa[j] + kappa[i] >= t[i] - M
10              for j in range(n) for i in range(n) if j != i),
11              name="precedence")
12 m.addConstrs((-kappa[j] + tau[j] >= -d[j] for j in range(n)),
13              name="tardiness")
14 m.addConstrs((kappa[j] >= t[j] for j in range(n)), name="start")
15 m.optimize()

```

The instance

$n = 3$ jobs, $M = 5 + 4 + 6 = 15$:

	$j = 1$	$j = 2$	$j = 3$
t_j	5	4	6
d_j	3	4	10

Model (7.14) for the instance:

Model 7.7 on the instance

$$\begin{array}{llllllll}
 \min & & & & & & & \tau_1 + \tau_2 + \tau_3 \\
 \text{subject to} & s_{12} & +s_{21} & & & & & = 1 \\
 & & s_{13} & & +s_{31} & & & = 1 \\
 & & & s_{23} & & +s_{32} & & = 1 \\
 & -15s_{12} & & & & -\kappa_1 + \kappa_2 & & \geq -11 \\
 & & -15s_{13} & & & -\kappa_1 & +\kappa_3 & \geq -9 \\
 & & & -15s_{21} & & +\kappa_1 - \kappa_2 & & \geq -10 \\
 & & & & -15s_{23} & & -\kappa_2 + \kappa_3 & \geq -9 \\
 & & & & & -15s_{31} & +\kappa_1 & -\kappa_3 \geq -10 \\
 & & & & & & -15s_{32} & +\kappa_2 - \kappa_3 \geq -11 \\
 & & & & & & & -\kappa_1 & +\tau_1 \geq -3 \\
 & & & & & & & -\kappa_2 & +\tau_2 \geq -4 \\
 & & & & & & & & -\kappa_3 & +\tau_3 \geq -10 \\
 & & & & & & & & \kappa_1 & \geq 5 \\
 & & & & & & & & \kappa_2 & \geq 4 \\
 & & & & & & & & \kappa_3 & \geq 6 \\
 & s_{12}, & s_{13}, & s_{21}, & s_{23}, & s_{31}, & s_{32} & & & \in \{0, 1\} \\
 & & & & & & & \kappa_1, \kappa_2, \kappa_3 & & \geq 0 \\
 & & & & & & & & \tau_1, \tau_2, \tau_3 & \geq 0
 \end{array}$$

Constructive heuristic: the primal bound

The simplest: process the jobs in the given order $1 \rightarrow 2 \rightarrow 3$ (FIFO); a better rule orders by increasing due date (EDD, *earliest due date*), which here coincides.

- **Step 1.** Job 1: $\kappa_1 = 5$, $\tau_1 = \max\{0, 5 - 3\} = 2$.
- **Step 2.** Job 2: $\kappa_2 = 5 + 4 = 9$, $\tau_2 = \max\{0, 9 - 4\} = 5$.
- **Step 3.** Job 3: $\kappa_3 = 9 + 6 = 15$, $\tau_3 = \max\{0, 15 - 10\} = 5$.

Solution $\bar{s}_{12} = \bar{s}_{13} = \bar{s}_{23} = 1$ (the others 0), $\bar{\kappa} = (5, 9, 15)$, $\bar{\tau} = (2, 5, 5)$, value 12: $z(MILP) \leq 12$. It is a constructive heuristic of the family seen in chapter 5: a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

LP relaxation and dual: the dual bound

Relaxing (7.14f) to $s_{ji} \geq 0$, with duals α_{ji} free (7.14b), $\beta_{ji} \geq 0$ (7.14c), $\gamma_j \geq 0$ (7.14d) and $\delta_j \geq 0$ (7.14e):

Dual of the LP relaxation of model 7.7

$$\max \sum_{j < i} \alpha_{ji} + \sum_{j \neq i} (t_i - M) \beta_{ji} \quad (7.15a)$$

$$- \sum_{j=1}^n d_j \gamma_j + \sum_{j=1}^n t_j \delta_j$$

$$\text{subject to } \alpha_{ji} - M \beta_{ji} \leq 0, \quad \forall i, j \in \{1, 2, \dots, n\}, j < i, \quad (7.15b)$$

$$\alpha_{ji} - M \beta_{ij} \leq 0, \quad \forall i, j \in \{1, 2, \dots, n\}, j < i, \quad (7.15c)$$

$$- \sum_{i \neq j} \beta_{ji} + \sum_{i \neq j} \beta_{ij} - \gamma_j + \delta_j \leq 0, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.15d)$$

$$\gamma_j \leq 1, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.15e)$$

$$\alpha_{ji} \geq 0, \quad \forall i, j \in \{1, 2, \dots, n\}, j < i, \quad (7.15f)$$

$$\beta_{ji} \geq 0, \quad \forall i, j \in \{1, 2, \dots, n\}, j \neq i, \quad (7.15g)$$

$$\gamma_j \geq 0, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.15h)$$

$$\delta_j \geq 0, \quad \forall j \in \{1, 2, \dots, n\}. \quad (7.15i)$$

Description. The variable α_{ji} is the price of the ordering choice between jobs j and i , β_{ji} the price of the precedence written with the big-M, γ_j the price of the constraint that defines the tardiness of job j , and δ_j that of its minimum completion time. The objective (7.15a) collects these prices with the right-hand sides of the corresponding constraints. Constraints (7.15b) and (7.15c) are the columns of the s_{ji} and of the s_{ij} : the ordering choice enters the equality with coefficient 1 and the precedence with $-M$, and the balance cannot be positive because those variables do not appear in the objective. Constraints (7.15d) are the columns of the κ_j : the completion time enters the precedences with opposite signs, the tardiness with -1 and the minimum time with $+1$. Constraints (7.15e) are the columns of the τ_j : tardiness costs 1 in the primal objective, so its price cannot exceed 1.

For the instance the objective is $\max \alpha_{12} + \alpha_{13} + \alpha_{23} - 11\beta_{12} - 9\beta_{13} - 10\beta_{21} - 9\beta_{23} - 10\beta_{31} - 11\beta_{32} - 3\gamma_1 - 4\gamma_2 - 10\gamma_3 + 5\delta_1 + 4\delta_2 + 6\delta_3$.

A hand-built dual solution. Every β_{ji} has coefficient $t_i - M < 0$: it pays to keep them at zero, and then (7.15b) and (7.15c) impose $\alpha = 0$. What is left is γ and δ , with $\delta_j \leq \gamma_j \leq 1$ from constraints (7.15d) and (7.15e): every job contributes at most $t_j \delta_j - d_j \gamma_j \leq t_j - d_j$, positive only if the job is late even when processed first. Only job 1 is ($t_1 = 5 > d_1 = 3$): $\bar{\gamma}_1 = \bar{\delta}_1 = 1$, all the rest 0, value $-3 + 5 = 2$. Hence $2 \leq z(MILP) \leq 12$. The reading: job 1 will be at least 2 minutes late wherever it is placed.

What the solver says. $z(LP) = 2$: the hand-built solution is optimal for the dual, and the LP relaxation of this model is *extremely weak*, as for all big-M models — the relaxation sets $s_{ji} = 1/2$ and releases the precedences. The integer optimum is $z(MILP) = 11$, sequence $2 \rightarrow 1 \rightarrow 3$: $\tilde{\kappa} = (9, 4, 15)$, $\tilde{\tau} = (6, 0, 5)$. The given order cost 12; swapping the first two jobs saves 1.

LB (given order)	UB (hand dual)	$z(LP)$	$z(MILP)$	heuristic gap
12	2	2	11	9.1%

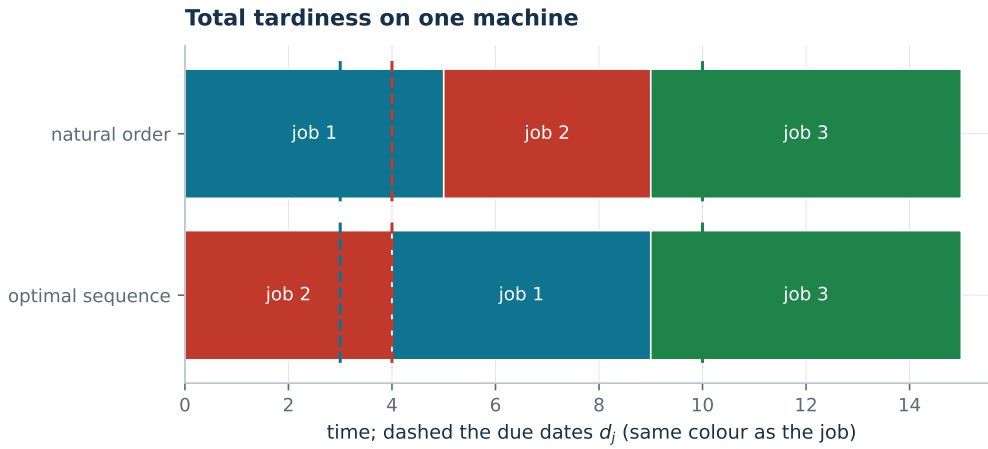


Figure 7.3: The two sequences on the instance: the given order (12) and the optimal sequence $2 \rightarrow 1 \rightarrow 3$ (11); dashed the due dates.

Additional considerations

- $s_{ji} \leq 1$ ($n(n-1)$) are valid and implied by (7.14b).
- *Transitivity*: for three distinct jobs j, i, k , if j precedes i and i precedes k then j precedes k :

$$s_{ji} \text{ AND } s_{ik} \implies s_{jk} \quad \text{and, by De Morgan,} \quad \text{NOT } s_{ji} \text{ OR NOT } s_{ik} \text{ OR } s_{jk},$$

i.e.

$$s_{ji} + s_{ik} - s_{jk} \leq 1$$

($n(n-1)(n-2)/6$ constraints per unordered triple, with its permutations). They are valid inequalities: in every real sequence transitivity holds, hence constraints (7.14c) imply them (a cycle $j \rightarrow i \rightarrow k \rightarrow j$ would give $\kappa_j \geq \kappa_j + t_i + t_k + t_j$, impossible). In the relaxation they cut off the $1/2$ solutions: they are the first step to strengthen a big-M model.

- The choice of M : $\sum_{j=1}^n t_j$ is the smallest value that works in general; larger M leave the same integer set and a weaker relaxation (Section 3.8).

Additional modelling questions

Modelling question 7.13 — release dates

Job 2 cannot start before time $\rho_2 = 2$ (the material arrives late); the others are available from the start. Model and find the new optimum.

Modelling question 7.14 — minimising the maximum tardiness

The company wants to minimise the tardiness of the latest job, not the sum. Model and solve; what changes in the link of τ_j ?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
UB	12	heuristic solution (UB)
LB	2	certified bound (LB)
$z(MILP)$	11	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 9.1\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 90.9\%.$$

The sandwich on variant 7a

The release dates add a family $\varepsilon_j \geq 0$ to the dual, entering the objective with its right-hand side $\rho_j + t_j$. With $\gamma_j = 1$, the column of κ_j imposes $\delta_j + \varepsilon_j \leq 1$, and it pays to put all the weight on ε_j : the release replaces the processing time. The heuristic is the same EDD rule, which now waits for the release.

	value	what it is
UB	12	heuristic solution
LB	2	dual certificate built by hand
$z(LP)$	4	relaxation without the bounds
$z(LP^+)$	4	relaxation with the bounds
$z(MILP)$	12	optimum of the MILP

Location and coverage

Class: BIP / MILP

Script: one per problem, `python/fam08_1_capacitated.py ...`

[Online page](#)

The family

Four problems in which we decide *where* to open a facility — a location, a hub — and how this constrains the variables that depend on that decision: where to ship goods (problem 1), which clients to serve (problem 2), which clients to cover (problem 3), which hub each terminal connects to (problem 4). What changes from problem to problem is the nature of the link between the opening variable and the variables that depend on it, and how the budget on openings enters the model.

The links from chapter 3 revisited here:

- **aggregated activation** (problem 1): a single family of constraints acts as both the link and the capacity constraint, $\sum_c y_{lc} \leq u_l x_l$;
- **disaggregated activation** (problem 2): $x_l \geq y_{lc}$ for every location and client, derived from the CNF of a Boolean implication as in problem 7.5, here with a budget k on the number of open locations instead of a time budget;
- **if and only if** (problem 3): one direction imposed by two families of constraints (signal threshold and interference), the other following from the objective;
- **aggregated activation and a maximum variable** (problem 4): the same activation as problem 1 together with the maximum link $z_j \geq c_{ij} x_{ij}$ already seen in problem 7.7.

For each: model, proof of the links, instance, constructive heuristic (a bound from the feasible-solutions side), dual of the LP relaxation with a hand-built dual solution (a bound from the other side), solver solution, additional considerations, and two additional modelling questions with solution. Every number comes from the scripts `fam08_1_capacitated.py ... fam08_4_hub.py`.

Family notation

Symbol	Type	Meaning
m	$\in \mathbb{Z}_{\geq 1}$	number of candidate locations/hubs, indexed by $l \in \{1, 2, \dots, m\}$
n	$\in \mathbb{Z}_{\geq 1}$	number of clients/terminals, indexed by $c \in \{1, 2, \dots, n\}$
t_{lc}	$\in \mathbb{Q}_{>0}$	transport/connection cost from location/hub l to client/terminal c
i_l, f_l	$\in \mathbb{Q}_{\geq 0}$	opening/activation cost of location/hub l
u_l	$\in \mathbb{Q}_{>0}$	capacity of location l
d_c	$\in \mathbb{Q}_{>0}$	demand of client c
k	$\in \mathbb{Z}_{\geq 1}$	maximum number of open locations / capacity of each hub
s_{lc}	$\in \mathbb{Q}_{\geq 0}$	signal strength from location l to client c
p_c	$\in \mathbb{Q}_{>0}$	profit if client c is covered
t, b	$\in \mathbb{Q}_{>0}$	signal threshold and interference limit

8.1 Capacitated facility location

Script: `python/fam08_1_capacitated.py`[Notebook in Colab](#)

Problem 8.1 — capacitated facility location

A company must serve $n \in \mathbb{Z}_{\geq 1}$ clients with its product and has identified $m \in \mathbb{Z}_{\geq 1}$ candidate locations where a facility can be installed. For each client $c \in \{1, 2, \dots, n\}$, the value $d_c \in \mathbb{Q}_{>0}$ is the demand, in liters of product, of that client. For each location $l \in \{1, 2, \dots, m\}$ and each client c , the value $t_{lc} \in \mathbb{Q}_{>0}$ is the cost, in euros, of transporting one liter of product from location l to client c . For each location l , the value $u_l \in \mathbb{Q}_{>0}$ is the capacity, in liters, of the facility installed there, and $i_l \in \mathbb{Q}_{>0}$ is the installation cost. The company wants to decide where to install facilities and how to serve the clients, at minimum total cost.

The problem in words. We decide where to install facilities and how much product to transport from each location to each client. *The objective:* minimum total cost (installation plus transport). *The constraints:* a location that is not installed ships nothing, and an installed location does not exceed its capacity; every client's demand must be satisfied exactly. This is the *capacitated facility location* problem.

Data (model input)

Symbol	Type	Meaning
m	$\in \mathbb{Z}_{\geq 1}$	number of candidate locations, $l \in \{1, 2, \dots, m\}$
n	$\in \mathbb{Z}_{\geq 1}$	number of clients, $c \in \{1, 2, \dots, n\}$
t_{lc}	$\in \mathbb{Q}_{>0}$	transport cost (per liter) from location l to client c
u_l	$\in \mathbb{Q}_{>0}$	capacity (liters) of location l
i_l	$\in \mathbb{Q}_{>0}$	installation cost of location l
d_c	$\in \mathbb{Q}_{>0}$	demand (liters) of client c

Decision variables

We introduce the following m binary variables:

$$x_l = \begin{cases} 1 & \text{if a facility is installed at location } l, \\ 0 & \text{otherwise,} \end{cases} \quad \forall l \in \{1, 2, \dots, m\},$$

and the following $m n$ non-negative continuous variables:

y_{lc} = liters of product transported from location l to client c

$$\forall l \in \{1, 2, \dots, m\}, \forall c \in \{1, 2, \dots, n\}.$$

Using these variables, a MILP model for the problem is the following:

Model 8.1 — capacitated facility location

$$\min \sum_{l=1}^m i_l x_l + \sum_{l=1}^m \sum_{c=1}^n t_{lc} y_{lc} \quad (8.1a)$$

$$\text{subject to } u_l x_l - \sum_{c=1}^n y_{lc} \geq 0, \quad \forall l \in \{1, 2, \dots, m\}, \quad (8.1b)$$

$$\sum_{l=1}^m y_{lc} = d_c, \quad \forall c \in \{1, 2, \dots, n\}, \quad (8.1c)$$

$$x_l \in \{0, 1\}, \quad \forall l \in \{1, 2, \dots, m\}, \quad (8.1d)$$

$$y_{lc} \geq 0, \quad \forall l \in \{1, 2, \dots, m\}, \forall c \in \{1, 2, \dots, n\}. \quad (8.1e)$$

Description of the objective function and constraints

- the linear objective function (8.1a) minimizes the total cost, the sum of the installation costs of the open locations and the transport costs;
- the linear constraints (8.1b) link the transport variables to the installation variables and impose the capacity restriction: they ensure that if a positive quantity is shipped from a location, that location is installed, and that an uninstalled location ships nothing; moreover, if a location is installed, the total quantity shipped does not exceed its capacity u_l (m linear constraints);
- the linear constraints (8.1c) ensure that every client's demand is satisfied exactly (n linear constraints);
- constraints (8.1d) and (8.1e) define the variables of the model.

The link between the variables. For each location l , if a positive quantity of product ships from location l , then the location must be installed:

$$\sum_{c=1}^n y_{lc} > 0 \implies x_l = 1.$$

The contrapositive gives

$$x_l = 0 \implies \sum_{c=1}^n y_{lc} = 0 :$$

an uninstalled location ships nothing. This implication — together with its contrapositive — is imposed directly by constraints (8.1b): if $\sum_{c=1}^n y_{lc} > 0$, the constraint forces $u_l x_l \geq \sum_{c=1}^n y_{lc} > 0$ and, since $u_l > 0$, $x_l > 0$, i.e., $x_l = 1$ (being binary). Conversely, if $x_l = 0$, the constraint becomes $\sum_{c=1}^n y_{lc} \leq 0$ and, since every $y_{lc} \geq 0$, $y_{lc} = 0$ for every c . (When $x_l = 1$, the constraint reduces to $\sum_{c=1}^n y_{lc} \leq u_l$: the capacity.)

The opposite direction — if nothing ships from a location, it is not installed:

$$x_l = 1 \implies \sum_{c=1}^n y_{lc} > 0$$

— is not imposed explicitly by the constraints, but follows from the objective function in every optimal solution: since $i_l > 0$, if $\sum_{c=1}^n y_{lc} = 0$ and $x_l = 1$, setting $x_l = 0$ remains feasible and decreases the cost by i_l . Therefore, in every optimal solution, no installed location goes unused. A single family of constraints thus acts as both the activation link and the capacity constraint.

The two bounds built below — the primal one, from the heuristic, and the dual one, from the relaxation — are what the solver works with.

The model in gurobipy

```

1 mod = gp.Model("capacitated_location")
2 x = mod.addVars(m, vtype=GRB.BINARY, name="x")
3 y = mod.addVars(m, n, name="y")
4 mod.setObjective(gp.quicksum(i[l] * x[l] for l in range(m))
5                  + gp.quicksum(t[l][c] * y[l, c] for l in range(m) for
6                  c in range(n)), GRB.MINIMIZE)
7 mod.addConstrs((u[l] * x[l] - gp.quicksum(y[l, c] for c in range(n)) >=
8                0
9                for l in range(m)), name="capacity")
10 mod.addConstrs((gp.quicksum(y[l, c] for l in range(m)) == d[c] for c in
11                range(n)), name="demand")

```

The instance

$m = 2$ locations, $n = 3$ clients:

t_{lc}	$c = 1$	$c = 2$	$c = 3$
$l = 1$	4	5	6
$l = 2$	6	4	3

	$l = 1$	$l = 2$
u_l	50	50
i_l	60	90

	$c = 1$	$c = 2$	$c = 3$
d_c	8	25	27

Constructive heuristic: the primal bound

Locations are scanned in natural order; for each, clients are scanned and each is shipped the minimum of the location's residual capacity and the client's residual demand, without exceeding either one; a location that shipped something is opened.

It is a constructive heuristic of the family seen in chapter 5: a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

Execution on the instance (script output): location 1 ships 8 to client 1 (demand exhausted, residual capacity 42), then 25 to client 2 (demand exhausted, residual capacity 17), then 17 to client 3 (location 1's capacity exhausted, 10 still owed); location 2 ships the remaining 10 to client 3. The feasible solution found is

$$\bar{x}_1 = \bar{x}_2 = 1, \quad \bar{y}_{11} = 8, \bar{y}_{12} = 25, \bar{y}_{13} = 17, \bar{y}_{23} = 10, \quad \text{all other } \bar{y}_{lc} = 0,$$

of value

$$\underbrace{60 + 90}_{\text{installation}} + \underbrace{4 \cdot 8 + 5 \cdot 25 + 6 \cdot 17 + 3 \cdot 10}_{\text{transport} = 32 + 125 + 102 + 30 = 289} = 150 + 289 = 439.$$

Hence

$$z(MILP) \leq UB = 439.$$

LP relaxation and dual: the dual bound

We associate a non-negative dual variable μ_l with each constraint (8.1b) ($\geq$ direction) and a free dual variable π_c with each constraint (8.1c) (an equality). The dual of the LP relaxation is:

Dual of the LP relaxation of model 8.1

$$\max \sum_{c=1}^n d_c \pi_c \tag{8.2a}$$

$$\text{subject to } u_l \mu_l \leq i_l, \quad \forall l \in \{1, 2, \dots, m\}, \tag{8.2b}$$

$$-\mu_l + \pi_c \leq t_{lc}, \quad \forall l \in \{1, 2, \dots, m\}, \forall c \in \{1, 2, \dots, n\}, \tag{8.2c}$$

$$\mu_l \geq 0, \quad \forall l \in \{1, 2, \dots, m\}, \tag{8.2d}$$

$$\pi_c \geq 0, \quad \forall c \in \{1, 2, \dots, n\}. \tag{8.2e}$$

Description. The variable μ_l is the price of one unit of capacity of site l and π_c the value of one unit of demand of customer c . The objective (8.2a) prices all the demand to be met at those values. Constraints (8.2b) are the columns of the x_l : the capacity u_l that opening site l makes available, priced at μ_l , cannot be worth more than the installation cost i_l . Constraints (8.2c) are the columns of the y_{lc} : carrying one unit from site l to customer c is worth π_c and consumes one unit of capacity at price μ_l ; the balance cannot exceed the transport cost t_{lc} .

A hand-built dual solution. Set $\bar{\mu}_l = i_l/u_l$: constraint (8.2b) holds with equality, spreading the installation cost over the capacity. On the instance,

$$\bar{\mu}_1 = 60/50 = 6/5, \quad \bar{\mu}_2 = 90/50 = 9/5.$$

Constraint (8.2c) becomes $\pi_c \leq t_{lc} + \bar{\mu}_l$ for every location l : the largest feasible value is $\bar{\pi}_c = \min_l(t_{lc} + \bar{\mu}_l)$:

$$\bar{\pi}_1 = \min(4 + 6/5, 6 + 9/5) = 26/5,$$

$$\bar{\pi}_2 = \min(5 + 6/5, 4 + 9/5) = 29/5,$$

$$\bar{\pi}_3 = \min(6 + 6/5, 3 + 9/5) = 24/5.$$

The value of this feasible dual solution is

$$8 \cdot \frac{26}{5} + 25 \cdot \frac{29}{5} + 27 \cdot \frac{24}{5} = \frac{208+725+648}{5} = \frac{1581}{5}.$$

By weak duality, $LB = 1581/5 \leq z(LP) \leq z(MILP) \leq UB = 439$. The recipe has a managerial meaning: spreading each location's fixed cost over its capacity and then serving every client at the minimum marginal cost is a lower bound anyone would write down; the dual formalizes it.

What the solver says. The optimum of the LP relaxation is exactly $z(LP) = 1581/5$: the hand-built dual solution above is already optimal (strong duality, no room for improvement). Strengthening the relaxation with $x_l \leq 1$ gives $z(LP^+) = 317$: a sizeable jump, a sign that the constraint $x_l \leq 1$ — implicit in the MILP but not in the relaxation without the bounds — matters here. The MILP has optimum $z(MILP) = 365$, with $\tilde{x}_1 = \tilde{x}_2 = 1$, $\tilde{y}_{11} = 8$, $\tilde{y}_{12} = 2$, $\tilde{y}_{22} = 23$, $\tilde{y}_{23} = 27$: both locations open, but location 1 serves only part of client 2's demand (the rest comes from location 2, which is cheaper for that client). The heuristic gap relative to the optimum is 20.3%.

UB (heuristic)	LB (hand dual)	$z(LP)$	$z(LP^+)$	$z(MILP)$	heuristic gap
439	1581/5	1581/5	317	365	20.3%

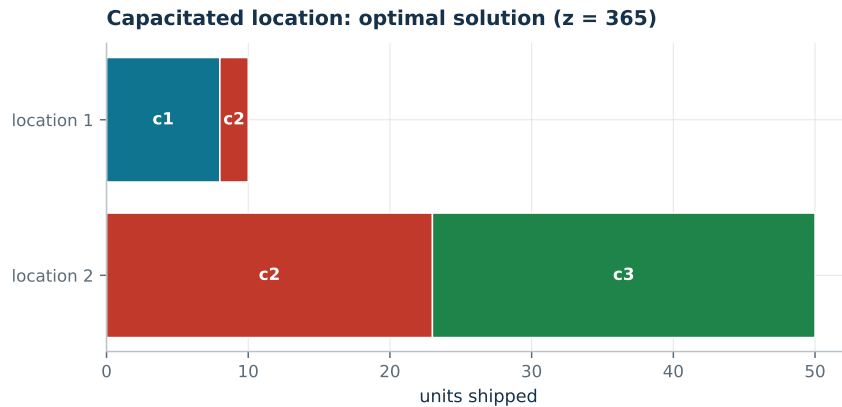


Figure 8.1: Optimal solution: for each open location, the liters shipped to each client.

Additional considerations

- For every location l and client c , if $u_l < d_c$ no single location can satisfy client c 's entire demand alone: not the case in this instance, but worth checking before assuming one location suffices.
- The constraints $y_{lc} \leq d_c x_l$ (for every l, c) are valid inequalities implied jointly by constraints (8.1b) and (8.1c): if $x_l = 0$ they force $y_{lc} = 0$, and if $x_l = 1$ they are never tighter than $y_{lc} \leq d_c$ (which follows anyway from demand satisfaction).

Additional modelling questions

Modelling question 8.1 — minimum lot for every open location

Every open location must ship at least 5 liters overall (it makes no sense to bear the installation cost for a negligible quantity). How does the model change? What is the new optimum for the instance?

Modelling question 8.2 — conditional opening

Location 2 can only be installed if location 1 is also installed (say, a supervision requirement). How does the model change? What is the new optimum?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
UB	439	heuristic solution (UB)
LB	$\frac{1581}{5}$	certified bound (LB)
$z(MILP)$	365	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 20.3\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 33.6\%.$$

The sandwich on variant 1b

"Site 2 opens only if site 1 opens" adds $\rho \leq 0$ to the dual, which moves fixed cost between the two sites: $\mu_1 = (i_1 + \rho)/u_1$ and $\mu_2 = (i_2 - \rho)/u_2$. One tries ρ among the values that change the minimum defining π_c ; on this instance the best is $\rho = 0$, because site 1 already has the lowest cost per litre.

	value	what it is
UB	439	heuristic solution
LB	$\frac{1581}{5}$	dual certificate built by hand
$z(LP)$	325	relaxation without the bounds
$z(LP^+)$	325	relaxation with the bounds
$z(MILP)$	365	optimum of the MILP

8.2 p-median: at most k locations

Script: `python/fam08_2_pmedian.py`

[Notebook in Colab](#)

Problem 8.2 — p-median

A company must choose at most $k \in \mathbb{Z}_{\geq 1}$ locations, among $m \in \mathbb{Z}_{\geq 1}$ candidates, where to open a service point, and assign each of the $n \in \mathbb{Z}_{\geq 1}$ clients to the most convenient open location. For each location $l \in \{1, 2, \dots, m\}$ and each client $c \in \{1, 2, \dots, n\}$, the value $d_{lc} \in \mathbb{Q}_{>0}$ is the distance (or service cost) between location l and client c . Every client must be assigned to exactly one open location. The company wants to decide which locations to open and how to assign clients, minimizing the sum of the distances between each client and the location it is assigned to.

The problem in words. We decide which locations to open (at most k) and which open location to assign each client to. *The objective:* minimum sum of client-location distances. *The constraints:* every client assigned to exactly one location; a client can only be assigned to an open location; at most k open locations. This is the classic *p-median* problem.

Data (model input)

Symbol	Type	Meaning
m	$\in \mathbb{Z}_{\geq 1}$	number of candidate locations, $l \in \{1, 2, \dots, m\}$
n	$\in \mathbb{Z}_{\geq 1}$	number of clients, $c \in \{1, 2, \dots, n\}$
d_{lc}	$\in \mathbb{Q}_{>0}$	distance between location l and client c
k	$\in \mathbb{Z}_{\geq 1}$	maximum number of open locations

Decision variables

We introduce the following m binary variables:

$$x_l = \begin{cases} 1 & \text{if location } l \text{ is open,} \\ 0 & \text{otherwise,} \end{cases} \quad \forall l \in \{1, 2, \dots, m\},$$

and the following $m n$ binary variables:

$$y_{lc} = \begin{cases} 1 & \text{if client } c \text{ is served by location } l, \\ 0 & \text{otherwise,} \end{cases} \quad \forall l \in \{1, 2, \dots, m\}, \forall c \in \{1, 2, \dots, n\}.$$

Using these variables, a BIP model for the problem is the following:

Model 8.2 — p-median

$$\min \sum_{l=1}^m \sum_{c=1}^n d_{lc} y_{lc} \quad (8.3a)$$

$$\text{subject to } \sum_{l=1}^m y_{lc} = 1, \quad \forall c \in \{1, 2, \dots, n\}, \quad (8.3b)$$

$$\sum_{l=1}^m x_l \leq k, \quad (8.3c)$$

$$x_l - y_{lc} \geq 0, \quad \forall l \in \{1, 2, \dots, m\}, \forall c \in \{1, 2, \dots, n\}, \quad (8.3d)$$

$$x_l \in \{0, 1\}, \quad \forall l \in \{1, 2, \dots, m\}, \quad (8.3e)$$

$$y_{lc} \in \{0, 1\}, \quad \forall l \in \{1, 2, \dots, m\}, \forall c \in \{1, 2, \dots, n\}. \quad (8.3f)$$

Description of the objective function and constraints

- the linear objective function (8.3a) minimizes the sum of the distances between every client and the location it is assigned to;
- the linear constraints (8.3b) ensure that every client is assigned to exactly one location (n linear constraints);
- the linear constraint (8.3c) limits to k the number of open locations (one linear constraint);
- the linear constraints (8.3d) link the assignment variables to the opening variables, in disaggregated form ($m n$ linear constraints);
- constraints (8.3e) and (8.3f) define the variables of the model.

The link between the variables. For every location l and client c , if client c is served by location l , then location l must be open:

$$y_{lc} = 1 \implies x_l = 1.$$

In Boolean form the implication is equivalent, by its CNF, to

$$y_{lc} \implies x_l \quad \equiv \quad \text{NOT } y_{lc} \text{ OR } x_l,$$

i.e. $x_l \geq y_{lc}$: exactly constraint (8.3d), imposed directly (the objective is not needed). From the contrapositive,

$$x_l = 0 \implies y_{lc} = 0 :$$

no assignment can leave a closed location — again imposed directly by the same constraint. Unlike problem 8.1, here there is no opening cost that would discourage open-but-unused locations: constraint (8.3c) only caps the number of open locations, so the direction “ $x_l = 1 \Rightarrow$ at least one client served by l ” is neither imposed nor guaranteed by optimality. The link is disaggregated — one row per pair (l, c) , as in problem 7.5 — because it is binary against binary for each pair, not a total against a single activation.

The two bounds built below — the primal one, from the heuristic, and the dual one, from the relaxation — are what the solver works with.

The model in gurobipy

```

1 mod = gp.Model("p_median")
2 x = mod.addVars(m, vtype=GRB.BINARY, name="x")
3 y = mod.addVars(m, n, vtype=GRB.BINARY, name="y")
4 mod.setObjective(gp.quicksum(dist[l][c] * y[l, c] for l in range(m) for
   c in range(n)), GRB.MINIMIZE)
5 mod.addConstrs((y.sum("*", c) == 1 for c in range(n)), name="assign")
6 mod.addConstr(x.sum() <= k, name="number_of_locations")
7 mod.addConstrs((x[l] - y[l, c] >= 0 for l in range(m) for c in range(n)
   ), name="link")

```

The instance

$m = 3$ locations, $n = 3$ clients, $k = 2$:

d_{lc}	$c = 1$	$c = 2$	$c = 3$
$l = 1$	5	6	10
$l = 2$	3	12	9
$l = 3$	10	9	4

Constructive heuristic: the primal bound

The first k locations are opened in natural order; then every client is served by the nearest open location.

Execution on the instance (script output): locations 1 and 2 are opened ($x_1 = x_2 = 1$). Client 1: the nearest open location is 2 (distance 3). Client 2: the nearest open location is 1 (distance 6). Client 3: the nearest open location is 2 (distance 9). The feasible solution found is

$$\bar{y}_{21} = 1, \quad \bar{y}_{12} = 1, \quad \bar{y}_{23} = 1, \quad \text{all other } \bar{y}_{lc} = 0,$$

of value $3 + 6 + 9 = 18$. Hence

$$z(MILP) \leq UB = 18.$$

LP relaxation and dual: the dual bound

We associate a free dual variable μ_c with each constraint (8.3b) (an equality), a non-positive dual variable ϱ with constraint (8.3c) (a $\leq$ direction in a minimization problem) and a non-negative dual variable π_{lc} with each constraint (8.3d) (a $\geq$ direction). The dual of the LP relaxation is:

Dual of the LP relaxation of model 8.2

$$\max \sum_{c=1}^n \mu_c + k \varrho \quad (8.4a)$$

$$\text{subject to } \varrho + \sum_{c=1}^n \pi_{lc} \leq 0, \quad \forall l \in \{1, 2, \dots, m\}, \quad (8.4b)$$

$$\mu_c - \pi_{lc} \leq d_{lc}, \quad \forall l \in \{1, 2, \dots, m\}, \forall c \in \{1, 2, \dots, n\}, \quad (8.4c)$$

$$\mu_c \geq 0, \quad \forall c \in \{1, 2, \dots, n\}, \quad (8.4d)$$

$$\varrho \leq 0, \quad (8.4e)$$

$$\pi_{lc} \geq 0, \quad \forall l \in \{1, 2, \dots, m\}, \forall c \in \{1, 2, \dots, n\}. \quad (8.4f)$$

Description. The variable μ_c is the value of the assignment of customer c , ϱ the (non-positive) price of the budget of k sites, and π_{lc} the price of the link "to serve c from l one must open l ". The objective (8.4a) prices the assignments and the budget. Constraints (8.4b) are the columns of the x_l : opening site l consumes one unit of budget and releases the links of all customers; the balance cannot be positive, because opening a site costs nothing in the objective of this model. Constraints (8.4c) are the columns of the y_{lc} : serving customer c from site l is worth μ_c and consumes the link π_{lc} ; the balance cannot exceed the distance d_{lc} .

A hand-built dual solution. With $\bar{\varrho} = 0$ and $\bar{\pi}_{lc} = 0$ for every pair, constraint (8.4b) holds ($0 \leq 0$) and constraint (8.4c) becomes $\mu_c \leq d_{lc}$ for every location l : the largest feasible value is $\bar{\mu}_c = \min_l d_{lc}$, the distance to the nearest location in absolute terms (open or not):

$$\bar{\mu}_1 = \min(5, 3, 10) = 3, \quad \bar{\mu}_2 = \min(6, 12, 9) = 6, \quad \bar{\mu}_3 = \min(10, 9, 4) = 4,$$

of value $3 + 6 + 4 + 2 \cdot 0 = 13$. By weak duality, $LB = 13 \leq z(LP) \leq z(MILP) \leq UB = 18$. The recipe has a meaning: "no client can cost less than its nearest location overall" is a trivial lower bound that ignores the budget constraint k ; the dual represents it exactly with $\varrho = 0$.

What the solver says. The optimum of the LP relaxation is $z(LP) = 15$, equal to the relaxation with the bounds $z(LP^+) = 15$: the constraint $y_{lc} \leq 1$ is never relevant here. The MILP has optimum $z(MILP) = 15$, with $\tilde{x}_1 = \tilde{x}_3 = 1$ (locations 1 and 3 open, not 1 and 2 as in the heuristic) and $\tilde{y}_{11} = \tilde{y}_{12} = \tilde{y}_{33} = 1$: the LP relaxation is already integral on this instance, and the heuristic — which always opens the first k locations regardless of cost — loses $18/15 - 1 = 20.0\%$.

UB (heuristic)	LB (hand dual)	$z(LP)$	$z(LP^+)$	$z(MILP)$	heuristic gap
18	13	15	15	15	20.0%

Additional considerations

- Constraint (8.3c) is "at most k ", not "exactly k ": opening fewer than k locations is never advantageous when distances are positive (opening one more location can only bring some client closer), but the model does not impose this explicitly. Question 2a below checks that, on this instance, the optimum does not change when equality is imposed.

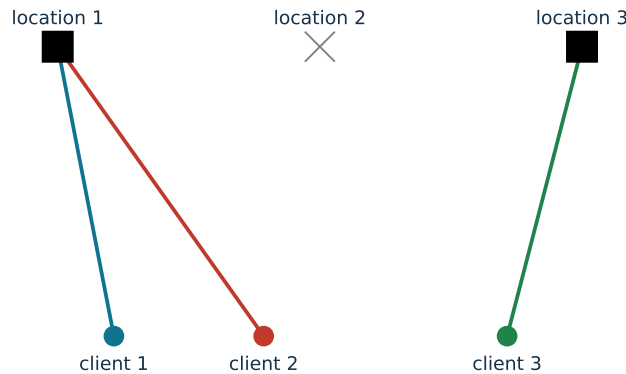
p-median: optimal solution ($z = 15$); square = open location

Figure 8.2: Optimal solution: square = open location, cross = closed location, lines = client–location assignment.

- Constraints (8.3d), summed over c for a fixed location l , give $n x_l \geq \sum_{c=1}^n y_{lc}$: an aggregated valid inequality, weaker than the disaggregated one used in the model.

Additional modelling questions

Modelling question 8.3 — exactly k open locations

For organisational reasons, exactly k locations must be open (not at most). How does the model change? What is the new optimum for the instance?

Modelling question 8.4 — proximity coverage for one client

Client 1 must be served within distance 4 (a service-level requirement). How is this modelled? What is the new optimum?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
UB	18	heuristic solution (UB)
LB	13	certified bound (LB)
$z(MILP)$	15	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 20.0\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 33.3\%.$$

The sandwich on variant 2a

Imposing *exactly* k sites instead of *at most* k adds $\sigma \geq 0$ with right-hand side k to the dual. The algebra settles it in one line: the column of the x_l imposes $\varrho + \sigma \leq 0$, and the objective contains $k(\varrho + \sigma)$, which is therefore never positive. The maximum is at $\varrho + \sigma = 0$ and the value is $\sum_c \mu_c$ again: the relaxation does not move, the integer optimum does.

	value	what it is
UB	18	heuristic solution
LB	13	dual certificate built by hand
$z(LP)$	15	relaxation without the bounds
$z(LP^+)$	15	relaxation with the bounds
$z(MILP)$	15	optimum of the MILP

8.3 Signal coverage with interference

Script: `python/fam08_3_coverage.py`

[Notebook in Colab](#)

Problem 8.3 — coverage with interference

An operator must choose at most $k \in \mathbb{Z}_{\geq 1}$ locations, among $m \in \mathbb{Z}_{\geq 1}$ candidates, where to install an antenna, to serve $n \in \mathbb{Z}_{\geq 1}$ clients. For each location $l \in \{1, 2, \dots, m\}$ and each client $c \in \{1, 2, \dots, n\}$, the value $s_{lc} \in \mathbb{Q}_{\geq 0}$ is the signal strength received by client c if location l is installed. A client c is *covered* if and only if the total signal strength received from all installed locations is at least the threshold $t \in \mathbb{Q}_{>0}$ *and* at most one installed location generates for that client a signal greater than or equal to the interference limit $b \in \mathbb{Q}_{>0}$. For each client c , the value $p_c \in \mathbb{Q}_{>0}$ is the profit if the client is covered. The operator wants to decide which locations to install to maximize the total profit.

The problem in words. We decide which locations to install (at most k). The objective: maximum total profit, the sum of the profits of the covered clients. The constraints: a client is covered if and only if it receives enough signal and not too much interference; the number of installed locations does not exceed k .

Data (model input)

Symbol	Type	Meaning
m	$\in \mathbb{Z}_{\geq 1}$	number of candidate locations, $l \in \{1, 2, \dots, m\}$
n	$\in \mathbb{Z}_{\geq 1}$	number of clients, $c \in \{1, 2, \dots, n\}$
s_{lc}	$\in \mathbb{Q}_{\geq 0}$	signal received by client c if location l is installed
p_c	$\in \mathbb{Q}_{>0}$	profit if client c is covered
t	$\in \mathbb{Q}_{>0}$	total signal threshold
b	$\in \mathbb{Q}_{>0}$	interference limit
k	$\in \mathbb{Z}_{\geq 1}$	maximum number of installed locations

For every client c define $\mathcal{L}_c = \{l : s_{lc} \geq b\}$, the “strong” locations for that client.

Decision variables

We introduce the following m binary variables:

$$x_l = \begin{cases} 1 & \text{if location } l \text{ is installed,} \\ 0 & \text{otherwise,} \end{cases} \quad \forall l \in \{1, 2, \dots, m\},$$

and the following n binary variables:

$$y_c = \begin{cases} 1 & \text{if client } c \text{ is covered,} \\ 0 & \text{otherwise,} \end{cases} \quad \forall c \in \{1, 2, \dots, n\}.$$

Using these variables, a BIP model for the problem is the following:

Model 8.3 — coverage with interference

$$\max \sum_{c=1}^n p_c y_c \quad (8.5a)$$

$$\text{subject to} \quad - \sum_{l=1}^m s_{lc} x_l + t y_c \leq 0, \quad \forall c \in \{1, 2, \dots, n\}, \quad (8.5b)$$

$$\sum_{l \in \mathcal{L}_c} x_l + (m-1) y_c \leq m, \quad \forall c \in \{1, 2, \dots, n\}, \quad (8.5c)$$

$$\sum_{l=1}^m x_l \leq k, \quad (8.5d)$$

$$x_l \in \{0, 1\}, \quad \forall l \in \{1, 2, \dots, m\}, \quad (8.5e)$$

$$y_c \in \{0, 1\}, \quad \forall c \in \{1, 2, \dots, n\}. \quad (8.5f)$$

Description of the objective function and constraints

- the linear objective function (8.5a) maximizes the total profit, the sum of the profits of the covered clients;
- the linear constraints (8.5b) link the coverage variables to the received signal: they ensure that if a client is covered, the total signal received is at least t (n linear constraints);
- the linear constraints (8.5c) link the coverage variables to interference: they ensure that if a client is covered, at most one location in $\mathcal{L}_c$ is installed (n linear constraints);
- the linear constraint (8.5d) limits to k the number of installed locations (one linear constraint);
- constraints (8.5e) and (8.5f) define the variables of the model.

The link between the variables: an “if and only if”. For every client c , the client is covered if and only if the installed locations jointly satisfy the threshold condition and the

interference condition:

$$y_c = 1 \iff \left(\sum_{l=1}^m s_{lc} x_l \geq t \right) \text{ AND } \left(\sum_{l \in \mathcal{L}_c} x_l \leq 1 \right).$$

One direction, imposed directly by the constraints. If $y_c = 1$, constraint (8.5b) forces $\sum_{l=1}^m s_{lc} x_l \geq t y_c = t$: the threshold is met. Constraint (8.5c) forces $\sum_{l \in \mathcal{L}_c} x_l \leq m - (m-1) y_c = 1$: at most one strong location is installed. So

$$y_c = 1 \implies \text{both conditions.}$$

The other direction, which follows from optimality. If both conditions are satisfied by a given set of installed locations, constraints (8.5b) and (8.5c) still allow both $y_c = 0$ and $y_c = 1$: nothing in the constraints forces $y_c = 1$. But since $p_c > 0$ and y_c appears only in these two constraints (with a positive coefficient on the side that bounds it from above), raising y_c from 0 to 1 remains feasible — by assumption both conditions hold — and raises the objective by p_c . Hence, in every optimal solution, $y_c = 1$ whenever both conditions are satisfied. The same pattern — one direction from the constraints, the other from optimality in a maximisation problem — as in problem 7.6.

The two bounds built below — the primal one, from the heuristic, and the dual one, from the relaxation — are what the solver works with.

The model in gurobipy

```

1 mod = gp.Model("coverage_interference")
2 x = mod.addVars(m, vtype=GRB.BINARY, name="x")
3 y = mod.addVars(n, vtype=GRB.BINARY, name="y")
4 mod.setObjective(gp.quicksum(p[c] * y[c] for c in range(n)), GRB.
    MAXIMIZE)
5 mod.addConstrs((-gp.quicksum(s[l][c] * x[l] for l in range(m)) + t * y[
    c] <= 0
6                 for c in range(n)), name="threshold")
7 mod.addConstrs((gp.quicksum(x[l] for l in L[c]) + (m - 1) * y[c] <= m
8                 for c in range(n)), name="interference")
9 mod.addConstr(x.sum() <= k, name="budget")

```

The instance

$m = 3$ locations, $n = 5$ clients, $t = 5$, $b = 4$, $k = 2$:

s_{lc}	$c = 1$	$c = 2$	$c = 3$	$c = 4$	$c = 5$		$c = 1$	$c = 2$	$c = 3$	$c = 4$	$c = 5$
$l = 1$	6	0	5	3	1						
$l = 2$	4	5	2	0	0						
$l = 3$	0	7	5	4	2						
						p_c	10	20	5	15	25

With $b = 4$: $\mathcal{L}_1 = \{1, 2\}$, $\mathcal{L}_2 = \{3\}$, $\mathcal{L}_3 = \{1, 3\}$, $\mathcal{L}_4 = \{3\}$, $\mathcal{L}_5 = \emptyset$.

Constructive heuristic: the primal bound

The first k locations are opened in natural order; a client is covered if the total signal received from the open locations reaches the threshold t and at most one open location in $\mathcal{L}_c$ is open.

Execution on the instance (script output): locations 1 and 2 are opened. Client 1: total signal $6 + 4 = 10 \geq 5$, but both strong locations in $\mathcal{L}_1 = \{1, 2\}$ are open ($2 > 1$): **not covered**. Client 2: total signal $0 + 5 = 5 \geq 5$, strong locations in $\mathcal{L}_2 = \{3\}$ open: none ($0 \leq 1$): **covered**. Client 3: total signal $5 + 2 = 7 \geq 5$, strong locations in $\mathcal{L}_3 = \{1, 3\}$ open: only location 1 ($1 \leq 1$): **covered**. Client 4: total signal $3 + 0 = 3 < 5$: **not covered**. Client 5: total signal $1 + 0 = 1 < 5$: **not covered**. The feasible solution found covers clients 2 and 3, of value $20 + 5 = 25$. Hence

$$z(MILP) \geq LB = 25.$$

LP relaxation and dual: the dual bound

We associate a non-negative dual variable π_c with each constraint (8.5b) (a $\leq$ direction in a maximisation problem), a non-negative dual variable λ_c with each constraint (8.5c), and a non-negative dual variable μ with constraint (8.5d). The dual of the LP relaxation is:

Dual of the LP relaxation of model 8.3

$$\min \quad m \sum_{c=1}^n \lambda_c + k \mu \quad (8.6a)$$

$$\text{subject to} \quad - \sum_{c=1}^n s_{lc} \pi_c + \sum_{c: l \in \mathcal{L}_c} \lambda_c + \mu \geq 0, \quad \forall l \in \{1, 2, \dots, m\}, \quad (8.6b)$$

$$t \pi_c + (m - 1) \lambda_c \geq p_c, \quad \forall c \in \{1, 2, \dots, n\}, \quad (8.6c)$$

$$\pi_c \geq 0, \quad \forall c \in \{1, 2, \dots, n\}, \quad (8.6d)$$

$$\lambda_c \geq 0, \quad \forall c \in \{1, 2, \dots, n\}, \quad (8.6e)$$

$$\mu \geq 0. \quad (8.6f)$$

Description. The variable π_c is the price of the threshold constraint of customer c , λ_c that of its interference constraint, and μ the price of one unit of budget. The objective (8.6a) adds up the right-hand sides of those constraints priced at those values. Constraints (8.6b) are the columns of the x_l : installing antenna l brings signal to the customers it covers, enters the interference constraints of the customers whose list contains it, and consumes one unit of budget; the balance must stay non-negative, because the x_l do not appear in the objective. Constraints (8.6c) are the columns of the y_c : declaring customer c served requires t units of signal at price π_c and $m - 1$ units of interference at price λ_c , and together they must cover the reward p_c .

A hand-built dual solution. With $\bar{\pi}_c = 0$ and $\bar{\mu} = 0$, constraint (8.6b) holds ($\sum \lambda_c \geq 0$), and constraint (8.6c) becomes $(m - 1)\bar{\lambda}_c \geq p_c$, i.e. $\bar{\lambda}_c = p_c/(m - 1) = p_c/2$:

$$\bar{\lambda}_1 = 5, \quad \bar{\lambda}_2 = 10, \quad \bar{\lambda}_3 = 5/2, \quad \bar{\lambda}_4 = 15/2, \quad \bar{\lambda}_5 = 25/2.$$

The value of this feasible dual solution is $m \sum_{c=1}^n \bar{\lambda}_c + k \cdot 0 = 3 \cdot (5 + 10 + 5/2 + 15/2 + 25/2) =$

$3 \cdot 75/2 = 225/2$. By weak duality, $LB = 25 \leq z(MILP) \leq z(LP) \leq UB = 225/2$ (a maximisation problem: the heuristic gives the lower bound, the dual the upper bound).

What the solver says. The optimum of the LP relaxation is $z(LP) = 41925/646 \approx 64.9$, while strengthening with $x_l \leq 1$ gives $z(LP^+) = 125/2 = 62.5$: a clear improvement. The MILP has optimum $z(MILP) = 45$, with $\tilde{x}_1 = \tilde{x}_3 = 1$ and $\tilde{y}_1 = \tilde{y}_2 = \tilde{y}_4 = 1$ (clients 1, 2 and 4 covered, not 3 or 5): a different configuration from the one found by the heuristic, which covered clients 2 and 3. The heuristic gap is 44.4%: on this maximisation problem, simply opening the first k locations is a much less effective choice than on the family's minimum-cost problems.

LB (heuristic)	UB (hand dual)	$z(LP)$	$z(LP^+)$	$z(MILP)$	heuristic gap
25	225/2	41925/646	125/2	45	44.4%

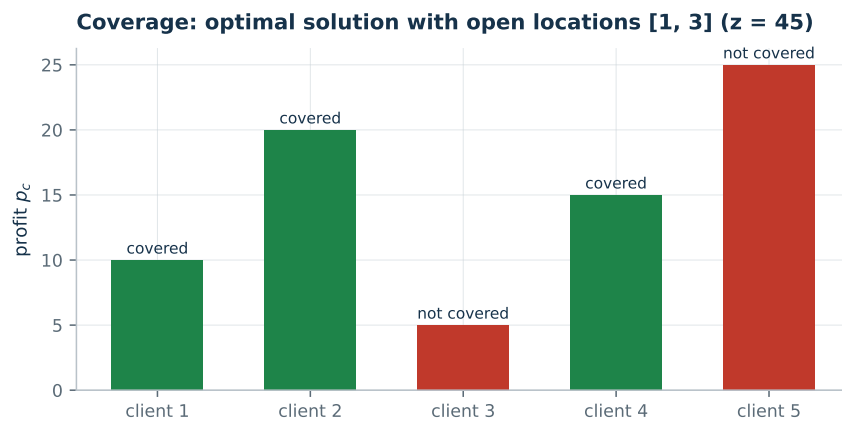


Figure 8.3: Optimal solution: profit of each client, green if covered, red if not covered.

Additional considerations

- Client 5 can never be covered in this instance: $\mathcal{L}_5 = \emptyset$, but the maximum receivable signal is $s_{15} + s_{25} + s_{35} = 1 + 0 + 2 = 3 < t = 5$ even opening all locations. Variable y_5 could be fixed to 0 upfront.
- Constraints (8.5c), for a client c with $|\mathcal{L}_c| \leq 1$, are redundant (never binding): the case for clients 2, 4 and 5 in this instance.

Additional modelling questions

Modelling question 8.5 — guaranteed minimum coverage

By contract, at least 3 clients must be covered. How does the model change? What is the new optimum?

Modelling question 8.6 — conditional installation

For maintenance reasons, location 1 can only be installed if location 3 is also installed. How is this modelled? What is the new optimum?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
LB	25	heuristic solution (LB)
UB	$\frac{225}{2}$	certified bound (UB)
$z(MILP)$	45	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|LB - z(MILP)|}{|z(MILP)|} = 44.4\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|LB - UB|}{|z(MILP)|} = 194.4\%.$$

The sandwich on variant 3a

"At least three clients covered" adds $\omega \leq 0$ to the dual. It is best left at zero, and the arithmetic says why: lowering it forces every λ_c up by $-\omega/(m-1)$, and in the objective those λ weigh more than the right-hand side saves. The heuristic, with few sites, tries every choice of k and keeps the best feasible one.

	value	what it is
UB	$\frac{225}{2}$	dual certificate built by hand
LB	45	heuristic solution
$z(LP)$	$\frac{41925}{646}$	relaxation without the bounds
$z(LP^+)$	$\frac{125}{2}$	relaxation with the bounds
$z(MILP)$	45	optimum of the MILP

8.4 Hub location with maximum cost

Script: `python/fam08_4_hub.py`

[Notebook in Colab](#)

Problem 8.4 — hub with maximum connection cost

In a computer network there are $n \in \mathbb{Z}_{\geq 1}$ terminals, each of which must be connected to exactly one hub. There are $m \in \mathbb{Z}_{\geq 1}$ hubs available, and each hub can serve at most $k \in \mathbb{Z}_{\geq 1}$ terminals. A hub must be activated before it can serve any terminals: for each hub $j \in \{1, 2, \dots, m\}$, the value $f_j \in \mathbb{Q}_{\geq 0}$ is the activation cost. For each terminal $i \in \{1, 2, \dots, n\}$ and each hub j , the value $c_{ij} \in \mathbb{Q}_{\geq 0}$ is the cost of connecting terminal i to hub j . The objective is to decide how to assign terminals to hubs to minimize the sum of the activation costs of the hubs used and the maximum connection cost of each hub, subject to the constraints described.

The problem in words. We decide which hubs to activate and which hub to connect each terminal to. *The objective:* activation costs plus, for each hub, the highest connection cost among the terminals assigned to it (not the sum: the bottleneck). *The constraints:* every terminal connected to exactly one hub; an inactive hub can serve no one, and an active one serves at most k terminals.

Data (model input)

Symbol	Type	Meaning
n	$\in \mathbb{Z}_{\geq 1}$	number of terminals, $i \in \{1, 2, \dots, n\}$
m	$\in \mathbb{Z}_{\geq 1}$	number of hubs, $j \in \{1, 2, \dots, m\}$
k	$\in \mathbb{Z}_{\geq 1}$	capacity (terminals) of each hub
c_{ij}	$\in \mathbb{Q}_{\geq 0}$	cost of connecting terminal i to hub j
f_j	$\in \mathbb{Q}_{\geq 0}$	activation cost of hub j

Decision variables

We introduce the following nm binary variables:

$$x_{ij} = \begin{cases} 1 & \text{if terminal } i \text{ is connected to hub } j, \\ 0 & \text{otherwise,} \end{cases} \quad \forall i \in \{1, 2, \dots, n\}, \forall j \in \{1, 2, \dots, m\},$$

the following m binary variables:

$$y_j = \begin{cases} 1 & \text{if hub } j \text{ is activated,} \\ 0 & \text{otherwise,} \end{cases} \quad \forall j \in \{1, 2, \dots, m\},$$

and the following m non-negative continuous variables:

$$z_j = \text{maximum connection cost among the terminals assigned to hub } j$$

$$\forall j \in \{1, 2, \dots, m\}.$$

Using these variables, a MILP model for the problem is the following:

Model 8.4 — hub with maximum cost

$$\min \sum_{j=1}^m f_j y_j + \sum_{j=1}^m z_j \quad (8.7a)$$

$$\text{subject to } \sum_{j=1}^m x_{ij} = 1, \quad \forall i \in \{1, 2, \dots, n\}, \quad (8.7b)$$

$$-\sum_{i=1}^n x_{ij} + k y_j \geq 0, \quad \forall j \in \{1, 2, \dots, m\}, \quad (8.7c)$$

$$-c_{ij} x_{ij} + z_j \geq 0, \quad \forall i \in \{1, 2, \dots, n\}, \forall j \in \{1, 2, \dots, m\}, \quad (8.7d)$$

$$x_{ij} \in \{0, 1\}, \quad \forall i \in \{1, 2, \dots, n\}, \forall j \in \{1, 2, \dots, m\}, \quad (8.7e)$$

$$y_j \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, m\}, \quad (8.7f)$$

$$z_j \geq 0, \quad \forall j \in \{1, 2, \dots, m\}. \quad (8.7g)$$

Description of the objective function and constraints

- the linear objective function (8.7a) minimizes the total cost, the sum of the activation costs of the hubs used and the maximum connection cost of each hub;
- the linear constraints (8.7b) ensure that every terminal is connected to exactly one hub (n linear constraints);
- the linear constraints (8.7c) link the assignment variables to the activation variables and impose the capacity: if a hub is not activated it can serve no one, and if it is activated it serves at most k terminals (m linear constraints);
- the linear constraints (8.7d) link the assignment variables to the maximum-cost variable: if terminal i is connected to hub j , then z_j cannot be smaller than c_{ij} (nm linear constraints);
- constraints (8.7e), (8.7f) and (8.7g) define the variables of the model.

First link: activation (aggregated). For every hub j , if at least one terminal is connected to j , then j must be activated: $\sum_{i=1}^n x_{ij} \geq 1 \Rightarrow y_j = 1$; from the contrapositive, $y_j = 0 \Rightarrow \sum_{i=1}^n x_{ij} = 0$. This implication — together with its contrapositive — is imposed directly, in aggregated form, by constraint (8.7c): if $y_j = 0$ the constraint becomes $\sum_{i=1}^n x_{ij} \leq 0$, i.e. $x_{ij} = 0$ for every i ; if $y_j = 1$ it becomes $\sum_{i=1}^n x_{ij} \leq k$, the capacity. The opposite direction — an activated hub serves at least one terminal, $y_j = 1 \Rightarrow \sum_{i=1}^n x_{ij} \geq 1$ — is not imposed by the constraints: $y_j = 1$ with all $x_{ij} = 0$ satisfies (8.7c). It follows from optimality, with a strength that depends on the sign of f_j . If $f_j > 0$: given an optimal solution with $\sum_{i=1}^n x_{ij} = 0$ and $y_j = 1$, setting $y_j = 0$ stays feasible and reduces the cost by $f_j > 0$, contradicting optimality; hence **in every optimum** no activated hub is left empty. If instead $f_j = 0$ — which the statement allows, having declared $f_j \in \mathbb{Q}_{\geq 0}$ — the exchange does not worsen but does not improve either, and the correct conclusion is the weaker one: **there exists an optimum** in which the empty hubs are switched off. On the instance $f = (5, 6, 7)$ is strictly positive and the strong version holds; but a proof must be written with the assumption it uses, not with the one that is convenient. Exactly the activation pattern already seen in problem 7.2, here in aggregated form.

Second link: maximum variable. For every terminal i and hub j , if terminal i is connected to hub j , then $z_j \geq c_{ij}$: $x_{ij} = 1 \Rightarrow z_j \geq c_{ij}$, imposed directly by constraint (8.7d) (equivalent to $z_j \geq c_{ij}x_{ij}$). Combining over all terminals connected to j , $z_j \geq \max_{i: x_{ij}=1} c_{ij}$ (with the convention that the maximum over the empty set is 0, guaranteed by $z_j \geq 0$). Since the objective minimizes $\sum_{j=1}^m z_j$ and z_j appears in no other constraint, in every optimal solution z_j takes the smallest feasible value: if $z_j > \max_{i: x_{ij}=1} c_{ij}$, decreasing it to that maximum remains feasible and decreases the objective. Hence, in every optimum, $z_j = \max_{i: x_{ij}=1} c_{ij}$ exactly: the same pattern as the maximum variable of problem 7.7.

The two bounds built below — the primal one, from the heuristic, and the dual one, from the relaxation — are what the solver works with.

The model in gurobipy

```

1 mod = gp.Model("hub_max")
2 x = mod.addVars(n, m, vtype=GRB.BINARY, name="x")
3 y = mod.addVars(m, vtype=GRB.BINARY, name="y")
4 z = mod.addVars(m, name="z")
5 mod.setObjective(gp.quicksum(f[j] * y[j] for j in range(m)) + z.sum(),
6                     GRB.MINIMIZE)
7 mod.addConstrs((gp.quicksum(x[i, j] for j in range(m)) == 1 for i in
8                     range(n)), name="assignment")
9 mod.addConstrs((-gp.quicksum(x[i, j] for i in range(n)) + k * y[j] >= 0
10                    for j in range(m)), name="activation")
11 mod.addConstrs((-c[i][j] * x[i, j] + z[j] >= 0 for i in range(n) for j
12                    in range(m)), name="maximum")

```

The instance

$n = 3$ terminals, $m = 3$ hubs, $k = 2$:

c_{ij}	$j = 1$	$j = 2$	$j = 3$
$i = 1$	5	10	2
$i = 2$	5	4	6
$i = 3$	5	4	6

	$j = 1$	$j = 2$	$j = 3$
f_j	5	6	7

Constructive heuristic: the primal bound

A *next-fit* heuristic, inspired by bin packing: one hub at a time is filled up to k terminals, then the algorithm moves to the next one; if all hubs have been used before every terminal is assigned, the algorithm fails. This is the same generic heuristic used for the scheduling problems of the previous family (hubs are the “machines” with capacity k , terminals the “jobs” with unit time independent of the machine): the script calls it from `euristiche.py` instead of rewriting it.

Execution on the instance (script output): terminal 1 goes to hub 1 (residual capacity $2 \rightarrow 1$); terminal 2 goes to hub 1 (residual capacity $1 \rightarrow 0$); terminal 3 no longer fits on hub 1 (full), it moves to hub 2. Hubs 1 and 2 are activated: the maximum connection cost of hub 1 is $\max(c_{11}, c_{21}) = \max(5, 5) = 5$, that of hub 2 is $c_{32} = 4$. The feasible solution found is

$$\bar{x}_{11} = \bar{x}_{21} = \bar{x}_{32} = 1, \quad \bar{y}_1 = \bar{y}_2 = 1, \quad \bar{z}_1 = 5, \bar{z}_2 = 4,$$

of value $5 + 6 + 5 + 4 = 20$. Hence

$$z(MILP) \leq UB = 20.$$

LP relaxation and dual: the dual bound

We associate a free dual variable α_i with each constraint (8.7b) (an equality), a non-negative dual variable β_j with each constraint (8.7c) (a $\geq$ direction), and a non-negative dual variable γ_{ij} with each constraint (8.7d) (a $\geq$ direction). The dual of the LP relaxation is:

Dual of the LP relaxation of model 8.4

$$\max \sum_{i=1}^n \alpha_i \quad (8.8a)$$

$$\text{subject to } \alpha_i - \beta_j - c_{ij} \gamma_{ij} \leq 0, \quad \forall i \in \{1, 2, \dots, n\}, \forall j \in \{1, 2, \dots, m\}, \quad (8.8b)$$

$$k \beta_j \leq f_j, \quad \forall j \in \{1, 2, \dots, m\}, \quad (8.8c)$$

$$\sum_{i=1}^n \gamma_{ij} \leq 1, \quad \forall j \in \{1, 2, \dots, m\}, \quad (8.8d)$$

$$\alpha_i \geq 0, \quad \forall i \in \{1, 2, \dots, n\}, \quad (8.8e)$$

$$\beta_j \geq 0, \quad \forall j \in \{1, 2, \dots, m\}, \quad (8.8f)$$

$$\gamma_{ij} \geq 0, \quad \forall i \in \{1, 2, \dots, n\}, \forall j \in \{1, 2, \dots, m\}. \quad (8.8g)$$

Description. The variable α_i is the value of terminal i , β_j the price of one of the k slots of hub j , and γ_{ij} the price of the link that pushes the maximum cost of hub j above c_{ij} . The objective (8.8a) prices the terminals, all of which must be assigned. Constraints (8.8b) are the columns of the x_{ij} : assigning terminal i to hub j satisfies its assignment constraint, takes one slot and forces the maximum cost of the hub up to c_{ij} ; the balance cannot be positive. Constraints (8.8c) are the columns of the y_j : the k slots that opening hub j makes available cannot be worth more than the fixed cost f_j . Constraints (8.8d) are the columns of the z_j : the maximum cost of hub j weighs 1 in the primal objective, so the prices of the links that push it up sum to at most one.

A hand-built dual solution. Set $\bar{\gamma}_{ij} = 0$ for every pair (constraint (8.8d) is automatically satisfied) and $\bar{\beta}_j = f_j/k$, the largest value allowed by constraint (8.8c): on the instance $\bar{\beta} = (5/2, 3, 7/2)$. Constraint (8.8b) then becomes $\alpha_i \leq \bar{\beta}_j$ for *every* hub j (not only the most convenient one): the largest feasible value is $\bar{\alpha}_i = \min_j \bar{\beta}_j = 5/2$ for every terminal i . The value of this feasible dual solution is $3 \cdot 5/2 = 15/2$. By weak duality, $LB = 15/2 \leq z(LP) \leq z(MILP) \leq UB = 20$.

Setting $\bar{\gamma}_{ij} = 0$ only for the “inconvenient” hubs and hoping that constraint (8.8b) bounds α_i only against the cheapest one is a mistake: the constraint holds for *every* j , so α_i is bounded by the *minimum* over all hubs, not by a single one. A dual solution that ignores this — say, setting $\bar{\gamma}_{i1} = 1/n$ only for the cheapest hub and leaving $\bar{\beta}_j = 0$ elsewhere — is not feasible: it would violate constraint (8.8b) for the other hubs.

What the solver says. The optimum of the LP relaxation is $z(LP) = 25/2$, which strengthening with $x_{ij} \leq 1$ improves to $z(LP^+) = 1015/78 \approx 13.0$. The MILP has optimum $z(MILP) = 19$, with $\tilde{x}_{13} = \tilde{x}_{21} = \tilde{x}_{31} = 1$, $\tilde{y}_1 = \tilde{y}_3 = 1$, $\tilde{z}_1 = 5$, $\tilde{z}_3 = 2$: hubs 1 and 3 are activated (not 1 and 2 as in the heuristic), with terminal 1 alone on hub 3 (the cheapest for it, $c_{13} = 2$) and terminals 2 and 3 together on hub 1. The heuristic gap is 5.3%: here next-fit, which ignores costs, still comes reasonably close to the optimum.

UB (heuristic)	LB (hand dual)	$z(LP)$	$z(LP^+)$	$z(MILP)$	heuristic gap
20	15/2	25/2	1015/78	19	5.3%

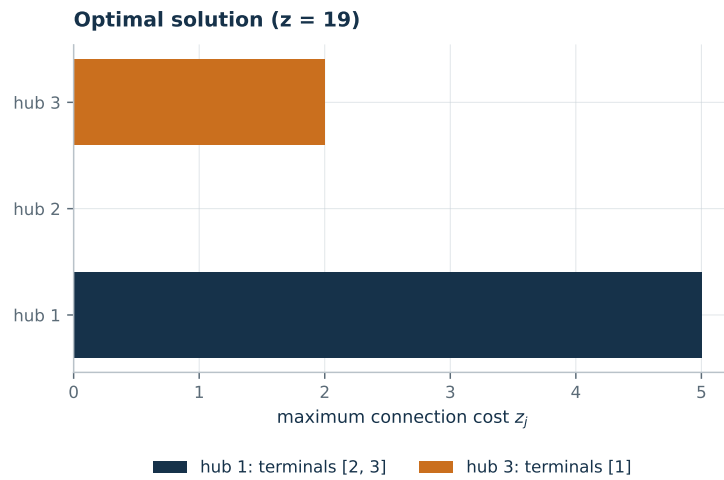


Figure 8.4: Optimal solution: maximum connection cost of each activated hub, with the terminals assigned.

Additional considerations

- The disaggregated constraints $x_{ij} \leq y_j$ (for every i, j) are valid inequalities implied by the aggregated constraint (8.7c): they give the same activation logic in a different form, and question 4a below checks that they do not change the optimum on this instance.
- Setting $M_j = \max_i c_{ij}$, the constraints $z_j \leq M_j y_j$ are not valid inequalities for the model (which allows $z_j > 0$ even with $y_j = 0$), but they are *optimality-preserving constraints*: in every optimum $z_j = 0$ when $y_j = 0$, because the objective minimizes z_j .

Additional modelling questions

Modelling question 8.7 — disaggregated activation link

Add the disaggregated links $x_{ij} \leq y_j$ to model (8.7). Does the optimum for the instance change? Does the relaxation change? And what happens if, instead of adding them, one replaces the aggregated constraint (8.7c) by them?

Modelling question 8.8 — forbidden connection

Terminal 1 cannot be connected to hub 2 (a security requirement). How is this modelled? What is the new optimum?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
UB	20	heuristic solution (UB)
LB	$\frac{15}{2}$	certified bound (LB)
$z(MILP)$	19	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 5.3\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 65.8\%.$$

The sandwich on variant 4b

Forbidding a connection means removing a column from the primal, hence *removing* a constraint from the dual: α_1 no longer has to stand the comparison with the forbidden hub and can rise. It is given one γ on each of the hubs left to it — each hub has a budget of 1 and no other terminal uses it — and the bound grows. This is the case where one more constraint in the primal *improves* the certificate, instead of leaving it still.

	value	what it is
UB	20	heuristic solution
LB	$\frac{21}{2}$	dual certificate built by hand
$z(LP)$	$\frac{25}{2}$	relaxation without the bounds
$z(LP^+)$	$\frac{79}{6}$	relaxation with the bounds
$z(MILP)$	19	optimum of the MILP

Production planning

Class: MILP

Script: one per problem, `python/fam09_1_lotsizing.py` ...

[Online page](#)

The family

Three problems in which one decides *how much* to produce, not only *whether* to do something. The quantity variables are continuous or integer, and binaries sit on top of them to switch on, to limit or to reward: the production run with its fixed cost (problem 1), the workforce with the hirings (2), the minimum lot per type with a bonus for variety (3).

The common feature is the *balance*: a quantity that comes in, one that goes out and one that stays. In problems 1 and 2 it is the inventory balance, $x_t + s_{t-1} - s_t = d_t$; in problem 2 there is also the workforce balance, $y_t - y_{t-1} - z_t = 0$. These are equality constraints, and their duals are free variables: it is the first family of the course in which this happens systematically.

The links between variables of chapter 3 that come back here:

- **fixed cost** (problem 1): $x_t \leq M_t y_t$, with M_t read off the data (the residual demand) and not picked at random;
- **integer counts** (problem 2): the workforce y_t and the hirings z_t are numbers of people, and the hours available are $r y_t$;
- **minimum lot** (problem 3): the semicontinuous variable $q_j y_j \leq x_j \leq M_j y_j$, which is zero or at least q_j ;
- **counting the types and if and only if** (problem 3): the variety bonus is collected only if the count of active types reaches two, and the missing direction is imposed by optimality because the bonus is positive.

For each of them: model, proof of the links, instance, constructive heuristic, dual of the LP relaxation with a hand-built dual solution, solver solution, additional considerations and two modelling questions with their answers. All the numbers come from the scripts `fam09_1_lotsizing.py`, `fam09_2_workforce.py` and `fam09_3_vehicles.py` in the `python/` folder, one per problem, each with its own notebook.

Notation of the family

Symbol	Type	Meaning
n	$\in \mathbb{Z}_{\geq 1}$	number of periods, indexed by $t \in \{1, 2, \dots, n\}$
d_t	$\in \mathbb{Q}_{\geq 0}$	demand of period t
p_t	$\in \mathbb{Q}_{> 0}$	unit production cost in period t
q_t	$\in \mathbb{Q}_{\geq 0}$	fixed set-up cost of a run in period t
h_t	$\in \mathbb{Q}_{\geq 0}$	storage cost of one unit at the end of period t
r_0, r_n	$\in \mathbb{Q}_{\geq 0}$	initial inventory and required final inventory
M_t	$\in \mathbb{Q}_{> 0}$	largest useful production in period t : $\sum_{\tau \geq t} d_\tau + r_n$
m_0	$\in \mathbb{Z}_{\geq 0}$	workers on duty at the start of the horizon
w	$\in \mathbb{Q}_{> 0}$	wage of one worker per period
u	$\in \mathbb{Q}_{\geq 0}$	cost of hiring one worker
r	$\in \mathbb{Q}_{> 0}$	hours worked by one worker in one period
g	$\in \mathbb{Q}_{> 0}$	labour hours needed for one unit of product
s	$\in \mathbb{Z}_{\geq 1}$	number of product types, indexed by j
k	$\in \mathbb{Z}_{\geq 1}$	number of resources, indexed by i
a_{ij}	$\in \mathbb{Q}_{\geq 0}$	amount of resource i per unit of type j
b_i	$\in \mathbb{Q}_{> 0}$	availability of resource i
$\bar{p}_j$	$\in \mathbb{Q}_{> 0}$	unit profit of type j
$\bar{q}_j$	$\in \mathbb{Z}_{\geq 1}$	minimum lot of type j , if it is produced
$\bar{r}$	$\in \mathbb{Q}_{> 0}$	bonus if at least two different types are produced

9.1 Lot sizing with a fixed set-up cost

Script: `python/fam09_1_lotsizing.py`[Notebook in Colab](#)

Problem 9.1 — lot sizing with a fixed cost

A company must plan the production of a single product over $n \in \mathbb{Z}_{\geq 1}$ periods. For every period $t \in \{1, 2, \dots, n\}$, the value $d_t \in \mathbb{Q}_{\geq 0}$ is the demand to be met, $p_t \in \mathbb{Q}_{> 0}$ the cost of producing one unit and $q_t \in \mathbb{Q}_{\geq 0}$ the fixed cost to be paid if production is started in that period. For every period $t \in \{1, 2, \dots, n-1\}$, the value $h_t \in \mathbb{Q}_{\geq 0}$ is the cost of keeping one unit in stock from the end of period t to the start of the next one. The initial inventory is $r_0 \in \mathbb{Q}_{\geq 0}$ and the one required at the end of the horizon is $r_n \in \mathbb{Q}_{\geq 0}$. The company wants to decide how much to produce in every period so as to meet all the demand at minimum total cost.

The problem in words. We *decide* how much to produce in every period, and therefore how much stays in stock. *The objective*: minimum total cost (production, set-ups and storage). *The constraints*: the demand of every period must be met exactly, with what is produced and with what has been kept aside; and one cannot produce without paying the fixed set-up cost. It is the *lot sizing* problem with set-up.

Data (input of the model)

Symbol	Type	Meaning
n	$\in \mathbb{Z}_{\geq 1}$	number of periods, $t \in \{1, 2, \dots, n\}$
d_t	$\in \mathbb{Q}_{\geq 0}$	demand of period t
p_t	$\in \mathbb{Q}_{> 0}$	unit production cost in period t
q_t	$\in \mathbb{Q}_{\geq 0}$	fixed set-up cost in period t
h_t	$\in \mathbb{Q}_{\geq 0}$	storage cost at the end of period t , $t \leq n - 1$
r_0, r_n	$\in \mathbb{Q}_{\geq 0}$	initial and required final inventory

Decision variables

We introduce the following n non-negative continuous variables:

$$x_t = \text{units produced in period } t, \quad \forall t \in \{1, 2, \dots, n\},$$

the following $n - 1$ non-negative continuous variables:

$$s_t = \text{units in stock at the end of period } t, \quad \forall t \in \{1, 2, \dots, n - 1\},$$

and the following n binary variables:

$$y_t = \begin{cases} 1 & \text{if production is started in period } t, \\ 0 & \text{otherwise,} \end{cases} \quad \forall t \in \{1, 2, \dots, n\}.$$

Using these variables, a MILP model for the problem reads as follows:

Model 9.1 — lot sizing with a fixed cost

$$\min \sum_{t=1}^n p_t x_t + \sum_{t=1}^n q_t y_t + \sum_{t=1}^{n-1} h_t s_t \quad (9.1a)$$

$$\text{subject to } x_1 - s_1 = d_1 - r_0, \quad (9.1b)$$

$$x_t + s_{t-1} - s_t = d_t, \quad \forall t \in \{2, \dots, n - 1\}, \quad (9.1c)$$

$$x_n + s_{n-1} = d_n + r_n, \quad (9.1d)$$

$$-x_t + M_t y_t \geq 0, \quad \forall t \in \{1, 2, \dots, n\}, \quad (9.1e)$$

$$x_t \geq 0, \quad \forall t \in \{1, 2, \dots, n\}, \quad (9.1f)$$

$$s_t \geq 0, \quad \forall t \in \{1, 2, \dots, n - 1\}, \quad (9.1g)$$

$$y_t \in \{0, 1\}, \quad \forall t \in \{1, 2, \dots, n\}. \quad (9.1h)$$

Description. The objective function (9.1a) adds up the production cost, the cost of the set-ups and the storage cost. Constraints (9.1b)–(9.1d) are the inventory balance, one per period (n equality constraints): what one had plus what one produces minus what one sells is what is left. Constraints (9.1e) link quantity and set-up (n linear constraints).

The link between the variables. Constraint (9.1e) reads $x_t \leq M_t y_t$. If $y_t = 0$ then $x_t \leq 0$ and, together with $x_t \geq 0$, one gets $x_t = 0$: without a set-up nothing is produced. If $y_t = 1$ the constraint becomes $x_t \leq M_t$ and is not restrictive, provided M_t is chosen large enough. The other direction — if $x_t = 0$ then $y_t = 0$ — is not imposed by any constraint, but follows from optimality: if $x_t = 0$ and $y_t = 1$, setting $y_t = 0$ stays feasible and saves $q_t \geq 0$.

A valid M_t must be at least the largest quantity that it is worth producing in period t . In an optimal solution one never produces more than the demand still to be covered, final inventory included:

$$M_t = \sum_{\tau=t}^n d_{\tau} + r_n.$$

Any larger value is still valid but weakens the LP relaxation, and any smaller value may cut off optimal solutions. On the instance, $M = (110, 90, 80, 50, 10)$: the big-M of the last period is 10, not 110.

The two bounds built below — the primal one, from the heuristic, and the dual one, from the relaxation — are what the solver works with.

The model in gurobipy

```

1 m = gp.Model("lot_sizing")
2 x = m.addVars(n, name="x")
3 s = m.addVars(n - 1, name="s")
4 y = m.addVars(n, vtype=GRB.BINARY, name="y")
5 m.setObjective(gp.quicksum(p[t] * x[t] for t in range(n))
6               + gp.quicksum(q[t] * y[t] for t in range(n))
7               + gp.quicksum(h[t] * s[t] for t in range(n - 1)), GRB.
               MINIMIZE)
8 m.addConstr(x[0] - s[0] == d[0] - r0, name="balance[0]")
9 m.addConstrs((x[t] + s[t - 1] - s[t] == d[t] for t in range(1, n - 1)),
               name="balance")
10 m.addConstr(x[n - 1] + s[n - 2] == d[n - 1] + rn, name=f"balance[{n -
               1}]")
11 m.addConstrs((-x[t] + M[t] * y[t] >= 0 for t in range(n)), name="run")

```

The instance

$n = 5$ periods (days), $r_0 = r_n = 0$:

	$t = 1$	$t = 2$	$t = 3$	$t = 4$	$t = 5$
d_t	20	10	30	40	10
p_t	2	3	2	3	2
q_t	50	50	50	50	50
M_t	110	90	80	50	10

	$t = 1$	$t = 2$	$t = 3$	$t = 4$
h_t	1	1	1	1

The total demand is 110 units.

Model 9.1 on the instance

$$\begin{aligned}
& \min 2x_1 + 3x_2 + 2x_3 + 3x_4 + 2x_5 + s_1 + s_2 + s_3 + s_4 + 50y_1 + 50y_2 + 50y_3 + 50y_4 + 50y_5 \\
& \text{subject to} \quad x_1 \qquad \qquad \qquad -s_1 \qquad \qquad \qquad = 20 \\
& \qquad \qquad x_2 \qquad \qquad \qquad +s_1 - s_2 \qquad \qquad \qquad = 10 \\
& \qquad \qquad \qquad x_3 \qquad \qquad \qquad +s_2 - s_3 \qquad \qquad \qquad = 30 \\
& \qquad \qquad \qquad \qquad x_4 \qquad \qquad \qquad +s_3 - s_4 \qquad \qquad \qquad = 40 \\
& \qquad \qquad \qquad \qquad \qquad x_5 \qquad \qquad \qquad +s_4 \qquad \qquad \qquad = 10 \\
& \qquad -x_1 \qquad \qquad \qquad \qquad \qquad \qquad +110y_1 \qquad \qquad \geq 0 \\
& \qquad \qquad -x_2 \qquad \qquad \qquad \qquad \qquad \qquad +90y_2 \qquad \qquad \geq 0 \\
& \qquad \qquad \qquad -x_3 \qquad \qquad \qquad \qquad \qquad \qquad +80y_3 \qquad \qquad \geq 0 \\
& \qquad \qquad \qquad \qquad -x_4 \qquad \qquad \qquad \qquad \qquad \qquad +50y_4 \qquad \geq 0 \\
& \qquad \qquad \qquad \qquad \qquad -x_5 \qquad \qquad \qquad \qquad \qquad \qquad +10y_5 \geq 0 \\
& \qquad \qquad x_1, \quad x_2, \quad x_3, \quad x_4, \quad x_5 \qquad \qquad \qquad \geq 0 \\
& \qquad \qquad \qquad \qquad \qquad \qquad s_1, \quad s_2, \quad s_3, \quad s_4 \qquad \qquad \geq 0 \\
& \qquad \qquad \qquad \qquad \qquad \qquad \qquad \qquad y_1, \quad y_2, \quad y_3, \quad y_4, \quad y_5 \in \{0, 1\}
\end{aligned}$$

Dual on the instance

$$\begin{aligned}
& \max 20\mu_1 + 10\mu_2 + 30\mu_3 + 40\mu_4 + 10\mu_5 \\
& \text{subject to} \quad \mu_1 \qquad \qquad \qquad -\pi_1 \qquad \qquad \leq 2 \\
& \qquad \qquad \mu_2 \qquad \qquad \qquad -\pi_2 \qquad \qquad \leq 3 \\
& \qquad \qquad \qquad \mu_3 \qquad \qquad \qquad -\pi_3 \qquad \qquad \leq 2 \\
& \qquad \qquad \qquad \qquad \mu_4 \qquad \qquad \qquad -\pi_4 \qquad \leq 3 \\
& \qquad \qquad \qquad \qquad \qquad \mu_5 \qquad \qquad \qquad -\pi_5 \leq 2 \\
& \qquad \qquad \qquad \qquad \qquad \qquad 110\pi_1 \qquad \leq 50 \\
& \qquad \qquad \qquad \qquad \qquad \qquad \qquad 90\pi_2 \qquad \leq 50 \\
& \qquad \qquad \qquad \qquad \qquad \qquad \qquad \qquad 80\pi_3 \qquad \leq 50 \\
& \qquad \qquad \qquad \qquad \qquad \qquad \qquad \qquad \qquad 50\pi_4 \leq 50 \\
& \qquad \qquad \qquad \qquad \qquad \qquad \qquad \qquad \qquad \qquad 10\pi_5 \leq 50 \\
& \qquad -\mu_1 \quad +\mu_2 \qquad \qquad \qquad \leq 1 \\
& \qquad \qquad -\mu_2 \quad +\mu_3 \qquad \qquad \leq 1 \\
& \qquad \qquad \qquad -\mu_3 \quad +\mu_4 \qquad \leq 1 \\
& \qquad \qquad \qquad \qquad -\mu_4 \quad +\mu_5 \leq 1 \\
& \qquad \qquad \mu_1, \quad \mu_2, \quad \mu_3, \quad \mu_4, \quad \mu_5 \qquad \geq 0 \\
& \qquad \qquad \qquad \qquad \qquad \qquad \pi_1, \quad \pi_2, \quad \pi_3, \quad \pi_4, \quad \pi_5 \geq 0
\end{aligned}$$

Constructive heuristics: the primal bound

Two classic rules, both feasible by construction.

(a) **Lot-for-lot.** Every day one produces exactly the demand of the day: no inventory, but a set-up every day. The cost is

$$\sum_{t=1}^5 p_t d_t + \sum_{t=1}^5 q_t = 270 + 250 = 520.$$

(b) **Least unit cost.** One starts from the first uncovered period and chooses to cover with a single run the k periods that minimise the average cost per unit, that is

$$\frac{q + \sum_{i=1}^{k-1} h \cdot (\text{units kept } i \text{ periods})}{\sum_{i=0}^{k-1} d_{t+i}},$$

then one restarts from the first period that is still uncovered.

It is a constructive heuristic of the family seen in chapter 5: a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

The running time is $O(n^2)$. On the instance:

- period 1: covers up to period 2, quantity 30, average cost 2 per unit;
- period 3: covers up to period 4, quantity 70, average cost ≈ 1.286 ;
- period 5: covers period 5 alone, quantity 10, average cost 5.

Runs are started on days 1, 3, 5; the total cost is 420. Keeping the better of the two heuristics,

$$z(MILP) \leq UB = 420.$$

LP relaxation and dual: the dual bound

We associate a free dual variable μ_t with each balance constraint (9.1b)–(9.1d) (equalities) and a non-negative dual variable π_t with each constraint (9.1e) ($\geq$ direction). With $b_1 = d_1 - r_0$, $b_t = d_t$ for $2 \leq t \leq n-1$ and $b_n = d_n + r_n$, the dual of the LP relaxation is:

Dual of the LP relaxation of model 9.1

$$\max \sum_{t=1}^n b_t \mu_t \quad (9.2a)$$

$$\text{subject to } \mu_t - \pi_t \leq p_t, \quad \forall t \in \{1, 2, \dots, n\}, \quad (9.2b)$$

$$M_t \pi_t \leq q_t, \quad \forall t \in \{1, 2, \dots, n\}, \quad (9.2c)$$

$$-\mu_t + \mu_{t+1} \leq h_t, \quad \forall t \in \{1, 2, \dots, n-1\}, \quad (9.2d)$$

$$\mu_t \geq 0, \quad \forall t \in \{1, 2, \dots, n\}, \quad (9.2e)$$

$$\pi_t \geq 0, \quad \forall t \in \{1, 2, \dots, n\}. \quad (9.2f)$$

Description. The variable μ_t is the value attached to one unit available in period t , and π_t is the price of one unit of setup capacity. The objective (9.2a) prices the demand of each period at those values. Constraints (9.2b), one per period, are the columns of the x_t : producing one unit in period t is worth μ_t and consumes setup capacity at price π_t , and the balance cannot exceed the unit cost p_t . Constraints (9.2c) are the columns of the y_t : all the capacity M_t that a setup makes available, priced at π_t , cannot be worth more than the fixed cost q_t . Constraints (9.2d) are the columns of the s_t : carrying one unit from period t to $t + 1$ earns $\mu_{t+1} - \mu_t$, which cannot exceed the holding cost h_t . The μ_t are free because the balances are equalities; the π_t are non-negative because the setup constraints are of type $\geq$ in a minimisation.

A hand-built dual solution. Set $\bar{\pi}_t = 0$ for every t : constraints (9.2c) are satisfied because $q_t \geq 0$, and the set-ups become free. What is left is $\mu_t \leq p_t$ and $\mu_{t+1} \leq \mu_t + h_t$: the largest feasible value is built forward,

$$\bar{\mu}_1 = p_1, \quad \bar{\mu}_t = \min(\bar{\mu}_{t-1} + h_{t-1}, p_t) \quad \text{for } t \geq 2.$$

The reading is direct: $\bar{\mu}_t$ is the lowest unit cost of having one unit available in period t , either by producing it then, or by producing it earlier and keeping it in stock. On the instance $h = 1$ is low and the production costs are 2, 3, 2, 3, 2, so

$$\bar{\mu} = (2, 3, 2, 3, 2),$$

and the dual value is

$$LB = \sum_{t=1}^5 d_t \bar{\mu}_t = 2 \cdot 20 + 3 \cdot 10 + 2 \cdot 30 + 3 \cdot 40 + 2 \cdot 10 = 270.$$

It is the production cost if the set-ups were free: a valid bound, and deliberately optimistic. The LP relaxation, which pays for the set-ups in fractions ($\pi_t = q_t/M_t$ is feasible), goes higher:

$$z(LP) = z(LP^+) = \frac{3890}{11} \approx 353.64.$$

Optimal solution

The optimum is obtained by solving the model: the two bounds just built, the primal and the dual one, are what helps the solver get there.

	$t = 1$	$t = 2$	$t = 3$	$t = 4$	$t = 5$
run y_t	1	0	1	0	0
production x_t	30	0	80	0	0
inventory s_t	10	0	50	10	—

The cost is $z(MILP) = 390$, and the chain of bounds is

$$270 = LB \leq z(LP) = \frac{3890}{11} \leq z(MILP) = 390 \leq UB = 420.$$

The heuristic gap is $(420 - 390)/390 = 7.7\%$; the certified gap, before solving the MILP, is $(420 - 270)/390 = 38.5\%$.

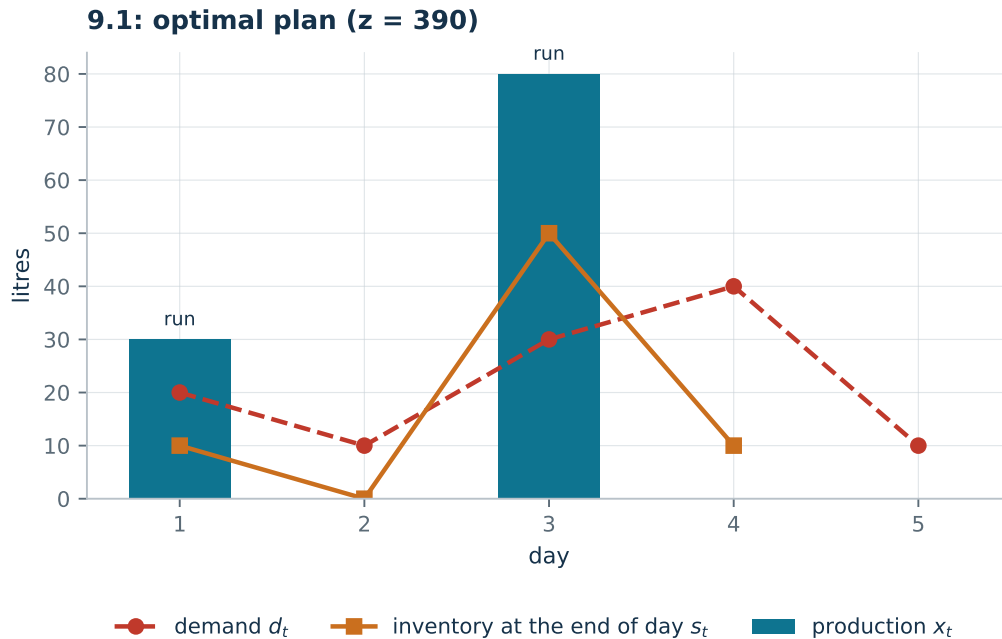


Figure 9.1: Optimal plan: production, demand and inventory at the end of the day. The two runs cover, the first one, days 1–2 and, the second one, days 3–5.

Additional considerations

- The model is a special case of the problem solved in $O(n^2)$ time by the Wagner–Whitin dynamic program. Least unit cost is *not* that algorithm: it is a myopic rule that looks at one run at a time, and in general it does not find the optimum. On the instance it indeed stops at 420 against 390.
- The valid inequality $x_t \leq M_t$, obtained from constraint (9.1e) with $y_t \leq 1$, adds nothing: M_t is already the largest useful production. Indeed $z(LP) = z(LP^+)$.
- An alternative formulation, much tighter, uses the variables $x_{t\tau}$ = "units produced in t and sold in τ " (the *facility location* formulation of lot sizing). Its LP relaxation is integral, but it has $O(n^2)$ variables: it is the typical trade-off between the size of the model and the quality of the relaxation.

Additional modelling questions

Modelling question 9.1 — daily capacity

The plant cannot produce more than 35 units a day. How does the model change? What is the new optimum for the instance?

Modelling question 9.2 — minimum lot

If one produces on a given day, at least 25 units must be produced (below that threshold the set-up is not justified). How does the model change? What is the new optimum?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
UB	420	heuristic solution (UB)
LB	270	certified bound (LB)
$z(MILP)$	390	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 7.7\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 38.5\%.$$

The sandwich on variant 1a

The cap $x_t \leq 35$ adds to the dual a family $\nu_t \geq 0$ with right-hand side -35 : it enters the objective with a negative sign and loosens the column of x_t . Raising ν_t by one unit lifts the cap on μ_t and collects d_t , but costs 35, so it only pays where the demand exceeds the capacity; and on this instance not even the busiest day justifies it, because the chain of the stocks already keeps μ_t below the cap. The certificate is therefore the one of the base problem, $LB = 270$. What changes is the heuristic: on day four lot-for-lot would exceed the cap, and the 5 excess units are anticipated to day three, which holds them in stock.

	value	what it is
UB	520	heuristic solution
LB	270	dual certificate built by hand
$z(LP)$	$\frac{305365}{792}$	relaxation without the bounds
$z(LP^+)$	$\frac{305365}{792}$	relaxation with the bounds
$z(MILP)$	470	optimum of the MILP

9.2 Production and workforce: two equivalent formulations

Script: `python/fam09_2_workforce.py`

[Notebook in Colab](#)

Problem 9.2 — production and workforce

A shoe factory must plan its production over $n \in \mathbb{Z}_{\geq 1}$ months. For every month $t \in \{1, 2, \dots, n\}$, the value $d_t \in \mathbb{Z}_{\geq 0}$ is the demand in pairs and $p_t \in \mathbb{Q}_{>0}$ the cost of the raw materials for one pair. For every month $t \in \{1, 2, \dots, n-1\}$, the value $h_t \in \mathbb{Q}_{\geq 0}$ is the cost of keeping one pair in stock at the end of the month. At the start of the horizon the company has $m_0 \in \mathbb{Z}_{\geq 0}$ workers; every worker works $r \in \mathbb{Q}_{>0}$ hours a month and costs $w \in \mathbb{Q}_{>0}$ euros a month, and producing one pair requires $g \in \mathbb{Q}_{>0}$ labour hours. At the start of every month workers may be hired, at a cost of $u \in \mathbb{Q}_{\geq 0}$ euros each; nobody is laid off. The company wants to decide how much to produce and how many workers to employ

in every month, at minimum total cost.

The problem in words. We *decide* how much to produce, how much to keep in stock and how many workers to hire. *The objective:* minimum total cost (raw materials, storage, wages and hirings). *The constraints:* the demand must be met exactly; the production of a month cannot require more hours than the workers on duty can do.

Data (input of the model)

Symbol	Type	Meaning
n	$\in \mathbb{Z}_{\geq 1}$	number of months, $t \in \{1, 2, \dots, n\}$
d_t	$\in \mathbb{Z}_{\geq 0}$	demand (pairs) of month t
p_t	$\in \mathbb{Q}_{>0}$	cost of the raw materials for one pair
h_t	$\in \mathbb{Q}_{\geq 0}$	storage cost at the end of month t
m_0	$\in \mathbb{Z}_{\geq 0}$	workers on duty at the start
w	$\in \mathbb{Q}_{>0}$	monthly wage of one worker
u	$\in \mathbb{Q}_{\geq 0}$	cost of hiring one worker
r	$\in \mathbb{Q}_{>0}$	hours worked by one worker in one month
g	$\in \mathbb{Q}_{>0}$	labour hours for one pair

Two formulations

The same decision can be written in two ways, and it is worth putting them side by side because it is the first case in the course in which two apparently different models describe the same set of plans.

Formulation A: the hirings. The staff variables are

$$z_t = \text{workers hired at the start of month } t, \quad \forall t \in \{1, 2, \dots, n\},$$

integer and non-negative. A worker hired in month t stays on duty until the end of the horizon, so they cost u once plus w for each of the remaining $n - t + 1$ months. The wage of the m_0 initial workers, $m_0 w n$, is a constant term that does not depend on the decisions: it is left out of the model and added to the final value.

Formulation B: the workforce. The staff variables are

$$y_t = \text{workers on duty in month } t, \quad \forall t \in \{1, 2, \dots, n\},$$

integer and non-negative, with $y_t \geq y_{t-1}$ because nobody is laid off. The cost is $w y_t$ every month, plus u for every hiring, that is $u(y_n - m_0)$ in total, because the hirings are the increments of the workforce and the telescoping sum leaves only the endpoints.

Model 9.2A — with the hirings

$$\min \sum_{t=1}^n p_t x_t + \sum_{t=1}^{n-1} h_t s_t + \sum_{t=1}^n (u + w(n-t+1)) z_t \quad (9.3a)$$

$$\text{subject to } x_1 - s_1 = d_1, \quad (9.3b)$$

$$x_t + s_{t-1} - s_t = d_t, \quad \forall t \in \{2, \dots, n-1\}, \quad (9.3c)$$

$$x_n + s_{n-1} = d_n, \quad (9.3d)$$

$$-g x_t + r \sum_{j=1}^t z_j \geq -r m_0, \quad \forall t \in \{1, 2, \dots, n\}, \quad (9.3e)$$

$$x_t \in \mathbb{Z}_{\geq 0}, \quad \forall t \in \{1, 2, \dots, n\}, \quad (9.3f)$$

$$s_t \in \mathbb{Z}_{\geq 0}, \quad \forall t \in \{1, 2, \dots, n-1\}, \quad (9.3g)$$

$$z_t \in \mathbb{Z}_{\geq 0}, \quad \forall t \in \{1, 2, \dots, n\}. \quad (9.3h)$$

Model 9.2B — with the workforce

$$\min \sum_{t=1}^n p_t x_t + \sum_{t=1}^{n-1} h_t s_t + \sum_{t=1}^n w y_t + u(y_n - m_0) \quad (9.4a)$$

$$\text{subject to } x_1 - s_1 = d_1, \quad (9.4b)$$

$$x_t + s_{t-1} - s_t = d_t, \quad \forall t \in \{2, \dots, n-1\}, \quad (9.4c)$$

$$x_n + s_{n-1} = d_n, \quad (9.4d)$$

$$-g x_t + r y_t \geq 0, \quad \forall t \in \{1, 2, \dots, n\}, \quad (9.4e)$$

$$y_1 \geq m_0, \quad (9.4f)$$

$$-y_{t-1} + y_t \geq 0, \quad \forall t \in \{2, \dots, n\}, \quad (9.4g)$$

$$x_t \in \mathbb{Z}_{\geq 0}, \quad \forall t \in \{1, 2, \dots, n\}, \quad (9.4h)$$

$$s_t \in \mathbb{Z}_{\geq 0}, \quad \forall t \in \{1, 2, \dots, n-1\}, \quad (9.4i)$$

$$y_t \in \mathbb{Z}_{\geq 0}, \quad \forall t \in \{1, 2, \dots, n\}. \quad (9.4j)$$

Description. The two formulations share the stock balance constraints, (9.3b)–(9.3d) and (9.4b)–(9.4d): one per month, they say that what is produced plus what is in stock covers the demand exactly. Constraints (9.3e) and (9.4e), one per month, are the available hours: a month's production cannot require more hours than the workers on the payroll can do. Formulation *B* has in addition constraint (9.4f), which starts from the m_0 initial workers, and

constraints (9.4g), one per month from the second on, which forbid layoffs. In A those same facts are hidden in the domain of the z_t , which are non-negative.

The two formulations describe the same problem

The correspondence is

$$y_t = m_0 + \sum_{j=1}^t z_j, \quad \text{that is} \quad z_t = y_t - y_{t-1} \quad (\text{with } y_0 = m_0).$$

Feasibility. With this substitution constraint (9.3e) becomes exactly (9.4e). The conditions $z_t \geq 0$ become $y_t \geq y_{t-1}$, that is (9.4g), and $y_1 \geq m_0$, that is (9.4f). The balance constraints contain no staff variables and stay identical. The correspondence is therefore a bijection between the feasible plans of the two models.

Cost. It is enough to compute the staff cost in the two ways. In B it is

$$\sum_{t=1}^n w y_t + u (y_n - m_0) = \sum_{t=1}^n w \left(m_0 + \sum_{j \leq t} z_j \right) + u \sum_{t=1}^n z_t.$$

The first term equals $m_0 w n + w \sum_{t=1}^n (n - t + 1) z_t$, because z_j appears in every month from j onwards, that is $n - j + 1$ times. Adding the second one gives

$$m_0 w n + \sum_{t=1}^n (u + w (n - t + 1)) z_t,$$

that is the constant $m_0 w n$ plus exactly the third term of (9.3a). The two objective functions therefore differ by that constant alone, and the two models have the same optima.

Why keep both

Formulation A has fewer constraints (no monotonicity) but cost coefficients that depend on the period; formulation B has uniform coefficients and extends better if layoffs are added (one only needs a second family of variables $\ell_t \geq 0$ and the balance $y_t = y_{t-1} + z_t - \ell_t$). On the instance even the *relaxations* coincide: $z(LP) = 15\,960$ for both. Two formulations that are equivalent on the integers are not always equivalent on the relaxation; here they are, and the check must be made, not assumed.

The model in gurobipy

```

1 m = gp.Model("workforce_B")
2 x = m.addVars(n, vtype=GRB.INTEGER, name="x")
3 s = m.addVars(n - 1, vtype=GRB.INTEGER, name="s")
4 y = m.addVars(n, vtype=GRB.INTEGER, name="y")
5 m.setObjective(gp.quicksum(p[t] * x[t] for t in range(n))
6                 + gp.quicksum(h[t] * s[t] for t in range(n - 1))
7                 + gp.quicksum(w * y[t] for t in range(n)) + u * (y[n -
8                     1] - m0), GRB.MINIMIZE)
9 m.addConstr(x[0] - s[0] == d[0], name="balance[0]")
10 m.addConstrs((x[t] + s[t - 1] - s[t] == d[t] for t in range(1, n - 1)),
11              name="balance")

```

```

10 m.addConstr(x[n - 1] + s[n - 2] == d[n - 1], name=f"balance[{n - 1}]" )
11 m.addConstrs((-g * x[t] + r * y[t] >= 0 for t in range(n)), name="hours
    ")
12 m.addConstr(y[0] >= m0, name="initial_workforce")
13 m.addConstrs((-y[t - 1] + y[t] >= 0 for t in range(1, n)), name="
    no_layoffs")

```

The instance

$n = 3$ months, $m_0 = 2$ workers, $w = 1500$, $u = 100$, $r = 160$ hours, $g = 4$ hours per pair, $h_t = 3$:

	$t = 1$	$t = 2$	$t = 3$
d_t	60	100	140
p_t	15	15	15

With two workers the initial capacity is $2 \cdot 160/4 = 80$ pairs a month: enough for the first month, not for the other two.

Model 9.2 on the instance

$$\begin{aligned}
 & \min 15x_1 + 15x_2 + 15x_3 + 3s_1 + 3s_2 + 1500y_1 + 1500y_2 + 1600y_3 \\
 & \text{subject to} \quad x_1 - s_1 = 60 \\
 & \quad x_2 + s_1 - s_2 = 100 \\
 & \quad x_3 + s_2 = 140 \\
 & \quad -4x_1 + 160y_1 \geq 0 \\
 & \quad -4x_2 + 160y_2 \geq 0 \\
 & \quad -4x_3 + 160y_3 \geq 0 \\
 & \quad y_1 \geq 2 \\
 & \quad -y_1 + y_2 \geq 0 \\
 & \quad -y_2 + y_3 \geq 0 \\
 & \quad x_1, x_2, x_3 \in \mathbb{Z}_{\geq 0} \\
 & \quad s_1, s_2 \in \mathbb{Z}_{\geq 0} \\
 & \quad y_1, y_2, y_3 \in \mathbb{Z}_{\geq 0}
 \end{aligned}$$

Dual on the instance

$$\begin{aligned}
& \max 60\mu_1 + 100\mu_2 + 140\mu_3 - 320\nu_1 - 320\nu_2 - 320\nu_3 \\
& \text{subject to } \mu_1 \qquad \qquad \qquad -4\nu_1 \qquad \qquad \leq 15 \\
& \qquad \qquad \mu_2 \qquad \qquad \qquad -4\nu_2 \qquad \qquad \leq 15 \\
& \qquad \qquad \qquad \mu_3 \qquad \qquad \qquad -4\nu_3 \leq 15 \\
& \qquad -\mu_1 \quad +\mu_2 \qquad \qquad \qquad \leq 3 \\
& \qquad \qquad -\mu_2 \quad +\mu_3 \qquad \qquad \leq 3 \\
& \qquad \qquad \qquad 160\nu_1 + 160\nu_2 + 160\nu_3 \leq 4600 \\
& \qquad \qquad \qquad \qquad 160\nu_2 + 160\nu_3 \leq 3100 \\
& \qquad \qquad \qquad \qquad \qquad 160\nu_3 \leq 1600 \\
& \qquad \mu_1, \quad \mu_2, \quad \mu_3 \qquad \qquad \qquad \geq 0 \\
& \qquad \qquad \nu_1, \quad \nu_2, \quad \nu_3 \geq 0
\end{aligned}$$

Constructive heuristic: the primal bound

"Just in time" production: every month one produces exactly the demand and hires the minimum number of workers needed.

It is a constructive heuristic of the family seen in chapter 5: a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

On the instance: month 1, $\lceil 4 \cdot 60/160 \rceil = 2$ workers, no hiring; month 2, $\lceil 4 \cdot 100/160 \rceil = 3$, one hiring; month 3, $\lceil 4 \cdot 140/160 \rceil = 4$, one more hiring. The cost, including the constant term $m_0 w n = 9000$, is

$$z(MILP) \leq UB = 18\,200.$$

LP relaxation and dual: the dual bound

On formulation A , we associate a free dual variable μ_t with each balance and a non-negative one ν_t with each constraint (9.3e). The dual of the LP relaxation is:

Dual of the LP relaxation of model 9.2A

$$\max \sum_{t=1}^n d_t \mu_t - r m_0 \sum_{t=1}^n \nu_t \quad (9.5a)$$

$$\text{subject to } \mu_t - g \nu_t \leq p_t, \quad \forall t \in \{1, 2, \dots, n\}, \quad (9.5b)$$

$$-\mu_t + \mu_{t+1} \leq h_t, \quad \forall t \in \{1, 2, \dots, n-1\}, \quad (9.5c)$$

$$r \sum_{t=j}^n \nu_t \leq u + w(n-j+1), \quad \forall j \in \{1, 2, \dots, n\}, \quad (9.5d)$$

$$\mu_t \geq 0, \quad \forall t \in \{1, 2, \dots, n\}, \quad (9.5e)$$

$$\nu_t \geq 0, \quad \forall t \in \{1, 2, \dots, n\}. \quad (9.5f)$$

Description. The variable μ_t is the value of one pair available in month t and ν_t the price of one working hour in month t . The objective (9.5a) prices the demand at those values and

subtracts $r m_0 \sum_t \nu_t$, that is, the hours the m_0 initial workers supply for free. Constraints (9.5b) are the columns of the x_t : producing one pair in month t is worth μ_t and consumes g hours at price ν_t , and the balance cannot exceed the raw-material cost p_t . Constraints (9.5c) are the columns of the s_t : holding one pair in stock earns $\mu_{t+1} - \mu_t$, which cannot exceed h_t . Constraints (9.5d) are the columns of the z_j : a worker hired at the beginning of month j supplies r hours in each month from j onwards, and their total value cannot exceed what that hire costs, namely $u + w(n - j + 1)$.

A hand-built dual solution. Set $\bar{\nu}_t = 0$: the working hours are not priced, constraints (9.5d) are satisfied because the right-hand side is positive, and what is left is $\mu_t \leq p_t$ and $\mu_{t+1} \leq \mu_t + h_t$. As in problem 9.1, the largest feasible value is

$$\bar{\mu}_1 = p_1, \quad \bar{\mu}_t = \min(\bar{\mu}_{t-1} + h_{t-1}, p_t).$$

On the instance $\bar{\mu} = (15, 15, 15)$ and

$$\sum_{t=1}^n d_t \bar{\mu}_t = 15 \cdot 300 = 4500,$$

to which the constant term 9000 must be added:

$$LB = 4500 + 9000 = 13\,500.$$

It is the cost of the raw materials alone plus the wage of the workers already on duty: the extra labour is given away. The LP relaxation, which pays for it in fractions of a person, reaches $z(LP) = z(LP^+) = 15\,960$.

Optimal solution

The optimum is obtained by solving the model: the two bounds just built, the primal and the dual one, are what helps the solver get there.

	$t = 1$	$t = 2$	$t = 3$
production x_t	60	120	120
workforce y_t	2	3	3
hirings z_t	0	1	0
inventory s_t	0	20	—

The cost is $z(MILP) = 16\,660$, with the chain

$$13\,500 = LB \leq z(LP) = 15\,960 \leq z(MILP) = 16\,660 \leq UB = 18\,200.$$

The heuristic gap is 9.2%. The optimal plan hires *one* worker instead of two and moves twenty pairs from the third month to the second: paying 3 euros of storage for twenty pairs costs 60 euros against the 1600 of a hiring in the third month.

Additional considerations

- The monotonicity constraint (9.4g) is what makes the problem non-trivial: if one could lay off at no cost, formulation B would split into n independent problems, one per month.

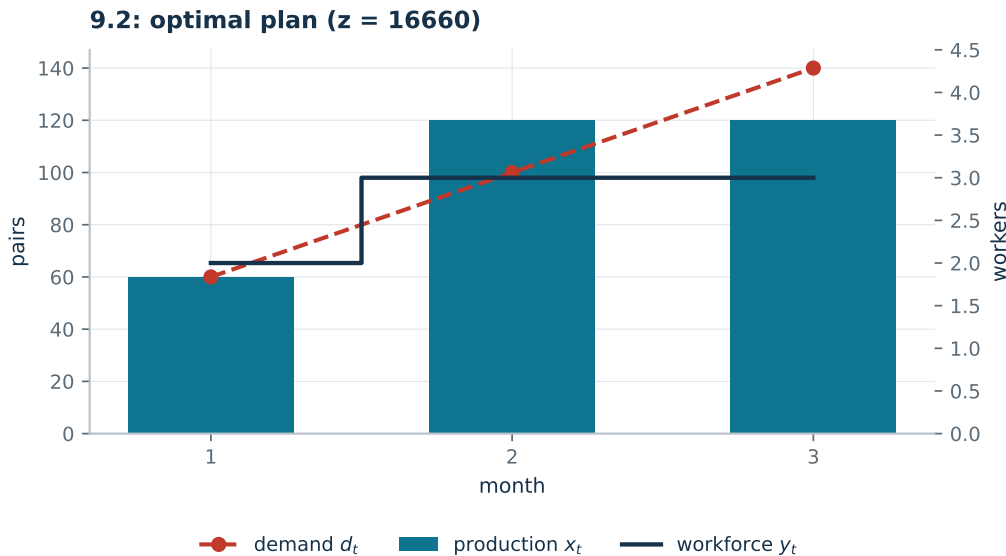


Figure 9.2: Optimal plan: production, demand and workforce. The workforce goes up once only, and the early production covers the final peak.

- The variables x_t and s_t are declared integer because pairs of shoes cannot be split. In this model they could be left continuous without changing the optimum (the data are integer and the balance matrix is totally unimodular), but the declaration is nevertheless the correct one from the modelling point of view.
- The constant term $m_0 w n$ must be remembered in every comparison: forgetting it makes formulation *A* look much cheaper than *B*.

Additional modelling questions

Modelling question 9.3 — very expensive hirings

The hiring cost goes up from 100 to 3000 euros (selection and training). How does the optimal plan change?

Modelling question 9.4 — overtime

Every worker may do up to 40 hours of overtime a month, paid 25 euros an hour. How does the model change? Is it worth using them?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
UB	18200	heuristic solution (UB)
LB	13500	certified bound (LB)
$z(MILP)$	16660	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 9.2\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 28.2\%.$$

The sandwich on variant 2a

The variant adds no constraint: it changes a datum, $u = 3000$ instead of 100. Model, dual and recipe are those of the base problem, and the certificate does not notice the change, because μ_t is the lowest cost of having a pair available in month t and the hiring cost does not appear there: the bound stays 13500, while the optimum rises from 16660 to 19560. It is the limiting case of this chapter: a correct dual bound can be completely insensitive to the datum that moves the problem, and here it is the heuristic that follows it.

	value	what it is
UB	24000	heuristic solution
LB	13500	dual certificate built by hand
$z(LP)$	17490	relaxation without the bounds
$z(LP^+)$	17490	relaxation with the bounds
$z(MILP)$	19560	optimum of the MILP

9.3 Vehicles: minimum lot and a bonus for variety

Script: `python/fam09_3_vehicles.py`

[Notebook in Colab](#)

Problem 9.3 — vehicles with minimum lot and bonus

A company produces $s \in \mathbb{Z}_{\geq 1}$ types of vehicle using $k \in \mathbb{Z}_{\geq 1}$ resources. For every resource $i \in \{1, 2, \dots, k\}$ and every type $j \in \{1, 2, \dots, s\}$, the value $a_{ij} \in \mathbb{Q}_{\geq 0}$ is the amount of resource i needed for one unit of type j , and $b_i \in \mathbb{Q}_{> 0}$ is the availability of resource i . For every type j , the value $\bar{p}_j \in \mathbb{Q}_{> 0}$ is the profit of one unit and $\bar{q}_j \in \mathbb{Z}_{\geq 1}$ the minimum quantity to be produced if that type is chosen. If the production includes at least two different types, the company collects a bonus $\bar{r} \in \mathbb{Q}_{> 0}$ (a grant for diversification). The company wants to maximise the total profit.

The problem in words. We *decide* how many units of each type to produce. *The objective*: maximum profit, bonus included. *The constraints*: the resources must not be exceeded; and of a given type one produces either zero units or at least $\bar{q}_j$.

Data (input of the model)

Symbol	Type	Meaning
s	$\in \mathbb{Z}_{\geq 1}$	number of types, $j \in \{1, 2, \dots, s\}$
k	$\in \mathbb{Z}_{\geq 1}$	number of resources, $i \in \{1, 2, \dots, k\}$
a_{ij}	$\in \mathbb{Q}_{\geq 0}$	resource i for one unit of type j
b_i	$\in \mathbb{Q}_{> 0}$	availability of resource i
$\bar{p}_j$	$\in \mathbb{Q}_{> 0}$	unit profit of type j
$\bar{q}_j$	$\in \mathbb{Z}_{\geq 1}$	minimum lot of type j
$\bar{r}$	$\in \mathbb{Q}_{> 0}$	variety bonus
M_j	$\in \mathbb{Z}_{\geq 1}$	$\min_i \lfloor b_i / a_{ij} \rfloor$: largest producible amount of type j alone

Decision variables

We introduce s non-negative integer variables, s binaries and one binary:

$$x_j = \text{units produced of type } j, \quad \forall j \in \{1, 2, \dots, s\},$$

$$y_j = \begin{cases} 1 & \text{if type } j \text{ is produced,} \\ 0 & \text{otherwise,} \end{cases} \quad z = \begin{cases} 1 & \text{if the bonus is collected,} \\ 0 & \text{otherwise.} \end{cases}$$

Model 9.3 — vehicles with minimum lot and bonus

$$\max \sum_{j=1}^s \bar{p}_j x_j + \bar{r} z \quad (9.6a)$$

$$\text{subject to } \sum_{j=1}^s a_{ij} x_j \leq b_i, \quad \forall i \in \{1, 2, \dots, k\}, \quad (9.6b)$$

$$x_j - \bar{q}_j y_j \geq 0, \quad \forall j \in \{1, 2, \dots, s\}, \quad (9.6c)$$

$$x_j - M_j y_j \leq 0, \quad \forall j \in \{1, 2, \dots, s\}, \quad (9.6d)$$

$$-\sum_{j=1}^s y_j + 2z \leq 0, \quad (9.6e)$$

$$x_j \in \mathbb{Z}_{\geq 0}, \quad \forall j \in \{1, 2, \dots, s\}, \quad (9.6f)$$

$$y_j \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, s\}, \quad (9.6g)$$

$$z \in \{0, 1\}. \quad (9.6h)$$

Description. The objective (9.6a) adds up the profits of the vehicles produced and the bonus for variety. Constraints (9.6b), one per resource, are the availabilities. Constraints (9.6c) and (9.6d), one per type each, make x_j semicontinuous: either zero, or at least the minimum lot $\bar{q}_j$ and at most M_j . Constraint (9.6e), a single one, says that the bonus is collected only if at least two types are active.

The link between the variables: semicontinuity. The two constraints (9.6c) and (9.6d) together say

$$\bar{q}_j y_j \leq x_j \leq M_j y_j.$$

If $y_j = 0$ both give $x_j = 0$. If $y_j = 1$ one gets $\bar{q}_j \leq x_j \leq M_j$. The variable x_j is therefore zero or between the minimum lot and the maximum: it is a *semicontinuous* variable. Neither constraint alone is enough: without (9.6d) a type could be produced with $y_j = 0$; without (9.6c) the minimum lot would be empty.

The bonus is collected only with at least two types

Constraint (9.6e) reads $2z \leq \sum_{j=1}^s y_j$. If $z = 1$ then $\sum_{j=1}^s y_j \geq 2$: at least two types are activated, and by semicontinuity at least two types are actually produced (in quantity $\geq \bar{q}_j > 0$). The other direction — if there are two active types then $z = 1$ — is not imposed by the constraint, but follows from optimality: if $\sum_{j=1}^s y_j \geq 2$ and $z = 0$, setting $z = 1$ stays feasible and increases the objective by $\bar{r} > 0$. Hence in every optimal solution z is a faithful indicator. Question 3b below shows that with $\bar{r} = 0$ this argument breaks down, and z stops being faithful.

The two bounds built below — the primal one, from the heuristic, and the dual one, from the relaxation — are what the solver works with.

The model in gurobipy

```

1 mm = gp.Model("vehicles")
2 x = mm.addVars(n, vtype=GRB.INTEGER, name="x")
3 y = mm.addVars(n, vtype=GRB.BINARY, name="y")
4 z = mm.addVar(vtype=GRB.BINARY, name="z")
5 mm.setObjective(gp.quicksum(p[j] * x[j] for j in range(n)) + r * z, GRB
    .MAXIMIZE)
6 mm.addConstrs((gp.quicksum(a[i][j] * x[j] for j in range(n)) <= b[i]
    for i in range(m)), name="resource")
7 mm.addConstrs((x[j] - q[j] * y[j] >= 0 for j in range(n)), name="
    minimum_lot")
8 mm.addConstrs((x[j] - M[j] * y[j] <= 0 for j in range(n)), name="
    activate")
9 mm.addConstr(-gp.quicksum(y[j] for j in range(n)) + 2 * z <= 0, name="
    bonus")

```

The instance

$s = 3$ types, $k = 2$ resources (steel in tons and labour hours), $\bar{q}_j = 10$ for every type, $\bar{r} = 500$:

a_{ij}	$j = 1$	$j = 2$	$j = 3$		$i = 1$	$i = 2$
$i = 1$ (steel)	2	3	5			
$i = 2$ (hours)	30	25	40			
					b_i	100 1200

	$j = 1$	$j = 2$	$j = 3$
$\bar{p}_j$	200	250	300
M_j	40	33	20

The big-Ms are computed from the data: $M_1 = \min(\lfloor 100/2 \rfloor, \lfloor 1200/30 \rfloor) = 40$, and likewise $M_2 = 33$, $M_3 = 20$.

Constructive heuristic: the primal bound

The problem is a maximisation, so the heuristic gives the *primal* bound, which sits below the optimum. The types are sorted by profit relative to the consumption of the tightest resource, the two best ones are activated at their minimum lot (so the bonus is guaranteed) and then one fills up with the most profitable among those activated.

It is a constructive heuristic of the family seen in chapter 5: a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

On the instance types 2 and 1 are activated at their minimum lot, then one fills up: the production is $(11, 26, 0)$, the steel runs out and 220 hours are left. The profit is 8700 plus the bonus of 500:

$$z(MILP) \geq LB = 9200.$$

LP relaxation and dual: the dual bound

The primal is a maximisation with $\leq$ and $\geq$ constraints: following the conversion table of chapter 4, we associate $\pi_i \geq 0$ with constraints (9.6b), $\lambda_j \leq 0$ with (9.6c), $\beta_j \geq 0$ with (9.6d) and $\gamma \geq 0$ with (9.6e). Writing $\lambda_j = -\ell_j$ with $\ell_j \geq 0$, the dual is:

Dual of the LP relaxation of model 9.3

$$\min \sum_{i=1}^k b_i \pi_i \quad (9.7a)$$

$$\text{subject to } \sum_{i=1}^k a_{ij} \pi_i - \ell_j + \beta_j \geq \bar{p}_j, \quad \forall j \in \{1, 2, \dots, s\}, \quad (9.7b)$$

$$\bar{q}_j \ell_j - M_j \beta_j - \gamma \geq 0, \quad \forall j \in \{1, 2, \dots, s\}, \quad (9.7c)$$

$$2\gamma \geq \bar{r}, \quad (9.7d)$$

$$\pi_i \geq 0, \quad \forall i \in \{1, 2, \dots, k\}, \quad (9.7e)$$

$$\ell_j \geq 0, \quad \forall j \in \{1, 2, \dots, s\}, \quad (9.7f)$$

$$\beta_j \geq 0, \quad \forall j \in \{1, 2, \dots, s\}, \quad (9.7g)$$

$$\gamma \geq 0. \quad (9.7h)$$

Description. The variable π_i is the price of one unit of resource i ; ℓ_j and β_j are the prices of the two semicontinuity constraints of type j , and γ is the price of the bonus. The objective (9.7a) prices all the available resources at those values. Constraints (9.7b) are the columns of the x_j : the resources one unit of type j consumes, priced at π_i and corrected by the two semicontinuity constraints, must cover the profit $\bar{p}_j$ of that unit. Constraints (9.7c) are the columns of the y_j : switching on type j forces at least $\bar{q}_j$ units and allows at most M_j , and the balance of the two prices must cover the bonus γ . Constraint (9.7d) is the column of z : the bonus $\bar{r}$ is collected only with two active types, and indeed it has to be covered by 2γ .

A hand-built dual solution. For example one can proceed in three steps.

1. Constraint (9.7d) forces $\gamma \geq \bar{r}/2$: take the minimum, $\bar{\gamma} = \bar{r}/2 = 250$.
2. Set $\bar{\beta}_j = 0$ and read from constraint (9.7c) the smallest feasible $\bar{\ell}_j$,

$$\bar{\ell}_j = \frac{\bar{\gamma}}{\bar{q}_j} = \frac{250}{10} = 25 \quad \text{for every } j.$$

The reading is that every activated type "carries" its share of the bonus.

3. What is left is (9.7b), which becomes $\sum_{i=1}^k a_{ij} \pi_i \geq \bar{p}_j + \bar{\ell}_j$. One prices *a single* resource, so that it covers all the types:

$$\bar{\pi}_i = \max_j \frac{\bar{p}_j + \bar{\ell}_j}{a_{ij}}, \quad \text{bound} = b_i \bar{\pi}_i,$$

and keeps the resource giving the smaller bound.

On the instance:

$$\begin{aligned} \bar{\pi}_1 &= \max\left(\frac{225}{2}, \frac{275}{3}, \frac{325}{5}\right) = \frac{225}{2}, & b_1 \bar{\pi}_1 &= 100 \cdot \frac{225}{2} = 11\,250, \\ \bar{\pi}_2 &= \max\left(\frac{225}{30}, \frac{275}{25}, \frac{325}{40}\right) = 11, & b_2 \bar{\pi}_2 &= 1200 \cdot 11 = 13\,200. \end{aligned}$$

The better bound is the one from the steel:

$$z(MILP) \leq UB = 11\,250.$$

Optimal solution

The optimum is obtained by solving the model: the two bounds just built, the primal and the dual one, are what helps the solver get there. The optimal production is (26, 16, 0): types 1 and 2 are activated, the bonus is collected, all 100 tons of steel are consumed and 1180 hours out of the 1200 available. The value is

$$z(MILP) = 200 \cdot 26 + 250 \cdot 16 + 500 = 9700,$$

and the chain of bounds is

$$9200 = LB \leq z(MILP) = 9700 \leq z(LP^+) = 9750 \leq z(LP) = \frac{20625}{2} \leq UB = 11\,250.$$

The heuristic gap is 5.2%.

Here the relaxation with the bounds beats the hand-built dual

It is the only problem in the chapter in which $z(LP^+) = 9750$ is *better* than the hand-built dual bound (11 250), and only slightly worse than the optimum (9700). The reason is that the relaxation without the bounds lets y_j and z grow above 1, and with them the bonus: $z(LP) = 20625/2 \approx 10\,312$. Adding $y_j \leq 1$ and $z \leq 1$ removes exactly that freedom. When a model contains rewarded indicators, the relaxation with the bounds is not a detail.

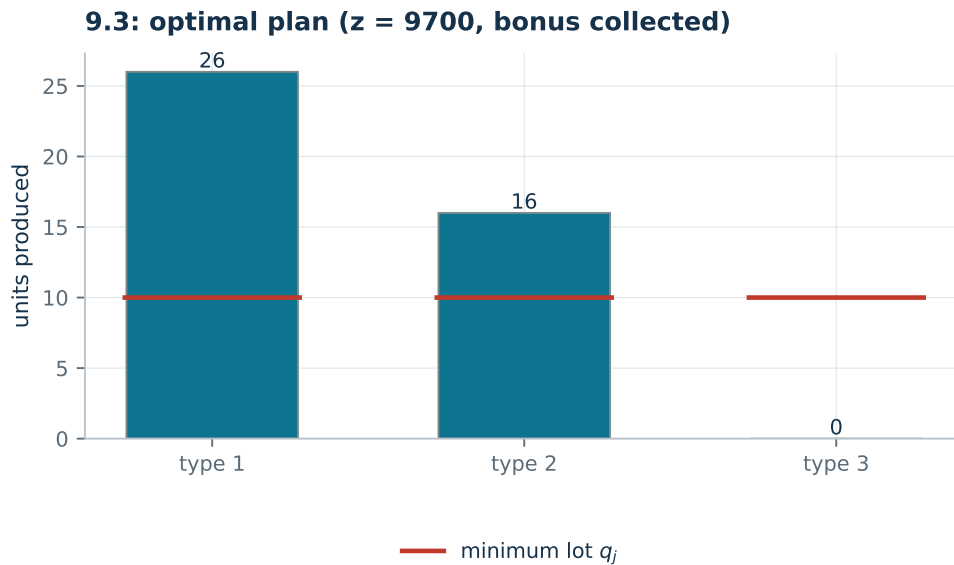


Figure 9.3: Optimal plan: units produced per type, with the minimum lot $\bar{q}_j$ marked in red. Type 3 stays at zero.

Additional considerations

- The big-M M_j is the largest producible amount of type j *alone*, not of the whole plan: it is already a much tighter value than an arbitrary constant, and indeed $M_3 = 20$ while the steel alone would allow 20 units and the hours 30.
- If for some type we had $\bar{q}_j > M_j$, that type would be impossible to produce and could be removed from the model by a check on the data. It is a pre-processing step always worth doing.
- The bonus is modelled with *one* binary and *one* constraint. With a different threshold (at least f types) one only replaces the coefficient 2 with f ; with several bonuses in brackets one would need one variable per bracket and a chain of constraints, as in the piecewise linear functions of section 3.14.

Additional modelling questions

Modelling question 9.5 — a more demanding bonus

The bonus is collected only if at least *three* different types are produced. How does the model change? What is the new optimum?

Modelling question 9.6 — zero bonus

The diversification grant is abolished, that is $\bar{r} = 0$. What happens to the variable z ?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
LB	9200	heuristic solution (LB)
UB	11250	certified bound (UB)
$z(MILP)$	9700	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|LB - z(MILP)|}{|z(MILP)|} = 5.2\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|LB - UB|}{|z(MILP)|} = 21.1\%.$$

The sandwich on variant 3a

The bonus constraint touches a single column of the dual, the one of z , which now sees $\bar{r}/3$ instead of $\bar{r}/2$; the rest of the dual is identical. The recipe is the one of the base problem with $\gamma = \bar{r}/3 = 500/3$, and the bound drops from 11250 to $32500/3$: asking for three types instead of two spreads the bonus over more activations, so each type carries less of it and the dual prices fall. The heuristic starts from the base plan, tries to switch on a third type at its minimum lot and with these resources does not manage: it stays at 8700, and the optimum drops from 9700 to 9200.

	value	what it is
UB	$\frac{32500}{3}$	dual certificate built by hand
LB	8700	heuristic solution
$z(LP)$	$\frac{29875}{3}$	relaxation without the bounds
$z(LP^+)$	$\frac{28750}{3}$	relaxation with the bounds
$z(MILP)$	9200	optimum of the MILP

Mixed models

Class: BIP / MILP

Script: one per problem, `python/fam10_1_prizes.py` ...

[Online page](#)

Why a chapter of mixed models

The three families of the previous chapters have a recognisable structure: one assigns, one locates, one plans. The nine problems in this chapter do not have one, and none should be forced on them: each puts together different pieces, and the modelling work consists precisely in recognising which.

- **Selection with alternative modes** (problems 10.1 and 10.2): a subset is chosen, but each object has more than one way of being chosen, and the ways exclude one another.
- **Counts with a minimum lot** (problem 10.3): the variables are not binary but integer quantities, and a quantity may be zero or else above a threshold.
- **Covering with containers** (problems 10.4 and 10.5): a requirement must be covered by buying packs of fixed composition, and the excess is either paid for or wasted.
- **Splitting and balancing** (problems 10.6–10.9): a set must be divided among several containers and the quality is measured by how much the containers resemble one another.

Four of these problems share a trait that did not appear in the three families: the **LP relaxation is weak**, and in two cases it is exactly zero. The reason is always the same: a fractional solution can split every object in half and put one half in each container, levelling everything. The useful dual bound must then be looked for elsewhere:

- in **parity**, when a count cannot but be even;
- in the **number of containers** needed, read off the capacities;
- in the **dominance of one class** of objects over the others.

These are *combinatorial* arguments: they come from integrality, and the dual of the relaxation cannot see them.

For each problem, as in the families: description, variables, symbolic model, model on the instance, a feasible solution with its primal bound, the linear relaxation with its dual, a dual solution with its dual bound, the optimal solution and the final comparison.

10.1 Prizes obtainable in two ways

Script: `python/fam10_1_prizes.py`

[Notebook in Colab](#)

Problem 10.1 — prizes with two modes

A loyalty programme offers $s \in \mathbb{Z}_{\geq 1}$ prizes and a customer has $p \in \mathbb{Q}_{>0}$ points. Every prize $i \in \{1, 2, \dots, s\}$ can be obtained in two alternative ways: with points only, spending $a_i \in \mathbb{Q}_{>0}$ of them; or spending only $b_i \in \mathbb{Q}_{>0}$ points (with $b_i < a_i$) and adding a cash contribution of $c_i \in \mathbb{Q}_{\geq 0}$ euros. Every prize can be obtained at most once, and in one mode only. Every prize carries a preference value $d_i \in \mathbb{Q}_{>0}$, and the customer wants an overall preference of at least $\ell \in \mathbb{Q}_{>0}$. The customer wants to minimise the money spent.

The problem in words. We *decide* which prizes to take and, for each of them, in which of the two modes. *The objective*: minimum cash contribution. *The constraints*: the points are not enough for everything; the overall preference must reach the threshold; and the same prize is not taken twice.

Data (input of the model)

Symbol	Type	Meaning
s	$\in \mathbb{Z}_{\geq 1}$	number of prizes, $i \in \{1, 2, \dots, s\}$
a_i	$\in \mathbb{Q}_{>0}$	points required by the points-only mode
b_i	$\in \mathbb{Q}_{>0}$	points required by the mode with contribution, $b_i < a_i$
c_i	$\in \mathbb{Q}_{\geq 0}$	contribution in euros of the second mode
d_i	$\in \mathbb{Q}_{>0}$	preference value of prize i
p	$\in \mathbb{Q}_{>0}$	points available
ℓ	$\in \mathbb{Q}_{>0}$	minimum preference required

Decision variables

Two families of binaries are needed, one per mode:

$$x_i = \begin{cases} 1 & \text{if prize } i \text{ is taken with points only,} \\ 0 & \text{otherwise,} \end{cases}$$

$$y_i = \begin{cases} 1 & \text{if prize } i \text{ is taken with points and contribution,} \\ 0 & \text{otherwise,} \end{cases}$$

for every $i \in \{1, 2, \dots, s\}$: $2s$ binary variables in total.

Model 10.1 — prizes with two modes

$$\min \sum_{i=1}^s c_i y_i \quad (10.1a)$$

$$\text{subject to } x_i + y_i \leq 1, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.1b)$$

$$\sum_{i=1}^s (a_i x_i + b_i y_i) \leq p, \quad (10.1c)$$

$$\sum_{i=1}^s d_i (x_i + y_i) \geq \ell, \quad (10.1d)$$

$$x_i, y_i \in \{0, 1\}, \quad \forall i \in \{1, 2, \dots, s\}. \quad (10.1e)$$

Description. The objective (10.1a) counts money only: the points-only mode does not appear, because it costs no euros. Constraints (10.1b) are the mutual exclusion (s linear constraints); constraint (10.1c) is the points budget (a single one); constraint (10.1d) is the preference threshold (a single one).

What mutual exclusion does and does not say

The constraint $x_i + y_i \leq 1$ allows *three* configurations: $(0, 0)$, $(1, 0)$ and $(0, 1)$. It only forbids $(1, 1)$. In particular:

- it does not say "prize i must be taken": the configuration $(0, 0)$ is legitimate and means that the prize is given up;
- it does not say "if I do not take it with points then I take it with a contribution": the converse

$$x_i = 0 \implies y_i = 1$$

is false, and the counterexample is precisely $(0, 0)$;

- if one wanted to force every prize to be taken, the constraint would have to be written as an equality, $x_i + y_i = 1$: that is a set *partitioning* instead of a set *packing*, and the problem changes.

The quantity $x_i + y_i$ is therefore the indicator "prize i has been taken, one way or the other", and it is exactly that sum which appears in the preference constraint.

The two bounds built below — the primal one, from the heuristic, and the dual one, from the relaxation — are what the solver works with.

The model in gurobipy

```
1 m = gp.Model("prizes")
2 x = m.addVars(s, vtype=GRB.BINARY, name="x")
3 y = m.addVars(s, vtype=GRB.BINARY, name="y")
4 m.setObjective(gp.quicksum(c[i] * y[i] for i in range(s)), GRB.MINIMIZE)
5 m.addConstrs((x[i] + y[i] <= 1 for i in range(s)), name="one_mode")
```

```

6 m.addConstr(gp.quicksum(a[i] * x[i] + b[i] * y[i] for i in range(s)) <=
  p, name="points")
7 m.addConstr(gp.quicksum(d[i] * (x[i] + y[i]) for i in range(s)) >= ell,
  name="preference")

```

The instance

$s = 5$ prizes, $p = 20$ points, $\ell = 16$:

	$i = 1$	$i = 2$	$i = 3$	$i = 4$	$i = 5$
a_i	8	6	10	5	7
b_i	4	3	6	2	4
c_i	10	8	15	5	9
d_i	5	4	7	3	6

Constructive heuristic: the primal bound

The prizes are scanned by decreasing preference. Each one is taken with points only if they suffice, otherwise with the contribution if the reduced points suffice, otherwise it is skipped; one stops as soon as the required preference is reached.

It is a constructive heuristic of the family seen in chapter 5: a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

On the instance the order by preference is 3, 5, 1, 2, 4.

- Prize 3 (preference 7): the points alone are enough ($10 \leq 20$), it is taken; 10 points are left, preference 7.
- Prize 5 (preference 6): the points alone are enough ($7 \leq 10$), it is taken; 3 points are left, preference 13.
- Prize 1 (preference 5): the points are not enough for mode a ($8 > 3$) nor for mode b ($4 > 3$): it is skipped.
- Prize 2 (preference 4): the points are not enough for mode a ($6 > 3$) but they are for mode b ($3 \leq 3$): it is taken with a contribution of 8 euros; the preference reaches $17 \geq 16$ and we stop.

The total contribution is

$$z(MILP) \leq UB = 8.$$

LP relaxation and dual: the dual bound

We associate $\sigma_i \geq 0$ with constraints (10.1b), $\pi \geq 0$ with the points constraint (10.1c) and $\rho \geq 0$ with the preference threshold (10.1d). The primal is a minimisation, so the $\leq$ constraints give duals of negative sign: we write $-\sigma_i$ and $-\pi$ with $\sigma_i, \pi \geq 0$. The dual of the LP relaxation is:

Dual of the LP relaxation of model 10.1

$$\max \quad -\sum_{i=1}^s \sigma_i - p\pi + \ell\rho \quad (10.2a)$$

$$\text{subject to} \quad -\sigma_i - a_i\pi + d_i\rho \leq 0, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.2b)$$

$$-\sigma_i - b_i\pi + d_i\rho \leq c_i, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.2c)$$

$$\sigma_i \geq 0, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.2d)$$

$$\pi \geq 0, \quad (10.2e)$$

$$\rho \geq 0. \quad (10.2f)$$

Description. The variable π is the price of one point, ρ the value of one unit of preference and σ_i the price of the mutual exclusion of prize i . The objective (10.2a) collects the threshold ℓ priced at ρ and pays the point budget p priced at π together with the multipliers σ_i . Constraints (10.2b) are the columns of the x_i : taking prize i with points only yields $d_i\rho$ of preference, consumes $a_i\pi$ of points and σ_i of mutual exclusion, and the balance cannot exceed the money cost of that option, which is zero. Constraints (10.2c) say the same for the second option, which consumes only b_i points: there the balance may reach c_i , the contribution that option charges.

A hand-built dual solution. Here the recipe has *two* free parameters, not one: the price π of a point and the price ρ of one unit of preference. Once these are fixed, the multipliers of the mutual exclusion follow from constraint (10.2b) by taking the smallest feasible value,

$$\bar{\sigma}_i = \max(0, d_i\rho - a_i\pi),$$

and only constraints (10.2c) on the second mode remain to be checked. The function

$$V(\pi, \rho) = -\sum_{i=1}^s \bar{\sigma}_i(\pi, \rho) - p\pi + \ell\rho$$

is concave and piecewise linear: it can be explored on a grid. On the instance the maximum is at

$$\bar{\pi} = 2, \quad \bar{\rho} = 3, \quad \bar{\sigma} = (0, 0, 1, 0, 4),$$

with value

$$LB = -(0 + 0 + 1 + 0 + 4) - 20 \cdot 2 + 16 \cdot 3 = -5 - 40 + 48 = 3.$$

This solution is *optimal* for the relaxation without the bounds: indeed $z(LP) = z(LP^+) = 3$.

Here $LB = 3$ and $z(MILP) = 5$: the gap between the dual bound and the integer optimum is 40%, and the certified gap between heuristic and dual is 100%. Nothing is wrong: the LP relaxation may take "half a prize" at half preference, and that freedom is worth a lot. It is the most extreme case in the course, and it is a reminder that a valid bound is not automatically a useful bound.

Optimal solution

The optimum is obtained by solving the model: the two bounds just built, the primal and the dual one, are what helps the solver get there.

	$i = 1$	$i = 2$	$i = 3$	$i = 4$	$i = 5$
points only x_i	0	0	1	0	1
with contribution y_i	0	0	0	1	0

Prizes 3 and 5 are taken with points only ($10 + 7 = 17$) and prize 4 with the contribution (2 points and 5 euros): 19 points are used out of 20, the preference is $7 + 6 + 3 = 16$, exactly the threshold, and the contribution is

$$z(MILP) = 5.$$

The chain of bounds is

$$3 = LB = z(LP) \leq z(MILP) = 5 \leq UB = 8,$$

with a heuristic gap of 60%. The heuristic goes wrong because it only looks at the preference: it takes prize 5 with points (correctly), but then it is left with 3 points and has to buy prize 2 for 8 euros, whereas the optimum saves the points for prize 4, which only costs 5 euros of contribution.

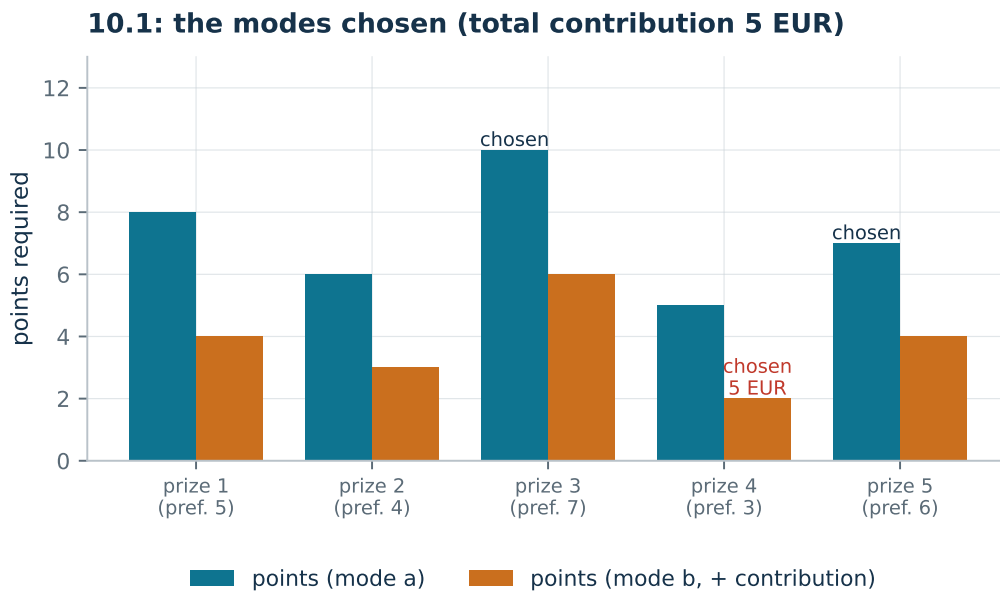


Figure 10.1: For every prize, the points required by the two modes; the labels mark which mode is chosen at the optimum and the contribution paid.

Additional considerations

- The valid inequalities $x_i \leq 1$ and $y_i \leq 1$ are implied by constraints (10.1b): indeed $z(LP) = z(LP^+)$.
- If for some prize we had $c_i = 0$, the second mode would dominate the first one (fewer points, same cost) and the variable x_i could be removed. It is a check on the data that is worth doing.

- The model extends effortlessly to k modes: k families of binaries and the constraint $\sum_{m=1}^k x_i^{(m)} \leq 1$. The structure stays a set packing by rows.

Additional modelling questions

Modelling question 10.1 — alternative prizes

Prizes 3 and 5 come from the same supplier and are alternatives: at most one of them may be taken, in either mode. How does the model change? What is the new optimum?

Modelling question 10.2 — at least four prizes

Besides the preference threshold, the customer wants at least four different prizes. How does the model change? What is the new optimum?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
UB	8	heuristic solution (UB)
LB	3	certified bound (LB)
$z(MILP)$	5	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 60.0\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 100.0\%.$$

The sandwich on variant 1b

The constraint $\sum_i (x_i + y_i) \geq 4$ adds to the dual a $\kappa \geq 0$ with right-hand side 4: it enters the objective and appears in *both* columns of every prize, because a prize counts however it was taken. The recipe of the base problem extends by searching κ too on the grid of quarters, and gives $\pi = \varrho = 3/2$, $\kappa = 9/2$, that is $LB = 6$. The heuristic enumerates the $3^5 = 243$ possible states — each prize not taken, on points or with a contribution — and finds 13, which is the optimum: with five objects enumeration is the most honest rule.

	value	what it is
UB	13	heuristic solution
LB	6	dual certificate built by hand
$z(LP)$	$\frac{25}{2}$	relaxation without the bounds
$z(LP^+)$	$\frac{25}{2}$	relaxation with the bounds
$z(MILP)$	13	optimum of the MILP

10.2 Combinatorial auction

Script: `python/fam10_2_auction.py`

[Notebook in Colab](#)

Problem 10.2 — combinatorial auction

An auctioneer has a set $S = \{1, 2, \dots, n\}$ of $n \in \mathbb{Z}_{\geq 1}$ items to sell and has received $r \in \mathbb{Z}_{\geq 1}$ bids. For every bid $j \in \{1, 2, \dots, r\}$, the set $B_j \subseteq S$ is the subset of items requested and $p_j \in \mathbb{Q}_{>0}$ the profit in euros if the bid is accepted. Each item can be sold at most once, and a bid can be accepted only if all its items are available. The auctioneer wants to select a set of bids of maximum total profit.

The problem in words. We *decide* which bids to accept. *The objective:* maximum profit. *The constraint:* two accepted bids cannot ask for the same item. A bid is all-or-nothing: one does not accept part of it.

Data (input of the model)

Symbol	Type	Meaning
n	$\in \mathbb{Z}_{\geq 1}$	number of items, $i \in S = \{1, 2, \dots, n\}$
r	$\in \mathbb{Z}_{\geq 1}$	number of bids, $j \in \{1, 2, \dots, r\}$
B_j	$\subseteq S$	items requested by bid j
p_j	$\in \mathbb{Q}_{>0}$	profit of bid j

Decision variables

A single family of r binary variables:

$$x_j = \begin{cases} 1 & \text{if bid } j \text{ is accepted,} \\ 0 & \text{otherwise,} \end{cases} \quad \forall j \in \{1, 2, \dots, r\}.$$

Model 10.2 — combinatorial auction

$$\max \sum_{j=1}^r p_j x_j \quad (10.3a)$$

$$\text{subject to } \sum_{j: i \in B_j} x_j \leq 1, \quad \forall i \in S, \quad (10.3b)$$

$$x_j \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, r\}. \quad (10.3c)$$

Description. It is a pure *set packing*: one constraint per item (n linear constraints) and one variable per bid. Constraint (10.3b) says that item i is sold at most once.

By rows, not by pairs

An alternative, and worse, way of writing the same constraint is to list all the *pairs* of conflicting bids: $x_j + x_{j'} \leq 1$ for every $j \neq j'$ with $B_j \cap B_{j'} \neq \emptyset$. On the instance that would be nine constraints instead of four, and on real instances the number of pairs grows like r^2 while the constraints per item stay n . The row form is also tighter: if three bids all ask for item i , the row constraint says $x_1 + x_2 + x_3 \leq 1$, whereas the three pairwise inequalities allow $x_1 = x_2 = x_3 = 1/2$. It is the same comparison between aggregated and disaggregated form already seen in section 3.1, but here the direction of the advantage is

reversed.

The two bounds built below — the primal one, from the heuristic, and the dual one, from the relaxation — are what the solver works with.

The model in gurobipy

```

1 m = gp.Model("auction")
2 x = m.addVars(r, vtype=GRB.BINARY, name="x")
3 m.setObjective(gp.quicksum(p[j] * x[j] for j in range(r)), GRB.MAXIMIZE
4 )
5 m.addConstrs((gp.quicksum(x[j] for j in range(r) if i in B[j]) <= 1
6                 for i in range(n)), name="item")

```

The instance

$n = 4$ items, $r = 6$ bids:

	$j = 1$	$j = 2$	$j = 3$	$j = 4$	$j = 5$	$j = 6$
B_j	{1}	{2}	{3, 4}	{1, 3}	{2, 4}	{1, 3, 4}
p_j	6	3	12	12	10	16

The model of the instance is

$$\max 6x_1 + 3x_2 + 12x_3 + 12x_4 + 10x_5 + 16x_6$$

subject to

$$\begin{aligned}
 x_1 + x_4 + x_6 &\leq 1 && \text{(item 1)} \\
 x_2 + x_5 &\leq 1 && \text{(item 2)} \\
 x_3 + x_4 + x_6 &\leq 1 && \text{(item 3)} \\
 x_3 + x_5 + x_6 &\leq 1 && \text{(item 4)}
 \end{aligned}$$

with $x_j \in \{0, 1\}$.

Constructive heuristic: the primal bound

The problem is a maximisation, so the heuristic gives the *primal* bound, which sits below the optimum. The bids are sorted by decreasing profit *per item* and those whose items are still free are accepted.

It is a constructive heuristic of the family seen in chapter 5: a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

The running time is $O(r \log r + r n)$. On the instance the ratios are 6, 3, 6, 6, 5 and $16/3 \approx 5.33$, so the order is 1, 3, 4, 6, 5, 2:

- bid 1 {1}: accepted (profit 6);
- bid 3 {3, 4}: accepted (profit 12);
- bid 4 {1, 3}: rejected, items 1 and 3 are already sold;
- bid 6 {1, 3, 4}: rejected;

- bid 5 {2, 4}: rejected, item 4 is already sold;
- bid 2 {2}: accepted (profit 3).

Bids 1, 2, 3 are accepted:

$$z(MILP) \geq LB = 21.$$

LP relaxation and dual: the dual bound

We associate a non-negative dual variable λ_i with each constraint (10.3b). The dual of the LP relaxation is:

Dual of the LP relaxation of model 10.2

$$\min \sum_{i \in S} \lambda_i \quad (10.4a)$$

$$\text{subject to } \sum_{i \in B_j} \lambda_i \geq p_j, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.4b)$$

$$\lambda_i \geq 0, \quad \forall i \in S. \quad (10.4c)$$

Description. The variable λ_i is the price the auctioneer puts on item i . The objective (10.4a) is the total value of the lots at those prices. Constraints (10.4b) are the columns of the x_j , one per bid: the items bid j asks for, priced at λ_i , must be worth at least what the bid pays, namely p_j . In short, no bid may turn out to be a bargain.

A hand-built dual solution. For example one can proceed as follows: spread the profit of every bid over its items and take, for every item, the maximum over the bids that ask for it,

$$\bar{\lambda}_i = \max_{j: i \in B_j} \frac{p_j}{|B_j|}.$$

Feasibility is immediate: for every bid j ,

$$\sum_{i \in B_j} \bar{\lambda}_i \geq \sum_{i \in B_j} \frac{p_j}{|B_j|} = |B_j| \cdot \frac{p_j}{|B_j|} = p_j.$$

On the instance:

$$\bar{\lambda}_1 = \max\left(\frac{6}{1}, \frac{12}{2}, \frac{16}{3}\right) = 6, \quad \bar{\lambda}_2 = \max\left(\frac{3}{1}, \frac{10}{2}\right) = 5,$$

$$\bar{\lambda}_3 = \max\left(\frac{12}{2}, \frac{12}{2}, \frac{16}{3}\right) = 6, \quad \bar{\lambda}_4 = \max\left(\frac{12}{2}, \frac{10}{2}, \frac{16}{3}\right) = 6,$$

and

$$z(MILP) \leq UB = 6 + 5 + 6 + 6 = 23.$$

The "obvious" recipe $\bar{\lambda}_i = \max_{j: i \in B_j} p_j$ (without dividing by $|B_j|$) is feasible as well, but gives $16 + 10 + 16 + 16 = 58$: more than twice as much. Dividing by the cardinality of the requested set is what makes the bound useful. It is worth trying several recipes and keeping the best: they are all valid, not all informative.

Optimal solution

The optimum is obtained by solving the model: the two bounds just built, the primal and the dual one, are what helps the solver get there. At the optimum bids 4 ($\{1, 3\}$, profit 12) and 5 ($\{2, 4\}$, profit 10) are accepted: all four items are sold and

$$z(MILP) = 22.$$

The chain of bounds is

$$21 = LB \leq z(MILP) = 22 = z(LP^+) = z(LP) \leq UB = 23,$$

with a heuristic gap of 4.5%. The auctioneer sells everything, but not because a constraint forces it: the constraints are $\leq$, not $=$. With other bids the optimal solution could leave items on the shelf.

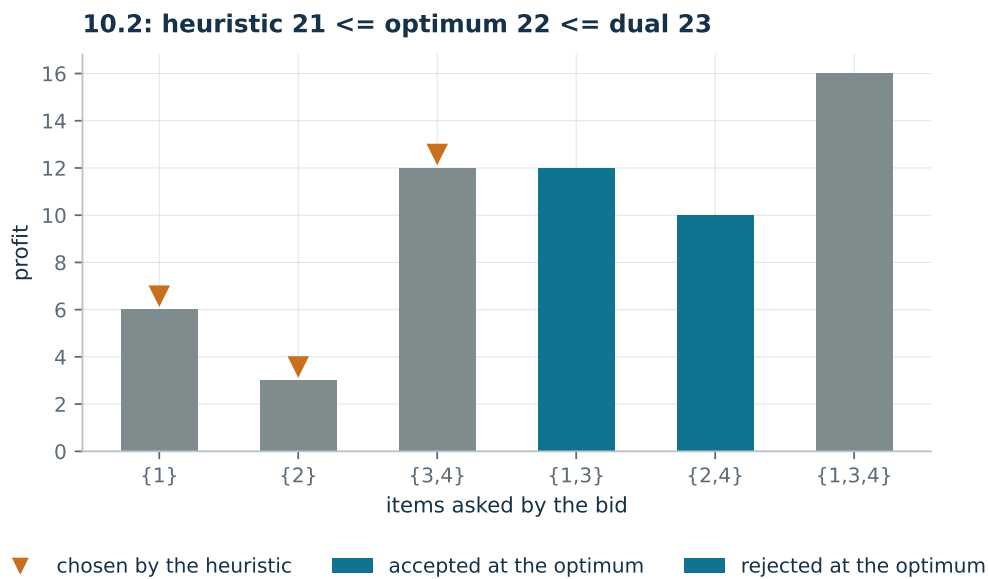


Figure 10.2: The six bids with their profit: in teal those accepted at the optimum, in grey those rejected; the triangle marks those chosen by the heuristic.

Additional considerations

- The valid inequalities $x_j \leq 1$ are implied by constraints (10.3b) whenever $B_j \neq \emptyset$: indeed $z(LP) = z(LP^+) = 22$.
- On this instance one also has $z(LP) = z(MILP)$: the relaxation lands on an integer vertex. It is a lucky case, not a property of set packing. The minimal counterexample is the *triangle*: three items and three bids asking for two of them each, all with profit 1. There $z(LP) = 3/2$ (with $x = 1/2$ on all three) against $z(MILP) = 1$.
- Set packing is NP-hard in general, but it becomes easy when the incidence matrix is *perfect* or *balanced*. The triangle is precisely the smallest graph that is not perfect in this sense, and it is the reason why *clique inequalities* are the classic cuts for this family.

Additional modelling questions

Modelling question 10.3 — bids from the same bidder

Bids 4 and 5 come from the same bidder, who may win at most one of them. How does the model change? What is the new optimum?

Modelling question 10.4 — limited deliveries

In this round the auctioneer can deliver at most two items in total. How does the model change? What is the new optimum?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
LB	21	heuristic solution (LB)
UB	23	certified bound (UB)
$z(MILP)$	22	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|LB - z(MILP)|}{|z(MILP)|} = 4.5\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|LB - UB|}{|z(MILP)|} = 9.1\%.$$

The sandwich on variant 2b

The cap on the items delivered, $\sum_j |B_j| x_j \leq 2$, adds to the dual a $\mu \geq 0$ with right-hand side 2, and in the column of every bid it appears multiplied by how many items that bid asks for. With μ fixed, every bid spreads over its items net of μ and λ_i is the largest among the bids asking for item i : trying μ among the per-item profits one reaches $\mu = 6$, $\lambda = 0$ and $UB = 12$. The heuristic accepts bid 3 alone and is worth 12 as well: the sandwich closes, and the optimum is known without solving the MILP.

	value	what it is
UB	12	dual certificate built by hand
LB	12	heuristic solution
$z(LP)$	12	relaxation without the bounds
$z(LP^+)$	12	relaxation with the bounds
$z(MILP)$	12	optimum of the MILP

10.3 Diet with a count of the foods and a minimum lot

Script: `python/fam10_3_diet.py`

[Notebook in Colab](#)

Problem 10.3 — diet with variety

A nutritionist must compose a monthly diet choosing among $s \in \mathbb{Z}_{\geq 1}$ foods and controlling $r \in \mathbb{Z}_{\geq 1}$ nutrients. For every food $i \in \{1, 2, \dots, s\}$, the value $w_i \in \mathbb{Q}_{>0}$ is the cost of one kilo; for every food i and every nutrient $j \in \{1, 2, \dots, r\}$, the value $g_{ij} \in \mathbb{Q}_{\geq 0}$ is the amount of nutrient j contained in one kilo of food i . For every nutrient j , the monthly intake must lie between $a_j \in \mathbb{Q}_{>0}$ and $b_j \in \mathbb{Q}_{>0}$. If a food enters the diet, at least $c_i \in \mathbb{Q}_{>0}$ kilos of it must be consumed and at most $d_i \in \mathbb{Q}_{>0}$. The diet must include at least $t \in \mathbb{Z}_{\geq 1}$ different foods. The goal is the diet of minimum cost.

The problem in words. We *decide* which foods to use and in what quantity. *The objective:* minimum cost. *The constraints:* every nutrient within its window; every chosen food in a quantity between the minimum lot and the cap; at least t different foods.

Data (input of the model)

Symbol	Type	Meaning
s	$\in \mathbb{Z}_{\geq 1}$	number of foods, $i \in \{1, 2, \dots, s\}$
r	$\in \mathbb{Z}_{\geq 1}$	number of nutrients, $j \in \{1, 2, \dots, r\}$
w_i	$\in \mathbb{Q}_{>0}$	cost of one kilo of food i
g_{ij}	$\in \mathbb{Q}_{\geq 0}$	grams of nutrient j per kilo of food i
a_j, b_j	$\in \mathbb{Q}_{>0}$	monthly minimum and maximum of nutrient j
c_i, d_i	$\in \mathbb{Q}_{>0}$	minimum and maximum quantity of food i , if chosen
t	$\in \mathbb{Z}_{\geq 1}$	minimum number of different foods

Decision variables

s non-negative continuous variables and s binaries:

x_i = kilos of food i consumed in the month,

$$y_i = \begin{cases} 1 & \text{if food } i \text{ enters the diet,} \\ 0 & \text{otherwise.} \end{cases}$$

Model 10.3 — diet with variety

$$\min \sum_{i=1}^s w_i x_i \quad (10.5a)$$

$$\text{subject to } \sum_{i=1}^s g_{ij} x_i \geq a_j, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.5b)$$

$$\sum_{i=1}^s g_{ij} x_i \leq b_j, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.5c)$$

$$x_i - c_i y_i \geq 0, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.5d)$$

$$x_i - d_i y_i \leq 0, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.5e)$$

$$\sum_{i=1}^s y_i \geq t, \quad (10.5f)$$

$$x_i \geq 0, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.5g)$$

$$y_i \in \{0, 1\}, \quad \forall i \in \{1, 2, \dots, s\}. \quad (10.5h)$$

Description. The objective (10.5a) is the total cost. Constraints (10.5b), one per nutrient, impose the minimum requirement; constraints (10.5c), again one per nutrient, impose the cap. Constraints (10.5d) and (10.5e), one per food each, make x_i a semicontinuous variable: either it is zero, or it lies between the minimum lot c_i and the maximum d_i . Constraint (10.5f), a single one, requires at least t different foods to be bought.

Why the count needs the minimum lot

The variety constraint (10.5f) counts the indicators y_i , not the quantities. If only the activation constraint (10.5e) were present, the indicator could be 1 with $x_i = 0$: foods would be "switched on" without a single gram being consumed, and the variety constraint would be satisfied by empty indicators. It is the one-way link of section 3.10, and here it produces a model that does not describe the problem.

The minimum lot constraint (10.5d) closes the loop: with $y_i = 1$ one gets $x_i \geq c_i > 0$, so every switched-on indicator corresponds to a food that is really consumed. On the instance, setting $c_i = 0$ the optimum drops from 48/5 to 46/5 and the solution "switches on" four foods, two of which with zero quantity: the count no longer says anything.

The model in gurobipy

```

1 m = gp.Model("diet")
2 x = m.addVars(s, name="x")
3 y = m.addVars(s, vtype=GRB.BINARY, name="y")
4 m.setObjective(gp.quicksum(w[i] * x[i] for i in range(s)), GRB.MINIMIZE
5 )
6 m.addConstrs((gp.quicksum(g[i][j] * x[i] for i in range(s)) >= a[j] for
7 j in range(r)),
8 name="minimum")

```

```

7 m.addConstrs((gp.quicksum(g[i][j] * x[i] for i in range(s)) <= b[j] for
  j in range(r)),
8              name="maximum")
9 m.addConstrs((x[i] - c[i] * y[i] >= 0 for i in range(s)), name="
  minimum_lot")
10 m.addConstrs((x[i] - d[i] * y[i] <= 0 for i in range(s)), name="
  activate")
11 m.addConstr(gp.quicksum(y[i] for i in range(s)) >= t, name="variety")

```

The instance

$s = 4$ foods, $r = 2$ nutrients, $c_i = 1$, $d_i = 8$, $t = 3$:

	milk	rice	bread	potatoes		iron	calcium
w_i (euros/kg)	2	3	1	4	a_j	60	40
iron (g/kg)	10	20	5	25	b_j	200	150
calcium (g/kg)	5	10	15	5			

Constructive heuristic: the primal bound

The t cheapest foods are switched on at their minimum lot, then the residual requirement of each nutrient is covered by adding the already active food with the lowest cost per gram.

It is a constructive heuristic of the family seen in chapter 5: a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

On the instance the three cheapest foods are bread (1), milk (2) and rice (3): they are switched on at their minimum lot, that is 1 kg each. With these quantities one has $5 + 10 + 20 = 35$ grams of iron (60 are needed) and $15 + 5 + 10 = 30$ of calcium (40 are needed).

- **Iron:** 25 grams are missing. Among the active foods the lowest cost per gram of iron is that of rice, $3/20 = 0.15$; $25/20 = 1.25$ extra kilos are needed, and the rice reaches 2.25 kg.
- **Calcium:** after that addition one has $30 + 12.5 = 42.5$ grams, already above the minimum of 40: nothing else is needed.

The final diet is milk 1 kg, rice 2.25 kg, bread 1 kg, at a cost of

$$z(MILP) \leq UB = \frac{39}{4} = 9.75.$$

LP relaxation and dual: the dual bound

We associate $\alpha_j \geq 0$ with the minima (10.5b), $\beta_j \geq 0$ with the maxima (10.5c), $\lambda_i \geq 0$ with the minimum lots (10.5d), $\mu_i \geq 0$ with the caps (10.5e) and $\tau \geq 0$ with the variety (10.5f). The dual of the LP relaxation is:

Dual of the LP relaxation of model 10.3

$$\max \sum_{j=1}^r a_j \alpha_j - \sum_{j=1}^r b_j \beta_j + t \tau \quad (10.6a)$$

$$\text{subject to } \sum_{j=1}^r g_{ij} (\alpha_j - \beta_j) + \lambda_i - \mu_i \leq w_i, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.6b)$$

$$-c_i \lambda_i + d_i \mu_i + \tau \leq 0, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.6c)$$

$$\alpha_j \geq 0, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.6d)$$

$$\beta_j \geq 0, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.6e)$$

$$\lambda_i \geq 0, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.6f)$$

$$\mu_i \geq 0, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.6g)$$

$$\tau \geq 0. \quad (10.6h)$$

Description. The variable α_j is the price of one unit of nutrient j when it serves to reach the minimum requirement, β_j what is paid not to exceed the cap, λ_i and μ_i the prices of the two semicontinuity constraints of food i , and τ the price of variety. The objective (10.6a) collects the requirements priced at α , pays the caps priced at β and collects the threshold t priced at τ . Constraints (10.6b) are the columns of the x_i : the nutritional content of one unit of food i , priced at $\alpha_j - \beta_j$ and corrected by the two semicontinuity constraints, cannot exceed the price w_i of that food. Constraints (10.6c) are the columns of the y_i : switching on food i grants c_i units of minimum lot at price λ_i , forces at most d_i at price μ_i , and must cover the price τ of variety.

A hand-built dual solution. Set $\beta = \mu = \tau = 0$: the maxima, the caps and the variety are not priced. Constraints (10.6c) become $-c_i \lambda_i \leq 0$, satisfied with $\lambda = 0$. What is left is $\sum_{j=1}^r g_{ij} \alpha_j \leq w_i$ for every food. The simplest recipe prices a *single* nutrient, at the lowest cost per gram among the foods:

$$\bar{\alpha}_j = \min_{i: g_{ij} > 0} \frac{w_i}{g_{ij}}, \quad \text{bound} = a_j \bar{\alpha}_j,$$

and one keeps the nutrient giving the larger bound. On the instance:

$$\text{iron: } \bar{\alpha}_1 = \min\left(\frac{2}{10}, \frac{3}{20}, \frac{1}{5}, \frac{4}{25}\right) = \frac{3}{20}, \quad 60 \cdot \frac{3}{20} = 9,$$

$$\text{calcium: } \bar{\alpha}_2 = \min\left(\frac{2}{5}, \frac{3}{10}, \frac{1}{15}, \frac{4}{5}\right) = \frac{1}{15}, \quad 40 \cdot \frac{1}{15} = \frac{8}{3}.$$

The better one is the iron:

$$z(MILP) \geq LB = 9.$$

Optimal solution

The optimum is obtained by solving the model: the two bounds just built, the primal and the dual one, are what helps the solver get there. The optimal diet is rice 1.4 kg, bread 1.4 kg,

potatoes 1 kg: three different foods, as required, with 60 grams of iron (exactly the minimum) and 40 of calcium (again exactly the minimum). The cost is

$$z(MILP) = \frac{48}{5} = 9.6,$$

and the chain of bounds is

$$9 = LB \leq z(LP) = \frac{46}{5} \leq z(LP^+) = \frac{48}{5} = z(MILP) \leq UB = \frac{39}{4}.$$

The heuristic gap is a mere 1.6%: it is the tightest sandwich of the chapter. Note that here the relaxation with the bounds coincides with the integer optimum while the pure one is below: adding $y_i \leq 1$ does matter, because the caps $d_i = 8$ are much wider than the quantities in play, and without $y_i \leq 1$ the relaxation would use fractional indicators larger than one.

10.3: heuristic diet (39/4 EUR) and optimal one (48/5 EUR)

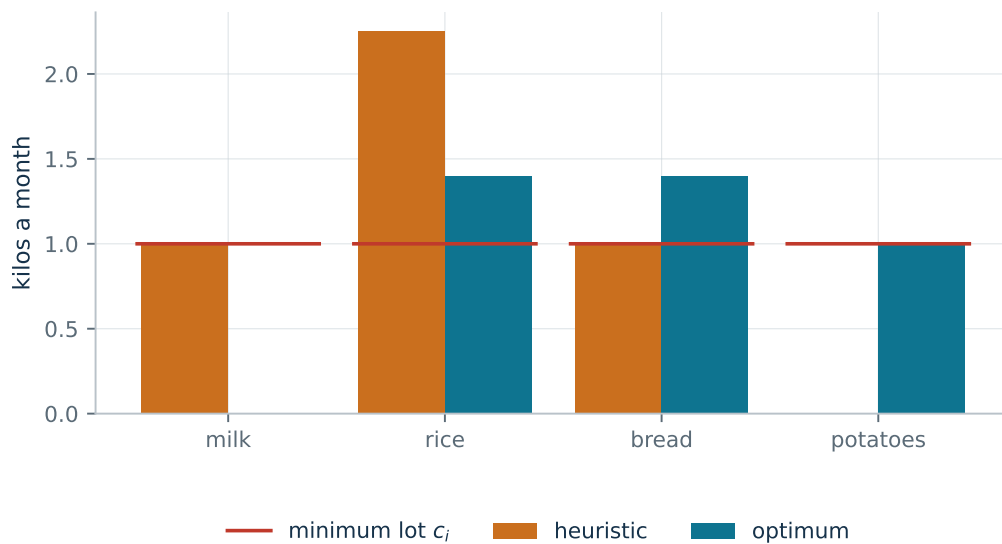


Figure 10.3: Quantity per food in the heuristic diet and in the optimal one; the red bar marks the minimum lot c_i .

Additional considerations

- The maximum constraints (10.5c) are not active at the optimum ($60 \leq 200$ and $40 \leq 150$): on this instance they could be removed without changing anything. They stay in the model because the problem requires them, and because on other instances they would bite.
- The classic form of the diet problem (Stigler, 1945) has neither minimum lots nor counts: it is a pure LP. It is exactly the two integer additions that turn it into a MILP, and that is why the problem belongs to this chapter.
- The variety constraint could also be written as a *covering* constraint: at least one food from each food group, that is $\sum_{i \in G_k} y_i \geq 1$ for every group k . It is a more informative formulation from the nutritional point of view, and no harder.

Additional modelling questions

Modelling question 10.5 — a higher minimum lot

The minimum lot goes up from 1 to 2 kilos for every chosen food. How does the model change? What is the new optimum?

Modelling question 10.6 — more variety

At least four different foods are wanted instead of three. How does the model change? What is the new optimum?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
UB	$\frac{39}{4}$	heuristic solution (UB)
LB	9	certified bound (LB)
$z(MILP)$	$\frac{48}{5}$	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 1.6\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 7.8\%.$$

The sandwich on variant 3b

The variant changes a datum, $t = 4$, not the structure. The count of the foods, however, is not priced: τ enters the objective with its right-hand side, but the column of the y_i imposes $\tau \leq c_i \lambda_i - d_i \mu_i$, and with the recipe of the base problem — $\lambda = \mu = 0$ and a single positive α , on iron — it stays $\tau = 0$. The dual bound does not move, the optimum does: the heuristic switches on the fourth food at its minimum lot, completes with bread and reaches $31/3$, which is the optimum.

	value	what it is
UB	$\frac{31}{3}$	heuristic solution
LB	9	dual certificate built by hand
$z(LP)$	$\frac{28}{3}$	relaxation without the bounds
$z(LP^+)$	$\frac{31}{3}$	relaxation with the bounds
$z(MILP)$	$\frac{31}{3}$	optimum of the MILP

10.4 Christmas trees and boxes of lights

Script: `python/fam10_4_lights.py`

[Notebook in Colab](#)

Problem 10.4 — trees and lights

The city administration wants to decorate $q \in \mathbb{Z}_{\geq 1}$ trees for the holidays. Every tree can be decorated according to one of the $n \in \mathbb{Z}_{\geq 1}$ possible configurations, each characterised by lights of $m \in \mathbb{Z}_{\geq 1}$ different colours. For every configuration $c \in \{1, 2, \dots, n\}$ and every colour $l \in \{1, 2, \dots, m\}$, the value $u_{cl} \in \mathbb{Z}_{\geq 0}$ is the number of lights of colour l required by configuration c , and $i_c \in \mathbb{Q}_{>0}$ is the installation cost of a tree decorated according to it. All the lights must be bought on the market, where they are sold in boxes of $k \in \mathbb{Z}_{\geq 1}$ types: for every type $b \in \{1, 2, \dots, k\}$ and every colour l , the value $v_{bl} \in \mathbb{Z}_{\geq 0}$ is the number of lights of colour l contained in a box of type b , and $p_b \in \mathbb{Q}_{\geq 0}$ is its cost. To ensure a pleasant visual variety at least $f \in \mathbb{Z}_{\geq 1}$ different configurations must be used. The goal is to decorate the trees at minimum total cost.

The problem in words. We *decide* how many trees to decorate with each configuration and how many boxes to buy of each type. *The objective:* minimum total cost (installation plus boxes). *The constraints:* all the trees must be decorated; the lights bought must be enough for those required, colour by colour; at least f different configurations.

Decision variables

n integer variables, k integer variables and n binaries:

$$x_c = \text{trees decorated with configuration } c, \quad y_b = \text{boxes bought of type } b,$$

$$z_c = \begin{cases} 1 & \text{if configuration } c \text{ is used,} \\ 0 & \text{otherwise.} \end{cases}$$

Model 10.4 — trees and lights

$$\min \sum_{c=1}^n i_c x_c + \sum_{b=1}^k p_b y_b \quad (10.7a)$$

$$\text{subject to } \sum_{c=1}^n x_c = q, \quad (10.7b)$$

$$\sum_{b=1}^k v_{bl} y_b - \sum_{c=1}^n u_{cl} x_c \geq 0, \quad \forall l \in \{1, 2, \dots, m\}, \quad (10.7c)$$

$$\sum_{c=1}^n z_c \geq f, \quad (10.7d)$$

$$x_c - z_c \geq 0, \quad \forall c \in \{1, 2, \dots, n\}, \quad (10.7e)$$

$$x_c \in \mathbb{Z}_{\geq 0}, \quad \forall c \in \{1, 2, \dots, n\}, \quad (10.7f)$$

$$y_b \in \mathbb{Z}_{\geq 0}, \quad \forall b \in \{1, 2, \dots, k\}, \quad (10.7g)$$

$$z_c \in \{0, 1\}, \quad \forall c \in \{1, 2, \dots, n\}. \quad (10.7h)$$

Description. The objective (10.7a) adds up the cost of the installations and that of the boxes bought. Constraint (10.7b), a single one, says that the decorated trees are exactly q . Constraints (10.7c), one per colour, say that the lights bought are enough to cover those used. Constraint (10.7d), a single one, imposes at least f different configurations. Constraints (10.7e), one per configuration, tie the indicator z_c to the actual use of the configuration.

The link between the two storeys. Constraint (10.7c) is the availability: for every colour, the lights bought must be at least those required. There is no big-M, because both sides are sums of real quantities. The count of the boxes comes from here: if the lights of colour l required are u_l and a box contains v_{bl} of them, buying only boxes of type b one would need $\lceil u_l/v_{bl} \rceil$ boxes. The model does not write the ceiling: it obtains it by making y_b integer.

The "configuration used" link

Constraint (10.7e) reads $z_c \leq x_c$. If $z_c = 1$ then $x_c \geq 1$: the configuration is really used on at least one tree. The other direction is not imposed, but it is not needed: the minimisation objective has no interest in raising z_c , and constraint (10.7d) forces it to do so exactly f times. Note that no big-M is needed here because x_c and z_c are directly comparable: x_c is a count, not a continuous quantity.

A frequent mistake is to write $x_c \leq M z_c$ thinking of activation: that would be the link in the *other* direction, and would impose that an unused configuration has $x_c = 0$ — true but useless, because it is already guaranteed by $\sum_{c=1}^n x_c = q$ and by $x_c \geq 0$.

The model in gurobipy

```

1 m = gp.Model("lights")
2 x = m.addVars(nc, vtype=GRB.INTEGER, name="x")
3 y = m.addVars(nb, vtype=GRB.INTEGER, name="y")
4 z = m.addVars(nc, vtype=GRB.BINARY, name="z")
5 m.setObjective(gp.quicksum(i[c] * x[c] for c in range(nc))
6                 + gp.quicksum(p[b] * y[b] for b in range(nb)), GRB.
7                 MINIMIZE)
8 m.addConstr(x.sum() == q, name="trees")
9 m.addConstrs((gp.quicksum(v[b][l] * y[b] for b in range(nb))
10               - gp.quicksum(u[c][l] * x[c] for c in range(nc)) >= 0
11               for l in range(nl)), name="lights")
12 m.addConstr(z.sum() >= f, name="variety")
13 m.addConstrs((x[c] - z[c] >= 0 for c in range(nc)), name="used")

```

The instance

$q = 20$ trees, $n = 3$ configurations, $m = 2$ colours, $k = 2$ box types, $f = 2$:

u_{cl}	colour 1	colour 2	i_c
$c = 1$	4	2	7
$c = 2$	2	3	6
$c = 3$	2	2	8

v_{bl}	colour 1	colour 2	p_b
$b = 1$	10	2	100
$b = 2$	15	4	200

The price of one light depends on the colour and on the box type: colour 1 at 10 in box 1 and at 40/3 in box 2; colour 2 at 50 in both. Colour 2 is much more expensive, and it is what drives the solution.

Constructive heuristic: the primal bound

Two phases. First the configurations: $q - f + 1$ trees with the one that is cheapest to install and one tree for each of the other $f - 1$, so that variety is met at the lowest installation cost. Then the boxes: while some light is missing, buy the box with the lowest price per missing light.

It is a constructive heuristic of the family seen in chapter 5: a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

On the instance the cheapest configuration to install is number 2 (cost 6), the second one is number 1 (cost 7): 19 trees are decorated with the 2 and one with the 1. Then 42 lights of colour 1 and 59 of colour 2 are needed, and the constructive heuristic buys 30 boxes of type 1. The total cost is

$$z(MILP) \leq UB = 3121.$$

The heuristic picks the configuration that is cheapest to install, number 2, which is however the greediest for the expensive colour: the boxes pay the bill. It is the typical mistake of a constructive heuristic that looks at a single cost item.

LP relaxation and dual: the dual bound

We associate α free with constraint (10.7b) (an equality), $\beta_l \geq 0$ with constraints (10.7c), $\gamma \geq 0$ with (10.7d) and $\delta_c \geq 0$ with (10.7e). The dual is:

Dual of the LP relaxation of model 10.4

$$\max \quad q\alpha + f\gamma \quad (10.8a)$$

$$\text{subject to} \quad \alpha - \sum_{l=1}^m u_{cl} \beta_l + \delta_c \leq i_c, \quad \forall c \in \{1, 2, \dots, n\}, \quad (10.8b)$$

$$\sum_{l=1}^m v_{bl} \beta_l \leq p_b, \quad \forall b \in \{1, 2, \dots, k\}, \quad (10.8c)$$

$$\gamma - \delta_c \leq 0, \quad \forall c \in \{1, 2, \dots, n\}, \quad (10.8d)$$

$$\alpha \geq 0, \quad (10.8e)$$

$$\beta_l \geq 0, \quad \forall l \in \{1, 2, \dots, m\}, \quad (10.8f)$$

$$\gamma \geq 0, \quad (10.8g)$$

$$\delta_c \geq 0, \quad \forall c \in \{1, 2, \dots, n\}. \quad (10.8h)$$

Description. The variable α is the value of one decorated tree, β_l the price of one light of colour l , γ the price of variety and δ_c that of the link between configuration c and its indicator. The objective (10.8a) prices the q trees at α and the variety threshold f at γ . Constraints (10.8b) are the columns of the x_c : using configuration c on a tree is worth α , consumes u_{cl} lights of each colour and releases δ_c , and the balance cannot exceed the installation cost i_c . Constraints (10.8c)

are the columns of the y_b : a box of type b supplies v_{bl} lights of each colour, and their total value cannot exceed the price p_b of the box. Constraints (10.8d) are the columns of the z_c : the price of variety cannot exceed that of the link which makes it claimable.

A hand-built dual solution. Set $\gamma = 0$ and $\delta_c = 0$: the variety is not priced and constraint (10.8d) is satisfied. What is left are constraints (10.8c) on the boxes, which cap β , and constraints (10.8b) on the configurations, which cap α .

One prices *a single* colour, at the highest price per light that no box can beat:

$$\bar{\beta}_l = \min_{b: v_{bl} > 0} \frac{p_b}{v_{bl}},$$

and then takes the largest feasible α ,

$$\bar{\alpha} = \min_c (i_c + u_{cl} \bar{\beta}_l),$$

that is: every tree costs at least the installation of the most convenient configuration, lights included. On the instance the better colour is number 2, where both box types give the same price per light, $\bar{\beta}_2 = 50$:

$$i_1 + 2 \cdot 50 = 107, \quad i_2 + 3 \cdot 50 = 156, \quad i_3 + 2 \cdot 50 = 108,$$

hence $\bar{\alpha} = 107$ and

$$z(MILP) \geq LB = 20 \cdot 107 = 2140.$$

Optimal solution

The optimum is obtained by solving the model: the two bounds just built, the primal and the dual one, are what helps the solver get there. 19 trees are decorated with configuration 1 and one with configuration 3, and 20 boxes of type 1 are bought:

$$z(MILP) = 19 \cdot 7 + 8 + 20 \cdot 100 = 2141.$$

The lights needed are 78 of colour 1 (200 are bought: many are left over) and 40 of colour 2 (exactly 40 are bought). The chain of bounds is

$$2140 = LB = z(LP) \leq z(LP^+) = 2141 = z(MILP) \leq UB = 3121.$$

The dual bound is off by *one* unit out of 2141, and the strengthened relaxation is exact. The heuristic gap is 45.8%.

Additional considerations

- Colour 1 is left over in abundance: the boxes are bought for colour 2, and colour 1 comes "for free". That is why the dual bound pricing colour 2 alone is almost exact.
- Constraint (10.7b) is an equality: all the trees must be decorated. If it were $\leq q$ the problem would become trivial ($x = 0$ and zero cost); if it were $\geq q$ nothing would change, because decorating more trees costs more.
- The structure "two storeys tied by an availability" appears identically in *cutting stock* problems: below the pieces to be cut, above the bars to be bought.

Additional modelling questions

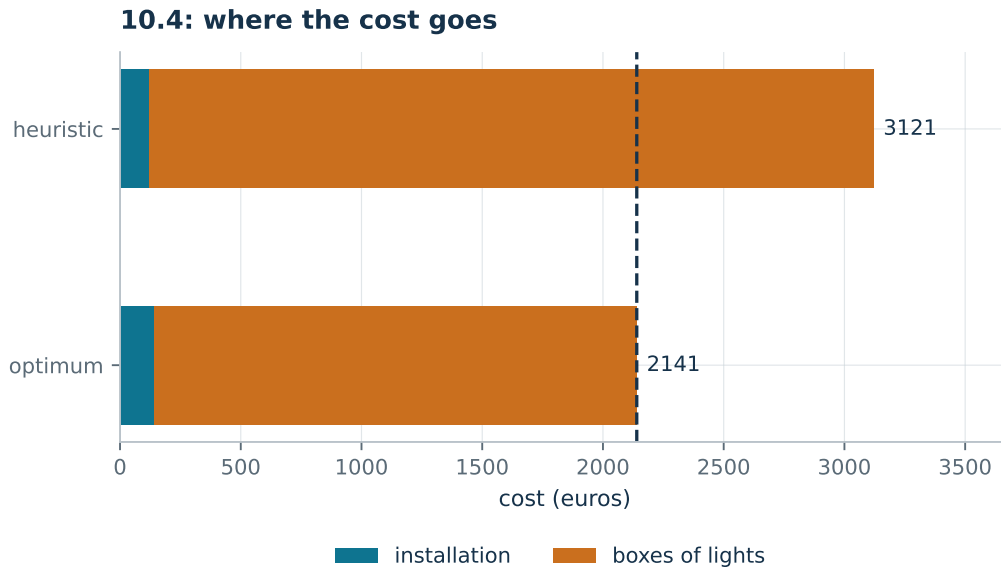


Figure 10.4: Where the cost goes in the two solutions: installation and boxes, with the dual bound dashed.

Modelling question 10.7 — all the configurations

All three configurations are required to appear. How does the model change? What is the new optimum?

Modelling question 10.8 — minimum lot per configuration

Every configuration used must decorate at least three trees (below that threshold it is not worth equipping the crew). How does the model change? What is the new optimum?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
UB	3121	heuristic solution (UB)
LB	2140	certified bound (LB)
$z(MILP)$	2141	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 45.8\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 45.8\%.$$

The sandwich on variant 4a

Asking for all three configurations changes the right-hand side of the variety constraint, not the structure. But that constraint is not priced: the recipe of the base problem keeps $\gamma = \delta = 0$ and puts the whole price on a single colour, so the bound stays 2140 while the optimum rises from 2141 to 2239. The heuristic is the same, and to respect the variety it installs the two missing configurations on one tree each: it costs 3023, and the certified gap stays wide. Here the MILP really is needed.

	value	what it is
UB	3023	heuristic solution
LB	2140	dual certificate built by hand
$z(LP)$	2140	relaxation without the bounds
$z(LP^+)$	2190	relaxation with the bounds
$z(MILP)$	2239	optimum of the MILP

10.5 Shipments in boxes

Script: `python/fam10_5_shipments.py`

[Notebook in Colab](#)

Problem 10.5 — shipments in boxes

A company needs to organise the shipments from its $n \in \mathbb{Z}_{\geq 1}$ production plants to its $m \in \mathbb{Z}_{\geq 1}$ customers. The company produces $k \in \mathbb{Z}_{\geq 1}$ product types. Products are shipped in boxes: every box travels directly from one plant to one customer and has maximum capacity $w \in \mathbb{Z}_{\geq 1}$, expressed as the maximum number of product units it can carry. For every product $p \in \{1, 2, \dots, k\}$ and every customer $c \in \{1, 2, \dots, m\}$, the value $d_{pc} \in \mathbb{Z}_{\geq 0}$ is the number of units ordered. For every product p and every plant $s \in \{1, 2, \dots, n\}$, the value $a_{ps} \in \mathbb{Z}_{\geq 0}$ is the number of units available. The company wants a shipment plan that minimises the total number of boxes used.

The problem in words. We *decide* how many units of each product to ship from each plant to each customer, and how many boxes are needed on each route. *The objective*: the minimum number of boxes. *The constraints*: every order met exactly, no plant beyond its stock, and enough boxes on every route for the units shipped.

For brevity write $P = \{1, 2, \dots, k\}$ for the set of products, $S = \{1, 2, \dots, n\}$ for the set of plants and $C = \{1, 2, \dots, m\}$ for the set of customers.

Decision variables

$k n m$ non-negative integer variables for the flow and $n m$ for the boxes:

$$x_{psc} = \text{units of product } p \text{ shipped from } s \text{ to } c, \quad y_{sc} = \text{boxes shipped from } s \text{ to } c.$$

Model 10.5 — shipments in boxes

$$\min \sum_{s=1}^n \sum_{c=1}^m y_{sc} \quad (10.9a)$$

$$\text{subject to } \sum_{s=1}^n x_{psc} = d_{pc}, \quad \forall p \in P, \forall c \in C, \quad (10.9b)$$

$$\sum_{c=1}^m x_{psc} \leq a_{ps}, \quad \forall p \in P, \forall s \in S, \quad (10.9c)$$

$$-\sum_{p=1}^k x_{psc} + w y_{sc} \geq 0, \quad \forall s \in S, \forall c \in C, \quad (10.9d)$$

$$x_{psc} \in \mathbb{Z}_{\geq 0}, \quad \forall p \in P, \forall s \in S, \forall c \in C, \quad (10.9e)$$

$$y_{sc} \in \mathbb{Z}_{\geq 0}, \quad \forall s \in S, \forall c \in C. \quad (10.9f)$$

Description. The objective (10.9a) counts the boxes used on all the legs. Constraints (10.9b), one per product–customer pair, say that every order is met exactly. Constraints (10.9c), one per product–plant pair, do not allow shipping more than the plant has. Constraints (10.9d), one per leg, say that the boxes shipped from s to c are enough to hold the units loaded.

The link is a rounding up

Constraint (10.9d) reads

$$\sum_{p=1}^k x_{psc} \leq w y_{sc} \quad \Longleftrightarrow \quad y_{sc} \geq \frac{\sum_{p=1}^k x_{psc}}{w},$$

and since y_{sc} is a non-negative integer this is equivalent to

$$y_{sc} \geq \left\lceil \frac{\sum_{p=1}^k x_{psc}}{w} \right\rceil.$$

The ceiling never appears in the model: integrality produces it. The same constraint also imposes the implication

$$\sum_{p=1}^k x_{psc} > 0 \implies y_{sc} \geq 1,$$

that is: if even a single unit leaves a plant towards a customer, then at least one box must travel on that route. The contrapositive is easier to read: if $y_{sc} = 0$ then $\sum_{p=1}^k x_{psc} = 0$. The other direction — if nothing is shipped then no box — is not imposed by the constraint but follows from optimality, because every box costs 1 and the objective is a minimisation.

The model in gurobipy

```
1 m = gp.Model("shipments")
2 x = m.addVars(nk, nn, nm, vtype=GRB.INTEGER, name="x")
3 y = m.addVars(nn, nm, vtype=GRB.INTEGER, name="y")
```

```

4 m.setObjective(y.sum(), GRB.MINIMIZE)
5 m.addConstrs((x.sum(p, "*", c) == d[p][c] for p in range(nk) for c in
    range(nm)),
6             name="demand")
7 m.addConstrs((x.sum(p, s, "*") <= a[p][s] for p in range(nk) for s in
    range(nn)),
8             name="availability")
9 m.addConstrs((w * y[s, c] - x.sum("*", s, c) >= 0 for s in range(nn)
    for c in range(nm)),
10            name="capacity")

```

The instance

$n = 2$ plants, $m = 2$ customers, $k = 2$ products, $w = 10$:

d_{pc}	$c = 1$	$c = 2$	a_{ps}	$s = 1$	$s = 2$
$p = 1$	5	0	$p = 1$	8	6
$p = 2$	2	4	$p = 2$	5	7

The units to ship are $5 + 0 + 2 + 4 = 11$ in total.

Constructive heuristic: the primal bound

Customer by customer: one tries to serve them from a single plant, the one that has everything they need; if no plant is enough, the order is split among several plants. At the end one counts $\lceil \cdot / w \rceil$ boxes per route.

It is a constructive heuristic of the family seen in chapter 5: a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

On the instance customer 1 asks for 5 units of product 1 and 2 of product 2: plant 1 has 8 and 5, so it is enough on its own. Customer 2 asks for 4 units of product 2: after the first shipment plant 1 has 3 left, plant 2 has 7, so the shipment leaves from there. There are two routes, with 7 and 4 units: one box each.

$$z(MILP) \leq UB = 2.$$

LP relaxation and dual: the dual bound

We associate α_{pc} free with the demand constraints (10.9b), $\beta_{ps} \leq 0$ with the availability ones (10.9c) and $\gamma_{sc} \geq 0$ with the link (10.9d). The dual is:

Dual of the LP relaxation of model 10.5

$$\max \sum_{p=1}^k \sum_{c=1}^m d_{pc} \alpha_{pc} + \sum_{p=1}^k \sum_{s=1}^n a_{ps} \beta_{ps} \quad (10.10a)$$

$$\text{subject to } \alpha_{pc} + \beta_{ps} - \gamma_{sc} \leq 0, \quad \forall p \in P, \forall s \in S, \forall c \in C, \quad (10.10b)$$

$$w \gamma_{sc} \leq 1, \quad \forall s \in S, \forall c \in C, \quad (10.10c)$$

$$\alpha_{pc} \geq 0, \quad \forall p \in P, \forall c \in C, \quad (10.10d)$$

$$\beta_{ps} \leq 0, \quad \forall p \in P, \forall s \in S, \quad (10.10e)$$

$$\gamma_{sc} \geq 0, \quad \forall s \in S, \forall c \in C. \quad (10.10f)$$

Description. The variable α_{pc} is the value of one unit of product p delivered to customer c , β_{ps} the (non-positive) price of the availability of product p at plant s , and γ_{sc} the price of one unit of space on the leg from s to c . The objective (10.10a) prices orders and availabilities at those values. Constraints (10.10b) are the columns of the x_{psc} : shipping one unit of product p from s to c is worth $\alpha_{pc} + \beta_{ps}$ and takes up one unit of space at price γ_{sc} ; the balance cannot be positive, because those variables do not appear in the objective. Constraints (10.10c) are the columns of the y_{sc} : one box offers w units of space, and their value cannot exceed 1, which is what the box costs.

A hand-built dual solution. Constraint (10.10c) caps $\gamma_{sc} \leq 1/w$: take the maximum, $\bar{\gamma}_{sc} = 1/w$. With $\bar{\beta} = 0$ constraint (10.10b) becomes $\alpha_{pc} \leq \bar{\gamma}_{sc} = 1/w$: take $\bar{\alpha}_{pc} = 1/w$. The value is

$$LB = \frac{1}{w} \sum_{p=1}^k \sum_{c=1}^m d_{pc} = \frac{11}{10}.$$

The reading is that every unit ordered takes $1/w$ of a box: it is the "volumetric" bound, and it coincides with $z(LP)$.

A stronger integer bound

The volumetric bound ignores that a box cannot be split between two customers. Every customer c with at least one unit ordered receives at least

$$\left\lceil \frac{\sum_{p=1}^k d_{pc}}{w} \right\rceil$$

boxes, and these boxes are different from those of the other customers. Summing over all the customers:

Customer	units ordered	minimum boxes
1	7	1
2	4	1
total	11	2

$$z(MILP) \geq LB = 2,$$

almost twice the volumetric bound $11/10$.

Optimal solution

The optimum is obtained by solving the model: the two bounds just built, the primal and the dual one, are what helps the solver get there. Plant 2 serves both customers: customer 1 with one box containing 5 units of product 1 and 2 of product 2, customer 2 with one box containing 4 units of product 2.

$$z(MILP) = 2, \quad 2 = LB \leq z(MILP) = 2 \leq UB = 2, \quad z(LP) = z(LP^+) = \frac{11}{10}.$$

The heuristic gap is zero, and the integer bound certifies optimality. The difference $2 - 11/10 = 9/10$ is entirely due to integrality: 45% of the optimal value.

10.5: optimal plan with 2 boxes

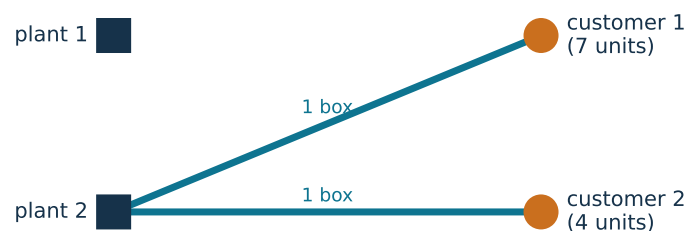


Figure 10.5: The optimal plan: which plants serve which customers and with how many boxes.

Additional considerations

- The model is a multi-product flow with costs on the containers only. If the costs were on the flow (per unit carried) and there were no boxes, the problem would be a pure transportation problem, solvable in polynomial time and with an integral relaxation.
- The variables x_{psc} could be continuous without changing the optimum, because the data are integer and the transportation matrix is totally unimodular *for fixed* y . They stay integer because the problem speaks of units of product.
- Constraint (10.9d) uses the same capacity w on every route. With boxes of different sizes one would need several families $y_{sc}^{(t)}$, one per type, as in problem 10.4.

Additional modelling questions

Modelling question 10.9 — smaller boxes

The boxes hold 4 units instead of 10. What is the new optimum?

Modelling question 10.10 — separate products

Different products cannot travel in the same box. How does the model change? What is the new optimum?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
UB	2	heuristic solution (UB)
LB	2	certified bound (LB)
$z(MILP)$	2	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 0.0\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 0.0\%.$$

The sandwich on variant 5a

The capacity of the box changes, 4 instead of 10, not the structure: the recipe stays $\beta = 0$ and $\gamma = \alpha = 1/w$, the dual constraints stay satisfied by construction and the bound of the relaxation is still “units ordered divided by the capacity of the box”, that is $11/4$ instead of $11/10$ — inversely proportional to the capacity, so smaller boxes raise it. As in the base problem, though, the per-customer count is stronger: every customer receives at least $\lceil (\text{its units})/4 \rceil$ boxes and boxes are not shared between customers, so $LB = 2 + 1 = 3$. The heuristic uses 3: the sandwich closes, and the optimum goes from 2 to 3.

	value	what it is
UB	3	heuristic solution
LB	3	count of the boxes per customer
$z(LP)$	$\frac{11}{4}$	relaxation without the bounds
$z(LP^+)$	$\frac{11}{4}$	relaxation with the bounds
$z(MILP)$	3	optimum of the MILP

10.6 Summer camps

Script: `python/fam10_6_camps.py`

[Notebook in Colab](#)

Problem 10.6 — summer camps

A company runs $r \in \mathbb{Z}_{\geq 1}$ summer camps to host children during the holidays. For every camp $j \in \{1, 2, \dots, r\}$, the value $d_j \in \mathbb{Z}_{\geq 1}$ is the maximum number of children that can be hosted. The company has received requests from children of $s \in \mathbb{Z}_{\geq 1}$ different nationalities: for every nationality $i \in \{1, 2, \dots, s\}$ there are $f_i \in \mathbb{Z}_{\geq 0}$ girls and $g_i \in \mathbb{Z}_{\geq 0}$ boys. In every camp the number of girls must be greater than or equal to the number of boys, and the number of children of nationality $c \in \{1, 2, \dots, s\}$ must be greater than or equal to that of every other nationality. The company wants to maximise the total number of children accepted.

The problem in words. We *decide* how many children of each nationality and of each sex to send to each camp. *The objective:* the largest number of children accepted. *The constraints:* neither the availabilities nor the capacities may be exceeded; and in every camp the two composition rules must hold.

Data (input of the model)

Symbol	Type	Meaning
s	$\in \mathbb{Z}_{\geq 1}$	number of nationalities, $i \in \{1, 2, \dots, s\}$
r	$\in \mathbb{Z}_{\geq 1}$	number of camps, $j \in \{1, 2, \dots, r\}$
f_i	$\in \mathbb{Z}_{\geq 0}$	girls available of nationality i
g_i	$\in \mathbb{Z}_{\geq 0}$	boys available of nationality i
d_j	$\in \mathbb{Z}_{\geq 1}$	capacity of camp j
c	$\in \{1, 2, \dots, s\}$	nationality that must be the majority

Decision variables

The variables are not binary but *counts*: $2rs$ non-negative integer variables,

$$x_{ij} = \text{girls of nationality } i \text{ in camp } j, \quad y_{ij} = \text{boys of nationality } i \text{ in camp } j.$$

Model 10.6 — summer camps

$$\max \sum_{i=1}^s \sum_{j=1}^r (x_{ij} + y_{ij}) \quad (10.11a)$$

$$\text{subject to } \sum_{j=1}^r x_{ij} \leq f_i, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.11b)$$

$$\sum_{j=1}^r y_{ij} \leq g_i, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.11c)$$

$$\sum_{i=1}^s (x_{ij} + y_{ij}) \leq d_j, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.11d)$$

$$\sum_{i=1}^s (x_{ij} - y_{ij}) \geq 0, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.11e)$$

$$x_{cj} + y_{cj} - \sum_{i \neq c} (x_{ij} + y_{ij}) \geq 0, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.11f)$$

$$x_{ij} \in \mathbb{Z}_{\geq 0}, \quad \forall i \in \{1, 2, \dots, s\}, \forall j \in \{1, 2, \dots, r\}, \quad (10.11g)$$

$$y_{ij} \in \mathbb{Z}_{\geq 0}, \quad \forall i \in \{1, 2, \dots, s\}, \forall j \in \{1, 2, \dots, r\}. \quad (10.11h)$$

Description. Constraints (10.11b) and (10.11c) cap the availabilities ($2s$ constraints); constraints (10.11d) the capacities (r constraints); (10.11e) impose "girls $\geq$ boys" in every camp (r constraints); (10.11f) impose that nationality c is not in the minority (r constraints).

The majority constraint with more than two nationalities

With $s = 2$ nationalities the constraint "nationality c is not fewer than any other" is written once only: there is a single "other" nationality. With $s > 2$ the statement of the problem asks for

$$x_{cj} + y_{cj} \geq x_{ij} + y_{ij} \quad \forall i \in \{1, 2, \dots, s\}, i \neq c, \forall j \in \{1, 2, \dots, r\},$$

that is $(s - 1)r$ inequalities. The form (10.11f) written here is *stronger*: it imposes that nationality c is not fewer than *all the others together*, that is that it takes at least half of the places in every camp. The two readings coincide for $s = 2$ and diverge for $s > 2$; the choice must be made explicitly, by reading the statement, and not for convenience of writing. Here we keep the aggregated form because on the instance $s = 2$ and because it immediately yields a combinatorial argument (see below).

The two bounds built below — the primal one, from the heuristic, and the dual one, from the relaxation — are what the solver works with.

The model in gurobipy

```

1 m = gp.Model("camps")
2 x = m.addVars(s, r, vtype=GRB.INTEGER, name="x")
3 y = m.addVars(s, r, vtype=GRB.INTEGER, name="y")
4 m.setObjective(gp.quicksum(x[i, j] + y[i, j] for i in range(s) for j in
   range(r)),
5               GRB.MAXIMIZE)
6 m.addConstrs((x.sum(i, "*") <= f[i] for i in range(s)), name="girls")
7 m.addConstrs((y.sum(i, "*") <= g[i] for i in range(s)), name="boys")
8 m.addConstrs((gp.quicksum(x[i, j] + y[i, j] for i in range(s)) <= d[j]
9               for j in range(r)), name="capacity")
10 m.addConstrs((gp.quicksum(x[i, j] - y[i, j] for i in range(s)) >= 0
11               for j in range(r)), name="balance")
12 m.addConstrs((x[c, j] + y[c, j] - gp.quicksum(x[i, j] + y[i, j]
13               for i in range(s) if i != c) >= 0 for j in range(r)),
               name="majority")

```

The instance

$s = 2$ nationalities, $r = 2$ camps, $c = 1$:

	$i = 1$	$i = 2$		$j = 1$	$j = 2$
f_i	8	10	d_j	15	8
g_i	4	12			

In total there are 34 children available and 23 places.

Constructive heuristic: the primal bound

The problem is a maximisation. One camp is filled at a time, taking first the majority nationality (girls and then boys) and then the others, stopping as soon as one of the three constraints would break.

It is a constructive heuristic of the family seen in chapter 5: a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

On the instance:

- **Camp 1** (capacity 15): all 8 girls and all 4 boys of nationality 1 are taken, then 3 girls of nationality 2. The camp is full: 15 places, 12 of nationality 1 against 3 of nationality 2 (majority respected), 11 girls against 4 boys (balance respected).
- **Camp 2** (capacity 8): nationality 1 is exhausted, so any child of nationality 2 would violate the majority. The camp stays empty.

$$z(MILP) \geq LB = 15.$$

LP relaxation and dual: the dual bound

We associate $\alpha_i, \beta_i, \gamma_j \geq 0$ with the three groups of $\leq$ constraints and $\delta_j, \varepsilon_j \geq 0$ with the two groups of composition constraints. The dual of the LP relaxation is:

Dual of the LP relaxation of model 10.6

$$\min \sum_{i=1}^s f_i \alpha_i + \sum_{i=1}^s g_i \beta_i + \sum_{j=1}^r d_j \gamma_j \quad (10.12a)$$

$$\text{subject to } \alpha_i + \gamma_j - \delta_j + \sigma_i \varepsilon_j \geq 1, \quad \forall i \in \{1, 2, \dots, s\}, \forall j \in \{1, 2, \dots, r\}, \quad (10.12b)$$

$$\beta_i + \gamma_j + \delta_j + \sigma_i \varepsilon_j \geq 1, \quad \forall i \in \{1, 2, \dots, s\}, \forall j \in \{1, 2, \dots, r\}, \quad (10.12c)$$

$$\alpha_i \geq 0, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.12d)$$

$$\beta_i \geq 0, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.12e)$$

$$\gamma_j \geq 0, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.12f)$$

$$\delta_j \geq 0, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.12g)$$

$$\varepsilon_j \geq 0, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.12h)$$

Description. Here $\sigma_i = -1$ for $i = c$ and $\sigma_i = +1$ otherwise. The variable α_i is the price of a place for the girls of nationality i and β_i that for the boys; γ_j is the price of a place in camp j , δ_j the price of the balance constraint and ε_j that of the majority constraint. The objective (10.12a) prices the availability of each nationality and the capacity of each camp. Constraints (10.12b) are the columns of the x_{ij} : accepting one girl of nationality i in camp j uses one place of her nationality and one of the camp, raises the balance by one unit and moves the majority by σ_i ; the total value must cover the unit that girl contributes to the primal objective. Constraints (10.12c) say the same for the boys, with the sign of the balance reversed.

A hand-built dual solution. The simplest recipe prices only the capacity: $\alpha = \beta = \delta = \varepsilon = 0$ and $\gamma_j = 1$ for every camp. All the dual constraints become $\gamma_j \geq 1$ and are satisfied, and

$$UB = \sum_{j=1}^r d_j = 15 + 8 = 23.$$

The reading is obvious: every accepted child takes one place, so no more children can be accepted than there are places. And it is also *optimal*: on the relaxation without the bounds $z(LP) = 23$.

Two more combinatorial arguments

The bound 23 is not the only one that can be read off the data. Constraint (10.11f) says that in every camp nationality c takes at least half of the places; since there are $f_c + g_c = 12$ children of that nationality in total, at most $2 \cdot 12 = 24$ children can be accepted. Likewise constraint (10.11e) says that in every camp the girls are at least half, and there are 18 girls: at most $2 \cdot 18 = 36$ children can be accepted.

Argument	upper bound
capacity of the camps	23
majority nationality	24
girls available	36

On this instance the capacity wins, but that is not a rule: variant 11a below enlarges camp 1 and hands the lead to the majority nationality.

Optimal solution

The optimum is obtained by solving the model: the two bounds just built, the primal and the dual one, are what helps the solver get there.

	camp 1		camp 2	
	girls	boys	girls	boys
nationality 1	8	0	0	4
nationality 2	1	6	4	0
total	15 out of 15		8 out of 8	

Both camps are full: $z(MILP) = 23$, and the chain of bounds is

$$15 = LB \leq z(MILP) = 23 = z(LP) = UB.$$

The heuristic gap is 34.8%, and the dual bound settles the problem: the whole gap was on the solution side, not on the certificate side. The mistake of the heuristic is clear: it uses up the majority nationality in the first camp, and in the second one nobody is left who can form the majority.

Additional considerations

- The variables are integer but not binary: it is the first family of the course in which the counts take large values, and the relaxation stays tight anyway because all the constraints are sums.
- The balance constraint and the majority constraint are *independent*: one can have camps with many girls and few children of nationality c , and vice versa. On the instance it is the second one that bites.
- If a nationality had zero children available, the majority constraint would automatically exclude it from every camp in which anyone else appears: a limit case worth checking on the data.

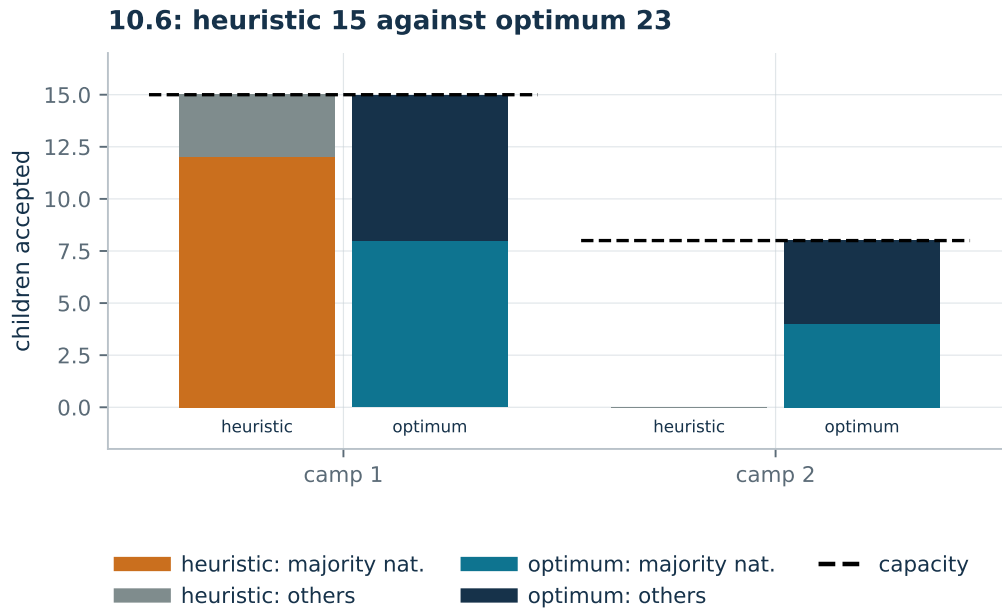


Figure 10.6: Children accepted per camp, split between majority nationality and the others, in the heuristic solution and in the optimal one; the dashed line is the capacity.

Additional modelling questions

Modelling question 10.11 — a bigger camp

Camp 1 is enlarged and reaches 20 places. What is the new optimum?

Modelling question 10.12 — a nationality that cannot be split

For organisational reasons the children of nationality 1 must all stay in the same camp. How does the model change? What is the new optimum?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
LB	15	heuristic solution (LB)
UB	23	certified bound (UB)
$z(MILP)$	23	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|LB - z(MILP)|}{|z(MILP)|} = 34.8\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|LB - UB|}{|z(MILP)|} = 34.8\%.$$

The sandwich on variant 6a

Camp 1 goes from 15 to 20 places: a datum changes, not the structure. The recipe is the one of the base problem — $\gamma_j = 1$ and all the other dual variables at zero, because every child accepted takes a place — and it returns $\sum_j d_j$, which is now 28 instead of 23. The certificate does not change shape: the datum it adds up does. The heuristic fills the camps in order and reaches 20, the optimum is 24.

	value	what it is
UB	28	dual certificate built by hand
LB	20	heuristic solution
$z(LP)$	24	relaxation without the bounds
$z(LP^+)$	24	relaxation with the bounds
$z(MILP)$	24	optimum of the MILP

10.7 Antitrust split

Script: `python/fam10_7_antitrust.py`

[Notebook in Colab](#)

Problem 10.7 — antitrust split

A company has $s \in \mathbb{Z}_{\geq 1}$ branches and sells $r \in \mathbb{Z}_{\geq 1}$ products. For every branch $i \in \{1, 2, \dots, s\}$ and every product $j \in \{1, 2, \dots, r\}$, the value $v_{ij} \in \mathbb{Q}_{\geq 0}$ is the revenue in millions of euros generated by that branch with that product. Because of a new antitrust regulation the company must split into two smaller companies; every branch is indivisible and must be assigned to exactly one of the two. The company wants to partition the branches minimising the maximum difference, over all the products, between the revenues of the two new companies.

The problem in words. We *decide* which of the two companies each branch goes to. *The objective*: to make the two companies as similar as possible, measuring similarity on the product where they do worst. *The constraint*: every branch to a single company.

Decision variables

One family of s binaries is enough:

$$x_i = \begin{cases} 1 & \text{if branch } i \text{ goes to company } A, \\ 0 & \text{if it goes to company } B, \end{cases} \quad \forall i \in \{1, 2, \dots, s\},$$

plus a free variable z for the min-max.

One family or two?

The classic statement introduces *two* families, x_i and y_i , with the constraint $x_i + y_i = 1$. The two writings are equivalent, because $y_i = 1 - x_i$: substituting one obtains exactly the model below, with s fewer variables and s fewer constraints. It is better to keep the aggregated form as long as the companies are *two*; with three or more companies the disaggregated form $\sum_{k=1}^2 x_{ik} = 1$ is the only possible one, and it is the one that extends. Recognising when two models are the same thing is part of the craft: here the check is a substitution,

in other cases (problem 9.2) it requires a proof.

With $T_j = \sum_{i=1}^s v_{ij}$ the total revenue on product j , the revenue of A is $\sum_{i=1}^s v_{ij} x_i$ and that of B is $T_j - \sum_{i=1}^s v_{ij} x_i$: their difference is $2 \sum_{i=1}^s v_{ij} x_i - T_j$.

Model 10.7 — antitrust split

$$\min z \quad (10.13a)$$

$$\text{subject to } z - 2 \sum_{i=1}^s v_{ij} x_i + T_j \geq 0, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.13b)$$

$$z + 2 \sum_{i=1}^s v_{ij} x_i - T_j \geq 0, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.13c)$$

$$x_i \in \{0, 1\}, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.13d)$$

$$z \geq 0. \quad (10.13e)$$

Description. The objective (10.13a) is the worst imbalance. Constraints (10.13b) and (10.13c), one per product each, say that z is no smaller than the imbalance of product j nor than its opposite: together they impose $z \geq |2 \sum_i v_{ij} x_i - T_j|$, and the minimisation turns them into an equality on the worst product.

Absolute value and min-max in two lines

The two inequalities (10.13b) and (10.13c) say

$$-z \leq 2 \sum_{i=1}^s v_{ij} x_i - T_j \leq z,$$

that is $z \geq |2 \sum_{i=1}^s v_{ij} x_i - T_j|$ for every product j . Since z appears alone in the minimisation objective, at the optimum it equals exactly the maximum of the absolute values: it is technique 3.6 (min-max) applied to technique 3.7 (absolute value). The direction " z is no larger than necessary" is not imposed by any constraint, but by optimality: if z were larger, it could be lowered while staying feasible.

The model in gurobipy

```

1 m = gp.Model("antitrust")
2 x = m.addVars(s, vtype=GRB.BINARY, name="x")
3 z = m.addVar(lb=-GRB.INFINITY, name="z")
4 m.setObjective(z, GRB.MINIMIZE)
5 for j in range(r):
6     tot = sum(v[i][j] for i in range(s))
7     m.addConstr(z - 2 * gp.quicksum(v[i][j] * x[i] for i in range(s)) +
8                 tot >= 0)
9     m.addConstr(z + 2 * gp.quicksum(v[i][j] * x[i] for i in range(s)) -
10                 tot >= 0)

```

The instance

$s = 4$ branches, $r = 3$ products:

v_{ij}	$j = 1$	$j = 2$	$j = 3$	branch total
$i = 1$	3	3	2	8
$i = 2$	6	8	5	19
$i = 3$	3	4	4	11
$i = 4$	2	7	9	18
T_j	14	22	20	

Constructive heuristic: the primal bound

The branches are assigned in decreasing order of total revenue, each one to the company that currently has the smaller total. It is the analogue of LPT for scheduling.

It is a constructive heuristic of the family seen in chapter 5: a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

On the instance the totals are 8, 19, 11, 18, so the order is 2, 4, 3, 1: branch 2 to A ($A = 19$), branch 4 to B ($B = 18$), branch 3 to B ($B = 29$), branch 1 to A ($A = 27$). One gets $A = \{1, 2\}$, $B = \{3, 4\}$, with differences 4, 0 and 6 on the three products:

$$z(MILP) \leq UB = 6.$$

LP relaxation and dual: the bound is zero

We associate $\lambda_j \geq 0$ with constraints (10.13b) and $\mu_j \geq 0$ with those (10.13c). The column of z (a free variable) gives an equality constraint:

Dual of the LP relaxation of model 10.7

$$\max \sum_{j=1}^r T_j (\mu_j - \lambda_j) \quad (10.14a)$$

$$\text{subject to } \sum_{j=1}^r (\lambda_j + \mu_j) = 1, \quad (10.14b)$$

$$2 \sum_{j=1}^r v_{ij} (\mu_j - \lambda_j) \leq 0, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.14c)$$

$$\lambda_j \geq 0, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.14d)$$

$$\mu_j \geq 0, \quad \forall j \in \{1, 2, \dots, r\}. \quad (10.14e)$$

Description. The variables λ_j and μ_j are the prices of the two constraints that squeeze the imbalance of product j , one from above and one from below. The objective (10.14a) prices the total T_j of each product at those values. Constraint (10.14b) is the column of z : the variable appears in every constraint with coefficient 1 and in the primal objective with cost 1, so the prices of the $2r$ constraints must sum to exactly one — the dual distributes a single unit of weight among the products. Constraints (10.14c) are the columns of the x_i : moving branch i

from one company to the other changes the imbalance of product j by $2v_{ij}$, and the weighted sum of these moves cannot be positive, because the x_i do not appear in the objective.

A hand-built dual solution. Take $\bar{\lambda}_1 = \bar{\mu}_1 = 1/2$ and everything else zero: constraint (10.14b) is satisfied, $\mu_j - \lambda_j = 0$ for every j and therefore so are constraints (10.14c). The value is $LB = 0$.

No feasible dual solution is worth more than zero. Indeed, setting $\theta_j = \mu_j - \lambda_j$, the objective is $\sum_{j=1}^r T_j \theta_j$ and constraints (10.14c) impose $\sum_{j=1}^r v_{ij} \theta_j \leq 0$ for every branch; summing over all the branches one gets $\sum_{j=1}^r T_j \theta_j \leq 0$. The dual is therefore worth at most 0, and by strong duality $z(LP) = 0$.

The primal certificate is even simpler: the fractional solution $x_i = 1/2$ for every branch with $z = 0$ is feasible for the relaxation and balances every product exactly. Half a branch to each company: legitimate for the LP, meaningless for the problem.

A combinatorial bound, product by product

If the relaxation says nothing, one must look elsewhere. For every product j one can compute, looking at *that product alone*, the smallest imbalance achievable:

$$g_j = \min_{S \subseteq \{1,2,\dots,s\}} \left| 2 \sum_{i \in S} v_{ij} - T_j \right|.$$

It is the classic partition problem on a single column, and with a small s it is solved by enumeration (2^s subsets). Every feasible partition of the complete problem is in particular a partition for product j , so its imbalance on product j is at least g_j ; since the objective is the maximum over all products,

$$z(MILP) \geq \max_j g_j.$$

On the instance:

Product	total T_j	g_j
1	14	2
2	22	0
3	20	2

(on product 1 the values are 3, 6, 3, 2: the sum 7 is not reachable by any subset, and the best is 6 against 8, that is $g_1 = 2$). Hence

$$z(MILP) \geq LB = 2,$$

a bound that the LP relaxation cannot see, because it comes from integrality and not from the constraints.

Optimal solution

The optimum is obtained by solving the model: the two bounds just built, the primal and the dual one, are what helps the solver get there. The optimal partition is $A = \{2, 3\}$ and $B = \{1, 4\}$, with differences 4, 2 and 2 on the three products:

$$z(MILP) = 4,$$

and the chain of bounds is

$$2 = LB \leq z(MILP) = 4 \leq UB = 6, \quad z(LP) = z(LP^+) = 0.$$

The heuristic gap is 50%. Note that LB is not the value of the dual (0) but the combinatorial bound: in the summary table of the chapter the column is called "certified bound" precisely for this reason.

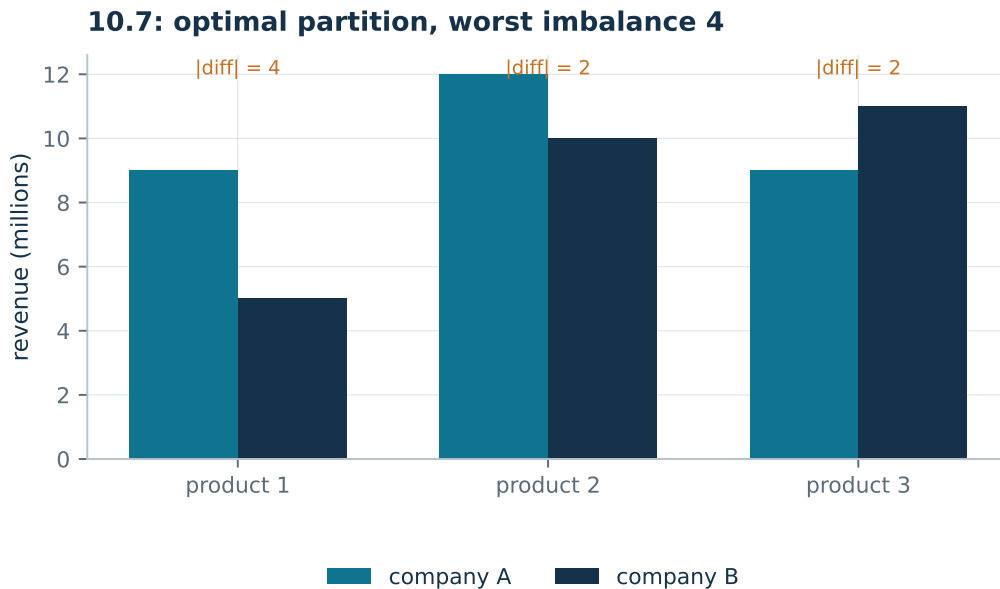


Figure 10.7: Revenue of the two optimal companies, product by product, with the difference highlighted.

Additional considerations

- The problem with a single product is *number partitioning*, one of Karp's twenty-one NP-complete problems. With several products it is its vector version, and it stays NP-hard.
- The combinatorial bound can be strengthened: instead of the maximum of the g_j one can look, for every pair of products, for the minimum of the maximum of the two imbalances. The cost grows but the bound improves, and it is the same idea as *surrogate relaxation*.
- The min-max objective is not the only possible one: minimising the sum of the differences is another legitimate choice and gives a model with r auxiliary variables instead of one (see question 10.9).

Additional modelling questions

Modelling question 10.13 — two inseparable branches

Branches 1 and 2 share their premises and must stay in the same company. How does the model change? What is the new optimum?

Modelling question 10.14 — min-sum instead of min-max

One wants to minimise the *sum* of the differences over all the products instead of the worst difference. How does the model change? Is the optimal partition the same?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
UB	6	heuristic solution (UB)
LB	2	certified bound (LB)
$z(MILP)$	4	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 50.0\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 100.0\%.$$

The sandwich on variant 7a

The equality $x_1 - x_2 = 0$ adds to the dual a free σ , which appears with a plus sign in the column of branch 1 and with a minus in that of branch 2. Its right-hand side is zero, so the objective does not change: the relaxation is worth 0 as in the base problem, and for the same reason. The useful bound is combinatorial again: keeping branches 1 and 2 together, the best possible imbalance on the three products is 4, 0 and 2, and every feasible partition must respect all of them, so $z(MILP) \geq 4$. With the branches free it was 2: tying two branches raises the bound.

	value	what it is
UB	6	heuristic solution
LB	4	smallest imbalance with branches 1 and 2 tied
$z(LP)$	0	relaxation without the bounds
$z(LP^+)$	0	relaxation with the bounds
$z(MILP)$	6	optimum of the MILP

10.8 Songs on CDs

Script: `python/fam10_8_cds.py`

[Notebook in Colab](#)

Problem 10.8 — songs on CDs

A music production company publishes a collection of $n \in \mathbb{Z}_{\geq 1}$ songs. For every song $i \in \{1, 2, \dots, n\}$, the value $d_i \in \mathbb{Q}_{\geq 1}$ is the duration in minutes. The collection must be distributed over $m \in \mathbb{Z}_{\geq 1}$ compact discs; every song must be assigned to exactly one CD, and every CD $j \in \{1, 2, \dots, m\}$ must contain at least $w_j \in \mathbb{Z}_{\geq 1}$ songs. The company wants

to minimise the difference between the total duration of the longest CD and that of the shortest one.

The problem in words. We *decide* which CD every song ends up on. *The objective:* CDs as homogeneous as possible in duration. *The constraints:* every song on one CD only, and no CD too sparse.

Decision variables

$n m$ binary variables and two non-negative continuous ones:

$$x_{ij} = \begin{cases} 1 & \text{if song } i \text{ goes on CD } j, \\ 0 & \text{otherwise,} \end{cases}$$

y = duration of the longest CD, z = duration of the shortest CD.

Model 10.8 — songs on CDs

$$\min \quad y - z \quad (10.15a)$$

$$\text{subject to} \quad \sum_{j=1}^m x_{ij} = 1, \quad \forall i \in \{1, 2, \dots, n\}, \quad (10.15b)$$

$$\sum_{i=1}^n x_{ij} \geq w_j, \quad \forall j \in \{1, 2, \dots, m\}, \quad (10.15c)$$

$$-\sum_{i=1}^n d_i x_{ij} + y \geq 0, \quad \forall j \in \{1, 2, \dots, m\}, \quad (10.15d)$$

$$\sum_{i=1}^n d_i x_{ij} - z \geq 0, \quad \forall j \in \{1, 2, \dots, m\}, \quad (10.15e)$$

$$x_{ij} \in \{0, 1\}, \quad \forall i \in \{1, 2, \dots, n\}, \forall j \in \{1, 2, \dots, m\}, \quad (10.15f)$$

$$y, z \geq 0. \quad (10.15g)$$

Description. The objective (10.15a) is the difference between the longest and the shortest duration. Constraints (10.15b), one per song, say that every song ends up on exactly one CD. Constraints (10.15c), one per CD, impose the minimum number of songs. Constraints (10.15d), one per CD, push y above every duration, and constraints (10.15e), again one per CD, push z below every duration: at the optimum the first becomes the maximum of the durations and the second the minimum.

Two auxiliary variables, two opposite directions

Constraints (10.15d) say $y \geq \sum_{i=1}^n d_i x_{ij}$ for every CD: y is *at least* the maximum. Constraints (10.15e) say $z \leq \sum_{i=1}^n d_i x_{ij}$ for every CD: z is *at most* the minimum. The objective $\min(y - z)$ pushes y down and z up, so at the optimum y equals exactly the largest duration and z exactly the smallest one. It is technique 3.5 used twice, in two opposite directions: note that one needs an objective that pushes *both* variables in the right direction, otherwise

one of the two would stay undetermined.

The model in gurobipy

```

1 mod = gp.Model("cds")
2 x = mod.addVars(n, m, vtype=GRB.BINARY, name="x")
3 y = mod.addVar(name="y")
4 z = mod.addVar(name="z")
5 mod.setObjective(y - z, GRB.MINIMIZE)
6 mod.addConstrs((x.sum(i, "*") == 1 for i in range(n)), name="song")
7 mod.addConstrs((x.sum("*", j) >= w[j] for j in range(m)), name="minimum
8     ")
9 mod.addConstrs((y - gp.quicksum(d[i] * x[i, j] for i in range(n)) >= 0
10     for j in range(m)), name="maximum")
11 mod.addConstrs((gp.quicksum(d[i] * x[i, j] for i in range(n)) - z >= 0
12     for j in range(m)), name="minimum_duration")

```

The instance

$n = 6$ songs, $m = 2$ CDs, $w_j = 1$:

	$i = 1$	$i = 2$	$i = 3$	$i = 4$	$i = 5$	$i = 6$
d_i	5	6	7	3	4	10

The total duration is 35 minutes.

Two heuristics compared: the primal bound

The insertion rule is always the same — every song on the currently shortest CD — but the order in which the songs are scanned changes the result a lot.

It is a constructive heuristic of the family seen in chapter 5: a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

(a) Natural order. $5 \rightarrow \text{CD1}$, $6 \rightarrow \text{CD2}$, $7 \rightarrow \text{CD1}$, $3 \rightarrow \text{CD2}$, $4 \rightarrow \text{CD2}$, $10 \rightarrow \text{CD1}$. Final durations 22 and 13: difference 9.

(b) Decreasing duration (LPT). $10 \rightarrow \text{CD1}$, $7 \rightarrow \text{CD2}$, $6 \rightarrow \text{CD2}$, $5 \rightarrow \text{CD1}$, $4 \rightarrow \text{CD2}$, $3 \rightarrow \text{CD1}$. Final durations 18 and 17: difference 1.

The same rule, with two different orders, gives 9 and 1. The better one is kept:

$$z(\text{MILP}) \leq UB = 1.$$

LP relaxation and dual: zero again

We associate α_i free with constraints (10.15b) (equalities), $\beta_j \geq 0$ with (10.15c), $\gamma_j \geq 0$ with (10.15d) and $\delta_j \geq 0$ with (10.15e). The columns of y and z give $\sum_{j=1}^m \gamma_j = 1$ and $\sum_{j=1}^m \delta_j = 1$.

Dual of the LP relaxation of model 10.8

$$\max \sum_{i=1}^n \alpha_i + \sum_{j=1}^m w_j \beta_j \quad (10.16a)$$

$$\text{subject to } \sum_{j=1}^m \gamma_j = 1, \quad (10.16b)$$

$$\sum_{j=1}^m \delta_j = 1, \quad (10.16c)$$

$$\alpha_i + \beta_j - d_i \gamma_j + d_i \delta_j \leq 0, \quad \forall i \in \{1, 2, \dots, n\}, \forall j \in \{1, 2, \dots, m\}, \quad (10.16d)$$

$$\alpha_i \geq 0, \quad \forall i \in \{1, 2, \dots, n\}, \quad (10.16e)$$

$$\beta_j \geq 0, \quad \forall j \in \{1, 2, \dots, m\}, \quad (10.16f)$$

$$\gamma_j \geq 0, \quad \forall j \in \{1, 2, \dots, m\}, \quad (10.16g)$$

$$\delta_j \geq 0, \quad \forall j \in \{1, 2, \dots, m\}. \quad (10.16h)$$

Description. The variable α_i is the value of song i , β_j the price of the minimum number of songs on CD j , while γ_j and δ_j are the weights with which CD j enters the maximum and the minimum duration respectively. The objective (10.16a) prices the songs and the thresholds w_j . Constraints (10.16b) and (10.16c) are the columns of y and z : each of the two variables appears in one constraint per CD and in the primal objective with coefficient ± 1 , so the respective weights must sum to one. Constraints (10.16d) are the columns of the x_{ij} : putting song i on CD j satisfies its assignment constraint (α_i), contributes to the minimum number of songs (β_j) and moves both durations by d_i ; the balance cannot be positive, because the x_{ij} do not appear in the objective.

A hand-built dual solution. With $\bar{\gamma}_j = \bar{\delta}_j = 1/m$ and $\alpha = \beta = 0$ the two equality constraints are satisfied and constraints (10.16d) become $-d_i/m + d_i/m = 0 \leq 0$. The value is $LB = 0$.

That the relaxation is worth zero is best seen from the primal side: putting $x_{ij} = 1/m$ for every song and every CD, every CD "lasts" $35/2 = 17.5$ minutes and the difference is zero. It is feasible for the relaxation and meaningless for the problem, because a song cannot be split. The LP relaxation of balancing models is almost always like this: it levels everything and is worth zero.

A parity argument that settles the problem

The durations are integers and the CDs are two: their durations q_1 and q_2 are integers adding up to $q = 35$, which is *odd*. Two integers adding up to an odd number cannot be equal, and their difference $|q_1 - q_2|$ has the same parity as q , that is it is odd. An odd non-negative difference is at least 1:

$$z(MILP) \geq LB = 1.$$

The LPT heuristic reaches exactly 1: the two bounds coincide, and the heuristic solution is optimal. One knows this *before* calling the solver, and it is the clearest case in the course in which a two-line combinatorial argument does the job that the LP relaxation cannot do.

Optimal solution

The optimum is obtained by solving the model: the two bounds just built, the primal and the dual one, are what helps the solver get there.

	songs	duration
CD 1	1, 2, 3	18
CD 2	4, 5, 6	17

$$1 = LB \leq z(MILP) = 1 \leq UB = 1, \quad z(LP) = z(LP^+) = 0.$$

The heuristic gap is zero. Note that the solution found by the solver is not the one of the LPT heuristic (which put songs 6, 1, 4 on one CD and 3, 2, 5 on the other), but has the same value: the problem has several optima.

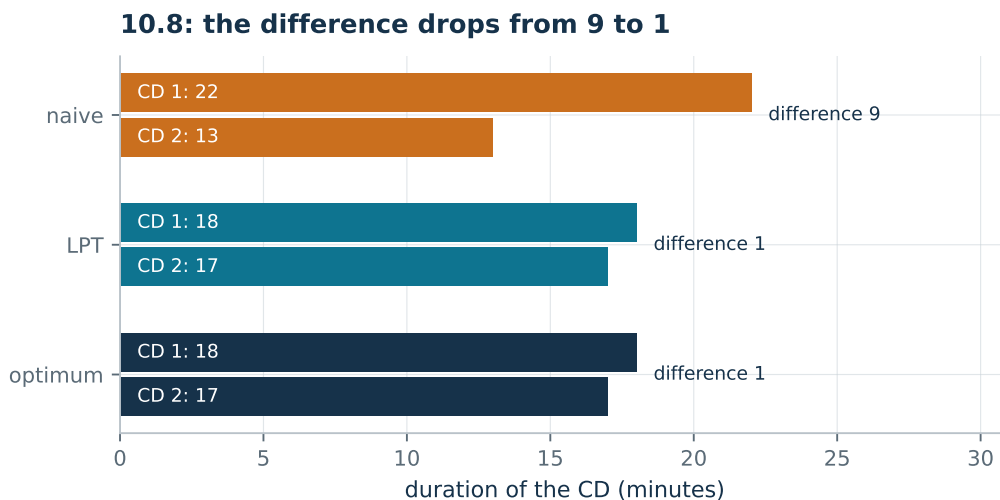


Figure 10.8: Duration of the two CDs in the three solutions: naive heuristic, LPT and optimum.

Additional considerations

- The parity argument holds as written for $m = 2$. With $m = 3$ it must be redone: 35 is not divisible by 3, so the three durations cannot all be equal, but the minimum spread is no longer immediate. On the instance with three CDs the optimum is 2 (durations 13, 11, 11).
- The constraint $\sum_{i=1}^n x_{ij} \geq w_j$ with $w_j = 1$ never bites at the optimum: a solution leaving a CD empty would have $z = 0$ and therefore a difference equal to the total duration. It is needed in the model, though, because without it the relaxation would have degenerate solutions.
- Minimising $y - z$ is not the same as minimising y (the makespan): the latter is the classic formulation of scheduling on parallel machines, and it has different optima. It is the same difference as between "balancing" and "finishing early".

Additional modelling questions

Modelling question 10.15 — a smaller medium

CD 1 is a reduced medium and cannot exceed 15 minutes. How does the model change? What is the new optimum?

Modelling question 10.16 — three CDs

The collection is distributed over three CDs instead of two. How does the model change? What is the new optimum?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
UB	1	heuristic solution (UB)
LB	1	certified bound (LB)
$z(MILP)$	1	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 0.0\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 0.0\%.$$

The sandwich on variant 8b

With three CDs instead of two the structure does not change: the columns of y and z ask the γ and the δ to sum to one, and spreading them evenly, $\gamma_j = \delta_j = 1/3$, is the choice that favours no CD. The dual is worth 0, like the relaxation, for the usual reason: a fractional solution splits every track into three equal parts and levels the durations. The useful bound comes from integrality, as in the base problem: the durations are integers summing to 35, which is not a multiple of 3, so they cannot all be equal and the difference is at least 1. The LPT heuristic leaves 2, which is the optimum: the certified gap stays open by one unit.

	value	what it is
UB	2	heuristic solution
LB	1	parity of the durations on three CDs
$z(LP)$	0	relaxation without the bounds
$z(LP^+)$	0	relaxation with the bounds
$z(MILP)$	2	optimum of the MILP

10.9 Books on shelves

Script: `python/fam10_9_shelves.py`

[Notebook in Colab](#)

Problem 10.9 — books on shelves

A library needs to arrange $n \in \mathbb{Z}_{\geq 1}$ books on shelves. Every book $b \in \{1, 2, \dots, n\}$ has width $w_b \in \mathbb{Z}_{\geq 1}$ and height $h_b \in \mathbb{Z}_{\geq 1}$. There are $m \in \mathbb{Z}_{\geq 1}$ shelves available, each of maximum width $c \in \mathbb{Q}_{>0}$. Every book must be assigned to exactly one shelf, and the total width of the books on a shelf cannot exceed c . The height of a shelf is that of the tallest book placed on it. The library wants to minimise the sum of the heights of the shelves.

The problem in words. We *decide* which shelf every book goes on. *The objective*: minimum sum of the heights (it is the timber that is saved). *The constraints*: every book on one shelf only, and no shelf wider than c .

Decision variables

n m binary variables and m non-negative continuous ones:

$$x_{bs} = \begin{cases} 1 & \text{if book } b \text{ goes on shelf } s, \\ 0 & \text{otherwise,} \end{cases} \quad y_s = \text{height of shelf } s.$$

Model 10.9 — books on shelves

$$\min \sum_{s=1}^m y_s \quad (10.17a)$$

$$\text{subject to } \sum_{s=1}^m x_{bs} = 1, \quad \forall b \in \{1, 2, \dots, n\}, \quad (10.17b)$$

$$\sum_{b=1}^n w_b x_{bs} \leq c, \quad \forall s \in \{1, 2, \dots, m\}, \quad (10.17c)$$

$$-h_b x_{bs} + y_s \geq 0, \quad \forall b \in \{1, 2, \dots, n\}, \forall s \in \{1, 2, \dots, m\}, \quad (10.17d)$$

$$x_{bs} \in \{0, 1\}, \quad \forall b \in \{1, 2, \dots, n\}, \forall s \in \{1, 2, \dots, m\}, \quad (10.17e)$$

$$y_s \geq 0, \quad \forall s \in \{1, 2, \dots, m\}. \quad (10.17f)$$

Description. The objective (10.17a) is the total height of the shelves. Constraints (10.17b), one per book, say that every book sits on exactly one shelf. Constraints (10.17c), one per shelf, are the available width. Constraints (10.17d), one per book–shelf pair, push the height y_s above that of every book placed on that shelf.

The link between the variables. Constraints (10.17d) say $y_s \geq h_b x_{bs}$: if book b is on shelf s then $y_s \geq h_b$, otherwise the constraint is empty ($y_s \geq 0$). Since y_s appears in the minimisation objective with a positive coefficient, at the optimum it equals exactly the largest height among the books present, and it is 0 if the shelf is empty. It is technique 3.5, the maximum variable, in its disaggregated form: one constraint per book instead of a single one with a big-M.

An empty shelf costs nothing

With $y_s \geq 0$ and no other constraint, an empty shelf has $y_s = 0$ and does not contribute to the objective. It follows that giving the library more shelves can never worsen the optimum, but it need not improve it: splitting the books over more shelves means paying more heights. On the instance, with three shelves instead of two, the optimum stays 15.

The model in gurobipy

```

1 mod = gp.Model("shelves")
2 x = mod.addVars(n, m, vtype=GRB.BINARY, name="x")
3 y = mod.addVars(m, name="y")
4 mod.setObjective(y.sum(), GRB.MINIMIZE)
5 mod.addConstrs((x.sum(b, "*") == 1 for b in range(n)), name="book")
6 mod.addConstrs((gp.quicksum(w[b] * x[b, s] for b in range(n)) <= c
7                  for s in range(m)), name="width")
8 mod.addConstrs((y[s] - h[b] * x[b, s] >= 0 for b in range(n) for s in
9                  range(m)),
10                name="height")

```

The instance

$n = 4$ books, $m = 2$ shelves, $c = 10$:

	$b = 1$	$b = 2$	$b = 3$	$b = 4$
w_b	3	5	4	6
h_b	8	5	7	4

The total width of the books is 18, the overall capacity $2 \cdot 10 = 20$.

Constructive heuristic: two orders, two outcomes

The rule is first-fit: every book on the first shelf where it fits. As in problem 10.8 the order changes everything — but here, in one order, the heuristic *fails*.

It is a constructive heuristic of the family seen in chapter 5: a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

(a) Decreasing height (1, 3, 2, 4). Book 1 (width 3) on shelf 1; book 3 (width 4) on shelf 1, which reaches 7; book 2 (width 5) on shelf 2. Book 4, of width 6, is left: shelf 1 has 3 of residual width, shelf 2 has 5. **The heuristic fails.**

(b) Decreasing width (4, 2, 3, 1). Book 4 (width 6) on shelf 1; book 2 (width 5) on shelf 2; book 3 (width 4) on shelf 1, which reaches 10; book 1 (width 3) on shelf 2, which reaches 8. Heights: $\max(4, 7) = 7$ and $\max(5, 8) = 8$, sum 15.

$$z(MILP) \leq UB = 15.$$

The decreasing-height order is the one suggested by the *objective*, but the constraint that can make an insertion infeasible is the *width*: that is what one must sort on. It is the same logic as first-fit decreasing for bin packing, where one sorts by size and not by value. A constructive heuristic that may fail is not wrong in itself — one only has to foresee the case and change the order — but it must be said explicitly, because a heuristic that "sometimes returns nothing" provides no bound at all.

LP relaxation and dual: the dual bound

We associate α_b free with constraints (10.17b) (equalities), $\beta_s \leq 0$ with (10.17c) ($\leq$ direction in a minimisation) and $\gamma_{bs} \geq 0$ with (10.17d). The dual is:

Dual of the LP relaxation of model 10.9

$$\max \sum_{b=1}^n \alpha_b + c \sum_{s=1}^m \beta_s \quad (10.18a)$$

$$\text{subject to } \sum_{b=1}^n \gamma_{bs} \leq 1, \quad \forall s \in \{1, 2, \dots, m\}, \quad (10.18b)$$

$$\alpha_b + w_b \beta_s - h_b \gamma_{bs} \leq 0, \quad \forall b \in \{1, 2, \dots, n\}, \forall s \in \{1, 2, \dots, m\}, \quad (10.18c)$$

$$\alpha_b \geq 0, \quad \forall b \in \{1, 2, \dots, n\}, \quad (10.18d)$$

$$\beta_s \leq 0, \quad \forall s \in \{1, 2, \dots, m\}, \quad (10.18e)$$

$$\gamma_{bs} \geq 0, \quad \forall b \in \{1, 2, \dots, n\}, \forall s \in \{1, 2, \dots, m\}. \quad (10.18f)$$

Description. The variable α_b is the value of book b , β_s the (non-positive) price of the width of shelf s , and γ_{bs} the price of the link "the height of shelf s covers book b ". The objective (10.18a) prices the books and the width available on each shelf. Constraints (10.18b) are the columns of the y_s : the height of shelf s costs 1 in the primal objective, and the prices of the links that push it up cannot be worth more. Constraints (10.18c) are the columns of the x_{bs} : putting book b on shelf s satisfies its assignment constraint (α_b), takes up w_b of width and forces the height up to h_b ; the balance cannot be positive, because the x_{bs} do not appear in the objective.

A hand-built dual solution. Set $\beta = 0$ (the width is not priced) and concentrate all the "weight" γ on the tallest book, number 1 with $h_1 = 8$: $\bar{\gamma}_{1s} = 1$ for every s and $\bar{\gamma}_{bs} = 0$ for $b \neq 1$. Constraints (10.18b) become $1 \leq 1$. Constraints (10.18c) become $\alpha_1 \leq 8$ for $b = 1$ and $\alpha_b \leq 0$ for $b \neq 1$: take $\bar{\alpha}_1 = 8$ and $\bar{\alpha}_b = 0$. The value is

$$LB = 8.$$

It is the obvious remark: the shelf holding the tallest book is at least as tall as that book.

A stronger combinatorial bound

The total width of the books is 18 and every shelf holds 10: at least

$$\left\lceil \frac{18}{10} \right\rceil = 2$$

shelves must be non-empty. One of them holds the tallest book and measures at least 8; the other one holds at least one book, so it measures at least $\min_{b \neq 1} h_b = 4$. Summing up,

$$z(MILP) \geq LB = 8 + 4 = 12,$$

better than the dual bound 8. Here too the jump comes from integrality: the relaxation can put half a book on each shelf and pay half the height twice, that is 8 in total again.

Optimal solution

The optimum is obtained by solving the model: the two bounds just built, the primal and the dual one, are what helps the solver get there.

	books	width	height
shelf 1	1, 2	8 out of 10	8
shelf 2	3, 4	10 out of 10	7

$$z(MILP) = 15, \quad 12 = LB \leq z(MILP) = 15 \leq UB = 15, \quad z(LP) = z(LP^+) = 8.$$

The heuristic gap is zero: first-fit by decreasing width finds the optimum. The certified gap, before solving the MILP, is $(15 - 12)/15 = 20\%$.

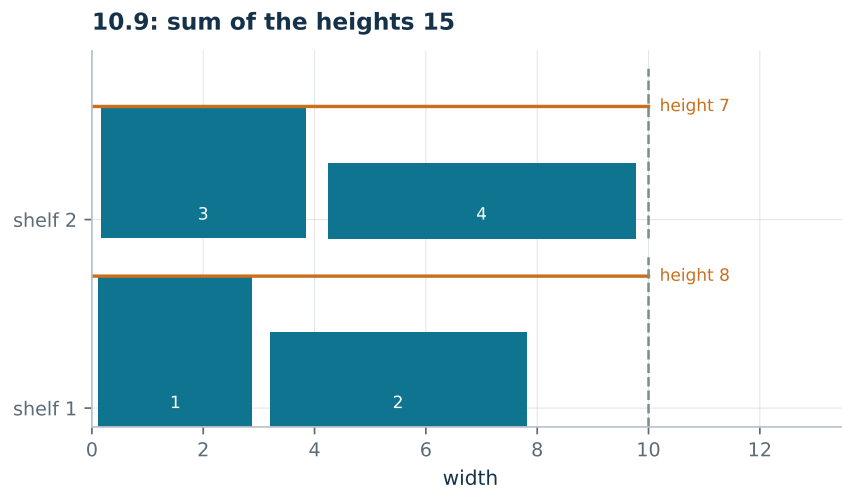


Figure 10.9: Optimal arrangement: the books on each shelf, with the shelf height marked in orange and the maximum width dashed.

Additional considerations

- The problem is a variant of bin packing in which the cost of a container is not fixed but depends on its contents. It is known as *bin packing with item fragmentation* or, in the warehousing literature, as *shelf space allocation*.
- The model has an annoying symmetry: swapping the two shelves gives the same solution with the same value. On larger instances it is worth breaking it, for instance by imposing $y_1 \geq y_2 \geq \dots \geq y_m$.
- The disaggregated form (10.17d) uses $n m$ constraints. The aggregated form with a big-M, $y_s \geq h_b - M(1 - x_{bs})$, would use just as many and is weaker: here the disaggregated one costs nothing and is preferable.

Additional modelling questions

Modelling question 10.17 — a third shelf

The library buys a third shelf, as wide as the others. What is the new optimum?

Modelling question 10.18 — wider shelves

The shelves are 12 wide instead of 10. What is the new optimum?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
UB	15	heuristic solution (UB)
LB	12	certified bound (LB)
$z(MILP)$	15	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 0.0\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 20.0\%.$$

The sandwich on variant 9b

The width does not enter the dual certificate: the recipe puts the whole γ weight on the tallest book — number 1, 8 tall — because whatever shelf holds it must be at least as tall, and the dual bound stays 8. The combinatorial argument of the base problem is needed here too: the total width is 18 and one shelf holds 12, so the non-empty shelves are still at least two, one at least 8 tall and the other at least 4, that is $LB = 12$. What changes is the heuristic: first-fit by decreasing width now fits the four books on two shelves 5 and 8 tall and drops from 15 to 13; the optimum from 15 to 12, and the bound certifies it.

	value	what it is
UB	13	heuristic solution
LB	12	shelves used and minimum heights
$z(LP)$	8	relaxation without the bounds
$z(LP^+)$	8	relaxation with the bounds
$z(MILP)$	12	optimum of the MILP

10.10 Two problems to model

The nine preceding problems reach the reader already dissected: variables, model, instance, bounds. The two that close the chapter do not. They are given as they really arrive — a text, some data, a question — and the work of recognising the structure is left to be done.

The steps are always the same:

1. list the **decisions**, one by one, and give each of them a variable with its domain;
2. translate every sentence of the statement into **one constraint**, and count how many constraints there are as a function of the data;
3. check the **links** between variables with a truth table;
4. build by hand a **feasible solution** and a **dual solution**, to have the two bounds;
5. solve the model and **compare**.

The solutions to these questions, like all the others in the course, are in the document reserved for instructors and are not published.

The first: a depot and its vans

Problem 10.10 — the depot's week

A depot supplies three shops for five days. Each day each shop has a known requirement, in pallets. The depot has an initial stock and can receive pallets from the factory with a daily order: an order costs a fixed amount to activate the transport, plus an amount per pallet. Pallets not delivered stay in the depot, and the end-of-day stock costs. Every delivery to a shop travels on a van that carries at most a given number of pallets, and every van that leaves costs. On a given day the depot cannot send out more than a given number of vans in total. The minimum-cost weekly plan is wanted.

Modelling question 10.19 — the decisions and the variables

List the decisions of the problem and, for each of them, the type of variable that corresponds to it. How many families of variables are needed, and with which indices?

Modelling question 10.20 — the model

Write a complete MILP model. Say, for each family of constraints, which sentence of the statement it translates and how many constraints there are as a function of the data.

Modelling question 10.21 — the links

At least three links between variables from chapter 3 appear in the model. Which ones? For each of them write the truth table and say whether the missing direction is imposed by a constraint or by optimality.

Modelling question 10.22 — the bounds

Build by hand a feasible solution and a dual solution of the LP relaxation. Say which of the two values is LB and which is UB , and why.

The second: shifts and skills

Problem 10.11 — the technical desk

A support service covers seven days. Each day a known number of technicians must be present. Technicians are hired on weekly contracts of fixed shape: a contract states which days are worked, and has a known weekly cost. Each technician has one of three skills, and each day at least one technician of each of the three is needed. Two contracts are incompatible with the rarest skill. The minimum-cost plan is wanted.

Modelling question 10.23 — the decisions and the variables

The variables count technicians, not choices: why? What difference does it make, in the model and in the relaxation, to declare them integer instead of binary?

Modelling question 10.24 — the model

Write a complete MILP model, with the indices always accompanied by the set they range over.

Modelling question 10.25 — a bound read off the data

Build a certified bound without writing the dual, by counting the man-days covered by one contract. Compare it with the value of the LP relaxation.

Modelling question 10.26 — a strengthening inequality

Find a valid inequality that the model does not contain and measure how much gap it closes: solve the two relaxations, with and without it, and report the difference.

Part III

The course

Course organization

11.1 The three-part path

	Content	Learning objectives
Part I	Modelling	recognise a link between variables (activation, maximum, big-M, if and only if . . .) and prove that the model really imposes it
Part II	The problems	apply the links to three families of real problems and to the mixed models, from the model to Gurobi code
Part III	The course	put what was learned to the test with the additional modelling questions

11.2 The format of every exercise (and of the exam)

Every problem of the course — and every exam question — follows the same four-question scheme:

1. **Model.** Write the MILP: variables (with their count), objective, constraints with their description, and — if two linked families of variables are involved — explain the link: what is the implication, and how the constraint (or the optimum) imposes it, in both directions.
2. **Instance.** Write the model for a small numerical instance.
3. **Heuristic.** Design a constructive algorithm that finds a feasible solution of the instance, and run it step by step to obtain an upper bound (lower bound if the problem is a maximisation).
4. **Dual.** Write the dual of the LP relaxation, for the general model and for the instance, and hand-build a feasible dual solution to obtain the bound from the other side.

The resulting $LB \leq z(MILP) \leq UB$ is the thread running through the course: a model is not just written down, it is squeezed from both sides before it is handed to the solver.

11.3 The additional questions and their solutions

Every problem closes with two or three *additional modelling questions*: they change a datum or add a constraint, and ask how the model changes and what the new optimum is. They are the part worth practising on, because the base model is read, the variant is written.

The solutions of the additional questions are reserved for instructors: they live in a separate booklet, which is not published. What stays available to everyone — and it is what one needs in order to know what is expected — is the *sandwich worked out in full* on one variant of each exercise of chapters 7 and 8: the model of the variant, a feasible constructive heuristic run step by step, a dual certificate built by hand and the bound table. It is the model answer the exam expects.

11.4 Grading criteria

Dimension	Weight
Correctness of the model (variables, objective, constraints)	35%
Proof of the link between the variables (both directions)	25%
Constructive heuristic and correct execution	20%
Dual of the relaxation and feasible dual solution	20%

11.5 Typical discussion questions

- Is the link constraint aggregated or disaggregated? What difference does it make to the LP relaxation?
- Is the opposite direction of the implication imposed by the constraint or does it follow from the optimum? How is it proved?
- What is the smallest big-M that can be justified from the data?
- Does the heuristic find the optimum? How can one know without the solver?
- Is the hand-built dual optimal for the relaxation, or only feasible?

11.6 The most common mistakes

1. Writing an implication whose thesis is true regardless (vacuous contrapositive): before writing it, check that antecedent and consequent are both genuine.
2. Proving only one direction of an implication “imposed by the constraint” — both “if”s are always needed.
3. Confusing a relation “imposed by the constraint” with one that “follows from the optimum”: the second requires the six-step exchange argument, not just “it’s clearly worth it”.
4. Concluding “in every optimal solution” when the coefficient is only ≥ 0 (not > 0): the correct conclusion is weaker, “there exists an optimum in which...”.
5. Using a huge big-M “to be safe”: it needlessly worsens the LP relaxation.

6. Forgetting that a mutual-exclusion constraint ($A + B \leq 1$) does not imply that at least one of the two must equal 1.
7. Writing the dual of the relaxation with the wrong direction or sign of a dual variable relative to the direction of the primal constraint.
8. Building a feasible dual solution without checking its feasibility on all the constraints.

11.7 Reproducibility

```
python3 -m pip install gurobipy matplotlib pandas
python3 python/run_all.py           # regenerates data, results, figures and
    notebooks
python3 python/check_numbers.py     # checks every number quoted in the notes
```

Index

- absolute value, 11, 228
- activation, 22, 26, 28, 49, 88, 102, 109, 127, 128, 146, 149, 154, 163, 165, 168, 206, 212
- big-M, 24, 32, 34, 93, 94, 102, 126, 139, 140, 143, 144, 173, 189, 191, 212, 238, 241, 247
- binary expansion, 36
- capacitated location, 28
- certified gap, 12, 45, 50, 55, 176, 197, 241
- constructive heuristic, 45, 47, 51, 52, 56, 57, 76, 78, 79, 86, 97, 102, 105, 112, 119, 125, 130, 131, 136, 142, 146, 149, 170, 175, 183, 189, 196, 201, 207, 213, 218, 224, 229, 234, 239
- dual bound, 11, 12, 43, 48, 53, 55, 66, 68, 76, 86, 93, 94, 190, 193, 197, 214, 225, 241
- exchange argument, 25, 30–32, 37, 40, 247
- fixed cost, 27, 102, 108, 113, 120, 127, 151, 170, 171, 176
- inventory balance, 170, 172
- knapsack, 46, 48, 50, 55, 94, 103, 127, 131
- LP relaxation, 9, 10, 86, 102, 106, 112, 143, 146, 150, 156, 161, 166, 170, 173, 175, 177, 183, 193, 196, 202, 207, 224, 230, 235, 236, 243, 244, 246, 247
- maximum variable, 102, 146, 238
- minimum lot, 29, 76, 170, 187, 189, 191, 193, 206–210
- p-median, 153
- piecewise linear function, 197
- precedences, 143
- price of integrality, 95, 98
- primal bound, 12, 52, 66, 193
- semi-continuous variable, 28, 75, 170, 187, 206
- set covering, 12, 44, 77, 86, 97
- set packing, 12, 86, 200, 203
- set partitioning, 12, 36, 68
- strong duality, 11, 43, 45, 107, 151, 230
- tolerances, 11
- valid inequalities, 51, 90, 114, 121, 126, 132, 144, 168, 198, 203
- weak duality, 107, 120, 151, 156, 167